# A Structural Approach to Interpretable Deep Learning

*From DC Decompositions to Two-Player Games*

Thesis in partial fulfillment of the requirements for the degree
Master of Science (M.Sc.)

submitted by
Jakob Zimmermann

Under the scientific supervision of
**Prof. Dr. Wojciech Samek**

Under the scientific guidance of
**Georg Loho**

Berlin, November 2026




Technische Universität Berlin
Fakultät IV – Elektrotechnik und Informatik
Fraunhofer Heinrich-Hertz-Institut
Department of Artificial Intelligence

# Affidavit

## Deutsch

1) Hiermit versichere ich, dass ich die vorliegende Arbeit ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen, die den benutzten Quellen und Hilfsmitteln unverändert oder inhaltlich entnommen sind, habe ich als solche kenntlich gemacht.
2) Sofern generative KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.
3) Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023[1] habe ich zur Kenntnis genommen.
4) Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Jakob Zimmermann
Berlin, 10. November 2026

[1] https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

## English

1) I hereby declare that I have written this thesis without the help of third parties and exclusively using the sources and aids listed. I have marked as such all passages that are taken from the sources and aids used, either unchanged or in terms of content.
2) Where generative AI tools were used, I have stated the product name, manufacturer, the software version used and the respective purposes (e.g. linguistic review and improvement of texts, systematic research). I am fully responsible for the selection, adoption and all results of the AI-generated output I used.
3) I have taken note of the Statutes for Safeguarding Good Scientific Practice at TU Berlin dated February 15, 2023[2].
4) I further declare that I have not yet submitted the work in the same or a similar form to any other examination authority.

---

Jakob Zimmermann
Berlin, November 10, 2026

[2] https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

# Acknowledgements

I thank Gerrit Holzbach for giving me the opportunity to work on machine learning and gain a practical perspective. I am grateful to Georg Loho for opening my eyes to the theory of deep learning and polyhedral geometry.

I thank Grégoire Montavon for taking the time to discuss the idea of DC decomposition with me and for pointing me toward the Quantus framework and the intricacies of evaluating attribution methods. I am grateful to Wojciech Samek for his Responsible AI lecture, which made me aware of the problems in this area and strategies for addressing them. I also thank him for encouraging me to work on transformers and more modern machine learning models and paradigms, for asking me to study parameter randomization, for our conceptual discussions, and for his organizational support.

I thank Klaus Robert Müller for the interesting discussions of these results. I am also grateful to Shinichi Nakajima for the interesting discussions and for comparing my trajectory distribution view with the Normalized Relevance Measure framework and its signed-measure perspective on layer-wise relevance propagation and concept relevance propagation.

# Abstract


## Deutsch

Interpretierbarkeit dient zunehmend der Modellverbesserung und -steuerung neben der nachträglichen Analyse einzelner Vorhersagen. Wenn Attributionen die Korrektur unerwünschter Merkmale, das Training oder die Inferenz beeinflussen, muss ihre Berechnung verstanden und überprüft werden. Diese Arbeit verbindet dafür strukturierte Darstellungen interner Beiträge mit einer mathematischen Beschreibung ihrer Rückwärtspropagation.

Ein trainiertes neuronales Netz zu verstehen heißt, nachzuvollziehen, wie gegenläufige Einflüsse Eingaben in Ausgaben überführen. Diese Arbeit untersucht Interpretierbarkeit durch *Zerlegung*, die vorhandene interpretierbare Struktur eines trainierten ReLU-Netzes ohne zusätzliche Interpretierbarkeitsbeschränkungen offenlegt. Eine *Netzzerlegung* hebt die Berechnung in ein Paar koordinierter, nichtnegativer Ströme an, deren Differenz das ursprüngliche Netz exakt reproduziert. Dieser gemeinsame Begriff vereinheitlicht mehrere Strukturen und erhält jede Vorhersage.

Die erste ist die Differenz-konvexer-Funktionen-Zerlegung (DC-Zerlegung). Monotonie fördert Interpretierbarkeit und beschränkt Ausdrucksstärke. Wir zerlegen deshalb ein trainiertes ReLU-Netz — ein MLP oder CNN — in Teilnetze $g$ und $h$ mit $f = g - h$, die in verdoppelten Eingabekoordinaten konvex und monoton sind und positive und negative Evidenz auf getrennten Pfaden führen. Wir stabilisieren die ausgewertete ReLU-Zerlegung und konstruieren aus absoluter Krümmung exakte Paare für glatte Aktivierungen (GELU, Swish, Tanh, Sigmoid, Softplus). Explizite Softmax-, Produkt- und LayerNorm-Paare mit positivem $\varepsilon$ ergeben eine DC-Zerlegung deterministischer Transformer beliebiger endlicher Tiefe; daraus folgt keine Garantie numerisch stabiler Auswertung. Auf der ReLU-Zerlegung bauen wir die Saliency-Methoden SplitCAM und SplitLRP auf: Sie zeigen auf VGG-16 und ResNet-18 über ImageNet-S konfigurationsabhängige Verbesserungen bei Lokalisierung und Selectivity sowie Zielkonflikte bei Pixel Flipping und Robustheit. Zudem trainieren wir Differenzen konvexer oder monotoner Netze zur MNIST-Klassifikation und beobachten klassenabhängige Stromgradientenmuster als qualitative Illustration der Darstellung.

Die zweite ist die Stopp-Zerlegung: Sie teilt Gewichte nach Vorzeichen auf, $W = W^+ - W^-$, trennt exzitatorische und inhibitorische Pfadbeiträge und verbindet Rückwärtspropagation mit Spieltheorie. Aufbauend auf dem ReLU-Netz-Spiel von Gaubert und Vlassopoulos stellen wir Rückwärts-Attribution als Zwei-Spieler-Nullsummen-Stoppspiel auf einem erweiterten Netzgraphen dar. Gradienten und die vollständige $\alpha\beta$-LRP-Familie ergeben sich unter bestimmten Gleichgewichten als Integrale über Spieltrajektorien; Attributionskarten sind Projektionen der zugrunde liegenden Trajektorienverteilung. Erwünschte Lokalisierungsschärfe, Robustheit gegenüber Eingangsrauschen und stabiles Attention-Routing lassen sich durch Politik-Regularisierung, Risikoaversion und erweiterte Aktionsräume formulieren und unmittelbar in neue Rückwärtsregeln übersetzen. Auf ViT-B/16 übertrifft eine solche Anpassung von $\alpha\beta$-LRP alle betrachteten Vergleichsmethoden, einschließlich transformer-spezifischer Methoden, bei der Attributionslokalisierung. Die revidierte Evaluation wählt Lokalisierungs- und Faithfulness-Konfigurationen unabhängig auf Validierungsdaten aus und berichtet Konfidenzintervalle und gepaarte Tests; andere Metriken zeigen Zielkonflikte.

Ein Rahmen erschließt somit mehrere Lesarten derselben trainierten Gewichte. Unsere Trajektorienverteilungs-Sicht vergleicht das Routing im Rückwärtsdurchlauf durch eine Hellinger-Distanz zwischen normalisierten Routing-Verteilungen, bevor signierte, diskontierte Belegungsmaße modellspezifisch auf Heatmaps projiziert werden.

**Schlüsselwörter:** *DC-Zerlegung, Interpretierbarkeit, Neuronale Netze, Spieltheorie, Attribution*

## English

Interpretability increasingly supports model improvement and control alongside post hoc analysis of individual predictions. When attributions guide spurious-feature correction, training, or inference, their calculation must be understood and checked. This thesis connects structured representations of internal contributions with a mathematical account of their backward propagation.

Understanding a trained neural network means tracing how competing influences turn inputs into outputs. This thesis studies interpretability through *decomposition*, exposing interpretable structure already present in a trained ReLU network without imposing interpretability constraints. A *network decomposition* lifts the computation into a pair of coordinated, non-negative streams whose difference exactly reproduces the original network. This single notion unifies several structures while preserving every prediction.

The first is the difference-of-convex (DC) decomposition. Monotonicity aids interpretability while restricting expressivity. We therefore split a trained ReLU MLP or CNN into subnetworks $g$ and $h$ with $f = g - h$, convex and monotone in doubled input coordinates and carrying positive and negative evidence on separate pathways. We stabilize the evaluated ReLU split and construct exact smooth-activation pairs (GELU, Swish, tanh, sigmoid, softplus) from absolute curvature. Explicit softmax, product, and positive-epsilon LayerNorm pairs yield a DC decomposition of deterministic Transformers of arbitrary finite depth; this does not guarantee numerically stable evaluation. We build the saliency methods SplitCAM and SplitLRP on the ReLU representation, showing configuration-dependent gains in localization and Selectivity on VGG-16 and ResNet-18 over ImageNet-S, with trade-offs in Pixel Flipping and robustness. We also train differences of convex or monotone networks for MNIST classification and observe class-dependent stream-gradient patterns, qualitatively illustrating the representation.

The second is the stopping decomposition, which splits weights by sign, $W = W^+ - W^-$, separating excitatory and inhibitory path contributions and linking backward propagation to game theory. Building on Gaubert and Vlassopoulos' ReLU Net Game, we represent backward attribution as a two-player zero-sum stopping game on an extended network graph. Gradients and the full $\alpha\beta$-LRP family arise as integrals over game trajectories under specific equilibria; attribution maps project the underlying trajectory distribution. Desired localization focus, robustness to input noise, and stable attention routing can be formulated through policy regularization, risk aversion, and extended action sets, translating directly into new backward rules. On ViT-B/16, one such adaptation of $\alpha\beta$-LRP outperforms all evaluated baselines, including transformer-specific methods, in attribution localization. The revised evaluation selects localization and faithfulness configurations independently on validation data and reports confidence intervals and paired tests; other metrics expose trade-offs.

One framework thus offers several readings of the same trained weights. Our trajectory-distribution view compares backward routing through a Hellinger distance between normalized routing laws before the model-dependent projection of signed discounted occupation measures onto heatmaps.

**Keywords:** *DC Decomposition, Interpretability, Neural Networks, Game Theory, Attribution*

# Contents

**List of Figures** i

**List of Tables** ii

**List of Symbols** iii

**List of Abbreviations** v

**1 Introduction** 1
- 1.1 Motivation and Problem Statement . . . 1
- 1.2 Contributions of This Thesis . . . 4
- 1.3 Positioning within the Literature . . . 5

**2 Mathematical Foundations** 7
- 2.1 Neural Network Notation and PWL Mappings . . . 7
- 2.2 Convexity and Monotonicity in Neural Networks . . . 9
- 2.3 Game Theory and Markov Decision Processes . . . 11

**3 Network Decompositions and DC Formulations** 16
- 3.1 Generalized Network Decomposition . . . 16
- 3.2 Convex and Monotone DC Decompositions . . . 18
- 3.3 The Mixing Decomposition family . . . 29
- 3.4 The Stopping Decomposition . . . 30
- 3.5 Regularized Decompositions . . . 33
- 3.6 Numerical Stabilization of Split Passes . . . 35
- 3.7 Generalization to Convolutional Neural Networks (CNNs) . . . 39

**4 Game-Theoretic Framework** 43
- 4.1 Related Work . . . 43
- 4.2 Backward Attribution . . . 44
- 4.3 Attribution as a Game . . . 45
- 4.4 Game-Theoretic Modes . . . 46
- 4.5 Signed affine output logits . . . 47

**5 Extensions: Attention and Transformers** 49
- 5.1 Related Work: Interpretability in Transformers . . . 49
- 5.2 Decomposing Multiplicative Operations . . . 50
- 5.3 A Constructive DC Decomposition of Transformers . . . 52
- 5.4 Attention via a detached-logit oracle . . . 56
- 5.5 Extension: Vision Transformer Attention . . . 56

**6 Saliency and Attribution Methods** 60
- 6.1 Split-Based and Difference-of-Convex Attribution . . . 60
- 6.2 Game-Based Attribution: Modes and Technicalities . . . 70

**7 Experiments and Evaluation** 72
- 7.1 Experimental Setup . . . 72
- 7.2 Split Methods: VGG-16 . . . 72

7.3 Split Methods: ResNet-18 . . . 73
7.4 Split-Method Stabilization Ablations . . . 73
7.5 Game-Based Case Study . . . 75

**8 The Trajectory Distribution View** **79**
8.1 Comparison with Normalized Relevance Measures . . . 79
8.2 Trajectory Sanity Check . . . 80
8.3 Hellinger Distance and Disagreement Map . . . 81
8.4 Randomization Sanity Checks . . . 85

**9 Conclusion and Future Work** **91**
9.1 Summary of Contributions . . . 91
9.2 Limitations . . . 92
9.3 Broader Impact . . . 93
9.4 Future Research Directions . . . 93

**A Experimental Protocol and Reproducibility** **95**
A.1 Split-Method Evaluation . . . 95
A.2 Evaluation Protocol . . . 95
A.3 Formal Verification and Mathematical Audit . . . 98

**B Publication Provenance and Tool Use** **99**
B.1 Author's Contribution . . . 99
B.2 Disclosure in the Revised Game Paper . . . 99
B.3 AI Assistance in the Thesis Revision . . . 99

**List of References** **101**

# List of Figures

1.1 The backward pass as a two-player game . . . . . 3

3.1 The network splitting procedure . . . . . 16
3.2 Monotone, convex split streams and a standard ReLU layer . . . . . 22

6.1 Stream gradients of the inverted-input DIC model . . . . . 68
6.2 Second example of inverted-input DIC stream gradients . . . . . 69
6.3 Input-gradient visualizations for ten MNIST digits . . . . . 69

7.1 VGG-16 saliency map comparison . . . . . 73

8.1 Hellinger trajectory distance under parameter randomization . . . . . 80
8.2 Input-noise Hellinger trajectory distance . . . . . 84

# List of Tables


3.1 Closed scalar profiles for smooth activations . . . . . 23
3.2 Linear CNN chunks . . . . . 40

4.1 Modes of the games . . . . . 47

7.1 Selected interpretability evaluation configurations: VGG-16 . . . . . 74
7.2 Selected interpretability evaluation configurations: ResNet-18 . . . . . 75
7.3 Alpha comparison for SplitGrad and SplitCAM . . . . . 75
7.4 ViT-B/16 on the ImageNet-S test split . . . . . 77

8.1 Reference baselines for the input-noise sweep . . . . . 85
8.2 Parameter randomization sanity check . . . . . 87
8.3 Hellinger trajectory distance at the double-random baseline . . . . . 88
8.4 Per-image Spearman rank correlations . . . . . 89

# List of Symbols

## Greek Symbols

| Symbol | Description | Unit |
|---|---|---|
| $\alpha$ | Backward pass shift parameter | |
| $\delta$ | Network gradient (sensitivity) | |
| $\delta^g, \delta^h$ | Split stream gradients | |
| $\gamma$ | Discount factor / LRP emphasis parameter | |
| $\pi, \sigma$ | Policies of Max and Min players | |
| $\tau$ | Temperature / entropy regularization parameter | |
| $\varepsilon$ | Numerical stability constant | |

## Indices

| Symbol | Description |
|---|---|
| $c$ | Channel index |
| $i$,$j$ | Neuron indices |
| $l$ | Layer index |

## Latin Symbols

| Symbol | Description | Unit |
|---|---|---|
| $a^{(l)}$ | Post-activation at layer $l$ | |
| $a^{(l,+)}, a^{(l,-)}$ | Positive and negative activations of split network | |
| $d^{(l)}$ | Width of layer $l$ | |
| $f$ | Network function | |
| $g$ | Positive split stream (monotone, convex) | |
| $h$ | Negative split stream (monotone, convex) | |
| $L$ | Number of hidden layers | |
| $P^l_{i\to j}$ | Transition probability in game | |
| $Q[l]$ | Auxiliary network with min and absolute weights from layer $l$ | |
| $Q^l$ | $Q$-function (action-value function) | |
| $R^{(l)}$ | LRP relevance score at layer $l$ | |
| $V^l_{i,\pm}$ | Game value function at state ($l$,$i$,$\pm$) | |
| $W^{(l)}$ | Weight matrix of layer $l$ | |
| $W^{(l,+)}, W^{(l,-)}$ | Positive and negative parts of weight matrix | |
| $x$ | Network input | |
| $z^{(l)}$ | Pre-activation at layer $l$ | |

## Operators

| Symbol | Description | Definition |
|---|---|---|
| $\nabla$ | Gradient operator | $\left[\frac{\partial}{\partial x_1}, \ldots, \frac{\partial}{\partial x_n}\right]^T$ |
| $\odot$ | Hadamard (element-wise) product | |
| ReLU | Rectified linear unit | $\max\{0, x\}$ |

# List of Abbreviations

## Methods

| | |
|---|---|
| **ADF** | Assumed Density Filtering |
| **AL** | Attribution Localization |
| **CAM** | **C**lass **A**ctivation **M**apping |
| **CPWL** | **C**ontinuous **P**iece**w**ise **L**inear |
| **DC** | **D**ifference of **C**onvex |
| **GAE** | Generic Attention Explainability |
| **ICNN** | **I**nput-**C**onvex **N**eural **N**etwork |
| **LRP** | **L**ayer-wise **R**elevance **P**ropagation |
| **MDP** | **M**arkov **D**ecision **P**rocess |
| **NRM** | **N**ormalized **R**elevance **M**easure |
| **PG** | Pointing Game |
| **RG** | **R**outing **G**ame |
| **Sel** | Selectivity |
| **SG** | **S**topping **G**ame |
| **TIBAV** | Transformer Interpretability Beyond Attention Visualization |

## Architectures

| | |
|---|---|
| **CNN** | **C**onvolutional **N**eural **N**etwork |
| **GELU** | Gaussian Error Linear Unit |
| **LLM** | **L**arge **L**anguage **M**odel |
| **MLP** | Multi**l**ayer **P**erceptron |
| **ReLU** | **R**ectified **L**inear **U**nit |
| **ViT** | **Vi**sion **T**ransformer |

# 1 Introduction

## 1.1 Motivation and Problem Statement

Interpretability is moving from post hoc inspection toward practical model improvement and control. As interpretations guide model correction, training, and inference (Anders et al., 2022; Komorowski et al., 2026; Pahde et al., 2023; Ross et al., 2017), we ask whether predictions are accurate, whether models use task-appropriate evidence, and whether we can understand, question, and correct that use. For example, attribution can investigate whether a language model draws on supplied context or knowledge acquired during training (Kahardipraja et al., 2025), and guide decoding toward supplied instructions or evidence (Komorowski et al., 2026).

For vision, this concerns shared visual grounding. When we ask a machine what it sees, seek its help, or entrust it with a task, we expect some alignment between its use of visual information and our understanding of the scene. A robot asked to grasp a cup should recognize its shape and position. A radiology assistant should attend to signs of disease, rather than clues about where the image was acquired—a documented shortcut (Zech et al., 2018). We seek decisions grounded in task-relevant objects, features, and relations that we can understand and question. Muttenthaler et al. (2025) show that human similarity judgments can guide visual representations toward closer agreement with human perception. Attribution can investigate this alignment, providing evidence for communication, justified trust, and correction.

Heatmaps make this concrete: does a prediction rely on relevant features or incidental artifacts? Attribution has exposed "Clever Hans" strategies (Lapuschkin et al., 2019) that can undermine generalization when correlations change (Geirhos et al., 2020), posing safety concerns when vision systems guide physical actions. Physical attacks on road-sign classifiers demonstrate additional security risks from deliberate visual manipulations (Eykholt et al., 2018). Heatmaps can identify suspicious dependencies for further testing; visual inspection alone establishes neither robustness nor safety. Such scrutiny extends to spurious behavior in anomaly detection and explanations of one-class models (Kauffmann et al., 2020a,b).

Diagnosed dependencies on spurious artifacts can guide targeted corrections. Input-gradient constraints can enforce which inputs should be irrelevant (Ross et al., 2017) or encourage adversarial robustness (Ross and Doshi-Velez, 2018); Class Artifact Compensation suppresses identified artifact directions (Anders et al., 2022), while Reveal to Revise iterates detection and correction during model development (Pahde et al., 2023).

This evidence requires scrutiny. Randomization sanity checks show that some attribution methods produce similar maps after model parameters are randomized (Adebayo et al., 2018): apparently informative heatmaps may reflect input structure or architecture more than learned computation. Input dependence, perturbation fragility, and misleading modified-gradient structure raise related concerns (Ghorbani et al., 2019; Kindermans et al., 2018; Nie et al., 2018; Sixt et al., 2020). Beyond inspecting maps, we must understand their calculation and its relationship to the trained network.

Evaluation requires clarity about what metrics measure. Lipton (2018) distinguishes objectives grouped under interpretability; Doshi-Velez & Kim (2017) argue that evaluation must match the application and audience. Murdoch et al. (2019) separate predictive accuracy, descriptive accuracy, and user relevance. We study trained network computations and the backward rules attributing their outputs. We use Quantus (Hedström et al., 2023) to select faithfulness, localization, and robustness

metrics, complemented by numerical-stability and parameter-randomization checks; a convincing heatmap alone cannot establish reliance on intended evidence.

The network's arithmetic provides a structural starting point. Signed weights combine reinforcing and opposing contributions into a pre-activation; activation gates determine what passes onward. The resulting activation records their balance without showing how they accumulated. This suggests a *debate* perspective: which contributions support or oppose a class score, and how does their interaction determine it? The $\alpha\beta$ rule of ***L**ayer-wise **R**elevance **P**ropagation* (LRP) already separates positive and negative pre-activation contributions, normalizes them separately, and weights their backward redistribution through $\alpha$ and $\beta$ (Bach et al., 2015). Separating support and opposition connects how the network forms its output with how attribution traces it backward.

This thesis develops a unified mathematical framework connecting decompositions of trained network computations to backward attribution. Structured representations expose opposing contributions; games on the computational graph characterize backward attribution. Together, they make explicit what information attribution retains, how it propagates that information, and how its rules can be compared and systematically modified. Reliable visual inspection motivates the work; the relationship between network structure and attribution computation is its central research problem.

The first obstacle concerns the *representation* being interpreted. Monotonicity provides useful structure (Amos et al., 2017; Sill, 1997): with non-negative weights and monotone activations, increasing an input cannot decrease the downstream response. Within a non-negative stream, contributions accumulate without cancellation between positive and negative weighted terms (Ayinde and Zurada, 2018; Lee and Seung, 1999). Monotonicity constraints can encode chosen domain-specific fairness requirements (Wang and Gupta, 2020); these constraints alone do not guarantee fairness. For medical treatment or parole decisions, Rudin (2019) argues for inherently interpretable models with directly inspectable reasoning. Changing a trained, unconstrained network's weights to impose monotonicity generally changes its predictions.

The ***D**ifference of **C**onvex* (DC) decomposition structures the representation while preserving the learned predictor. Without requiring the network to be monotone or convex, we express it as $f = g - h$, constructing two stream functions layer by layer from its trained weights. The streams are convex and monotone in the paired, sign-separated input coordinates; their difference recovers the original network (Zimmermann and Loho, 2026). This realizes the debate perspective: one stream contributes positively to the class score, the other negatively. They separate contributions throughout the computation, distinct from the final score's positive and negative parts. These decomposition-defined roles alone do not establish a unique separation into human-interpretable concepts.

Using this representation at depth presents a computational challenge: rapidly growing streams can have a moderate difference, causing severe floating-point cancellation. *Hidden Monotonicity* stabilizes the forward and backward split passes to expose their signals without retraining (Zimmermann and Loho, 2026). Reconstructing the original computation does not ensure that numerical transformations preserve the exact DC representation's convexity and monotonicity in each stream.

To investigate the mechanisms of opposing streams exposed by DC decomposition of pretrained models, the MNIST experiments (Section 6.1.4) jointly train two small convex or monotone networks as a difference model. With complemented input $\mathbf{1} - x$ to the $h$-stream, the displayed gradients show class-dependent sensitivities on present strokes and candidate missing-stroke locations (Figure 6.1). These visual counterparts of debate motivate a factual/counterfactual interpretation requiring tests through complete-model interventions.

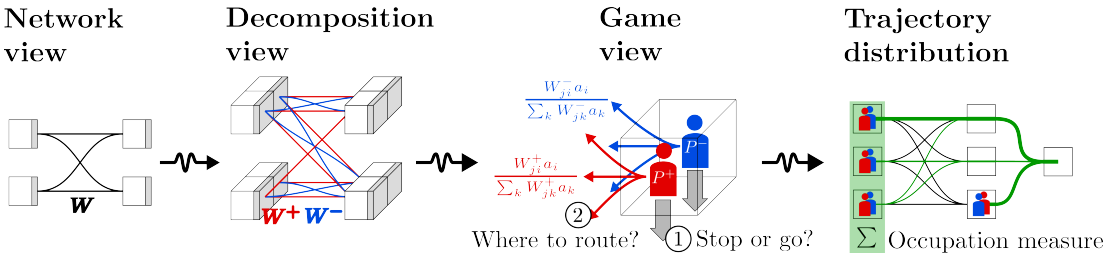



**Fig. 1.1:** The backward pass as a two-player game on a sign-separated network graph. Positive weights preserve the active player's role; negative weights switch it to the opponent. Activation states involve stopping decisions; in the routing game, additional decisions determine how play proceeds to predecessor states. Equilibrium policies induce a distribution over backward trajectories. With the prescribed signs, discounts, and output scaling, occupation measures recover the corresponding gradient or relevance attribution at the anchor settings.

The second obstacle is the zoo of backward attribution rules: local gradient sensitivities, LRP redistribution, local Deep Taylor decomposition, and transformer-specific extensions (Zimmermann et al., 2026). We unify gradients and the full $\alpha\beta$-LRP family through games of backward attribution. The stopping decomposition preserves their activation gates; the DC split retains opposing contributions even at inactive neurons.

Building on the ReLU Net Game of Gaubert & Vlassopoulos (2025), which represents forward evaluation as the value of a game played backward through the network, *Playing the Network Backward* develops this path structure into an attribution account (Zimmermann et al., 2026). Positive weights preserve the active player's role, negative weights switch it, and activation gates become stopping decisions. The stopping game represents gradients through differences of discounted occupation measures over backward trajectories. At its unit-temperature anchor, the entropy-regularized routing game recovers the full $\alpha\beta$-LRP family under the stated propagation conventions. Player roles, routing policies, and trajectory weights represent the positive and negative redistribution branches of $\alpha\beta$-LRP, embedding their separation in a two-player framework for comparison with gradients (Figure 1.1).

These representations support complementary ways to develop attribution methods. SplitCAM, SplitLRP, and SplitGrad use the DC representation; games expose stopping decisions, sign switches, and routing policies for principled modification. Policy regularization, risk aversion, and extended action sets translate desired properties into explicit changes of backward rules. Multiplicative and attention extensions reach beyond ReLU networks while distinguishing exact representations from conditioned attribution rules.

Trajectory analysis also addresses randomization sanity checks. Maps aggregate signed, discounted contributions from many backward trajectories and can obscure their routes through the model. Hellinger diagnostics compare normalized routing laws, complementing map comparisons: map comparisons test whether the presented explanation changes, and Hellinger diagnostics whether internal routing changes. Routing sensitivity alone cannot establish map faithfulness; recovering attribution also requires signs and model-dependent discounts. Experiments assess these distinct properties and trade-offs. Whether these methods improve oversight, correction, or deployment safety requires further empirical study.

Three research questions organize this account. *What structure can be exposed without changing the network's function?* We separate contributions into monotone convex streams and group paths by the sign of their weight product. *What does a backward attribution actually calculate?* Derivative identities and game occupation measures relate local rules to explicit global objects. *How can these*

*representations guide the design and evaluation of attribution methods for visual inspection?* Stable implementations, principled backward-rule modifications, and complementary map- and trajectory-level diagnostics provide a novel basis for assessing attribution's information and limitations.

**Publication basis.** This thesis brings together and extends two research works on which the thesis author is first author:

- *Hidden Monotonicity: Explaining Deep Neural Networks via their DC Decomposition*, coauthored with Georg Loho, was published at **CVPR 2026 and selected as a Highlight** (Zimmermann and Loho, 2026). [1]
- *Playing the Network Backward: A Game-Theoretic Attribution Framework*, coauthored with Jim Berend, Georg Loho, Sebastian Lapuschkin, and Wojciech Samek, is a **preprint**; this thesis uses arXiv:2605.06212v2, revised for AISTATS (Zimmermann et al., 2026).

Beyond the two papers, the thesis adds exact DC splits for smooth activations (GELU, Swish, sigmoid, and softplus), softmax, and LayerNorm, and combines them into a constructive Transformer decomposition for arbitrary finite depth. It connects the papers' decomposition and game frameworks, adds a partial Lean formalization, and reuses their experimental outputs. It corrects the uncertainty intervals in the VOC evaluation by resampling whole images. Appendix B records publication provenance and tool use.

**Author's contribution.** My principal contributions to both papers were developing the theory, developing and conducting the experiments, and writing all appendices and most text outside the introductions, in collaboration with the listed coauthors. Section B.1 distinguishes these contributions from the subsequent thesis integration and formalization.

## 1.2 Contributions of This Thesis

The thesis makes five contributions.

1) **A general framework of network decompositions.** We place DC, stopping, mixing, and regularized decompositions in a common framework and extend the DC construction to smooth activations, including GELU, Swish, sigmoid, and softplus.
2) **DC decompositions for ReLU networks, CNNs, and Transformers.** We stabilize the split forward and backward passes of pretrained ReLU networks and CNNs without changing their predictions. We also give a first exact Transformer DC decomposition algorithm within this framework, addressing the extension left open in *Hidden Monotonicity* through splits of softmax, LayerNorm, and non-piecewise-linear GELU (Section 5.3).
3) **A game-theoretic account of backward attribution.** We represent gradients and $\alpha\beta$-LRP through two-player games, providing a common basis for comparing and modifying their propagation rules and diagnosing changes in backward routing.
4) **New attribution methods with improved faithfulness and localization.** SplitCAM achieves the best Selectivity, a faithfulness metric, among the evaluated VGG-16 and ResNet-18 methods; we also introduce SplitLRP and SplitGrad. Game-derived attribution achieves the highest Attribution Localization among the evaluated ViT-B/16 methods on ImageNet-S (Gao et al., 2023), with this lead transferring to Pascal VOC without re-tuning.

[1] Official CVPR listing: https://cvpr.thecvf.com/virtual/2026/poster/36564.

5) **Directly trained difference models.** We train differences of convex or monotone networks and inspect their stream gradients on MNIST, as a counterpart to decomposing pretrained models.

## 1.3 Positioning within the Literature

This brief section positions the thesis along its three research questions; detailed chapter-opening discussions cover games and propagation rules (Section 4.1) and transformer interpretability (Section 5.1).

**What structure is hidden inside a trained network?** The use of monotonicity and convexity to structure interpretable models runs from monotonic networks (Daniels and Velikova, 2010; Sill, 1997) through certified and lattice-based monotone models (Gupta et al., 2016; Liu et al., 2020; Nguyen et al., 2023; Runje and Shankaranarayana, 2023; Sivaraman et al., 2020) to ***I**nput-**C**onvex **N**eural **N**etwork**s*** (ICNNs) (Amos et al., 2017) and convex network calculus (Bach, 2017; Calafiore et al., 2020; Sivaprasad et al., 2021). DC decompositions bridge this to unconstrained networks: classical DC programming (Le Thi and Pham Dinh, 2018; Sriperumbudur and Lanckriet, 2009; Tao and An, 1997) studies optimization with such decompositions, while the representation theory of piecewise-linear functions as differences of convex ones (Brandenburg et al., 2025; Griewank and Walther, 2020; Kazda and Li, 2024; Koutschan et al., 2025; Kripfganz and Schulze, 1987; Melzer, 1986; Schlüter and Schulze Darup, 2020; Zalgaller, 2000) guarantees the split exists; Awasthi et al. (2026) and Siahkamari et al. (2020) construct essentially the split used here, for optimization and regression, and CDiNN-type architectures make the difference structure trainable (Chen et al., 2025; Sankaranarayanan and Rengaswamy, 2022). This thesis studies the split's numerical instability in deep networks and its use for explanation. The geometric reading rests on the identification of ReLU networks with ***C**ontinuous **P**iece**w**ise **L**inear* (CPWL) maps (Arora et al., 2018; Balestriero and Baraniuk, 2021; Huchette et al., 2026) and their tropical representations (Zhang et al., 2018b); related piecewise-linear geometry informs functional identifiability (Phuong and Lampert, 2020).

**When can explanations be trusted?** The attribution toolbox spans gradient saliency (Simonyan et al., 2014), deconvolution and guided backpropagation (Springenberg et al., 2015; Zeiler and Fergus, 2014), LRP and Deep Taylor decomposition (Bach et al., 2015; Montavon et al., 2017, 2018), ***C**lass **A**ctivation **M**apping* (CAM) variants (Chattopadhay et al., 2018; Jiang et al., 2021; Selvaraju et al., 2017), axiomatic methods (Shrikumar et al., 2017; Sundararajan et al., 2017), noise averaging (Smilkov et al., 2017), and perturbation- or surrogate-based explanations (Fel et al., 2023; Fong and Vedaldi, 2017; Kolek et al., 2023; Lundberg and Lee, 2017; Ribeiro et al., 2016). A wave of critiques sharpened the trust question: sanity checks reveal parameter insensitivity (Adebayo et al., 2018), explanations can be fragile or manipulable (Alvarez-Melis and Jaakkola, 2018; Dombrowski et al., 2019; Ghorbani et al., 2019), modified backward rules can produce structure unrelated to the model's class decision (Nie et al., 2018; Sixt et al., 2020), and input dependence confounds attribution (Kindermans et al., 2018). In response, evaluation frameworks such as Quantus (Hedström et al., 2023) operationalize faithfulness, localization (Zhang et al., 2018a), robustness (Yeh et al., 2019), and complexity (Bhatt et al., 2020) as measurable desiderata — the standard this thesis holds its methods to in Chapter 7. The thesis' answer is structural rather than post hoc: attribution is computed on a representation (the split-pair) in which evidence cannot cancel invisibly, or derived from a game whose trajectory distribution records normalized backward routing and whose signed discounted occupation measures recover attribution, so sanity-check failures can be studied at both the trajectory and pixel levels

(Chapter 8); their different responses to randomization do not by themselves establish explanation faithfulness.

**How can attribution be made stable?** The closest precursor to the game framework is the ReLU Net Game of Gaubert & Vlassopoulos (2025), which identifies forward evaluation with the value of a zero-sum stopping game but connects it neither to the backward pass nor to attribution. Related stochastic views of relevance flow include Excitation Backprop's absorbing Markov chain (Zhang et al., 2018a), which the ***S**topping **G**ame* (SG) generalizes by adding an opponent, and the ***N**ormalized **R**elevance **M**easure* (NRM) framework (Xiong et al., 2026), a signed-measure calculus over walks compared with our trajectory distributions in Section 8.1. Model-agnostic Shapley-value methods such as KernelSHAP (Lundberg and Lee, 2017) play a *different*, cooperative game over input coalitions in a black-box setting, complementary to the white-box backward game here. Stability itself has been approached via curvature reduction with softplus activations (Dombrowski et al., 2022) — recovered here as entropy regularization of the stop decision. Risk-sensitive control (Fei et al., 2020) provides a separate motivation for the risk-averse game modes. On transformers, attention rollout and flow (Abnar and Zuidema, 2020), transformer relevance propagation (Achtibat et al., 2024; Ali et al., 2022; Chefer et al., 2021a,b), and artifact-aware constructions (Darcet et al., 2024; Wróbel et al., 2026) define the state of the art Chapter 5 engages with; Section 4.1 and Section 5.1 give the full accounts. The growing use of attribution as a control mechanism — model correction (Pahde et al., 2023), head identification (Kahardipraja et al., 2025; Voita et al., 2019), guided decoding (Komorowski et al., 2026), circuit localization (Hanna et al., 2024; Kramár et al., 2024; Syed et al., 2024) — raises the stakes: unstable attributions become failure modes of downstream procedures, not merely misleading pictures.

**Outline.** Chapter 2 assembles the foundations: network notation, CPWL mappings, convexity and monotonicity, ***M**arkov **D**ecision **P**rocesses* (MDPs) and zero-sum games, including the ReLU Net Game of Gaubert and Vlassopoulos. Chapter 3 develops the decomposition framework, numerical stabilization, and the CNN extension; Chapter 4 the two-player stopping and routing games, with gradients and $\alpha\beta$-LRP recovered as equilibrium occupation measures; Chapter 5 the extension and implementation on multiplicative operations and ***Vi**sion **T**ransformer*s (ViTs); and Chapter 6 the split methods SplitCAM, SplitLRP, and SplitGrad, inherently interpretable difference-of-convex architectures, and the attribution modes. Chapter 7 reports the evaluation on ImageNet-S and Pascal VOC across VGG-16, ResNet, and ViT-B/16; Chapter 8 develops the trajectory-distribution view and its Hellinger diagnostics; and Chapter 9 concludes with future directions from ***L**arge **L**anguage **M**odel*s (LLMs) to hardware-level acceleration.

# 2 Mathematical Foundations

This chapter establishes the notation and background. Section 2.1 introduces feedforward networks and their ReLU–CPWL equivalence. Section 2.2 defines monotone networks, ICNNs, and Maxout units, the building blocks of Chapter 3. Section 2.3 presents MDPs, zero-sum turn-based games, and the full ReLU Net Game of Gaubert and Vlassopoulos, distinguishing the framework inherited by Chapter 4 from its extensions. The absolute-curvature construction for smooth activations accompanies the network decomposition theorem in Section 3.2.2.

## 2.1 Neural Network Notation and PWL Mappings

The standard feedforward architecture serves as the reference model throughout; convolutional and attention-based architectures are treated in Section 3.7 and Chapter 5 as structured special cases and extensions.

**Definition 2.1 (Feedforward ReLU network).** Let $L \in \mathbb{N}$ and let $d^{(0)}, d^{(1)}, \dots, d^{(L)} \in \mathbb{N}$ denote the *layer widths*, with input dimension $d^{(0)}$ and output dimension $d^{(L)}$. A *feedforward ReLU network* of depth $L$ is given by weight matrices $W^{(l)} \in \mathbb{R}^{d^{(l)} \times d^{(l-1)}}$ and bias vectors $b^{(l)} \in \mathbb{R}^{d^{(l)}}$ for $l = 1, \dots, L$. On an input $x \in \mathbb{R}^{d^{(0)}}$ it computes the recursion

$$\begin{aligned} a^{(0)} &:= x, \\ z^{(l)} &:= W^{(l)} a^{(l-1)} + b^{(l)}, && l = 1, \dots, L, \\ a^{(l)} &:= \mathrm{ReLU}\big(z^{(l)}\big), && l = 1, \dots, L-1, \end{aligned} \tag{2.1}$$

where $\mathrm{ReLU}(z) := \max\{z, 0\}$ acts componentwise, and the output layer is affine, $a^{(L)} := z^{(L)}$. We call $z^{(l)}$ the *pre-activation* and $a^{(l)}$ the *(post-)activation* of layer $l$, and denote the *network function* by

$$f \colon \mathbb{R}^{d^{(0)}} \longrightarrow \mathbb{R}^{d^{(L)}}, \qquad f(x) := a^{(L)}. \tag{2.2}$$

Layers $1, \dots, L-1$ are the *hidden layers*. Throughout we fix the boundary convention $\mathrm{ReLU}'(0) := 0$, so that the gate indicator $\mathbf{1}\{z_i^{(l)} > 0\}$ returns 0 at $z_i^{(l)} = 0$.

For layers $l, \dots, L$, write $f^{(l)} \colon \mathbb{R}^{d^{(l)}} \to \mathbb{R}^{d^{(L)}}$ for the tail map, with $f = f^{(1)} \circ \big(x \mapsto \mathrm{ReLU}(W^{(1)}x + b^{(1)})\big)$ and $f^{(L)}$ the affine output map. Appending a constant *intercept input* 1 absorbs $b^{(l)}$ into a column of $W^{(l)}$; some proofs use this bias-free formulation implicitly.

Chapter 3 splits parameters and inputs by sign using the following notation.

**Definition 2.2 (Positive and negative parts).** For a scalar $u \in \mathbb{R}$ we write $u^+ := \max\{u, 0\}$ and $u^- := \max\{-u, 0\}$, so that $u = u^+ - u^-$ and $|u| = u^+ + u^-$, with $u^+ u^- = 0$. For a matrix or vector $W$ and a sign label $\sigma \in \{+, -\}$, the *entrywise part* $[W]^\sigma$ is obtained by applying $u \mapsto u^\sigma$ to every entry. For the layer weights and biases of Definition 2.1 we abbreviate

$$W^{(l,\sigma)} := [W^{(l)}]^\sigma, \qquad b^{(l,\sigma)} := [b^{(l)}]^\sigma, \qquad \sigma \in \{+, -\}, \tag{2.3}$$

so that $W^{(l)} = W^{(l,+)} - W^{(l,-)}$ and $|W^{(l)}| = W^{(l,+)} + W^{(l,-)}$ entrywise, and both $W^{(l,\sigma)}$ are entrywise non-negative.

*Remark 2.1 (Pair notation for decomposed activations).* The activation labels $a^{(l,\sigma)}$, $\sigma \in \{+,-\}$, denote the two non-negative *streams* defined in Section 3.1, distinct from the entrywise parts $[a^{(l)}]^{\sigma}$. The streams carry strictly more information than the sign pattern of $a^{(l)}$, supplying the sub-threshold signals exploited in Chapter 6.

### Continuous piecewise linear functions

The function class computed by ReLU networks admits an exact geometric description.

**Definition 2.3 (CPWL function).** A function $f\colon \mathbb{R}^d \to \mathbb{R}^m$ is ***C**ontinuous **P**iece**w**ise **L**inear* (CPWL) if it is continuous and there exist finitely many closed polyhedra $Q_1, \dots, Q_N \subseteq \mathbb{R}^d$ with $\bigcup_{r=1}^{N} Q_r = \mathbb{R}^d$ such that the restriction of $f$ to each $Q_r$ is an affine map. The full-dimensional polyhedra on which $f$ is affine, taken maximal with respect to inclusion, are called the *linear regions* of $f$.

For example, $|\cdot|\colon \mathbb{R} \to \mathbb{R}$ is CPWL with the two regions $(-\infty, 0]$ and $[0, \infty)$, and $x \mapsto \max\{x_1, \dots, x_d\}$ is CPWL with $d$ regions.

*Observation 2.1 (`Lean_1`).* Every feedforward ReLU network represents a CPWL function.

*Proof.* Affine maps and componentwise ReLU are CPWL. Composition preserves this class: if $f$ and $g$ are affine on polyhedral covers $Q_1, \dots, Q_N$ and $P_1, \dots, P_M$ of their respective domains, then $g \circ f$ is affine on the cover of at most $NM$ polyhedra $Q_r \cap f^{-1}(P_s)$. These are polyhedra because $f$ is affine on each $Q_r$; composition also preserves continuity. Induction along (2.1) proves the claim. □

Conversely, every CPWL function has an exact ReLU representation with a dimension-dependent depth bound (Arora et al., 2018). This equivalence underlies the difference-of-convex analysis of Chapter 3.

**Theorem 2.1 (CPWL–ReLU equivalence, (Arora et al., 2018), `Lean_2`).** *A function $f\colon \mathbb{R}^d \to \mathbb{R}$ is CPWL if and only if it is representable by a feedforward ReLU network. Moreover, every CPWL function on $\mathbb{R}^d$ is representable by a ReLU network of depth at most $\lceil \log_2(d+1) \rceil + 1$.*

*Proof.* Observation 2.1 proves the "if" direction. For the converse, every CPWL function $f$ admits a *difference-of-convex* representation

$$f = g - h, \qquad g = \max_{r \in [N_g]} \ell_r, \qquad h = \max_{s \in [N_h]} m_s, \tag{2.4}$$

with finitely many affine functions $\ell_r, m_s\colon \mathbb{R}^d \to \mathbb{R}$ (Kripfganz and Schulze, 1987; Melzer, 1986); their finite maxima $g$,$h$ are convex. The maximum of two scalars is a two-layer ReLU computation,

$$\max\{u, v\} = \frac{u+v}{2} + \frac{|u-v|}{2} = \frac{u+v}{2} + \frac{\mathrm{ReLU}(u-v) + \mathrm{ReLU}(v-u)}{2}, \tag{2.5}$$

absorbing the affine pre- and post-combinations into surrounding weight matrices. A balanced binary tree of pairwise maxima realizes $g$ at depth $\lceil \log_2 N_g \rceil + 1$, and likewise $h$; a final affine layer forms (2.4), proving representability. For the bound independent of the number of pieces, Arora et al. (2018) use the Wang–Sun representation of every CPWL function on $\mathbb{R}^d$ as a signed sum of maxima of at most $d+1$ affine functions each. These need at most $\lceil \log_2(d+1) \rceil$ hidden layers and a final affine output layer. □

*Remark 2.2 (Expressivity and linear regions).* Theorem 2.1 identifies the function *class*, not the *cost* of representation: deep networks can represent CPWL functions with exponentially many linear regions in the number of neurons, an interplay of depth, width, and region counts surveyed by Huchette et al. (2026). The representation (2.4) also foreshadows the central theme of this thesis: the convex CPWL streams $g$ and $h$ of the DC split can be extracted *constructively and locally* from a trained network, the subject of Section 3.2.3. For a stream $g(x) = \max_r(a_r^\top x + b_r)$, a uniquely active affine piece gives $\nabla g(x) = a_r$: the stream gradient records the slope of its current linear region.

## 2.2 Convexity and Monotonicity in Neural Networks

Chapter 3 splits general networks into sign-constrained components. We define the two classes and prove their monotonicity and convexity, respectively.

We order $\mathbb{R}^d$ coordinatewise: $x \le y$ if and only if $x_i \le y_i$ for all $i \in [d]$. A function $f\colon \mathbb{R}^d \to \mathbb{R}^m$ is *monotone* (non-decreasing) if $x \le y$ implies $f(x) \le f(y)$.

**Definition 2.4 (Monotone neural network).** A feedforward network with recursion (2.1) and monotone activation functions is called a *monotone neural network* if all its weight matrices are entrywise non-negative, $W^{(l)} \ge 0$ for all $l = 1, \dots, L$.

Weight-sign constraints are an established device for building monotone models (Daniels and Velikova, 2010; Sill, 1997) and remain central to modern monotone architectures (Runje and Shankaranarayana, 2023); monotonicity can also be certified without imposing such constraints (Liu et al., 2020).

**Proposition 2.2 (Monotone networks represent monotone functions, `Lean_3`).** *A monotone neural network represents a monotone function; each coordinate function is non-decreasing in every input variable.*

*Proof.* Induct from the monotone input $a^{(0)} = x$. Non-negative weights preserve coordinatewise order: $a \le a'$ implies $W^{(l)}a + b^{(l)} \le W^{(l)}a' + b^{(l)}$. Thus monotonicity of $a^{(l-1)}$ implies that of $z^{(l)}$; componentwise monotone activation gives monotonicity of $a^{(l)}$. The same order argument covers the affine output. □

**Definition 2.5 (Input-Convex Neural Network, (Amos et al., 2017)).** A feedforward network with recursion (2.1) whose activation functions are convex *and* monotone is called an ***I**nput-**C**onvex **N**eural **N**etwork* (ICNN) if all weight matrices except possibly the first are entrywise non-negative, $W^{(l)} \ge 0$ for all $l = 2, \dots, L$, while $W^{(1)}$ is unconstrained.

Definition 2.5 covers the plain feedforward core of the architecture of Amos et al. (2017); their sign-unconstrained skip ("passthrough") connections do not affect the statement below, since they enter each pre-activation affinely in $x$.

**Proposition 2.3 (ICNNs represent convex functions, `Lean_4`).** *An ICNN represents a convex function; each coordinate of every activation $a^{(l)}$, $l \ge 1$, is a convex function of the input $x$.*

*Proof.* We use two closure properties of convex functions: (i) a non-negative linear combination of convex functions plus a constant is convex; (ii) if $\varphi\colon \mathbb{R} \to \mathbb{R}$ is convex and non-decreasing and $u\colon \mathbb{R}^d \to \mathbb{R}$ is convex, then $\varphi \circ u$ is convex, since for $\lambda \in [0,1]$,

$$\varphi\big(u(\lambda x + (1-\lambda)y)\big) \le \varphi\big(\lambda u(x) + (1-\lambda)u(y)\big) \le \lambda\,\varphi(u(x)) + (1-\lambda)\,\varphi(u(y)),$$

where the first inequality uses convexity of $u$ and monotonicity of $\varphi$, and the second convexity of $\varphi$.

Induct over layers. The affine coordinates of $z^{(1)} = W^{(1)}x + b^{(1)}$ are convex, so (ii) gives convexity of $a^{(1)}$. If $a^{(l-1)}$ has convex coordinates, the non-negative weights for $l \geq 2$ make $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ convex by (i), and (ii) preserves convexity through the activation. The identity activation covers the affine output $a^{(L)} = z^{(L)}$. □

*Remark 2.3 (Monotonicity and convexity combine).* If, in addition, $W^{(1)} \geq 0$ in Definition 2.5, the network is simultaneously monotone and input-convex, hence non-decreasing *and* convex by Propositions 2.2 and 2.3. Both ReLU and Maxout (below) are convex and monotone, so non-negative-weight ReLU networks are of exactly this type. These are the building blocks of the DC splitting of Section 3.2.3: the streams $g$ and $h$ are computed by monotone networks acting on a signed embedding of the input, and are input-convex in the original input. The sign constraint also *limits* expressivity—a weight-constrained ReLU network represents only convex functions, precisely Proposition 2.3, which is why monotone architectures add concave counterparts of the activation (Runje and Shankaranarayana, 2023). The decompositions of this thesis turn the limitation into a feature: the difference of two such networks recovers full generality by Theorem 2.1 and (2.4).

### Maxout units

**Definition 2.6 (Maxout unit and Maxout network, (Goodfellow et al., 2013)).** A *rank-$k$ Maxout unit* with weights $w_1, \dots, w_k \in \mathbb{R}^d$ and biases $\beta_1, \dots, \beta_k \in \mathbb{R}$ computes

$$x \longmapsto \operatorname{Maxout}(x) := \max_{j \in [k]} \big(w_j^\top x + \beta_j\big). \tag{2.6}$$

A *rank-$k$ Maxout network* is a feedforward network whose neurons are rank-$k$ Maxout units; equivalently, layer $l$ carries $k$ weight matrices $W_1^{(l)}, \dots, W_k^{(l)}$ and biases $b_1^{(l)}, \dots, b_k^{(l)}$, and

$$a_i^{(l)} := \max_{j \in [k]} \big(W_j^{(l)} a^{(l-1)} + b_j^{(l)}\big)_i. \tag{2.7}$$

*Observation 2.2 (ReLU and max-pooling as Maxout,* `Lean_5`*).* The ReLU activation is the rank-2 Maxout unit with parameters $(w_1, \beta_1) = (w, \beta)$ and $(w_2, \beta_2) = (0, 0)$, since $\operatorname{ReLU}(w^\top x + \beta) = \max\{w^\top x + \beta, 0\}$. Max-pooling over a group $J$ of neurons is the rank-$|J|$ Maxout unit whose $j$-th argument is the coordinate projection onto the $j$-th element of $J$. Every rank-$k$ Maxout unit is convex in its input and non-decreasing in each pre-activation argument $w_j^\top x + \beta_j$; hence Maxout qualifies as an admissible activation in Definitions 2.4 and 2.5.

*Proof.* The first two claims follow from (2.6). A maximum of affine functions is convex. For argument-wise monotonicity, set $u_j = w_j^\top x + \beta_j$: $u_j \leq u'_j$ for all $j$ implies $\max_j u_j \leq \max_j u'_j$. □

*Remark 2.4.* Maxout networks are the natural piecewise-linear generalization of ReLU: a rank-$k$ unit expresses a maximum of $k$ affine functions in one neuron, matching the building blocks of the DC representation (2.4). The DC splitting of Chapter 3 is developed for ReLU networks and extended to Maxout units—and to the multiplicative interactions needed for attention—in Chapter 5, using the exchange identity $\max_{j \in [k]}(u_j - v_j) = \max_{j \in [k]}\big\{u_j + \sum_{i \neq j} v_i\big\} - \sum_{i \in [k]} v_i$ for pairs of non-negative streams.

## 2.3 Game Theory and Markov Decision Processes

Chapter 4 interprets backward propagation as a two-player zero-sum game on the decomposed network graph. We introduce MDPs, turn-based games, and their optimality equations, restricting throughout to finite state and action spaces.

**Definition 2.7 (Markov Decision Process).** A (finite) ***Markov Decision Process*** (MDP) (Puterman, 1994; Sutton and Barto, 2018) is a tuple $\mathscr{M} = (S, A, \mathbb{P}, R, \gamma)$ consisting of a finite state set $S$, a finite action set $A(s)$ for each $s \in S$, a *transition kernel* $\mathbb{P}(s' \mid s, a) \geq 0$ with $\sum_{s'} \mathbb{P}(s' \mid s, a) = 1$, a bounded *reward function* $R(s, a) \in \mathbb{R}$, and a *discount factor* $\gamma \in [0, 1)$. A *(stationary) policy* is a map $\pi$ assigning to each state $s$ a probability distribution $\pi(\cdot \mid s)$ over $A(s)$; $\pi$ is *pure* (deterministic) if each $\pi(\cdot \mid s)$ is a point mass. A policy induces, for every start state, a Markov chain on $S$ and hence a distribution over trajectories $(s_0, a_0, s_1, a_1, \ldots)$; the *value function* of $\pi$ is the expected discounted return

$$V^{\pi}(s) \coloneqq \mathbb{E}^{\pi}\Big[\sum_{t=0}^{\infty} \gamma^t \, R(s_t, a_t) \,\Big|\, s_0 = s\Big], \tag{2.8}$$

and the *optimal value function* is $V^{\star}(s) \coloneqq \max_{\pi} V^{\pi}(s)$.

Fixing an enumeration of $S$, a pure policy $\pi$ turns the kernel into a row-stochastic *transition matrix* $P^{\pi} \in \mathbb{R}^{S \times S}$ with $P^{\pi}_{ss'} = \mathbb{P}(s' \mid s, \pi(s))$ and the reward into a vector $R^{\pi}$, and (2.8) becomes the Neumann series $V^{\pi} = \sum_{t \geq 0} \gamma^t (P^{\pi})^t R^{\pi}$. This matrix picture recurs in Chapter 4, where the (sub)stochastic matrices are built from the normalized weight parts $W^{(l,\sigma)}$ of Definition 2.2.

**Proposition 2.4 (Bellman equations, `Lean_6`).** *Let $\mathscr{M}$ be a finite MDP with $\gamma \in [0,1)$. Then:*

*1) $V^{\pi}$ is the unique solution of the linear system*

$$V(s) = \sum_{a \in A(s)} \pi(a \mid s)\Big[R(s,a) + \gamma \sum_{s'} \mathbb{P}(s' \mid s, a)\, V(s')\Big], \qquad s \in S. \tag{2.9}$$

*2) $V^{\star}$ is the unique solution of the* Bellman optimality equation

$$V(s) = \max_{a \in A(s)} \Big\{ R(s, a) + \gamma \sum_{s'} \mathbb{P}(s' \mid s, a)\, V(s') \Big\}, \qquad s \in S, \tag{2.10}$$

*and any pure policy $\pi^{\star}$ that selects, in every state, an action attaining the maximum in (2.10) is optimal: $V^{\pi^{\star}} = V^{\star}$.*

*Proof.* Both claims are standard (Puterman, 1994; Sutton and Barto, 2018). The right-hand sides of (2.9) and (2.10) define $\gamma$-contractions on $\big(\mathbb{R}^S, \|\cdot\|_{\infty}\big)$, so each has a unique fixed point by the Banach fixed-point theorem; conditioning (2.8) on the first transition identifies these fixed points as $V^{\pi}$ and $V^{\star}$, and any policy greedy for $V^{\star}$ is optimal. The acyclic games below make even the contraction argument unnecessary (Remark 2.5). □

*Remark 2.5 (State-dependent structural discounts).* Our games use *state-dependent* discounts $\gamma(s) \geq 0$ that may exceed 1. These are *structural* normalizers: for neuron $i$ in layer $l$, the absolute weight mass

$$\gamma_i^{(l)} \coloneqq \sum_j \big(W^{(l,+)} + W^{(l,-)}\big)_{ij} = \sum_j |W_{ij}^{(l)}|, \tag{2.11}$$

restores the mass after normalizing row $i$ of $|W^{(l)}|$ into transition probabilities. Although contraction is unavailable, *layered acyclic* graphs ensure finite trajectories and value sums. Backward induction gives a unique value and optimal policies, possibly nonunique at ties, as the two-player theorem below shows.

### Zero-sum turn-based games

Two-player games enter because a decomposed network carries two interacting streams: a player $P^+$ who collects positive evidence and a player $P^-$ who collects negative evidence. We use the following conventions throughout: $p, q, r \in \{+, -\}$ denote player, turn, or sign labels; $p'$ denotes the opponent ($+' = -$, $-' = +$); the *sign-XOR* $p \oplus q$ is $+$ if $p = q$ and $-$ otherwise; and $\xi_p \in \{+1, -1\}$ is the numerical sign of the label $p$, so that $\xi_p \, \xi_q = \xi_{p \oplus q}$.

**Definition 2.8 (Zero-sum turn-based game).** A (finite) *zero-sum turn-based game* is a tuple $\mathscr{G} = (S_+, S_-, S_{\mathrm{ter}}, A, \mathbb{P}, R, \gamma)$ where the state set $S = S_+ \uplus S_- \uplus S_{\mathrm{ter}}$ is partitioned into states controlled by player $P^+$, states controlled by player $P^-$, and terminal states; the action sets $A(s)$, the transition kernel $\mathbb{P}$, and the reward $R$ are as in Definition 2.7 on non-terminal states; $\gamma \colon S \to \mathbb{R}_{\geq 0}$ is a (possibly state-dependent) discount; and each terminal state $s \in S_{\mathrm{ter}}$ carries a fixed one-time payoff $R(s)$. A *policy pair* $\pi = (\pi_+, \pi_-)$ consists of a policy $\pi_p$ for player $P^p$ on $S_p$. Play generates a trajectory as in an MDP, with the player in turn choosing the action; the expected total payoff $V^\pi(s)$, defined as in (2.8) with the discount accumulated multiplicatively along the trajectory ($\gamma^t$ replaced by $\prod_{m=0}^{t-1} \gamma(s_m)$), is paid *by* $P^-$ *to* $P^+$: player $P^+$ maximizes $V^\pi$ and player $P^-$ minimizes it.

**Theorem 2.5 (Shapley–Bellman optimality on layered games, `Lean_7`).** *Let $\mathscr{G}$ be a finite zero-sum turn-based game whose transition graph is acyclic. Then $\mathscr{G}$ has a* value*: there is a unique function $V^\star \colon S \to \mathbb{R}$ satisfying the* Shapley–Bellman optimality equations

$$V^\star(s) = \begin{cases} \max\limits_{a \in A(s)} \Big\{ R(s,a) + \gamma(s) \sum\limits_{s'} \mathbb{P}(s' \mid s, a)\, V^\star(s') \Big\}, & s \in S_+, \\ \min\limits_{a \in A(s)} \Big\{ R(s,a) + \gamma(s) \sum\limits_{s'} \mathbb{P}(s' \mid s, a)\, V^\star(s') \Big\}, & s \in S_-, \\ R(s), & s \in S_{\mathrm{ter}}, \end{cases} \tag{2.12}$$

*and there exist pure stationary policies $\pi^\star = (\pi^\star_+, \pi^\star_-)$, obtained by selecting maximizing respectively minimizing actions in (2.12), such that for all $s \in S$ and all policies $\pi_+, \pi_-$,*

$$V^{(\pi^\star_+, \pi_-)}(s) \;\geq\; V^{\pi^\star}(s) = V^\star(s) \;\geq\; V^{(\pi_+, \pi^\star_-)}(s). \tag{2.13}$$

*In particular, $\pi^\star$ is a Nash equilibrium of every subgame, i.e. a subgame perfect equilibrium.*

*Proof.* Induct backward along a topological order of the finite acyclic graph. At terminal states, (2.12) fixes $V^\star(s) = R(s)$ and (2.13) holds. For $s \in S_+$, assume uniqueness and the saddle inequalities at all successors. Each action's bracket in (2.12) is finite; maximizing over the finite action set uniquely defines $V^\star(s)$ and supplies a pure choice $\pi^\star_+(s)$. For any opponent policy $\pi_-$, condition on the first

transition and apply the successors' left saddle inequality with $\gamma(s) \geq 0$:

$$V^{(\pi_+^\star,\pi_-)}(s) = R\big(s,\pi_+^\star(s)\big) + \gamma(s)\sum_{s'}\mathbb{P}\big(s' \mid s,\pi_+^\star(s)\big)\,V^{(\pi_+^\star,\pi_-)}(s')$$
$$\geq R\big(s,\pi_+^\star(s)\big) + \gamma(s)\sum_{s'}\mathbb{P}\big(s' \mid s,\pi_+^\star(s)\big)\,V^\star(s') \;=\; V^\star(s),$$

The last equality uses the maximizing choice $\pi_+^\star(s)$. Against $\pi_-^\star$, any $\pi_+$ averages the bracket to at most its maximum; conditioning and the successors' right saddle inequality give $V^{(\pi_+,\pi_-^\star)}(s) \leq V^\star(s)$. For $s \in S_-$, exchange max with min and reverse the inequalities. Taking $\pi = \pi^\star$ in both inequalities gives $V^{\pi^\star} = V^\star$. The argument holds in every rooted subgame, proving subgame perfection. □

*Remark 2.6.* For general discounted stochastic games with cycles, existence of the value and of optimal stationary policies is Shapley's theorem (Shapley, 1953), proved by the contraction argument of Proposition 2.4 applied to the *Shapley operator*—the min-max analogue of the Bellman operator (2.12). Gaubert & Vlassopoulos (2025) formulate their network games in this operator language. The layered acyclic case of Theorem 2.5 is all we need, and it additionally delivers subgame perfection, on which Chapter 8 relies.

### 2.3.1 The ReLU Net Game of Gaubert and Vlassopoulos

Gaubert & Vlassopoulos (2025) represent the forward pass of a ReLU network as the value of a zero-sum turn-based stopping game played backward on its graph. We give their full construction in our notation; Chapter 4 retains its states, transitions, and stopping structure with revised payoff bookkeeping. Fix a network from Definition 2.1 with scalar output ($d^{(L)} = 1$), an input $x \in \mathbb{R}^{d^{(0)}}$, and nonzero weight rows, ensuring strictly positive discounts (2.11).

Read the identity $a_i^{(l)} = \max\{0, z_i^{(l)}\}$ as a decision: a player at neuron $i$ may *stop*, securing payoff 0, or *continue*, collecting the bias $b_i^{(l)}$ plus the weighted contributions of the previous layer. Continuing propagates the decision backward through the network; a negative weight flips the sign of its contribution, i.e. hands the turn to an adversary facing the mirrored problem. The optimal value at neuron $i$ is exactly the activation $a_i^{(l)}$.

**Definition 2.9 (ReLU Net Game, following (Gaubert and Vlassopoulos, 2025)).** The *ReLU Net Game* on the network $f$ at input $x$ is the zero-sum turn-based game with the following components.

- **States.** For every hidden layer $l \in \{1,\dots,L-1\}$, every neuron $i \in [d^{(l)}]$, and every turn label $q \in \{+,-\}$ there is an *activation state* $s_{i,q}^{(l)}$, controlled by player $P^q$, so that $S_q = \{s_{i,q}^{(l)} : l \in [L-1], i \in [d^{(l)}]\}$. For every input coordinate $k \in [d^{(0)}]$ and every $q \in \{+,-\}$ there is a *terminal state* $s_{k,q}^{(0)}$, and there is one absorbing *cemetery state* $\perp$ with zero payoff. Finally, there is a single *output state* $s^{(L)}$ at the affine output neuron, at which no decision is taken.
- **Actions.** At every activation state the player in turn chooses an action from $\{\text{stop}, \text{cont}\}$. The output state has the single action cont.
- **Transitions.** Stopping moves deterministically to the cemetery, $\mathbb{P}(\perp \mid s_{i,q}^{(l)}, \text{stop}) = 1$. Continuation at $s_{i,q}^{(l)}$ moves one layer backward, to neuron $j$ of layer $l-1$ with probability proportional to the absolute weight, keeping the turn on positive weights and passing it to the opponent on

negative weights:

$$\mathbb{P}\big(s_{j,r}^{(l-1)} \,\big|\, s_{i,q}^{(l)}, \mathrm{cont}\big) := \frac{W_{ij}^{(l,q\oplus r)}}{\gamma_i^{(l)}}, \qquad \gamma_i^{(l)} = \sum_j \big(W^{(l,+)} + W^{(l,-)}\big)_{ij}, \tag{2.14}$$

where for $l = 1$ the successor $s_{j,r}^{(0)}$ is the corresponding terminal state. The output state transitions by the same rule (2.14) with $l = L$ and turn label $q = +$.

– **Payoffs and discounts.** Continuation at $s_{i,q}^{(l)}$ pays the signed bias $\xi_q\, b_i^{(l)}$; stopping pays 0. A terminal state pays the signed input coordinate once, $R\big(s_{k,q}^{(0)}\big) := \xi_q\, x_k$, and the game ends. The output state pays $b^{(L)}$ upon its forced continuation. The discount at every activation state and at the output state is the structural discount $\gamma\big(s_{i,q}^{(l)}\big) := \gamma_i^{(l)}$ of (2.11).

Player $P^+$ maximizes and $P^-$ minimizes the expected discounted total payoff, as in Definition 2.8.

The game graph is layered and acyclic—every transition strictly decreases the layer index or ends in the cemetery—so Theorem 2.5 applies despite the discounts $\gamma_i^{(l)}$ being arbitrary positive numbers. Normalization and discounting cancel: the product of the transition probability (2.14) and the discount $\gamma_i^{(l)}$ is exactly the absolute weight $|W_{ij}^{(l)}|$, so the Shapley–Bellman recursion reproduces the affine algebra of the forward pass.

**Theorem 2.6 (Forward pass as game value, (Gaubert and Vlassopoulos, 2025), `Lean_8`).** *The ReLU Net Game of Definition 2.9 admits the subgame perfect equilibrium $\pi^\star = (\pi_+^\star, \pi_-^\star)$ in which both players use the same pure stopping rule,*

$$\pi_q^\star\big(s_{i,q}^{(l)}\big) = \mathrm{cont} \quad \Longleftrightarrow \quad z_i^{(l)} > 0, \tag{2.15}$$

*and the value function of the game evaluates the network:*

$$V^\star\big(s_{i,q}^{(l)}\big) = \xi_q\, a_i^{(l)} \quad \text{for } l \in \{0, 1, \dots, L-1\}, \qquad V^\star\big(s^{(L)}\big) = f(x), \tag{2.16}$$

*where for $l = 0$ we read $a_k^{(0)} = x_k$. The equilibrium is unique up to tie-breaking at neurons with $z_i^{(l)} = 0$, where both actions are optimal; the convention (2.15) stops at ties, matching $\mathrm{ReLU}'(0) = 0$.*

*Proof.* By Theorem 2.5, it suffices to check (2.16) and policy (2.15) against (2.12). Induct on layers.

*Base case $l = 0$.* The terminal payoff $R\big(s_{k,q}^{(0)}\big) = \xi_q x_k = \xi_q a_k^{(0)}$ satisfies (2.16).

*Inductive step.* Assume (2.16) at layer $l-1$, $l \geq 1$. At $s_{i,+}^{(l)} \in S_+$, stopping yields $0 + V^\star(\bot) = 0$; by (2.14) and induction, continuation yields

$$\begin{aligned} R\big(s_{i,+}^{(l)}, \mathrm{cont}\big) + \gamma_i^{(l)} \sum_{j,r} \frac{W_{ij}^{(l,+\oplus r)}}{\gamma_i^{(l)}}\, \xi_r\, a_j^{(l-1)} &= b_i^{(l)} + \sum_j \Big(W_{ij}^{(l,+)}\, a_j^{(l-1)} - W_{ij}^{(l,-)}\, a_j^{(l-1)}\Big) \\ &= b_i^{(l)} + \sum_j W_{ij}^{(l)}\, a_j^{(l-1)} \;=\; z_i^{(l)}, \end{aligned} \tag{2.17}$$

using $+ \oplus r = r$, $\xi_+ = 1$, $\xi_- = -1$, and $W^{(l)} = W^{(l,+)} - W^{(l,-)}$. The maximizing player $P^+$ therefore obtains

$$V^\star\big(s_{i,+}^{(l)}\big) = \max\big\{0, z_i^{(l)}\big\} = \mathrm{ReLU}\big(z_i^{(l)}\big) = a_i^{(l)} = \xi_+\, a_i^{(l)},$$

continuing exactly when $z_i^{(l)} > 0$. At the mirrored state $s_{i,-}^{(l)} \in S_-$ the continuation payoff is $\xi_- b_i^{(l)} = -b_i^{(l)}$, and the kernel labels satisfy $- \oplus r = r'$, so the computation (2.17) returns the continuation value

$$-b_i^{(l)} + \sum_j \Big( W_{ij}^{(l,-)}\, a_j^{(l-1)} - W_{ij}^{(l,+)}\, a_j^{(l-1)} \Big) = -z_i^{(l)}.$$

Thus $P^-$ obtains $V^\star\big(s_{i,-}^{(l)}\big) = \min\{0, -z_i^{(l)}\} = -\mathrm{ReLU}\big(z_i^{(l)}\big) = \xi_- a_i^{(l)}$, preferring continuation exactly when $-z_i^{(l)} < 0$, the gate (2.15). At $z_i^{(l)} = 0$, both actions give either player value 0, proving uniqueness up to ties. Forced continuation at the affine output gives, by (2.17) with $l = L$, $V^\star\big(s^{(L)}\big) = z^{(L)} = a^{(L)} = f(x)$. □

*Remark 2.7 (Softplus networks from entropy regularization).* Gaubert & Vlassopoulos (2025) also treat softplus networks, by *regularizing* the stop/continue decision. Replace the hard choice at a positive activation state by a mixed policy $\nu \in \Delta(\{\text{stop}, \text{cont}\})$ rewarded with a Shannon-entropy bonus at temperature $\theta > 0$. At a negative state, the entropy term enters with the opposite sign in the payoff to $P^+$. By the Legendre–Fenchel identity for the entropy (Boyd and Vandenberghe, 2004),

$$\max_{\nu \in \Delta_n} \Big\{ \textstyle\sum_{i=1}^n \nu_i r_i + \theta\, \mathscr{H}(\nu) \Big\} = \theta \log \textstyle\sum_{i=1}^n e^{r_i/\theta}, \qquad \mathscr{H}(\nu) = -\textstyle\sum_i \nu_i \log \nu_i, \tag{2.18}$$

with the Gibbs distribution $\nu_i^\star \propto e^{r_i/\theta}$ as unique maximizer, the regularized value at a positive state with continuation value $z_i^{(l)}$ (computed as in (2.17)) becomes $\theta \log\big(1 + e^{z_i^{(l)}/\theta}\big)$—the softplus activation at temperature $\theta$—and the equilibrium policy continues with the sigmoid probability $\big(1 + e^{-z_i^{(l)}/\theta}\big)^{-1}$. The hard game of Definition 2.9 is recovered in the limit $\theta \to 0$. Chapter 4 re-derives this mechanism from the regularized decompositions of Section 3.5 and reuses it at the routing states of the ***R**outing **G**ame* (RG), with the two temperatures $\theta$ (activation gates) and $\tau$ (routing) kept distinct.

*Remark 2.8 (What Chapter 4 inherits and what it changes).* The SG of Chapter 4 keeps the structural skeleton of Definition 2.9: per-neuron ± states, stop-versus-continue actions, the bias as continuation payoff and the input as final payoff, the transition kernel (2.14) driven by the weight parts $W^{(l,\sigma)}$, the cemetery state, and the structural discounts $\gamma_i^{(l)}$. Two changes are load-bearing for attribution. First, the original tracks only the single signed value (2.16); Chapter 4 additionally maintains a *non-negative payoff per player*, identifying the pair of player-specific values $V_p(s)$, $p \in \{+,-\}$, with the two streams of the Stopping Decomposition of Section 3.4 and recovering (2.16) as their difference. Second, the signed terminal payoffs $\xi_q\, x_k$ become the non-negative pair $(x_k^+, x_k^-)$ of Definition 2.2, keeping every player-specific quantity non-negative and exposing the parity structure of backward trajectories. Chapter 4 further extends the graph to skip connections and max-pooling and adds risk aversion and regularized policies, none of which appear in the original game.

# 3 Network Decompositions and DC Formulations

This chapter develops network decompositions that separate positive and negative contributions while preserving their difference. The DC instance exposes monotone, convex streams for the attribution methods of Chapter 6; the stopping instance preserves the gates underlying the games of Chapter 4. Mixing and regularized decompositions connect these choices. Their shared forward invariant supports comparison of retained information and backward calculations; numerical stabilization makes them usable in deep networks.

## 3.1 Generalized Network Decomposition

An ordinary signed activation combines opposing contributions into one number, hiding their magnitudes. Our decompositions replace it at every neuron and layer by a non-negative pair, exposing how supporting and opposing contributions accumulate while recovering the activation as their difference. Figure 3.1 illustrates the DC instance of Section 3.2.3.

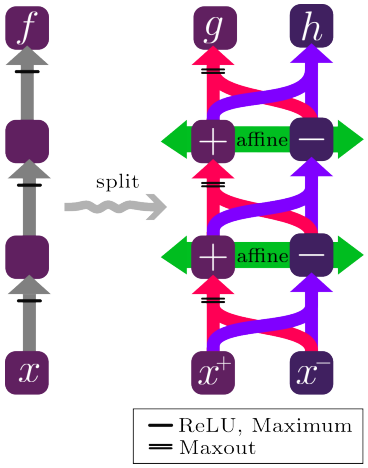



**Fig. 3.1:** The network splitting procedure: a ReLU network $f$ is decomposed into two monotone, convex streams $g$ and $h$ with $f = g - h$ (Section 3.2.3).

Consider an $L$-layer feedforward network with weights $W^{(l)}$, biases $b^{(l)}$, and entrywise activation $\varrho\colon \mathbb{R} \to \mathbb{R}$ (e.g. ReLU or softplus):

$$a^{(0)} = x, \qquad z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = \varrho_l\big(z^{(l)}\big), \qquad l = 1, \dots, L, \tag{3.1}$$

where $\varrho_l = \varrho$ for $1 \le l < L$ and $\varrho_L = \mathrm{id}$ gives the ungated affine readout $f(x) = a^{(L)} \in \mathbb{R}$. For multi-class networks, fix one target logit. We use the notation of Section 2.1, reserving $g$,$h$ for the split streams of Section 3.2.3.

A decomposition replaces each activation $a_i^{(l)}$ by a non-negative pair propagated through non-negative linear maps and *pairwise activation functions*. We first define this structure, then require its differences to recover the original network.

**Definition 3.1 (Decomposition).** A *decomposition* of the $L$-layer network (3.1) propagates non-negative activation pairs $\big(a_i^{(l,+)}, a_i^{(l,-)}\big) \in \mathbb{R}^2_{\ge 0}$ from input to output using only

1) **non-negative linear maps**—matrices with non-negative entries together with non-negative bias vectors—and
2) **pairwise activation functions** $\phi\colon \mathbb{R}^2_{\geq 0} \to \mathbb{R}^2_{\geq 0}$, $(z^+, z^-) \mapsto (a^+, a^-) = \phi(z^+, z^-)$, applied at every hidden neuron; the final affine readout uses the identity on the pair.

Activations are indexed by $l \in \{0, \dots, L\}$, affine parameters by $l \in \{1, \dots, L\}$. The layer-0 pair is any non-negative split of the input,

$$\big(a_k^{(0,+)}, a_k^{(0,-)}\big) \coloneqq \big(x_k^+, x_k^-\big), \qquad x_k^+ - x_k^- = x_k, \qquad x_k^+, x_k^- \geq 0, \tag{3.2}$$

and each affine layer $l$ maps the preceding activation pair to a non-negative pre-activation pair $\big(z_i^{(l,+)}, z_i^{(l,-)}\big)$. The pairwise activation applies for $l < L$; at $l = L$, $a^{(L,\pm)} = z^{(L,\pm)}$. We call $\big(a_i^{(l,+)}, a_i^{(l,-)}\big)$ the *decomposed activation* and $\big(z_i^{(l,+)}, z_i^{(l,-)}\big)$ the *decomposed pre-activation*.

**Canonical non-negative linear map.** We use the entrywise positive/negative split of the original weights and biases,

$$W^{(l,+)} \coloneqq [W^{(l)}]^+, \qquad W^{(l,-)} \coloneqq [W^{(l)}]^-, \qquad b^{(l,+)} \coloneqq [b^{(l)}]^+, \qquad b^{(l,-)} \coloneqq [b^{(l)}]^-, \tag{3.3}$$

where $[\cdot]^\pm$ are the entrywise positive and negative parts, so $W^{(l)} = W^{(l,+)} - W^{(l,-)}$ with $W^{(l,\pm)} \geq 0$. The resulting pre-activation pair at neuron $i$ is

$$z_i^{(l,+)} \coloneqq b_i^{(l,+)} + \sum_j W_{ij}^{(l,+)}\, a_j^{(l-1,+)} + \sum_j W_{ij}^{(l,-)}\, a_j^{(l-1,-)}, \tag{3.4}$$

$$z_i^{(l,-)} \coloneqq b_i^{(l,-)} + \sum_j W_{ij}^{(l,+)}\, a_j^{(l-1,-)} + \sum_j W_{ij}^{(l,-)}\, a_j^{(l-1,+)}. \tag{3.5}$$

Positive weights preserve streams; negative weights swap them, turning excitatory contributions into inhibitory ones and vice versa. The following definition connects the structure of Definition 3.1 to the original network.

**Definition 3.2 (Valid decomposition).** A decomposition (Definition 3.1) is *valid* for the network (3.1) with activation function $\varrho$ if the propagated activation pairs satisfy

$$a_i^{(l,+)} - a_i^{(l,-)} = \varrho_l\big(z_i^{(l)}\big) = a_i^{(l)} \tag{3.6}$$

for every layer $l \in \{0, \dots, L\}$ and every neuron $i$, where at $l = 0$ condition (3.6) is read as $a_k^{(0,+)} - a_k^{(0,-)} = x_k$.

Validity propagates through the canonical linear map, recovering the network output at the final layer.

**Proposition 3.1 (Validity recovers the network, `Lean_21`).** *Let a decomposition use the canonical non-negative linear map (3.4)–(3.5). If the validity condition (3.6) holds at layer $l-1$, then the decomposed pre-activations at layer $l$ satisfy*

$$z_i^{(l,+)} - z_i^{(l,-)} = z_i^{(l)} \qquad \text{for every neuron } i. \tag{3.7}$$

*Consequently, every valid decomposition recovers the original network output as the difference of the final decomposed activations,*

$$f(x) = a^{(L,+)} - a^{(L,-)}. \tag{3.8}$$

*Proof.* Subtracting (3.5) from (3.4) and regrouping by predecessor $j$,

$$z_i^{(l,+)} - z_i^{(l,-)} = b_i^{(l,+)} - b_i^{(l,-)} + \sum_j \big(W_{ij}^{(l,+)} - W_{ij}^{(l,-)}\big)\big(a_j^{(l-1,+)} - a_j^{(l-1,-)}\big) = b_i^{(l)} + \sum_j W_{ij}^{(l)}\, a_j^{(l-1)} = z_i^{(l)},$$

The second equality uses $W^{(l)} = W^{(l,+)} - W^{(l,-)}$, $b^{(l)} = b^{(l,+)} - b^{(l,-)}$, and validity at layer $l-1$, proving (3.7). At layer $L$, validity gives $a^{(L,+)} - a^{(L,-)} = \varrho_L\big(z^{(L)}\big) = z^{(L)} = f(x)$, proving (3.8). □

By Proposition 3.1, pre-activation differences recover the original pre-activations. A valid pairwise activation must therefore recover $\varrho(z^+ - z^-)$ and preserve non-negativity on the input quadrant. Section 3.2 gives conditions for convex, coordinatewise non-decreasing streams, constructs pairs from absolute curvature, and derives ReLU and smooth-activation formulas. Mixing (Section 3.3), stopping (Section 3.4), and regularized decompositions (Section 3.5) preserve the same difference with other structural properties.

## 3.2 Convex and Monotone DC Decompositions

The required activation-level object is a pair of functions $F_+,F_-\colon \mathbb{R}^2 \to \mathbb{R}$ satisfying

$$\begin{gathered}\varrho(x-y) = F_+(x,y) - F_-(x,y),\\ F_+,F_- \text{ convex and coordinatewise non-decreasing.}\end{gathered} \tag{3.9}$$

Convexity alone is insufficient for recursive composition: a convex function of convex inputs need not be convex unless the outer function is non-decreasing in those inputs. Conversely, coordinatewise monotonicity does not mean that a function has non-negative values everywhere. We require the pair to be non-negative only on $\mathbb{R}^2_{\geq 0}$, the domain visited by the split network. The global domain $\mathbb{R}^2$ in (3.9) makes the analytic requirements precise and permits signed inputs to the stream functions.

### 3.2.1 The profile criterion and network composition

Adding a common affine correction lifts a scalar DC decomposition $\varrho = p - q$ to such a pair. Here $L$ is a slope bound, independent of network depth. The criterion also covers ReLU's kink.

**Proposition 3.2 (Criterion for a convex, monotone pair, `Lean_22`).** *Let $L \geq 0$ and $C\colon \mathbb{R} \to \mathbb{R}$, and define*

$$T_{C,L}(x,y) := C(x-y) + Ly. \tag{3.10}$$

*Then $T_{C,L}$ is convex on $\mathbb{R}^2$ if and only if $C$ is convex. It is coordinatewise non-decreasing on $\mathbb{R}^2$ if and only if*

$$0 \leq C(v) - C(u) \leq L(v-u) \qquad (u \leq v). \tag{3.11}$$

*If $C$ is differentiable, condition (3.11) is equivalent to $0 \leq C' \leq L$. Consequently, given $p - q = \varrho$, the pair*

$$F_+(x,y) = p(x-y) + Ly, \qquad F_-(x,y) = q(x-y) + Ly \tag{3.12}$$

*is convex and coordinatewise non-decreasing exactly when both profiles are convex and satisfy (3.11). Under these conditions it maps $\mathbb{R}^2_{\geq 0}$ to itself exactly when $p(0),q(0) \geq 0$.*

*Proof.* Affine precomposition and affine addition preserve convexity; restriction to $y = 0$ gives the converse. Increasing $x$ gives the lower bound in (3.11); increasing $y$ by $v - u$ changes the argument

from $v$ to $u$, giving the upper bound. These inequalities also imply coordinatewise monotonicity. Difference quotients and the mean value theorem give the derivative criterion. The common $Ly$ cancels in $F_+ - F_-$. On the non-negative quadrant, monotonicity gives $F_\pm(x,y) \geq F_\pm(0,0)$; evaluation at the origin proves necessity. □

**Theorem 3.3 (Network DC decomposition from activation pairs, `Lean_23`).** *Consider the network (3.1) with hidden activation $\varrho$. Choose a pair $F_+,F_-$ satisfying (3.9) on all of $\mathbb{R}^2$ and $F_\pm(0,0) \geq 0$. Use this pair entrywise after the canonical affine split (3.4)–(3.5), and use the identity pair at the final affine readout. Then every decomposed activation, and each of the two output streams $G,H$, is convex and coordinatewise non-decreasing as a function of the independent doubled input coordinates $(x^+,x^-)$. Moreover,*

$$G(x^+,x^-) - H(x^+,x^-) = f(x^+ - x^-), \tag{3.13}$$

*and both streams are non-negative when $x^+,x^- \geq 0$. These conclusions also hold for every tail network starting at an intermediate layer.*

*If each component of an input split $x \mapsto (x^+(x),x^-(x))$ is convex, then $x \mapsto G(x^+(x),x^-(x))$ and $x \mapsto H(x^+(x),x^-(x))$ are convex. In particular, the usual split $x^\pm = [x]^\pm$ gives a DC representation on the original input space.*

*Proof.* The initial doubled coordinates are affine and non-decreasing. Non-negative affine combinations preserve convexity and coordinatewise monotonicity, as does composition with each convex, coordinatewise non-decreasing pair component. The final affine layer retains both properties. Induction proves the structural assertions for the network and every tail.

Subtracting each affine pair recovers the original affine map (Proposition 3.1); each hidden pair gives $F_+(z^+,z^-) - F_-(z^+,z^-) = \varrho(z^+ - z^-)$. The final identity pair gives the ungated pre-activation difference. Induction proves (3.13), including negative logits. Origin normalization and monotonicity preserve non-negativity at hidden layers, and canonical affine maps preserve it throughout. Composing a convex non-decreasing function with convex components proves the convex input-split assertion (Amos et al., 2017). □

Monotonicity refers to independent pair coordinates. Under $x^\pm = [x]^\pm$, convexity survives, while monotonicity in $x$ generally fails: $x^- = \max\{-x,0\}$ decreases with $x$. The theorem therefore gives monotone networks on doubled space and a DC representation on the original input space.

### 3.2.2 Construction from absolute second derivatives

Finite positive and negative curvature gives bounded profile slopes. We construct them from $|f''|$, with $f$ now denoting a scalar activation.

**Theorem 3.4 (Absolute-curvature construction, Lean_24).** *Let $f\colon \mathbb{R} \to \mathbb{R}$ be differentiable, let $f'$ be locally absolutely continuous, and assume $f'' \in L^1(\mathbb{R})$. Set*

$$\begin{aligned} s_- &:= \lim_{t\to-\infty} f'(t), \qquad V(t) := \int_{-\infty}^{t} |f''(u)|\ du, \\ D(z) &:= |f(0)| + |s_-|z + \int_0^z V(t)\ dt, \\ p &:= \frac{D+f}{2}, \qquad q := \frac{D-f}{2}. \end{aligned} \tag{3.14}$$

*The limits $s_-$ and $s_+ := \lim_{t\to+\infty} f'(t)$ exist. With*

$$L := \frac{\|f''\|_{L^1(\mathbb{R})} + |s_-| + |s_+|}{2}, \tag{3.15}$$

*the profiles $p$,$q$ are convex, satisfy $p - q = f$ and $0 \le p',q' \le L$, and have $p(0) = \max\{f(0),0\}$, $q(0) = \max\{-f(0),0\}$. Therefore (3.12) is a convex, coordinatewise non-decreasing pair on $\mathbb{R}^2$, is non-negative on $\mathbb{R}^2_{\ge 0}$, and recovers $f(x-y)$ exactly.*

*Proof.* Local absolute continuity gives $|f'(v) - f'(u)| \le \int_u^v |f''(t)|\ \mathrm{d}t$ for $u \le v$. Integrability makes $f'$ Cauchy at both ends of the real line, so its finite limits exist. Write

$$P = \frac{|f''| + f''}{2}, \qquad N = \frac{|f''| - f''}{2}. \tag{3.16}$$

Differentiating (3.14) yields

$$p'(z) = \max\{s_-,0\} + \int_{-\infty}^{z} P(u)\ \mathrm{d}u, \qquad q'(z) = \max\{-s_-,0\} + \int_{-\infty}^{z} N(u)\ \mathrm{d}u. \tag{3.17}$$

Both derivatives are non-negative, non-decreasing, and bounded, and $p'' = P$, $q'' = N$ almost everywhere. Their upper limits are $A = \max\{s_-,0\} + \int_{\mathbb{R}} P$ and $B = \max\{-s_-,0\} + \int_{\mathbb{R}} N$. Since $A + B = |s_-| + \|f''\|_1$ and $A - B = s_+$, their maximum is precisely (3.15). The difference and origin values follow directly from the definition. Proposition 3.2 now proves all pair properties. □

The functions $P$ and $N$ are the positive and negative parts of curvature. They are minimal: any twice weakly differentiable convex profiles $\widetilde{p},\widetilde{q}$ with $\widetilde{p} - \widetilde{q} = f$ satisfy $\widetilde{p}'' \ge P$ and $\widetilde{q}'' \ge N$ almost everywhere. Within the affine lift (3.12), the coefficient in (3.15) is also the least possible common slope bound for globally convex non-decreasing profiles. Indeed, their non-negative limiting slopes at $-\infty$ differ by $s_-$, so they are at least $\max\{s_-,0\}$ and $\max\{-s_-,0\}$; their curvature measures dominate $P\ \mathrm{d}z$ and $N\ \mathrm{d}z$. Both upper slope bounds are therefore at least $A$ and $B$ from the proof. This is a minimality statement for this profile-and-affine-lift construction, not for arbitrary bivariate decompositions or for the sizes of the resulting network streams.

*Remark 3.1 (Why the global domain matters).* Under the regularity assumptions on $f'$ above, integrability of $f''$ is also necessary for the existence of *finite-valued* convex, coordinatewise non-decreasing $F_+$,$F_-$ on all of $\mathbb{R}^2$ with difference $f(x-y)$. To see this, restrict each component to $(t,0)$ for $t \le 0$. The resulting convex non-decreasing function has slopes bounded between 0 and a finite one-sided slope

at the origin, hence finite curvature mass on the negative half-line. Next restrict each component to $(0, -t)$ for $t \geq 0$. These restrictions are convex and non-increasing, so their slopes lie between a finite slope at the origin and 0, giving finite curvature mass on the positive half-line. In both restrictions the difference is $f(t)$; the total variation of its curvature is bounded by the sum of the two positive curvature measures. Thus $\int_{\mathbb{R}} |f''| < \infty$. This necessity argument concerns the global domain and does not apply if the pair is defined only on the non-negative quadrant. For example,

$$(x-y)^2 = \left(2x^2 + 2y^2\right) - (x+y)^2$$

is a difference of two convex, coordinatewise non-decreasing functions on that quadrant, although the curvature of $f(z) = z^2$ is not integrable on $\mathbb{R}$.

**Corollary 3.5 (Exact splitting on a bounded activation interval).** *Let $a < b$ and let $f \in C^1([a,b])$ have absolutely continuous derivative. There exist finite-valued convex, coordinatewise non-decreasing functions $F_+, F_- : \mathbb{R}^2 \to \mathbb{R}$, non-negative on $\mathbb{R}^2_{\geq 0}$, such that $F_+(x,y) - F_-(x,y) = f(x-y)$ whenever $x - y \in [a,b]$.*

*Proof.* Extend $f$ by its tangent affine functions at $a$ and $b$. The extension is continuously differentiable, its derivative is locally absolutely continuous, and its curvature vanishes outside $[a,b]$. Apply Theorem 3.4 to this extension. □

This local statement applies, for example, to exponentiation on a fixed bounded score interval. It does not assert a global monotone convex split for $e^z$ on all of $\mathbb{R}$, whose curvature is not integrable.

### 3.2.3 ReLU: the maximum and identity pair

For $f(z) = \mathrm{ReLU}(z)$ choose $p(z) = \mathrm{ReLU}(z)$, $q(z) = 0$, and $L = 1$. Both profiles are convex and satisfy (3.11), even though $p$ is not differentiable at the origin. The profile lift gives the exact pair

$$\begin{aligned} F_+(a,b) = \mathrm{ReLU}(a-b) + b = \max\{a,b\}, \qquad F_-(a,b) = b, \\ \mathrm{ReLU}(a-b) = \max\{a,b\} - b. \end{aligned} \tag{3.18}$$

This specializes the general criterion to a positive Maxout component (Goodfellow et al., 2013) and a negative identity component. Replacing $f''\,\mathrm{d}z$ by $\delta_0$ in the absolute-curvature construction gives the same pair (Remark 3.5). This classical ReLU-network DC split (Awasthi et al., 2026; Siahkamari et al., 2020; Zhang et al., 2018b; Zimmermann and Loho, 2026) underlies our stabilized passes, CNN extension, and saliency methods.

The hidden-layer update is therefore

$$a^{(l,+)} = \max\{z^{(l,+)}, z^{(l,-)}\}, \qquad a^{(l,-)} = z^{(l,-)}, \qquad 1 \leq l < L, \tag{3.19}$$

with $a^{(L,\pm)} = z^{(L,\pm)}$ at the final affine readout. Theorem 3.3 gives validity and convexity. Section 3.3 identifies this as the mixing family's $\eta = 1$ endpoint.

**The stream functions $g$ and $h$.** The DC *split pair* consists of two monotone *split streams*, non-negative on the input quadrant (Figure 3.2). Like the tail network $f^{(l)} : \mathbb{R}^{d^{(l)}} \to \mathbb{R}$ on layers $l+1, \ldots, L$, define the restricted streams

$$g^{(l)} : \mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}} \to \mathbb{R}, \qquad h^{(l)} : \mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}} \to \mathbb{R}, \tag{3.20}$$

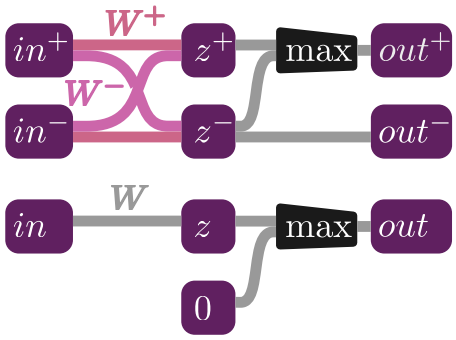



**Fig. 3.2:** Monotone, convex split streams (top) and a standard ReLU layer (bottom). Negative weights swap positive and negative input streams into opposite pre-activation streams; positive weights preserve them.

mapping an activation pair at layer $l$ to the $+$ and $-$ components of the decomposed output; we abbreviate $g \coloneqq g^{(0)}$, $h \coloneqq h^{(0)}$. On the output layer $g^{(L)}(x^+, x^-) = x^+$, $h^{(L)}(x^+, x^-) = x^-$, and for $0 \le l < L$,

$$g^{(l)}(x^+, x^-) = g^{(l+1)}\big(a^{(l+1,+)}, a^{(l+1,-)}\big), \qquad h^{(l)}(x^+, x^-) = h^{(l+1)}\big(a^{(l+1,+)}, a^{(l+1,-)}\big), \tag{3.21}$$

where (3.4)–(3.5) and (3.19) compute $\big(a^{(l+1,+)}, a^{(l+1,-)}\big)$ from $(x^+,x^-)$. These full tail functions, rather than individual layer activations, define $f = g - h$ and underpin Chapter 6. The ***Multilayer Perceptron*** (MLP) split pass starts from $\big(a^{(0,+)},a^{(0,-)}\big) = (x^+,x^-)$, then applies the affine pair (3.4)–(3.5) and the maximum/identity update (3.19) for $l = 1,\dots,L-1$. The final layer uses only the affine pair, allowing negative reconstructed logits.

Non-negative affine maps followed by Maxout gates and projections form monotone networks with convex, monotone activations. The composition principles of Section 2.2 (Amos et al., 2017) give:

**Proposition 3.6 (Monotonicity and convexity of the split streams, `Lean_25`).** *For every $0 \le l \le L$, the stream functions $g^{(l)}$ and $h^{(l)}$ of (3.20) are convex and non-decreasing in each coordinate of the pair $(x^+, x^-) \in \mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}}$.*

*Proof.* Apply Theorem 3.3 to each tail network using the pair $(\max\{a,b\},b)$ from (3.18). The final pair is the identity, so the same argument includes the ungated affine head. □

The correctness statement asserts that the difference of the two split-stream outputs equals the original tail network evaluated at the difference of their inputs—at *every* layer, not only at the input.

**Theorem 3.7 (Correctness of the split forward pass, `Lean_26`).** *For every $0 \le l \le L$ and all $x^+, x^- \in \mathbb{R}^{d^{(l)}}_{\ge 0}$,*

$$f^{(l)}\big(x^+ - x^-\big) = \big(g^{(l)} - h^{(l)}\big)(x^+, x^-). \tag{3.22}$$

*Proof.* Apply Theorem 3.3 to each tail. Hidden-pair subtraction gives $\mathrm{ReLU}(z^+ - z^-)$ by (3.18), and final-pair subtraction gives $z^+ - z^-$ since $\varrho_L = \mathrm{id}$. The identity tail at $l = L$ and recursion (3.21) give (3.22) at every layer. □

*Remark 3.2 (Preservation of sub-threshold signals).* At an inactive neuron, $z_i^{(l)} \le 0$, ReLU outputs $a_i^{(l)} = 0$, discarding the losing pre-activation's magnitude. Downstream computation cannot distinguish degrees of suppression. The split pair $\big(a_i^{(l,+)},a_i^{(l,-)}\big) = \big(z_i^{(l,-)},z_i^{(l,-)}\big)$ has the required zero difference while retaining the cancelled mass $z_i^{(l,-)}$ in both components. These sub-threshold contributions give the streams more information than the original activations, which Chapter 6 exploits. Retaining them also causes the numerical growth of Section 3.6: without subtractive cancellation, the streams accumulate absolute mass $z_i^{(l,+)} + z_i^{(l,-)}$ instead of the signed residual.

*Remark 3.3 (Flexibility of the input split).* Theorem 3.7 allows *any* non-negative representation $x = x^+ - x^-$, including $x^\pm = [x]^\pm$ and $x^\pm = [x]^\pm + c$ with $c \ge 0$. Numerical stabilization and Chapter 6's attribution baselines exploit this freedom. The half–half split $x^\pm = \pm x/2$ (Section 3.6) instead has signed components: it serves only as a backward chain-rule device along the unified input direction.

### 3.2.4 Closed formulas for smooth activations

The following activations admit explicit absolute-curvature profiles. Smooth curvature majorants yield shorter formulas, so we give both and distinguish exact recovery from minimal curvature. Write $\sigma(z) = (1+e^{-z})^{-1}$, let $\varphi,\Phi$ be the standard Gaussian density and distribution function, and set

$$s_\beta(z) \coloneqq \frac{1}{\beta}\log(1+e^{\beta z}), \qquad \beta > 0.$$

**Corollary 3.8 (Explicit convex, monotone activation pairs, Lean_27).** *For $f(z) = z\Phi(z)$ (GELU), $f(z) = z\sigma(\beta z)$ (Swish), $f = \sigma$ (sigmoid), and $f = s_{1/\theta}$ (softplus), choose the corresponding row of Table 3.1. Then the functions $F_+(x,y) = p(x-y) + Ly$ and $F_-(x,y) = q(x-y) + Ly$ are convex and coordinatewise non-decreasing on $\mathbb{R}^2$, map $\mathbb{R}^2_{\ge 0}$ to itself, and satisfy $F_+ - F_- = f(x-y)$.*

| Activation | $p(z)$ | $q(z)$ | $L$ |
|---|---|---|---|
| GELU | $2z\Phi(z) + 2\varphi(z)$ | $z\Phi(z) + 2\varphi(z)$ | 2 |
| Swish | $2s_\beta(z)$ | $2s_\beta(z) - z\sigma(\beta z)$ | 2 |
| Sigmoid | $\frac{1}{2}(s_1(z) + \sigma(z))$ | $\frac{1}{2}(s_1(z) - \sigma(z))$ | $\frac{1}{2}$ |
| Softplus | $s_{1/\theta}(z)$ | 0 | 1 |

**Tab. 3.1:** Closed profiles for (3.12), $\beta,\theta > 0$. GELU, Swish, and sigmoid trade additional curvature for shorter smooth formulas. GELU denotes the exact Gaussian activation, excluding its tanh approximation; SiLU is Swish with $\beta = 1$.

*Proof.* In every row $p - q = f$. For GELU (Hendrycks and Gimpel, 2016), direct differentiation gives

$$p' = 2\Phi, \quad p'' = 2\varphi, \qquad q' = \Phi - z\varphi, \quad q'' = z^2\varphi.$$

Hence both profiles are convex; $p'$ increases from 0 to 2 and $q'$ from 0 to 1. For Swish (Ramachandran et al., 2018), writing $u = \beta z$ gives

$$p' = 2\sigma(u), \quad p'' = 2\beta\sigma'(u), \qquad q' = \sigma(u) - u\sigma'(u), \quad q'' = \beta u\big(2\sigma(u) - 1\big)\sigma'(u) \ge 0.$$

The last inequality uses that $u$ and $2\sigma(u)-1$ have the same sign. The slopes again have limits 0,2 for $p'$ and 0,1 for $q'$, so they lie in [0,2]. For the sigmoid profiles,

$$p' = \tfrac{1}{2}\sigma(2-\sigma), \quad q' = \tfrac{1}{2}\sigma^2, \qquad p'' = \sigma'(1-\sigma) \geq 0, \quad q'' = \sigma'\sigma \geq 0,$$

with slopes in [0,1/2]. Softplus has $p' = \sigma(z/\theta) \in [0,1]$ and $p'' = \sigma'(z/\theta)/\theta \geq 0$, while $q = 0$. All profile values at zero are non-negative (for sigmoid, $s_1(0) = \log 2 > 1/2$). Proposition 3.2 proves the claim. □

These formulas require no numerical integration or auxiliary root solve. For example, the softplus row simplifies directly to

$$F_+(x,y) = \theta \log\bigl(e^{x/\theta} + e^{y/\theta}\bigr), \qquad F_-(x,y) = y, \tag{3.23}$$

whose limit as $\theta \downarrow 0$ is exactly $(\max\{x,y\},y)$. The sigmoid row gives

$$F_\pm(x,y) = \frac{\log(e^x + e^y) \pm \sigma(x-y)}{2}. \tag{3.24}$$

Writing $d = x - y$, the GELU and Swish pairs are explicitly

$$\begin{aligned} F_+^{\mathrm{GELU}}(x,y) &= 2\bigl(d\Phi(d) + \varphi(d) + y\bigr), \\ F_-^{\mathrm{GELU}}(x,y) &= d\Phi(d) + 2\varphi(d) + 2y, \end{aligned} \tag{3.25}$$

and

$$\begin{aligned} F_+^{\mathrm{Swish}}(x,y) &= \frac{2}{\beta}\log\bigl(e^{\beta x} + e^{\beta y}\bigr), \\ F_-^{\mathrm{Swish}}(x,y) &= F_+^{\mathrm{Swish}}(x,y) - d\sigma(\beta d). \end{aligned} \tag{3.26}$$

Stable implementations of the exponential formulas should evaluate log-sum-exp through its usual shifted form.

**Closed forms for the canonical absolute-curvature split.** The smaller coefficients in Theorem 3.4 can also be attained explicitly. For sigmoid,

$$p(z) = \begin{cases} \sigma(z), & z \leq 0, \\ \frac{1}{2} + \frac{1}{4}z, & z \geq 0, \end{cases} \qquad q(z) = p(z) - \sigma(z), \qquad L = \tfrac{1}{4}. \tag{3.27}$$

For both GELU and Swish, let $a > 0$ be the positive zero of $f''$, and define $m = f'(a) - 1$, $c = af'(a) - f(a)$ and $L = 1 + 2m$. Then

$$p(z) = \begin{cases} -c, & z \leq -a, \\ f(z) + mz, & -a \leq z \leq a, \\ Lz - c, & z \geq a, \end{cases} \qquad q(z) = p(z) - f(z). \tag{3.28}$$

The constants are

$$\begin{aligned} &\text{GELU:} \quad a = \sqrt{2}, \quad m = \Phi(\sqrt{2}) + \sqrt{2}\varphi(\sqrt{2}) - 1, \quad c = 2\varphi(\sqrt{2}), \\ &\text{Swish:} \quad a = r/\beta, \quad m = (r-2)/4, \quad c = (r^2-4)/(4\beta), \quad L = r/2, \end{aligned} \tag{3.29}$$

where $r$ uniquely solves $r\tanh(r/2) = 2$ on $r > 0$ ($r \approx 2.39936$): the left-hand side increases strictly from 0 to infinity. Fixing $r$ gives a pointwise Swish formula without quadrature. Softplus already uses the canonical profiles in Table 3.1.

For (3.28), both activations satisfy $f(-z) = f(z) - z$, $f'(-\infty) = 0$, $f'(+\infty) = 1$, and have positive curvature precisely on $(-a,a)$. Piecewise differentiation gives $p'' = \max\{f'',0\}$, $q'' = \max\{-f'',0\}$, with matching values and first derivatives at $\pm a$. Since $p(0) = q(0) = 0$, these are Theorem 3.4's normalized profiles. Sigmoid's curvature changes sign at zero, giving (3.27) analogously. Canonical GELU and Swish profiles can be negative on the negative half-line; origin normalization and coordinatewise monotonicity ensure their lifted pairs are non-negative on the required quadrant.

**Corollary 3.9 (Modern activation functions, `Lean_28`).** *Theorems 3.4 and 3.13 apply to the following activation functions, where $\varphi$ and $\Phi$ denote the standard Gaussian density and distribution function and $\sigma(x) = (1 + e^{-x})^{-1}$ the logistic sigmoid:*

1) GELU, $f(x) = x\,\Phi(x)$, *with* $f''(x) = (2 - x^2)\,\varphi(x)$; *the curvature is positive on* $(-\sqrt{2}, \sqrt{2})$ *and negative outside.*
2) Swish/SiLU, $f(x) = x\,\sigma(\beta x)$ *with* $\beta > 0$, *with* $f''(x) = \beta\,\sigma'(\beta x)\big(2 + \beta x\,(1 - 2\sigma(\beta x))\big)$; *the curvature is positive on a bounded interval around the origin and negative in both tails.*
3) Mish, $f(x) = x\tanh\big(\log(1 + e^x)\big)$.
4) Hyperbolic tangent, $f(x) = \tanh(x)$, *with* $f''(x) = -2\tanh(x)\operatorname{sech}^2(x)$, *so $P$ is supported on* $(-\infty, 0]$ *and $N$ on* $[0, \infty)$.
5) Logistic sigmoid, $f = \sigma$, *with* $f'' = \sigma'\,(1 - 2\sigma)$.
6) Softplus at temperature $\theta > 0$, $f(x) = \theta\log(1 + e^{x/\theta})$, *with* $f''(x) = \sigma'(x/\theta)/\theta \geq 0$; *here* $N = 0$, $C_2 = 0$, *and the decomposition degenerates to the statement that softplus itself is convex.*

*In each case $f(0) \geq 0$, and both constructions preserve non-negativity on the input quadrant (Remark 3.6).*

*Proof.* Direct differentiation gives the stated second derivatives. Each $f''$ is continuous and decays at least exponentially as $|x| \to \infty$ (Gaussian for GELU). For Swish and Mish, exponential decay of $\sigma'(\beta x)$ and the derivatives of $\tanh \circ \operatorname{softplus}$ dominates the polynomial factor $x$. Thus $f'' \in L^1(\mathbb{R})$ and $f'$ is locally absolutely continuous. Both constructions apply, with non-negative $f(0) \in \{0, \frac{1}{2}, \theta\log 2\}$. □

**Corollary 3.10 (Networks with smooth activations, `Lean_29`).** *Every finite feedforward network with GELU, Swish/SiLU, sigmoid, or softplus hidden activations, arbitrary signed weights and biases, and an affine output admits an exact decomposition into two convex, coordinatewise non-decreasing streams on doubled input space. Either the explicit profiles of Corollary 3.8 or the canonical profiles (3.27)–(3.28) and (3.23) may be used at every hidden neuron. With the usual non-negative input split, the streams remain non-negative and their pullbacks are convex functions of the original signed input. The same existence result holds for Mish and tanh by Theorem 3.4.*

*Proof.* The activation pairs have the properties required by Theorem 3.3, by Corollaries 3.8 and 3.9; apply that theorem with the identity pair at the output. □

The induction also permits different qualifying activations or profiles at each hidden layer. The theorem states the uniform-activation case used by the formal network interface.

*Remark 3.4 (Scope of the smooth extension).* The extension above is an exact structural theorem about the scalar activations and their composition into feedforward networks. It does not establish empirical attribution quality for these smooth networks, nor does it by itself extend the ReLU-specific hard-gate games or numerical stabilization formulas. The experiments later in the thesis evaluate the architectures and methods described in their protocols; the new smooth formulas are not additional experimental results.

### 3.2.5 Multivariate activations and softmax

For activations coupling coordinates, convexity means componentwise convexity of each vector-valued pair. This differentiable lift suffices for softmax.

**Proposition 3.11 (Multivariate profile lift, `Lean_30`).** *Let $C \in C^1(\mathbb{R}^n)$ and $\ell \in \mathbb{R}^n_{\geq 0}$. Define*

$$T_{C,\ell}(x,y) = C(x-y) + \langle \ell, y \rangle, \qquad x,y \in \mathbb{R}^n. \tag{3.30}$$

*Then $T_{C,\ell}$ is convex on $\mathbb{R}^{2n}$ if and only if $C$ is convex, and is coordinatewise non-decreasing if and only if $0 \leq \partial_j C(z) \leq \ell_j$ for all $z$ and $j$. Consequently, if $C_+ - C_- = f$, both profiles are convex, and both satisfy these derivative bounds, the common lift gives a convex, coordinatewise non-decreasing pair with difference $f(x-y)$. If $C_+(0), C_-(0) \geq 0$, the lifted pair is non-negative on $\mathbb{R}^{2n}_{\geq 0}$.*

*Proof.* Affine precomposition preserves convexity; restriction to $y = 0$ gives the converse. The derivatives $\partial_{x_j} T = \partial_j C$ and $\partial_{y_j} T = \ell_j - \partial_j C$ give the monotonicity criterion. The common affine term cancels, and monotonicity bounds values on the non-negative orthant by their origin values. □

Softmax admits a short pair because the log-sum-exp Hessian dominates its curvature. This uses multivariate curvature comparison, not the scalar integrability theorem.

**Theorem 3.12 (Explicit convex, monotone softmax pair, `Lean_31`).** *Let $n \geq 1$, and define*

$$H(z) = \log\left(\sum_{j=1}^{n} e^{z_j}\right), \qquad \pi_i(z) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}, \qquad c_n = \frac{\log n - 1/n}{2}.$$

*For each output coordinate $i$, set*

$$F_{i,\pm}(x,y) = \frac{H(x-y) \pm \pi_i(x-y) + \sum_{j=1}^{n} y_j}{2} - c_n. \tag{3.31}$$

*Both components are finite, convex, and coordinatewise non-decreasing on all of $\mathbb{R}^{2n}$. They are non-negative on $\mathbb{R}^{2n}_{\geq 0}$ and satisfy $F_{i,+}(x,y) - F_{i,-}(x,y) = \pi_i(x-y)$.*

*Proof.* Fix $z$ and a direction $v \in \mathbb{R}^n$, and abbreviate $\mu = \sum_j \pi_j(z) v_j$ and $V = \sum_j \pi_j(z)(v_j - \mu)^2$. Differentiation along $z + tv$ gives

$$\begin{aligned} D^2 H(z)[v,v] &= V, \\ D^2 \pi_i(z)[v,v] &= \sum_j \pi_j(z)\big(\mathbf{1}[j=i] - \pi_i(z)\big)(v_j - \mu)^2. \end{aligned} \tag{3.32}$$

Since $|\mathbf{1}[j=i]-\pi_i|\le 1$, it follows that $|D^2\pi_i(z)[v,v]|\le V$. Thus $H+\pi_i$ and $H-\pi_i$ have positive semidefinite Hessians and are convex. For the profiles $C_{i,\pm}=(H\pm\pi_i)/2-c_n$, their partial derivatives are

$$\begin{array}{c|cc} & \partial_j C_{i,+} & \partial_j C_{i,-} \\ \hline j=i & \pi_i-\pi_i^2/2 & \pi_i^2/2 \\ j\neq i & \pi_j(1-\pi_i)/2 & \pi_j(1+\pi_i)/2. \end{array} \tag{3.33}$$

Every entry lies in $[0,1/2]$; for the last entry, use $\pi_j\le 1-\pi_i$. Proposition 3.11 with $\ell_j=1/2$ gives global convexity and monotonicity. At the origin, $F_{i,+}(0,0)=1/n$ and $F_{i,-}(0,0)=0$, proving non-negativity on the orthant. Subtraction gives exact recovery. □

For fixed masked softmax, apply the theorem to each row's nonempty allowed set and assign masked outputs (0,0). Fully masked rows require a separate convention and lie outside this theorem. Fixed positive temperatures and fixed additive score biases enter the preceding affine map. Section 5.3 combines this pair with convex product and normalization pairs to split the full Transformer.

### 3.2.6 The symmetric-tilt variant and curvature minimality

A symmetric alternative uses the same curvature split with common tilt $M(a+b)$. Its fixed-profile minimality is independent of (3.12). Both constructions recover the same difference, generally with different streams; only the asymmetric normalization above gives the literal ReLU pair $(\max\{a,b\},b)$.

**Theorem 3.13 (DC–monotone splitting under integrable curvature, `Lean_32`).** *Let $f:\mathbb{R}\to\mathbb{R}$ be differentiable with locally absolutely continuous derivative $f'$ and $f''\in L^1(\mathbb{R})$. Then the limit $s_-:=\lim_{u\to-\infty}f'(u)$ exists and $f'$ is bounded. Write $P:=\max\{f'',0\}$ and $N:=\max\{-f'',0\}$, so that $f''=P-N$ with $P,N\ge 0$, and define*

$$C_1(x):=f(0)+\int_0^x\Big(s_-+\int_{-\infty}^t P(u)\,du\Big)\,dt,\qquad C_2(x):=\int_0^x\int_{-\infty}^t N(u)\,du\,dt, \tag{3.34}$$

*as well as $M:=\max\big\{\sup_{x\in\mathbb{R}}|C_1'(x)|,\sup_{x\in\mathbb{R}}|C_2'(x)|\big\}<\infty$. Then the functions*

$$g(a,b):=C_1(a-b)+M(a+b),\qquad h(a,b):=C_2(a-b)+M(a+b) \tag{3.35}$$

*are convex on $\mathbb{R}^2$, non-decreasing in each argument, and satisfy*

$$f(a-b)=g(a,b)-h(a,b)\qquad \text{for all } (a,b)\in\mathbb{R}^2. \tag{3.36}$$

*Proof. Well-definedness.* For $x\le y$ we have $|f'(y)-f'(x)|\le\int_x^y|f''(u)|\,\mathrm{d}u$, which tends to 0 as $x,y\to-\infty$ because $f''\in L^1(\mathbb{R})$; by the Cauchy criterion the limit $s_-$ exists. Consequently $|f'(x)|\le|s_-|+\|f''\|_{L^1(\mathbb{R})}$ for all $x$, so the boundedness of $f'$ is automatic and need not be assumed. Since $0\le P,N\le|f''|$, both $P$ and $N$ lie in $L^1(\mathbb{R})$, the inner integrals in (3.34) are well-defined, non-decreasing in $t$, and bounded by $\|P\|_{L^1(\mathbb{R})}$ and $\|N\|_{L^1(\mathbb{R})}$ respectively. Hence $C_1,C_2$ are well-defined with locally absolutely continuous derivatives

$$C_1'(x)=s_-+\int_{-\infty}^x P(u)\,\mathrm{d}u,\qquad C_2'(x)=\int_{-\infty}^x N(u)\,\mathrm{d}u,$$

and $|C_1'|\le|s_-|+\|P\|_{L^1(\mathbb{R})}$, $|C_2'|\le\|N\|_{L^1(\mathbb{R})}$, so $M<\infty$.

*Exactness.* For almost every $x$,

$$C_1'(x) - C_2'(x) = s_- + \int_{-\infty}^{x} \big(P(u) - N(u)\big)\,\mathrm{d}u = s_- + \int_{-\infty}^{x} f''(u)\,\mathrm{d}u = f'(x),$$

Integrating from 0, with $C_1(0) - C_2(0) = f(0)$, gives $C_1 - C_2 = f$ on $\mathbb{R}$. The common tilt cancels, proving (3.36) at $x = a - b$.

*Convexity.* Since $P,N \geq 0$, their (shifted) antiderivatives $C_1',C_2'$ are non-decreasing, so $C_1,C_2$ are convex. Linear precomposition by $(a,b) \mapsto a - b$ and addition of $M(a+b)$ preserve convexity, giving convex $g,h$ on $\mathbb{R}^2$. When $f''$ is continuous, the Hessian of $C_1(a-b)$ has eigenvalues 0 and $2C_1''(a-b) = 2P(a-b) \geq 0$.

*Monotonicity.* By the choice of $M$,

$$\frac{\partial g}{\partial a}(a,b) = C_1'(a-b) + M \geq 0, \qquad \frac{\partial g}{\partial b}(a,b) = -C_1'(a-b) + M \geq 0,$$

and analogously for $h$ with $C_2'$ in place of $C_1'$. Hence $g$ and $h$ are non-decreasing in each argument. □

The curvature split is canonical. For the fixed profiles (3.34), $M$ minimizes symmetric tilts $\lambda(a+b)$; this does not compare asymmetric corrections or other profile normalizations.

**Proposition 3.14 (Minimality of the curvature split, `Lean_33`).** *Let $f$ be as in Theorem 3.13 and suppose $f = c_1 - c_2$ for convex functions $c_1, c_2 \colon \mathbb{R} \to \mathbb{R}$ whose second distributional derivatives are locally integrable functions. Then $c_1'' \geq P$ and $c_2'' \geq N$ almost everywhere. In particular, $(C_1, C_2)$ is, up to a common affine summand, the unique pair of convex functions with $C_1 - C_2 = f$ of pointwise minimal curvature, and among all tilts of the form $(a,b) \mapsto \lambda(a+b)$, $\lambda \geq 0$, the constant $M$ in Theorem 3.13 is the smallest one rendering both components of (3.35) non-decreasing in each argument.*

*Proof.* Convexity gives $c_1'' = f'' + c_2'' \geq f''$ and $c_1'' \geq 0$, hence $c_1'' \geq \max\{f'',0\} = P$ almost everywhere; symmetrically, $c_2'' \geq N$. Equality a.e. in both bounds makes $c_1 - C_1$ and $c_2 - C_2$ affine. They coincide because $c_1 - c_2 = f = C_1 - C_2$. Monotonicity in both arguments requires $\lambda \geq \pm C_1'(x)$ for all $x$, equivalently $\lambda \geq \sup_x |C_1'(x)|$, and likewise for $C_2'$. The least admissible $\lambda$ is $M$. □

*Remark 3.5 (Bounded variation and the piecewise-linear case).* The construction extends to locally absolutely continuous $f$ whose weak derivative has a bounded-variation representative, with finite signed Radon curvature measure $\mu$. Substitute $|\mu|$ for $|f''(u)|\,\mathrm{d}u$ in Theorem 3.4, or Jordan parts $\mu_+,\mu_-$ for $P(u)\,\mathrm{d}u, N(u)\,\mathrm{d}u$. The resulting profiles are convex with bounded slopes; criterion (3.11) handles kinks. ReLU has $\mu = \delta_0$, $s_- = 0$, giving $p(z) = \max\{z,0\}$, $q = 0$, $L = 1$, hence exactly $(\max\{a,b\},b)$ under the asymmetric lift. The symmetric variant gives $(\mathrm{ReLU}(a-b) + a + b, a + b)$. Leaky ReLU with slope $\alpha$ has curvature $(1-\alpha)\delta_0$; standard ELU with unit negative-branch scale has curvature $e^z \mathbb{1}_{z<0}\,\mathrm{d}z$. Both fit this finite-measure construction, unifying piecewise-linear and smooth activations through bounded variation.

*Remark 3.6 (Compatibility with valid decompositions).* Both constructions fit Definition 3.1. The asymmetric pair has non-negative origin values $F_+(0,0) = \max\{f(0),0\}$, $F_-(0,0) = \max\{-f(0),0\}$; monotonicity gives non-negativity on $\mathbb{R}^2_{\geq 0}$. The symmetric pair (3.35) has $g(0,0) = f(0)$, $h(0,0) = 0$, so this holds whenever $f(0) \geq 0$, including Corollary 3.9. If $f(0) < 0$, adding $|f(0)|$ to both

components restores non-negativity and preserves their difference. Exactness gives activation-level validity; Theorem 3.3 gives network recovery, convexity, and monotonicity.

## 3.3 The Mixing Decomposition family

ReLU networks compute CPWL functions (Arora et al., 2018) admitting DC decompositions (Kripfganz and Schulze, 1987; Melzer, 1986; Zimmermann and Loho, 2026). We extend the ReLU pair to a one-parameter family of pairwise activations interpolating between two arithmetic extremes.

**Definition 3.3 (Mixing decomposition function, `Lean_34`).** For $\eta \in [0,1]$, the *mixing decomposition function* is

$$f^{\eta}_{\mathrm{mix}}(z^+, z^-) := \big(\, \eta \max\{z^+, z^-\} + (1-\eta)\, z^+,\;\; \eta\, z^- + (1-\eta) \min\{z^+, z^-\} \,\big). \tag{3.37}$$

The *Mixing Decomposition* uses $f^{\eta}_{\mathrm{mix}}$ at every hidden neuron, the identity pair at the affine output, the canonical map (3.4)–(3.5), and input convention (3.2). We tag its activations $a^{(l,\pm,\eta)}_{\mathrm{mix}}$ when comparing decompositions.

Convex combinations and maxima/minima preserve non-negativity on $\mathbb{R}^2_{\geq 0}$, so $f^{\eta}_{\mathrm{mix}}$ is a pairwise activation (Definition 3.1). It linearly interpolates between $\big(\max\{z^+, z^-\}, z^-\big)$ at $\eta = 1$ and $\big(z^+, \min\{z^+, z^-\}\big)$ at $\eta = 0$, giving a valid ReLU decomposition for every parameter.

**Theorem 3.15 (Layer-wise recovery of the Mixing Decomposition, `Lean_35`).** *Let the activation pairs be propagated by the canonical non-negative linear map (3.4)–(3.5), initialised by (3.2), and updated by $f^{\eta}_{\mathrm{mix}}$ for some $\eta \in [0,1]$. Then for every layer $l$ and neuron $i$,*

$$a^{(l,+,\eta)}_{\mathrm{mix},i} - a^{(l,-,\eta)}_{\mathrm{mix},i} = \varrho_l\big(z^{(l)}_i\big) = a^{(l)}_i, \tag{3.38}$$

*i.e. the Mixing Decomposition is a valid decomposition (Definition 3.2) of the ReLU network for every $\eta \in [0,1]$.*

*Proof.* Induct on $l$.

*Base case $l = 0$.* By the input convention (3.2), $a^{(0,+,\eta)}_{\mathrm{mix},k} - a^{(0,-,\eta)}_{\mathrm{mix},k} = x^+_k - x^-_k = x_k = a^{(0)}_k$.

*Inductive step.* Assuming (3.38) at $l-1$, Proposition 3.1 gives $z^{(l,+)}_i - z^{(l,-)}_i = z^{(l)}_i$, proving recovery at the affine output. At a hidden layer, write $u = z^{(l,+)}_i$, $v = z^{(l,-)}_i$. Definition (3.37) gives

$$a^{(l,+,\eta)}_{\mathrm{mix},i} - a^{(l,-,\eta)}_{\mathrm{mix},i} = \eta\big(\max\{u,v\} - v\big) + (1-\eta)\big(u - \min\{u,v\}\big) = \mathrm{ReLU}(u-v) = \mathrm{ReLU}\big(z^{(l)}_i\big) = a^{(l)}_i,$$

using the identities $\max\{u,v\} - v = \max\{u-v, 0\}$ and $u - \min\{u,v\} = \max\{u-v, 0\}$. □

Both endpoints recover $\mathrm{ReLU}(u-v)$: at $\eta = 1$ the $+$ stream carries the non-linearity through a maximum; at $\eta = 0$ the $-$ stream carries it through a minimum. The $\eta = 1$ endpoint produces *convex* activations at every layer.

**Theorem 3.16 ($\eta = 1$ convex characterization of the Mixing Decomposition, Lean_36).** *Under the input convention (3.2) with split components $x_k^{\pm}$ that are convex functions of $x$ (e.g. the entrywise split $x^{\pm} = [x]^{\pm}$, or any affine split), the $\eta = 1$ mixing decomposed activations*

$$a_{\mathrm{mix},i}^{(l,+,1)}(x) = \max\{z_i^{(l,+)}(x), z_i^{(l,-)}(x)\} \qquad \text{and} \qquad a_{\mathrm{mix},i}^{(l,-,1)}(x) = z_i^{(l,-)}(x)$$

*are convex in $x$ at every hidden layer $l$ and neuron $i$; at the final layer, both streams are non-negative affine combinations of the previous streams and remain convex. Consequently $a_i^{(l)} = a_{\mathrm{mix},i}^{(l,+,1)} - a_{\mathrm{mix},i}^{(l,-,1)}$ is a difference of convex functions, the standard DC representation of the CPWL function computed by the network (Arora et al., 2018; Zimmermann and Loho, 2026).*

*Proof.* For $\eta = 1$, the activation pair is $(\max\{z^+,z^-\},z^-)$, so Theorem 3.3 applies. Its final assertion gives convexity after any componentwise convex input split, and its identity output pair includes the final affine readout. □

*Remark 3.7 (On the $\eta = 0$ extreme).* The dual DC representation writes the output as $u(x) - v(x)$ with $v$ *concave*, and $f_{\mathrm{mix}}^0$ places $a_{\mathrm{mix},i}^{(l,-,0)} = \min\{z_i^{(l,+)}, z_i^{(l,-)}\}$ in the role of $v$. This is concave only when both decomposed pre-activations are concave (or affine): the minimum of two concave functions is concave, but of two convex functions is in general neither. Under the input convention (3.2), the first pre-activations are convex in $x$, but later $\eta = 0$ streams and pre-activations need not remain convex or concave. A canonical scalar example is $W^{(1)} = 1$, $b^{(1)} = 1$, giving the negative stream $v(x) = \min\{\max(x,0)+1, \max(-x,0)\}$. It satisfies $v(-2) = v(-1) = 1$, $v(0) = v(1) = 0$: the midpoint tests at $-1$ and 0 respectively exclude convexity and concavity. By Theorem 3.15, $f_{\mathrm{mix}}^0$ still yields a valid arithmetic decomposition, and the network-level concave representation is recoverable with an unsplit affine input convention, at the cost of the $\pm$ structure.

Only the $\eta = 1$ endpoint inherits per-layer convexity: it is the maximum/identity DC split of Section 3.2.3, with $\eta = 0$ its arithmetic dual.

**Corollary 3.17 (DC split as the $\eta = 1$ mixing endpoint, Lean_37).** *Set $\eta = 1$ in Definition 3.3 and drop the decomposition tag, writing $a^{(l,\pm)} := a_{\mathrm{mix}}^{(l,\pm,1)}$. The resulting DC split applies the update (3.19) entrywise at hidden layers $l = 1, \ldots, L-1$, and sets $a^{(L,\pm)} = z^{(L,\pm)}$ at the final affine readout, with the decomposed pre-activations (3.4)–(3.5) induced by the weight split $W^{(l)} = W^{(l,+)} - W^{(l,-)}$ and the input split $x = x^+ - x^-$. It is a valid decomposition of the ReLU network, and under the input convention of Theorem 3.16 its decomposed activations are convex functions of the input.*

*Proof.* At $\eta = 1$, (3.37) equals (3.18); Proposition 3.2 and Theorem 3.3 give its properties and network recovery. □

## 3.4 The Stopping Decomposition

The mixing family uses per-stream maxima and minima. The Stopping Decomposition applies the original network's hard gate as one *shared* decision to both streams.

**Definition 3.4 (Stopping decomposition function, Lean_38).** The *stopping decomposition function* is

$$f_{\mathrm{stop}}(z^+, z^-) := \big(\mathbf{1}[z^+ - z^- > 0]\, z^+,\ \ \mathbf{1}[z^+ - z^- > 0]\, z^-\big). \tag{3.39}$$

The *Stopping Decomposition* uses $f_{\text{stop}}$ at every hidden neuron, the identity pair at the affine output, the canonical map (3.4)–(3.5), and input convention (3.2). We tag its activations $a^{(l,\pm)}_{\text{stop}}$ when comparing decompositions.

Multiplying non-negative inputs by indicators makes $f_{\text{stop}}$ a pairwise activation $\mathbb{R}^2_{\geq 0} \to \mathbb{R}^2_{\geq 0}$ (Definition 3.1). Its gate becomes a stop/continue decision in Chapter 4.

**Theorem 3.18 (Validity of the Stopping Decomposition, `Lean_39`).** *Let the activation pairs be propagated by the canonical non-negative linear map (3.4)–(3.5), initialised by (3.2), and updated by $f_{\text{stop}}$. Then for every layer $l$ and neuron $i$,*

$$a^{(l,+)}_{\text{stop},i} - a^{(l,-)}_{\text{stop},i} = \varrho_l\big(z^{(l)}_i\big) = a^{(l)}_i, \tag{3.40}$$

*i.e. the Stopping Decomposition is a valid decomposition (Definition 3.2) of the ReLU network.*

*Proof.* Induction on $l$. The base case $l = 0$ is the input convention, and the final affine step follows directly from Proposition 3.1. For a hidden-layer step, assume (3.40) at $l-1$; Proposition 3.1 gives $z^{(l,+)}_i - z^{(l,-)}_i = z^{(l)}_i$. Since both components of (3.39) carry the *same* gate $\mathbf{1}\big[z^{(l)}_i > 0\big]$,

$$a^{(l,+)}_{\text{stop},i} - a^{(l,-)}_{\text{stop},i} = \mathbf{1}\big[z^{(l)}_i > 0\big]\big(z^{(l,+)}_i - z^{(l,-)}_i\big) = \mathbf{1}\big[z^{(l)}_i > 0\big]\, z^{(l)}_i = \mathrm{ReLU}\big(z^{(l)}_i\big) = a^{(l)}_i. \qquad \square$$

*Remark 3.8 (Shared gate versus per-stream maxima).* At an inactive neuron, $z^{(l)}_i \leq 0$, the DC split of Corollary 3.17 outputs $\big(z^{(l,-)}_i, z^{(l,-)}_i\big)$, preserving sub-threshold magnitude (Remark 3.2) and propagating cancelled mass. The Stopping Decomposition outputs $(0, 0)$, annihilating both streams exactly where the original ReLU vanishes. Thus the convex mixing streams ($\eta = 1$) retain dead-neuron contributions in downstream pairs, while stopping mass follows only active neurons. The discontinuous stopping gate sacrifices global convexity for exact forward-support agreement and retains regularity on each linear region.

**Lemma 3.19 (Monotonicity of the Stopping Decomposition on linear regions, `Lean_40`).** *On each linear region of the ReLU network—a maximal subset of input space on which all gate indicators $\mathbf{1}\big[z^{(l)}_j > 0\big]$ are constant—each stopping decomposed activation $a^{(l,\pm)}_{\text{stop},i}$ is a non-negative-linear (i.e. affine with non-negative coefficients) function of the input pair $(x^+, x^-) \in \mathbb{R}^{2d^{(0)}}_{\geq 0}$, hence monotone non-decreasing in each coordinate $x^+_k$ and $x^-_k$.*

*Proof.* Induction on $l$. At $l = 0$, $a^{(0,\pm)}_{\text{stop},k} = x^{\pm}_k$ is a coordinate of $(x^+, x^-)$. At $l \geq 1$, the map (3.4)–(3.5) has $W^{(l,\pm)}, b^{(l,\pm)} \geq 0$, so $z^{(l,\pm)}_i$ is non-negative-linear in the layer-$(l-1)$ activations and, by the hypothesis, in $(x^+, x^-)$. On the linear region the gate $\mathbf{1}\big[z^{(l)}_i > 0\big]$ is constant, so $a^{(l,\pm)}_{\text{stop},i} = \mathbf{1}\big[z^{(l)}_i > 0\big]\, z^{(l,\pm)}_i$ stays non-negative-linear. $\square$

**Excitatory and inhibitory path isolation.** The shared gate never mixes streams, allowing exact accounting of *which input mass reaches which stream.* Each decomposed pre-activation sums backward paths; the parity of their negative weights determines the excitatory (+) or inhibitory (−) destination.

**Lemma 3.20 (Parity path view on the Stopping Decomposition, Lean_41).** *Fix a ReLU network with weights $W^{(l)}$ and, for clarity, vanishing biases. For a neuron $i$ in layer $l \geq 1$, a* backward path *to the input is a list $P = (P_l, \dots, P_0)$ with $P_l = i$ and $P_t$ a neuron in layer $t$, using weights $W^{(t)}_{P_t,P_{t-1}}$. Define the absolute weight product, parity sign, and gate factor*

$$|w|(P) \coloneqq \prod_{t=1}^{l} \big|W^{(t)}_{P_t,P_{t-1}}\big|,$$
$$\operatorname{sgn}(P) = + \iff \#\big\{t : W^{(t)}_{P_t,P_{t-1}} < 0\big\} \text{ is even},$$
$$G_{<l}(P) \coloneqq \prod_{t=1}^{l-1} \mathbf{1}\big[z^{(t)}_{P_t} > 0\big].$$

*Then the decomposed pre-activations of the Stopping Decomposition admit the path-sum representation*

$$z_i^{(l,+)} = \sum_{P:\, P_l=i} G_{<l}(P)\, |w|(P)\, x_{P_0}^{\operatorname{sgn}(P)}, \qquad z_i^{(l,-)} = \sum_{P:\, P_l=i} G_{<l}(P)\, |w|(P)\, x_{P_0}^{\operatorname{sgn}(P)'}, \tag{3.41}$$

*where $\operatorname{sgn}(P)'$ denotes the opposite sign of $\operatorname{sgn}(P)$.*

*Proof.* Induction on $l$.

*Base case $l = 1$.* By (3.4), $z_i^{(1,+)} = \sum_k \big(W_{ik}^{(1,+)} x_k^+ + W_{ik}^{(1,-)} x_k^-\big)$. Paths with $P_1 = i$, $P_0 = k$ biject with predecessors $k$, each with $|w|(P) = \big|W_{ik}^{(1)}\big|$, $G_{<1}(P) = 1$, and $\operatorname{sgn}(P) = +$ iff $W_{ik}^{(1)} \geq 0$. If $W_{ik}^{(1)} > 0$, only $W_{ik}^{(1,+)}$ contributes, giving $|w|(P)\, x_k^+$ with $\operatorname{sgn}(P) = +$; if $W_{ik}^{(1)} < 0$, only $W_{ik}^{(1,-)}$ contributes, giving $|w|(P)\, x_k^-$ with $\operatorname{sgn}(P) = -$. A zero-weight edge has zero contribution regardless of its parity label. Both nonzero cases match (3.41); $z_i^{(1,-)}$ is identical with the parity swapped.

*Inductive step $l \geq 2$.* The recursion (3.4) takes stopping activations from layer $l-1$ as inputs,

$$z_i^{(l,+)} = \sum_j \Big(W_{ij}^{(l,+)}\, a_{\text{stop},j}^{(l-1,+)} + W_{ij}^{(l,-)}\, a_{\text{stop},j}^{(l-1,-)}\Big), \qquad a_{\text{stop},j}^{(l-1,\pm)} = \mathbf{1}\big[z_j^{(l-1)} > 0\big]\, z_j^{(l-1,\pm)}.$$

Each path $P$ through predecessor $j$ uniquely prepends $i$ to a sub-path $P'$. The edge multiplies its absolute weight by $\big|W_{ij}^{(l)}\big|$; the gate at $j$ extends $G_{<l-1}(P')$ to $G_{\leq l-1}(P') = G_{<l}(P)$. If $W_{ij}^{(l)} > 0$, only $W_{ij}^{(l,+)}\, a_{\text{stop},j}^{(l-1,+)}$ contributes to $z_i^{(l,+)}$, preserving $\operatorname{sgn}(P) = \operatorname{sgn}(P')$. The induction hypothesis therefore gives the terms of (3.41) carrying $x_{P_0}^{\operatorname{sgn}(P)}$. If $W_{ij}^{(l)} < 0$, only $W_{ij}^{(l,-)}\, a_{\text{stop},j}^{(l-1,-)}$ contributes and parity flips; the hypothesis for $z_j^{(l-1,-)}$ gives the same required terms through $x_{P'_0}^{\operatorname{sgn}(P')'} = x_{P_0}^{\operatorname{sgn}(P)}$. Zero-weight edges contribute nothing. Summing over $j$ proves the positive-stream formula; swapping streams proves the negative one. Biases add stub paths terminating at intermediate layers with the same parity rule. □

Subtracting the two path sums and using $x_k^+ - x_k^- = x_k$ recovers the standard active-path-sum representation $z_i^{(l)} = \sum_P G_{<l}(P)\, w(P)\, x_{P_0}$ with signed weight products $w(P)$: the Stopping Decomposition refines it by sorting every active path into the excitatory or inhibitory stream according to its parity. Chapter 4 reinterprets this decomposition as a two-player zero-sum stopping game, in which the gate of (3.39) becomes the players' optimal stop/continue decision.

## 3.5 Regularized Decompositions

The hard gate of the Stopping Decomposition solves an optimization problem: for any $\Delta \in \mathbb{R}$,

$$\mathrm{ReLU}(\Delta) = \max_{\pi\in[0,1]} \pi\Delta, \qquad \mathbf{1}[\Delta > 0] \in \mathrm{argmax}_{\pi\in[0,1]}\, \pi\Delta, \tag{3.42}$$

so the indicator in (3.39) is the *unregularized* stop ($\pi = 0$)/continue ($\pi = 1$) argmax for $\Delta = z^+ - z^-$. A convex regularization penalty can smooth this gate, replacing the recovered ReLU by the regularizer's Fenchel conjugate.

**Definition 3.5 (Regularized decomposition function, `Lean_42`).** Let $\Omega\colon [0,1] \to \mathbb{R}_{\le 0}$ be a proper, closed, convex regularizer with $\Omega \le 0$. Define the *regularized gate* and the *conjugate activation*

$$\pi_\Omega(\Delta) \in \mathrm{argmax}_{\pi\in[0,1]}\big\{\pi\Delta - \Omega(\pi)\big\}, \qquad \Omega^*(\Delta) \coloneqq \max_{\pi\in[0,1]}\big\{\pi\Delta - \Omega(\pi)\big\}, \tag{3.43}$$

where an arbitrary maximizer is fixed if several exist. The *regularized decomposition function* is

$$f^{\Omega}_{\mathrm{reg}}(z^+, z^-) \coloneqq \Big(\pi_\Omega(\Delta)\, z^+ - \Omega\big(\pi_\Omega(\Delta)\big),\ \ \pi_\Omega(\Delta)\, z^-\Big), \qquad \Delta \coloneqq z^+ - z^-. \tag{3.44}$$

Compactness of $[0,1]$ and upper semicontinuity of the objective ensure attainment in (3.43); $\Omega^*$ is the Fenchel conjugate of $\Omega$ extended by $+\infty$ outside $[0,1]$. Since $\pi_\Omega(\Delta) \in [0,1]$, $z^\pm \ge 0$, and $-\Omega \ge 0$, both components of (3.44) are non-negative, making $f^{\Omega}_{\mathrm{reg}}$ a pairwise activation (Definition 3.1). All maximizers recover $\Omega^*(\Delta)$, but their components can differ: for $\Omega = 0$, $z^+ = z^- = 1$, the choices $\pi = 0{,}1$ produce $(0,0),(1,1)$. Hence Definition 3.5 fixes a selection. Here the untagged pair $\big(a_i^{(l,+)}, a_i^{(l,-)}\big)$ denotes regularized-family activations, replacing the DC ($\eta = 1$) meaning of Corollary 3.17.

**Theorem 3.21 (Validity of regularized decompositions, `Lean_43`).** *Let $\Omega$ be as in Definition 3.5, and let the activation pairs be propagated by the canonical non-negative linear map (3.4)–(3.5), initialised by (3.2), and updated by $f^{\Omega}_{\mathrm{reg}}$ at hidden layers, with the identity pair at the final affine readout. Then for every hidden layer $l$ and neuron $i$,*

$$a_i^{(l,+)} - a_i^{(l,-)} = \Omega^*\big(z_i^{(l)}\big), \tag{3.45}$$

*At the final affine layer, the difference is $z^{(L)} = a^{(L)}$. Thus $f^{\Omega}_{\mathrm{reg}}$ is a valid decomposition of the network (3.1) with activation function $\varrho = \Omega^*$. Moreover, $\Omega^*$ is convex, non-decreasing, and 1-Lipschitz, and wherever $\Omega^*$ is differentiable the gate is its derivative, $\pi_\Omega(\Delta) = (\Omega^*)'(\Delta) \in [0,1]$.*

*Proof. Recovery.* Induction on $l$. The base case is the input convention. Assume (3.45) at $l-1$ for the network with activation $\Omega^*$; Proposition 3.1—whose proof uses only the difference identity—gives $\Delta_i \coloneqq z_i^{(l,+)} - z_i^{(l,-)} = z_i^{(l)}$. Subtracting the components of (3.44),

$$a_i^{(l,+)} - a_i^{(l,-)} = \pi_\Omega(\Delta_i)\,\big(z_i^{(l,+)} - z_i^{(l,-)}\big) - \Omega\big(\pi_\Omega(\Delta_i)\big) = \pi_\Omega(\Delta_i)\,\Delta_i - \Omega\big(\pi_\Omega(\Delta_i)\big) = \Omega^*\big(z_i^{(l)}\big),$$

where the last step evaluates (3.43) at its maximizer (Fenchel–Young).

*Properties of $\Omega^*$.* As a supremum of affine functions $\Delta \mapsto \pi\Delta - \Omega(\pi)$ with slopes $\pi \in [0,1]$, $\Omega^*$ is convex with subgradients in $[0,1]$, hence non-decreasing and 1-Lipschitz. At a point of differentiability

the subdifferential is $\{(\Omega^*)'(\Delta)\}$, and by Fenchel–Young every maximizer $\pi_\Omega(\Delta)$ is a subgradient; thus $\pi_\Omega(\Delta) = (\Omega^*)'(\Delta)$. □

**Corollary 3.22 (Unregularized case, Lean_44).** *For $\Omega \equiv 0$ on [0,1], Definition 3.5 with the maximizer $\pi_\Omega(\Delta) = \mathbf{1}[\Delta > 0]$ recovers the stopping decomposition function, $f^0_{\text{reg}} = f_{\text{stop}}$, and Theorem 3.21 specialises to Theorem 3.18 with $\Omega^*(\Delta) = \max_{\pi\in[0,1]} \pi\Delta = ReLU(\Delta)$.*

*Proof.* For $\Delta \neq 0$, $\pi\Delta$ uniquely maximizes at $\mathbf{1}[\Delta > 0]$ on [0,1]; for $\Delta = 0$, every $\pi$ maximizes, including the selected $\mathbf{1}[0 > 0] = 0$. Thus (3.44) with $\Omega = 0$ equals (3.39), with value ReLU($\Delta$). □

### 3.5.1 The Softplus Decomposition as the entropic subcase

The canonical smooth regularizer is the (negative) binary Shannon entropy at temperature $\theta > 0$,

$$\Omega_\theta(\pi) \coloneqq -\theta H(\pi), \qquad H(\pi) \coloneqq -\pi \log \pi - (1-\pi)\log(1-\pi), \tag{3.46}$$

which is strictly convex on [0,1] and satisfies $\Omega_\theta \leq 0$.

**Corollary 3.23 (Softplus Decomposition, Lean_45).** *For $\Omega = \Omega_\theta$ as in (3.46), the regularized gate and conjugate activation are*

$$\pi_{\Omega_\theta}(\Delta) = \sigma'_\theta(\Delta) \coloneqq \frac{1}{1+e^{-\Delta/\theta}}, \qquad \Omega^*_\theta(\Delta) = \sigma_\theta(\Delta) \coloneqq \theta \log\big(1+e^{\Delta/\theta}\big), \tag{3.47}$$

*the sigmoid gate and the softplus function at temperature $\theta$. The resulting decomposition function is the* softplus decomposition function

$$f^\theta_{\text{soft}}(z^+, z^-) = \Big(\sigma'_\theta(\Delta)\, z^+ + \theta\, H\big(\sigma'_\theta(\Delta)\big),\ \ \sigma'_\theta(\Delta)\, z^-\Big), \qquad \Delta = z^+ - z^-, \tag{3.48}$$

*and by Theorem 3.21 it yields a valid decomposition—the* Softplus Decomposition*—of the softplus network with parameter $\beta = 1/\theta$. In the limit $\theta \to 0$, $f^\theta_{\text{soft}} \to f_{\text{stop}}$ pointwise on $\{\Delta \neq 0\}$.*

*Proof.* The objective $\pi\Delta + \theta H(\pi)$ is strictly concave on (0,1) with derivative $\Delta - \theta \log \frac{\pi}{1-\pi}$; the first-order condition gives the unique maximizer $\pi = \sigma'_\theta(\Delta)$. Substituting back, with $\log \sigma'_\theta(\Delta) = \Delta/\theta - \log(1+e^{\Delta/\theta})$ and $\log(1-\sigma'_\theta(\Delta)) = -\log(1+e^{\Delta/\theta})$, yields the optimal value

$$\sigma'_\theta(\Delta)\,\Delta + \theta H\big(\sigma'_\theta(\Delta)\big) = \theta \log\big(1+e^{\Delta/\theta}\big) = \sigma_\theta(\Delta), \tag{3.49}$$

the Legendre–Fenchel form of the binary log-sum-exp at temperature $\theta$ over $\{0, \Delta\}$. Then (3.48) is (3.44) with $-\Omega_\theta(\pi_{\Omega_\theta}(\Delta)) = \theta H(\sigma'_\theta(\Delta))$, and validity is Theorem 3.21 with $\Omega^*_\theta = \sigma_\theta$. For the limit, $\sigma'_\theta(\Delta) \to \mathbf{1}[\Delta > 0]$ and $\theta H(\sigma'_\theta(\Delta)) = \sigma_\theta(\Delta) - \sigma'_\theta(\Delta)\Delta \to 0$ as $\theta \to 0$ for $\Delta \neq 0$. □

The + stream carries the entire entropy bonus $\theta H(\sigma'_\theta(\Delta))$ in (3.48), leaving the − stream purely gated. In Chapter 4, this bonus is the active player's surplus from mixing.

*Remark 3.9 (Loss of stream monotonicity).* The soft gate does not inherit the well-behavedness of the hard-gated Stopping Decomposition. Consider the negative stream $a^{(l,-)} = \sigma'_\theta(\Delta)\, z^-$ with $\Delta = z^+ - z^-$: holding $z^+$ fixed and increasing $z^-$ both raises the factor $z^-$ and, through $\Delta$, *lowers* the gate $\sigma'_\theta(\Delta)$. Differentiating, $\partial a^{(l,-)}/\partial z^- = \sigma'_\theta(\Delta)\big(1 - z^-\, \sigma''_\theta(\Delta)/\sigma'_\theta(\Delta)\big)$ turns negative once $z^-$ exceeds $\theta/\big(1-\sigma'_\theta(\Delta)\big)$, so $a^{(l,-)}_{\text{soft}}$ is *not* monotone non-decreasing in $z^-$. The per-region monotonicity

of the Stopping Decomposition (Lemma 3.19) therefore does not lift to the soft-gated family; the reader should not expect the Softplus Decomposition to inherit the monotone or convex structure of the hard-gated and DC decompositions. Only the difference recovery of Theorem 3.21 survives.

*Remark 3.10 (Probit regularizer and GELU).* The framework is not tied to the entropy. For the probit regularizer $\Omega_\Phi(\pi) := -\phi\big(\Phi^{-1}(\pi)\big)$, with $\phi, \Phi$ the standard Gaussian density and distribution function, one has $\frac{\mathrm{d}}{\mathrm{d}\pi}\big(-\Omega_\Phi\big)(\pi) = -\Phi^{-1}(\pi)$, so the first-order condition for (3.43) gives the gate $\pi_{\Omega_\Phi}(\Delta) = \Phi(\Delta)$ and the conjugate activation

$$\Omega_\Phi^*(\Delta) = \Delta\,\Phi(\Delta) + \phi(\Delta) = \mathbb{E}_{\varepsilon\sim\mathcal{N}(0,1)}\big[\max(\Delta+\varepsilon, 0)\big],$$

the Gaussian smoothing of the ReLU. This differs from GELU $\Delta\Phi(\Delta)$ (Hendrycks and Gimpel, 2016) by the term $\phi(\Delta)$, and the gate $\Phi(\Delta) = \mathbb{E}_\varepsilon\big[\mathbf{1}[\Delta+\varepsilon > 0]\big]$ is the hard stopping gate averaged over a Gaussian shift of its threshold—matching the original stochastic-gate motivation for GELU. The distinction matters for exact decomposability: the conjugate activation $\Omega_\Phi^*(\Delta) = \Delta\,\Phi(\Delta) + \phi(\Delta)$ is the object recovered by $f_{\mathrm{reg}}^{\Omega}$ and, being a Fenchel conjugate, is convex, non-decreasing, and 1-Lipschitz (Theorem 3.21). GELU $\Delta \mapsto \Delta\,\Phi(\Delta)$ is non-monotone—it dips below zero for $\Delta < 0$—and is therefore *not* itself a conjugate activation $\Omega^*$; the exactly decomposable object is the $+\phi(\Delta)$ variant, and networks using GELU are only approximately captured by this decomposition.

*Remark 3.11 (Connection to the integrable-curvature splitting).* Regularized gates connect to the convex activations covered by Theorem 3.13. Suppose $\Omega$ is strictly convex and the gate $\pi_\Omega$ is locally absolutely continuous on $\mathbb{R}$. Then $\Omega^*$ is differentiable with $(\Omega^*)' = \pi_\Omega$, which is non-decreasing with limits 0 and 1 at $-\infty$ and $+\infty$. Hence the weak curvature $(\Omega^*)'' \geq 0$ belongs to $L^1(\mathbb{R})$ and integrates to 1. A sufficient condition is $\Omega \in C^2((0,1))$ with $\Omega'' > 0$ there: the gate is the inverse of $\Omega'$ on its range, extended by 0 and 1 beyond finite endpoint slopes. Strict convexity and smoothness alone do not guarantee local absolute continuity of the gate; the distributional curvature can have a singular part. Under the stated regularity, Theorem 3.13 applies with $N = 0$, $C_2 = 0$, and $C_1 = \Omega^*$, as recorded for the softplus in Corollary 3.9(6). Regularizing the *gate* (this section) and the curvature-based smooth split of Section 3.2.2 thus yield compatible valid decompositions of the same networks under these hypotheses, cf. Remark 3.6.

## 3.6 Numerical Stabilization of Split Passes

Without cancellation, both streams accumulate the *absolute* mass $z^{(l,+)} + z^{(l,-)}$ (Remark 3.2). In deep networks, their common magnitude grows roughly geometrically with products of entrywise absolute weight matrices, quickly exceeding the representable range while $a^{(l,+)} - a^{(l,-)} = a^{(l)}$ stays moderate. We develop the affine stabilization of Zimmermann & Loho (2026): shifting forward activations and backward local sensitivities. Absorbing biases into weights, write the absolute-weight product above layer $l$ as

$$W_{\mathrm{abs}}^{[l,\to]} := \prod_{j=L,\dots,l+1} |W^{(j)}| = \prod_{j=L,\dots,l+1} \big(W^{(j,+)} + W^{(j,-)}\big), \tag{3.50}$$

with the empty product at $l = L$ read as the identity.

### 3.6.1 Shifting the forward pass

A *common* shift at any layer translates both stream outputs equally, preserving their difference and hence the network value (Theorem 3.7).

**Theorem 3.24 (Shift equivariance of the split streams,** `Lean_46`**).** *For any* $0 \le l \le L$ *and any* $c, a^{(+)}, a^{(-)} \in \mathbb{R}^{d(l)}$ *with* $a^{(\pm)}, a^{(\pm)} + c \ge 0$,

$$\begin{aligned} g^{(l)}\big(a^{(+)} + c, a^{(-)} + c\big) &= g^{(l)}\big(a^{(+)}, a^{(-)}\big) + W_{\mathrm{abs}}^{[l,\to]}\, c, \\ h^{(l)}\big(a^{(+)} + c, a^{(-)} + c\big) &= h^{(l)}\big(a^{(+)}, a^{(-)}\big) + W_{\mathrm{abs}}^{[l,\to]}\, c. \end{aligned} \tag{3.51}$$

*In particular,* $\big(g^{(l)} - h^{(l)}\big)\big(a^{(+)} + c, a^{(-)} + c\big) = \big(g^{(l)} - h^{(l)}\big)\big(a^{(+)}, a^{(-)}\big)$.

*Proof.* Induct backward on $l$ for $g^{(l)}$; $h^{(l)}$ follows analogously, then subtract. At $l = L$, $W_{\mathrm{abs}}^{[L,\to]} = I$ and $g^{(L)}$ is a projection. For $0 \le l < L$, the stream connections swap $W^{(l+1,+)}$ and $W^{(l+1,-)}$ (Figure 3.2), propagating a common shift to the next pair. At a hidden layer $l + 1 < L$,

$$\begin{aligned} a^{(l+1,+)}\big|_{+c} &= \max\big\{W^{(l+1,+)}a^{(+)} + W^{(l+1,-)}a^{(-)},\, W^{(l+1,-)}a^{(+)} + W^{(l+1,+)}a^{(-)}\big\} + \big|W^{(l+1)}\big|\, c, \\ a^{(l+1,-)}\big|_{+c} &= W^{(l+1,-)}a^{(+)} + W^{(l+1,+)}a^{(-)} + \big|W^{(l+1)}\big|\, c, \end{aligned}$$

since $\big(W^{(l+1,+)} + W^{(l+1,-)}\big)c = \big|W^{(l+1)}\big|c$ factors out of the maximum. At the ungated affine readout $l + 1 = L$, non-negative linearity gives the common shift $|W^{(L)}|c$. Applying the hypothesis at $l + 1$ with shift $\big|W^{(l+1)}\big|c$,

$$\begin{aligned} g^{(l)}\big(a^{(+)} + c, a^{(-)} + c\big) &= g^{(l+1)}\Big(a^{(l+1,+)} + \big|W^{(l+1)}\big|c, a^{(l+1,-)} + \big|W^{(l+1)}\big|c\Big) \\ &= g^{(l)}\big(a^{(+)}, a^{(-)}\big) + W_{\mathrm{abs}}^{[l,\to]}\, c. \end{aligned}$$

□

Theorem 3.24 permits *re-centering* large pairs by a common shift while preserving the function. The canonical choice

$$c^{(l)} = -\min\big\{a^{(l,+)}, a^{(l,-)}\big\}$$

(entrywise) zeros at least one component of each pair. A fixed shift preserves head Jacobians because Maxout branch comparisons are unchanged. Input-dependent shifts must be detached for these sensitivities: differentiating through them adds $W_{\mathrm{abs}}^{[l,\to]} Dc$ to both head derivatives, preserving their difference but potentially changing each derivative. The paper reports double-precision intermediates; the released evaluation command requires an explicit precision setting. The forward implementation offers several magnitude-reduction modes. Common shifts preserve the difference (Theorem 3.24); common scaling generally changes it. With cached original activations, the implementation then applies the symmetric correction

$$e^{(l)} = a^{(l)} - \big(a^{(l,+)} - a^{(l,-)}\big), \qquad a^{(l,+)} \leftarrow a^{(l,+)} + \tfrac{1}{2}e^{(l)}, \qquad a^{(l,-)} \leftarrow a^{(l,-)} - \tfrac{1}{2}e^{(l)}.$$

This restores the **forward invariant** in exact arithmetic. The default backward pass detaches the correction; differentiating through it changes stream sensitivities. Scaling modes, correction gradients, and convexity and monotonicity of stabilized streams require justification beyond the common-shift theorem. Floating-point reconstruction error still requires measurement.

### 3.6.2 Shifting the backward pass

All classical derivative statements in this subsection are restricted to points with strict gate inequalities in the relevant head. At a gate tie, a fixed branch convention defines a backpropagated selection

rather than asserting classical differentiability. The gradients of $g$ and $h$ involve only non-negative matrices, so they blow up like the forward activations. Let

$$\delta^{(l,+,g)} := \frac{\partial g^{(l)}}{\partial a^{(l,+)}}, \qquad \delta^{(l,-,g)} := \frac{\partial g^{(l)}}{\partial a^{(l,-)}}, \qquad \delta^{(l,+,h)} := \frac{\partial h^{(l)}}{\partial a^{(l,+)}}, \qquad \delta^{(l,-,h)} := \frac{\partial h^{(l)}}{\partial a^{(l,-)}} \tag{3.52}$$

denote the *local sensitivities* of the stream heads with respect to the split activations of layer $l$. Theorem 3.24 pins down their pairwise structure.

**Theorem 3.25 (Sensitivity complementarity,** `Lean_47`**).** *For every* $0 \le l \le L$,

$$\delta^{(l,+,g)} = W_{\mathrm{abs}}^{[l,\to]} - \delta^{(l,-,g)}, \qquad \delta^{(l,+,h)} = W_{\mathrm{abs}}^{[l,\to]} - \delta^{(l,-,h)}. \tag{3.53}$$

*Proof.* For a coordinate direction $e_j$, $j \in \{1, \dots, d^{(l)}\}$, Theorem 3.24 with $c = \epsilon\, e_j$ gives $g^{(l)}\big(a^{(l,+)} + \epsilon e_j, a^{(l,-)}\big) = g^{(l)}\big(a^{(l,+)}, a^{(l,-)} - \epsilon e_j\big) + \epsilon\, W_{\mathrm{abs}}^{[l,\to]} e_j$. Subtracting $g^{(l)}\big(a^{(l,+)}, a^{(l,-)}\big)$, dividing by $\epsilon$, and letting $\epsilon \to 0$ yields $\delta_j^{(l,+,g)} = W_{\mathrm{abs}}^{[l,\to]} e_j - \delta_j^{(l,-,g)}$. The claim for $h$ follows analogously. □

The chain rule gives four block Jacobians at hidden layers with strict gate patterns:

$$\begin{aligned}
\frac{\partial a^{(l+1,+)}}{\partial a^{(l,+)}} &= \frac{\partial a^{(l+1,+)}}{\partial z^{(l+1,+)}} \frac{\partial z^{(l+1,+)}}{\partial a^{(l,+)}} + \frac{\partial a^{(l+1,+)}}{\partial z^{(l+1,-)}} \frac{\partial z^{(l+1,-)}}{\partial a^{(l,+)}} \\
&= M^{(l+1,+)} W^{(l+1,+)} + M^{(l+1,-)} W^{(l+1,-)}, \\
\frac{\partial a^{(l+1,-)}}{\partial a^{(l,+)}} &= 0 \cdot W^{(l+1,+)} + W^{(l+1,-)}, \\
\frac{\partial a^{(l+1,+)}}{\partial a^{(l,-)}} &= \frac{\partial a^{(l+1,+)}}{\partial z^{(l+1,+)}} \frac{\partial z^{(l+1,+)}}{\partial a^{(l,-)}} + \frac{\partial a^{(l+1,+)}}{\partial z^{(l+1,-)}} \frac{\partial z^{(l+1,-)}}{\partial a^{(l,-)}} \\
&= M^{(l+1,+)} W^{(l+1,-)} + M^{(l+1,-)} W^{(l+1,+)}, \\
\frac{\partial a^{(l+1,-)}}{\partial a^{(l,-)}} &= 0 \cdot W^{(l+1,-)} + W^{(l+1,+)}.
\end{aligned} \tag{3.54}$$

Here $M^{(l,+)}$ and $M^{(l,-)}$ select the two complementary Maxout branches, defined below. For the final ungated affine layer, the same formulas hold with $M^{(L,+)} = I$ and $M^{(L,-)} = 0$. Biases contribute constant shifts to the pre-activations, so their derivatives with respect to the preceding activations vanish; fixed biases also preserve convexity, monotonicity, and the piecewise-affine structure.

By (3.53), each sensitivity pair sums to the exploding matrix $W_{\mathrm{abs}}^{[l,\to]}$, making at least one component large. Stabilization subtracts an $\alpha^{(l)}$-multiple of this bulk at each layer. Starting from $\delta_{\mathrm{shift}}^{(L,\pm,g)} := \delta^{(L,\pm,g)} - \alpha^{(L)}\big(\delta^{(L,+,g)} + \delta^{(L,-,g)}\big)$ and analogously for $h$, the *stabilized backward pass* iterates for $l = L-1, \dots, 0$: first the ordinary gradient recursion on the already shifted quantities,

$$\delta_{\mathrm{int}}^{(l,+,g)} := \delta_{\mathrm{shift}}^{(l+1,+,g)}\big(M^{(l+1,+)} W^{(l+1,+)} + M^{(l+1,-)} W^{(l+1,-)}\big) + \delta_{\mathrm{shift}}^{(l+1,-,g)}\, W^{(l+1,-)}, \tag{3.55}$$

$$\delta_{\mathrm{int}}^{(l,-,g)} := \delta_{\mathrm{shift}}^{(l+1,+,g)}\big(M^{(l+1,+)} W^{(l+1,-)} + M^{(l+1,-)} W^{(l+1,+)}\big) + \delta_{\mathrm{shift}}^{(l+1,-,g)}\, W^{(l+1,+)}, \tag{3.56}$$

and then the uniform shift

$$\delta_{\mathrm{shift}}^{(l,\pm,g)} := \delta_{\mathrm{int}}^{(l,\pm,g)} - \alpha^{(l)}\big(\delta_{\mathrm{int}}^{(l,+,g)} + \delta_{\mathrm{int}}^{(l,-,g)}\big), \tag{3.57}$$

with the same scheme for $h$. Here $M^{(l,+)} := \operatorname{diag}\big(\mathbf{1}\big[z^{(l)} \geq 0\big]\big)$ and $M^{(l,-)} := \operatorname{diag}\big(\mathbf{1}\big[z^{(l)} < 0\big]\big)$ are the Maxout gate patterns, and (3.55)–(3.56) are the exact chain-rule recursions of the unshifted sensitivities, obtained by differentiating (3.4)–(3.5) and (3.19): e.g. $\partial a^{(l+1,+)}/\partial a^{(l,+)} = M^{(l+1,+)}W^{(l+1,+)} + M^{(l+1,-)}W^{(l+1,-)}$ and $\partial a^{(l+1,-)}/\partial a^{(l,+)} = W^{(l+1,-)}$. For $\alpha^{(l)} \equiv 0$ the scheme reproduces the plain split-pair sensitivities. In the implementation, the gate patterns $M^{(l,\pm)}$ are taken from the cached original activations rather than the drifted split pre-activations, and cached original gradients give small corrections enforcing the **backward invariant** $\frac{1}{2}\big[(\delta^{(l,+,g)} - \delta^{(l,-,g)}) - (\delta^{(l,+,h)} - \delta^{(l,-,h)})\big] = \partial f^{(l)}/\partial a^{(l)}$.

Express the split gradients with respect to a unified input. Define $\tilde{g}^{(l)}(a^{(l)}) := g^{(l)}\big(a^{(l)}/2, -a^{(l)}/2\big)$, the head on the half–half split, and analogously $\tilde{h}^{(l)}$; by the chain rule

$$\delta^{(l,g)} := \frac{\partial \tilde{g}^{(l)}}{\partial a^{(l)}} = \frac{\delta^{(l,+,g)} - \delta^{(l,-,g)}}{2}, \qquad \delta^{(l,h)} := \frac{\partial \tilde{h}^{(l)}}{\partial a^{(l)}} = \frac{\delta^{(l,+,h)} - \delta^{(l,-,h)}}{2}, \tag{3.58}$$

and write $\delta^{(l,g)}_{\text{shift}} := \big(\delta^{(l,+,g)}_{\text{shift}} - \delta^{(l,-,g)}_{\text{shift}}\big)/2$, $\delta^{(l,h)}_{\text{shift}} := \big(\delta^{(l,+,h)}_{\text{shift}} - \delta^{(l,-,h)}_{\text{shift}}\big)/2$ for their stabilized counterparts, with $\delta^{g}_{\text{shift}} := \delta^{(0,g)}_{\text{shift}}$, $\delta^{h}_{\text{shift}} := \delta^{(0,h)}_{\text{shift}}$. For the final affine readout we use the uniform-notation convention $M^{(L,+)} = I$, $M^{(L,-)} = 0$; these are algebraic placeholders, not an additional gate. Standard backpropagation gives $\delta^{(l)} := \partial f^{(l)}/\partial a^{(l)} = \prod_{j=L,\dots,l+1} W^{(j,x)}$ with gated matrices $W^{(l,x)} := M^{(l,+)}W^{(l)}$.

**Theorem 3.26 (Closed formulas for the stabilized split gradients, `Lean_48`).** *With $\overline{\alpha}_j := \prod_{j\leq i\leq L}\big(1 - 2\alpha^{(i)}\big)$, the stabilized backward pass (3.55)–(3.57) computes, for every $0 \leq l \leq L$,*

$$\delta^{(l,g)}_{\text{shift}} = \frac{1}{2}\Big[\delta^{(l)} - \sum_{l\leq j<L} \overline{\alpha}_{j+1}\, W^{[j+1,\to]}_{\text{abs}}\big(M^{(j+1,-)}W^{(j+1)}\big) \prod_{i=(j-1),\dots,l} W^{(i+1,x)}\Big], \tag{3.59}$$

$$\delta^{(l,h)}_{\text{shift}} = -\frac{1}{2}\Big[\delta^{(l)} + \sum_{l\leq j<L} \overline{\alpha}_{j+1}\, W^{[j+1,\to]}_{\text{abs}}\big(M^{(j+1,-)}W^{(j+1)}\big) \prod_{i=(j-1),\dots,l} W^{(i+1,x)}\Big]. \tag{3.60}$$

*The $j = L-1$ correction term is zero because $M^{(L,-)} = 0$: no minimum or auxiliary Q source is inserted at the affine readout. In particular, $\delta^{(l,g)}_{\text{shift}} - \delta^{(l,h)}_{\text{shift}} = \delta^{(l)}$ for all $l$; for $\alpha^{(i)} \equiv 0$ (i.e. $\overline{\alpha}_j = 1$) the formulas describe the plain split-pair gradients, and for $\alpha^{(i)} \equiv 1/2$ all correction terms vanish and the stabilized quantities are $\pm\delta^{(l)}/2$, halves of the original gradient.*

*Proof.* We record two auxiliary identities.

*Claim 1 (bulk of the shifted sensitivities).* For every $l$,

$$\delta^{(l,+,g)}_{\text{shift}} + \delta^{(l,-,g)}_{\text{shift}} = \prod_{l\leq j\leq L}\big(1 - 2\alpha^{(j)}\big)\, W^{[l,\to]}_{\text{abs}} = \overline{\alpha}_l\, W^{[l,\to]}_{\text{abs}}. \tag{3.61}$$

At $l = L$, $\delta^{(L,+,g)} + \delta^{(L,-,g)} = I = W^{[L,\to]}_{\text{abs}}$, and the shift multiplies the sum by $(1 - 2\alpha^{(L)})$, since subtracting $\alpha^{(L)}$ times the sum from *each* component subtracts $2\alpha^{(L)}$ times it from the sum. For the step, adding (3.55), (3.56) and using $M^{(l+1,+)} + M^{(l+1,-)} = I$, $W^{(l+1,+)} + W^{(l+1,-)} = \big|W^{(l+1)}\big|$,

$$\delta^{(l,+,g)}_{\text{int}} + \delta^{(l,-,g)}_{\text{int}} = \big(\delta^{(l+1,+,g)}_{\text{shift}} + \delta^{(l+1,-,g)}_{\text{shift}}\big)\big|W^{(l+1)}\big| = \overline{\alpha}_{l+1}\, W^{[l+1,\to]}_{\text{abs}}\big|W^{(l+1)}\big| = \overline{\alpha}_{l+1}\, W^{[l,\to]}_{\text{abs}}$$

by induction, and the shift (3.57) multiplies this by $(1 - 2\alpha^{(l)})$, giving $\overline{\alpha}_l\, W^{[l,\to]}_{\text{abs}}$.

*Claim 2 (recursion of the stabilized difference).* Subtracting (3.56) from (3.55)—the shift cancels in the difference—and regrouping with $M^{(l+1,+)} - M^{(l+1,-)}$ applied to $W^{(l+1)} = W^{(l+1,+)} - W^{(l+1,-)}$,

$$\begin{aligned} 2\,\delta_{\text{shift}}^{(l,g)} &= \big(\delta_{\text{shift}}^{(l+1,+,g)} - \delta_{\text{shift}}^{(l+1,-,g)}\big) M^{(l+1,+)} W^{(l+1)} - \big(\delta_{\text{shift}}^{(l+1,+,g)} + \delta_{\text{shift}}^{(l+1,-,g)}\big) M^{(l+1,-)} W^{(l+1)} \\ &= 2\,\delta_{\text{shift}}^{(l+1,g)}\, W^{(l+1,x)} - \overline{\alpha}_{l+1}\, W_{\text{abs}}^{[l+1,\to]} M^{(l+1,-)} W^{(l+1)}, \end{aligned} \tag{3.62}$$

The first equality groups terms using $M^{(l+1,+)} + M^{(l+1,-)} = I$; the second uses Claim 1 at $l+1$.

*Induction.* At the base $l = L$ of (3.59), the initial shift cancels in the difference, so $\delta_{\text{shift}}^{(L,g)} = \big(\delta^{(L,+,g)} - \delta^{(L,-,g)}\big)/2 = I/2 = \delta^{(L)}/2$ with empty correction sum. For the step, unfolding (3.62) with the hypothesis at $l+1$,

$$\begin{aligned} 2\,\delta_{\text{shift}}^{(l,g)} &= \Big[\delta^{(l+1)} - \sum_{l+1\le j<L} \overline{\alpha}_{j+1} W_{\text{abs}}^{[j+1,\to]} \big(M^{(j+1,-)} W^{(j+1)}\big) \prod_{i=(j-1),\dots,l+1} W^{(i+1,x)}\Big] W^{(l+1,x)} \\ &\quad - \overline{\alpha}_{l+1}\, W_{\text{abs}}^{[l+1,\to]} M^{(l+1,-)} W^{(l+1)} \\ &= \delta^{(l)} - \sum_{l\le j<L} \overline{\alpha}_{j+1}\, W_{\text{abs}}^{[j+1,\to]} \big(M^{(j+1,-)} W^{(j+1)}\big) \prod_{i=(j-1),\dots,l} W^{(i+1,x)}, \end{aligned}$$

using $\delta^{(l+1)} W^{(l+1,x)} = \delta^{(l)}$ and absorbing the extra term as summand $j = l$. Equation (3.60) is identical up to the sign flip $\delta_{\text{shift}}^{(L,h)} = -I/2$. Subtracting (3.60) from (3.59) cancels the correction sums, giving the backward invariant $\delta_{\text{shift}}^{(l,g)} - \delta_{\text{shift}}^{(l,h)} = \delta^{(l)}$; the cases $\alpha \equiv 0,\ 1/2$ are immediate from $\overline{\alpha}_j$. □

*Remark 3.12 (Interpretation of the correction terms).* The correction summands in (3.59) are gradients of an auxiliary network: define $Q[j](x) := W_{\text{abs}}^{[j,\to]} \min\{z^{(j)}(x), 0\}$, with the original prefix through layer $j$ followed by a linear absolute-weight tail without biases or activation gates. In a bias-free tail this agrees in value with subsequent minimum activations, but the explicit definition also handles arbitrary prefix biases. Then the $j$-th summand equals $\overline{\alpha}_{j+1}\, \partial Q[j+1]/\partial x$ (at $l = 0$), capturing the mass that switches streams at layer $j+1$. The factors $\overline{\alpha}_j$ counter their rapid growth toward the input and control each layer's stream-switching contribution. In practice $\alpha^{(l)} = \alpha$: $\alpha = 1/2$ gives exact halves of the original gradient, $\alpha = 0$ the raw split-pair gradients, and intermediate values control the additional split signal. Chapter 6 uses these stabilized gradients for split saliency.

## 3.7 Generalization to Convolutional Neural Networks (CNNs)

Nothing in Definitions 3.1 and 3.2 is specific to dense layers: a decomposition only requires that every component propagate non-negative pairs through non-negative linear maps and pairwise non-linearities. We extend the DC split to standard CNNs (He et al., 2016; Simonyan and Zisserman, 2015), representing every component as a *chunk module* $(u^+, u^-) \mapsto (v^+, v^-)$ and keeping linear layers and ReLU gates as separate chunks, so that architectures with several linear operations between activations remain expressible; a converter implemented in PyTorch (Paszke et al., 2019) maps supported computation graphs to the split form. For each chunk we verify the same three properties—non-negative pairs map to non-negative pairs, the difference invariant $v^+ - v^- = \text{op}(u^+ - u^-)$ holds, and (where applicable) monotonicity and convexity of the streams are preserved (Proposition 3.6)—and chaining them yields Theorem 3.30. Two chunks carry the substance, split convolution and gated max-pooling; every other supported chunk is a non-negative linear map applied streamwise, covered by a single lemma.

**Convolution.** A convolutional layer with kernel $K$ and bias $b$ computes $u \mapsto K * u + b$, with $*$ the (multi-channel) cross-correlation. Splitting the parameters entrywise, $K = [K]^+ - [K]^-$ and $b = [b]^+ - [b]^-$, we define the split chunk

$$v^+ := [K]^+ * u^+ + [K]^- * u^- + [b]^+, \qquad v^- := [K]^- * u^+ + [K]^+ * u^- + [b]^-. \tag{3.63}$$

**Proposition 3.27 (Correctness of split convolution, Lean_49).** *The chunk (3.63) maps non-negative pairs to non-negative pairs, each output component is a non-negative linear (affine) function of $(u^+, u^-)$—hence convex and non-decreasing in each coordinate—and*

$$v^+ - v^- = K * (u^+ - u^-) + b.$$

*Proof.* Non-negative kernels and biases make $v^\pm$ affine maps with non-negative coefficients in $(u^+,u^-)$, hence convex and coordinatewise non-decreasing, with $v^\pm \geq 0$ for $u^\pm \geq 0$. Linearity of $*$ gives

$$v^+ - v^- = \big([K]^+ - [K]^-\big) * u^+ - \big([K]^+ - [K]^-\big) * u^- + [b]^+ - [b]^- = K * (u^+ - u^-) + b. \qquad \square$$

Convolution followed by a ReLU chunk—the entrywise Maxout gate (3.19)—is covered by the argument of Theorem 3.7, a convolution being a dense layer with a sparse, weight-shared matrix. The same entrywise weight split handles every linear chunk at once.

**Lemma 3.28 (Non-negative linear chunks preserve the invariants, Lean_50).** *Let a chunk compute an affine map $u \mapsto Au + c$ with real matrix $A = [A]^+ - [A]^-$ and bias $c = [c]^+ - [c]^-$, split streamwise as*

$$v^+ := [A]^+u^+ + [A]^-u^- + [c]^+, \qquad v^- := [A]^-u^+ + [A]^+u^- + [c]^-.$$

*Then the chunk maps non-negative pairs to non-negative pairs, each component is affine with non-negative coefficients in $(u^+, u^-)$—hence convex and non-decreasing in each coordinate—and $v^+ - v^- = A(u^+ - u^-) + c$.*

*Proof.* Identical to Proposition 3.27: the coefficients $[A]^\pm, [c]^\pm$ are non-negative, and subtracting the two lines gives $([A]^+ - [A]^-)(u^+ - u^-) + [c]^+ - [c]^- = A(u^+ - u^-) + c$. $\square$

Split convolution (3.63) is the instance $A = K * (\cdot)$; the remaining linear chunks are listed in Table 3.2. Residual connections combine the identity chunk ($A = I$) with an already-split branch by streamwise addition $v^\pm = u^\pm + F^\pm(u^+, u^-)$, preserving all three invariants whenever the branch does.

**Tab. 3.2:** Linear CNN chunks as instances of Lemma 3.28.

| Chunk | Affine map | Notes |
|---|---|---|
| Average pooling | $A = \frac{1}{q}\sum_{j\in[q]}(\cdot)_j$, $c = 0$ | non-negative $A$, applied streamwise |
| Batch norm (eval) | $A = \mathrm{diag}(c_{\mathrm{bn}})$, $c = d$ | scale $c_{\mathrm{bn}}$ may be signed; $1 \times 1$ diagonal conv |
| Dropout (train) | $A = \mathrm{diag}(m)$, $c = 0$ | mask $m \in \{0, \frac{1}{1-p}\}^d$ from the original pass |
| Residual add | $A = I$ plus split branch | streamwise sum with branch $(F^+, F^-)$ |

Evaluation-mode batch normalization has a per-channel scale $c_{\mathrm{bn}} = \gamma/\sqrt{\sigma^2 + \epsilon}$ that may carry either sign (Guillemot et al., 2020; Hui and Binder, 2019); in practice we merge it into an adjacent convolution

before splitting. Training-mode batch normalization, whose statistics depend on the batch, is genuinely multiplicative and requires the techniques of Chapter 5. Training-mode dropout uses the *same* realised mask $m$ on both streams and on $u = u^+ - u^-$, so the split network drops exactly the units the original does.

**Gated max-pooling.** Max-pooling over a window $[q]$ computes $\max_{j\in[q]} u_j$ per channel, which is neither linear nor a shared gate on a single pair. It is a CPWL function of the window and admits its own convexity-preserving splitting identity.

**Proposition 3.29 (Correctness of gated max-pooling, `Lean_51`).** *For a pooling window $[q]$, define the split chunk*

$$v^+ \coloneqq \max_{j\in[q]} \Big\{ u_j^+ + \sum_{i\in[q],\, i\neq j} u_i^- \Big\}, \qquad v^- \coloneqq \sum_{i\in[q]} u_i^-. \tag{3.64}$$

*Then $v^\pm \geq 0$ for $u^\pm \geq 0$, both components are convex and non-decreasing in each coordinate of $(u^+, u^-)$, and*

$$v^+ - v^- = \max_{j\in[q]} \big(u_j^+ - u_j^-\big). \tag{3.65}$$

*Proof.* Each candidate and the sum have non-negative coefficients, proving non-negativity; the linear combinations and maximum preserve convexity and monotonicity. For (3.65), add and subtract $u_j^-$ for each $j \in [q]$:

$$u_j^+ + \sum_{i\neq j} u_i^- = \big(u_j^+ - u_j^-\big) + \sum_{i\in[q]} u_i^-,$$

The second summand is independent of $j$, so maximizing and subtracting $v^- = \sum_i u_i^-$ yields $\max_j \big(u_j^+ - u_j^-\big)$. □

Identity (3.65) generalizes (3.18), the case $q = 2$ with the pair $\big(u^+ - u^-, 0\big)$. Optionally, (3.64) can be replaced by a *winner-takes-it-all* rule forwarding, in both streams, the entries where the *difference* is maximal; this preserves the difference invariant but sacrifices monotonicity and convexity, mirroring the shared-gate trade-off of Remark 3.8.

**Implementation and caches.** The pinned implementation uses PyTorch FX to trace supported operations into decomposition chunks. One cache accumulates inflowing activations and gradients per chunk; another mirrors the original model for layer-wise error analysis, optional correction, and reuse of its positivity patterns during backpropagation instead of drifted split maxima. Arithmetic is configurable as `float16`, `float32` (default), or `float64`; the original paper reports double precision. Conversion covers operations supported by the chunk implementations.

**Alternative nonlinearities and exactness.** The source implementation discussion proposes approximating nonlinearities such as ELU (Clevert et al., 2016) or Swish (Ramachandran et al., 2018) by small ReLU subnetworks before splitting. Finite ReLU networks are piecewise affine and cannot exactly reproduce these non-piecewise-affine activations. The split preserves the approximating network; its error relative to the original activation remains separate. The pinned repository's scalar approximation utility includes Swish, does not list ELU, and is unused by the inspected hybrid conversion path. Section 3.2.2 provides exact curvature-based scalar decompositions under its stated hypotheses.

**The CNN split pass.** The split forward pass processes the chunks of a computation graph $\mathscr{G}$ in topological order, applying to each the split rule of its type—Lemma 3.28 for the linear chunks, the Maxout gate (3.19) or max-pool split (3.64) for the non-linear ones—with optional entrywise re-centering $v^{\pm} \leftarrow v^{\pm} - \min\{v^+, v^-\}$ after each chunk. The stabilization of Section 3.6 applies unchanged, since Theorem 3.24 uses only non-negative linearity and the Maxout gate structure.

**Theorem 3.30 (Correctness of the CNN split pass, `Lean_52`).** *Let $\mathscr{G}$ be a feedforward computation graph composed of the chunk types above, computing the function $x \mapsto f(x)$. Then for every input split $x = x^+ - x^-$ with $x^{\pm} \geq 0$, the split pass produces non-negative pairs satisfying the difference invariant*

$$v_m^+ - v_m^- = v_m \qquad \text{for every chunk } C_m,$$

*where $v_m$ is the corresponding original activation; in particular $f(x) = v_M^+ - v_M^-$. If neither winner-takes-it-all pooling nor input-dependent re-centering is used, every $v_m^{\pm}$ is convex in $(x^+, x^-)$ and non-decreasing in each coordinate.*

*Proof.* Induction over the topological order of $\mathscr{G}$. At the input node the invariant is the input convention, non-negativity holds by assumption, and $x_k^{\pm}$ are affine, convex, non-decreasing. For the step, assume all predecessors of $C_m$ satisfy the three properties. Lemma 3.28 (linear chunks), Proposition 3.29 (max-pool), and Corollary 3.17 with (3.18) (the ReLU gate) establish for every type that (i) non-negative pairs map to non-negative pairs, (ii) the output difference equals the original operation on the input difference—by the invariant the activation $v_m$—and (iii) each output component is convex and non-decreasing, being built from non-negative linear combinations and entrywise maxima whose compositions preserve both properties (Amos et al., 2017). Re-centering subtracts a common shift, preserving the invariant and non-negativity (Theorem 3.24); it is omitted when convexity and coordinatewise monotonicity are required, since these properties need not survive the subtraction. Already re-centering the pair $(\max\{p,n\},n)$ gives $(\mathrm{ReLU}(p-n),0)$, whose first component decreases when $n$ increases with $p$ fixed. The output claim is the invariant at $C_M$. □

The toolbox now includes validity, a one-parameter family with a convex DC endpoint, curvature-based smooth-activation splitting (Section 3.2.2), hard- and regularized soft-gated decompositions, stabilized forward/backward passes, and CNN chunks. Chapter 4 builds games on the Stopping Decomposition; Chapter 6 derives attribution methods from $g$ and $h$.

# 4 Game-Theoretic Framework

The sign-separated streams of Chapter 3 expose accumulated opposing contributions. This chapter formalizes the backward-game picture of Figure 1.1, tracing backward rules through these contributions. The stopping decomposition supplies a common state graph for two games whose policies represent gradients and relevance propagation. Their trajectory distributions support method design (Chapter 6) and diagnostics (Chapter 8). Exact recovery concerns the anchor rules; myopic temperature changes and risk-sensitive policies deform these rules. The companion supplement, Chapter S.1, gives detailed definitions and proofs.

## 4.1 Related Work

Ancona et al. (2018) unify several gradient-based rules as modified gradients, a map-level construction excluding the $\alpha\beta$ rule. Our signed trajectory-space representation unifies gradients and the $\alpha\beta$ rule through their routing laws. Together with the split representation, it distinguishes which internal signals attribution retains and how it propagates them backward.

**Game and trajectory views.** The closest mathematical precursor, the ReLU Net Game of Gaubert & Vlassopoulos (2025), identifies forward ReLU-network evaluation with a backward Stopping Game's value, without connecting it to the backward pass or attribution. We reformulate it through a novel Stopping Decomposition whose weight split relates to the DC decomposition used directly for attribution by Zimmermann & Loho (2026). Our SG and RG use this weight split to recover backward attribution rules rather than the pure DC decomposition. The NRM framework of Xiong et al. (2026) assigns normalized signed relevance to network walks. Section 8.1 compares its treatment of LRP and concept relevance propagation with our trajectory probabilities and discounted occupations.

**Relevance propagation and stochastic explanations.** LRP, Deep Taylor, and Excitation Backprop form the core relevance propagation family (Bach et al., 2015; Montavon et al., 2017; Zhang et al., 2018a); variants modify redistribution for contrast, stability, or signed relevance (Binder et al., 2016; Gu et al., 2019; Kohlbrenner et al., 2020; Nam et al., 2020; Vukadin et al., 2024). Our common trajectory-space semantics recovers gradients through SG and the generalized $\alpha\beta$ rule through RG, whose analytical modes give principled modifications of backward rules. Excitation Backprop (Zhang et al., 2018a) is an absorbing Markov chain whose redistribution on non-negative activations gives the unstabilized $z^+$-rule with biases omitted: the $(\alpha,\beta) = (1,0)$ case of $\alpha\beta$-LRP. Our game adds an opponent for negative-weight influence and replaces input-dependent transitions with soft Bellman equations. Deep Taylor (Montavon et al., 2017) justifies $z^+$ on non-negative inputs; RG constructs a game for the generalized $\alpha\beta$ rule.

**Shapley-value attribution methods.** KernelSHAP explains marginal input-feature contributions relative to a baseline through a cooperative *black-box* game over feature coalitions (Lundberg and Lee, 2017). Our complementary *white-box* game traces relevance through network internals.

**Robust explanations via Softplus.** Dombrowski et al. (2022) propose Softplus replacement as a curvature-based defense against gradient-explanation manipulation. Section S.1.5 shows that entropy-regularizing the activation-state stop/continue choice in SG recovers Softplus-network gradients, situating their construction within our framework.

**Interpretability in Transformers and ViTs.** Attention Rollout / Flow use attention as a routing graph (Abnar and Zuidema, 2020); transformer-LRP propagates through attention and skip connections (Achtibat et al., 2024; Ali et al., 2022; Chefer et al., 2021b); DAVE addresses ViT tokenization

artifacts through conditioned operators and equivariant averaging (Wróbel et al., 2026). Our game semantics represents conditioned operators through a QK-softmax oracle reference policy and Value-routing attribution trajectories. Its RG+Eq mode uses Reynolds averaging to suppress DAVE's targeted tokenization-grid artifacts, with further control through the game modes.

## 4.2 Backward Attribution

Throughout the game, attention, and trajectory derivations, $W_{ij}^{(l)}$ denotes the edge from predecessor $i$ to successor $j$: the adjacency matrices transpose the DC recurrences' output-by-input matrices while representing the same network. The tag stop distinguishes stopping from DC streams. Consider a ReLU network with layers $l = 1, \ldots, L$, input $a^{(0)} = x$, pre-activations $z_j^{(l)} = \sum_i a_i^{(l-1)} W_{ij}^{(l)} + b_j^{(l)}$, and activations $a_j^{(l)} = \mathrm{ReLU}(z_j^{(l)})$. We initially explain a scalar activation $f(x) = a_u^{(L)}$; Section 4.5 extends this to ungated, possibly negative affine logits. Write $[u]^+ = \max(u,0)$, $[u]^- = \max(-u,0)$, hence $u = [u]^+ - [u]^-$, and $W^{(l,\pm)} = [W^{(l)}]^\pm$ entrywise.

**Layer-wise relevance propagation.** Starting from $R_u^{(L)} = f(x)$, LRP (Bach et al., 2015) redistributes each neuron's relevance $R_j^{(l)}$ backward, layer by layer, in proportion to each input's pre-activation contribution $a_i^{(l-1)} W_{ij}^{(l)}$, with bias and stabilizer shares absorbed. Rules weight these contributions differently. The $\alpha\beta$-rule uses $\alpha,\beta \geq 0$; $\alpha - \beta = 1$ gives conservation when both participating sign streams are nonempty and bias, stabilizer, and empty-stream absorption are accounted for. With $\varepsilon \geq 0$ added to each sign denominator, as in common implementations (Anders et al., 2026), positive and negative fractions are normalized separately and scaled by $\alpha$ and $\beta$:

$$
\begin{aligned}
R_i^{(l-1)} &= \sum_j \left( \alpha \, \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\Sigma_j^{(l,+)} + \varepsilon} - \beta \, \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\Sigma_j^{(l,-)} + \varepsilon} \right) R_j^{(l)}, \\
\Sigma_j^{(l,\pm)} &= \sum_k [a_k^{(l-1)} W_{kj}^{(l)}]^\pm + [b_j^{(l)}]^\pm .
\end{aligned}
\tag{4.1}
$$

Relevance assigned to the bias and to $\varepsilon$ is absorbed. For non-negative activations, $(\alpha,\beta) = (1,0)$ with $\varepsilon = 0$ and biases omitted from normalization gives the $z^+$-rule of Deep Taylor decomposition (Montavon et al., 2017) and Excitation Backprop (Zhang et al., 2018a); other rules (LRP-$\varepsilon$, LRP-$\gamma$) change the redistribution fractions.

**Transformers.** In a Vision Transformer (ViT), LayerNorm couples the features of a token through its input-dependent mean and variance, and the attention matrix $A = \mathrm{softmax}(QK^\top/\sqrt{d})$ depends on the input, which breaks conservation (Ali et al., 2022). Transformer-specific rules therefore *condition* on the forward pass: conservative propagation (Ali et al., 2022) and DAVE (Wróbel et al., 2026) hold $A$ and the LayerNorm denominator fixed, retain linear mean-centering, and propagate attention relevance through the Value path only; AttnLRP (Achtibat et al., 2024) also propagates through the softmax and the QK product.

These rules route mass backward from each neuron to its inputs; the games below make that routing explicit.

## 4.3 Attribution as a Game

**Setup and common state-space structure.** Using Section 4.2's notation, we construct a finite-horizon two-player game (Osborne and Rubinstein, 1994) played *backward* from the output. States record layer, neuron, and game-specific player-turn and stream-sign labels. At each non-terminal state, the active player stops at an absorbing cemetery state † or continues. Skip connections randomly select summand states; max-pooling greedily selects maximal-payoff states. Section S.1.1 gives the labels (Table S.1.1). For player $p$ and opponent $p'$, the SG value $V_p(s)$ is the $\gamma$-discounted expected payoff difference $r_p - r_{p'}$ along trajectories from $s$ under the policies. For $p$ in turn at $s$, the Bellman recursion is

$$V_p(s) = \max_{a \in \mathscr{A}_x(s)} \Big\{ r_p(s,a) - r_{p'}(s,a) + \gamma(s) \sum_{s'} \mathbb{P}_x(s' \mid s,a)\, V_p(s') \Big\}, \tag{4.2}$$

and the **R**outing **G**ame instead entropy-regularizes mixed routing decisions, yielding the soft game value (4.5). Both games start with unit mass at the positive output state. Player policies induce a backward trajectory measure $\Pi_x$ on $\zeta = (\zeta_0, \zeta_1, \dots)$. With game-specific cumulative discount $d_t(\zeta)$ up to time $t$, define the **discounted occupation measure**

$$\Gamma_x(v) := \mathbb{E}_{\zeta \sim \Pi_x}\big[\textstyle\sum_t d_t(\zeta)\, \mathbf{1}\{\zeta_t = v\}\big]\,. \tag{4.3}$$

It measures the discounted weight visiting each state.

### 4.3.1 The Stopping Game

Each layer-$l$ neuron $j$ has states $s^{(l)}_{j,+}$ and $s^{(l)}_{j,-}$; the sign identifies the player in turn. Continuation pays the bias and follows weight-magnitude environment transitions with discount $\gamma^{(l)}_j = \sum_i |W^{(l)}_{ij}|$ (positive- and negative-weight predecessors follow $\mathbb{P} \propto W^{(l,\pm)}/\gamma^{(l)}_j$, and a negative weight passes the turn to the opponent). If $\gamma^{(l)}_j = 0$, continuation pays the bias and enters the cemetery. We select $\mathrm{ReLU}'(0) = 0$ and the smallest-index maximizer at pooling ties. The expected payoffs realize the novel Stopping Decomposition into positive and negative paths to each activation (Lemma S.1.2); the gradient is the difference of two *non-negative* occupation measures:

**Theorem 4.1 (Gradient representation,** `Lean_53`**).** *The Stopping Game allows a subgame-perfect Nash equilibrium, unique up to tie-breaking at zero-activation neurons. On strict gate cells (and with unique max-pooling winners where applicable), under the stop-at-tie-breaks equilibrium the discounted occupation measure* $\Gamma_x(s^{(l)}_{i,p}) \geq 0$ *equals the output derivative with respect to the corresponding* Stopping Decomposition *activation. At ties, the corresponding identities refer to the backward quantities selected by the stated gate and winner conventions. On these cells, the gradient of* $f$ *with respect to the input is the difference* $\Gamma_x(s^{(0)}_{i,+}) - \Gamma_x(s^{(0)}_{i,-})$.

Section S.1.2 supplies the proof, an example, the full skip/pooling state space, and a comparison with the DC decomposition.

Modeling $\alpha\beta$-LRP or attention modules via the SG would require activation-dependent transitions. The RG moves activation-dependence into the players' policies, opening a continuous family of deformations.

### 4.3.2 The Routing Game

Each RG neuron has an activation state $s^{(l,\mathrm{act})}_{j,q}$ and linear routing states $s^{(l,\mathrm{lin})}_{j,q,+}$ and $s^{(l,\mathrm{lin})}_{j,q,-}$ for the weight parts $W^{(l,+)}$ and $W^{(l,-)}$. At the activation state, the player stops or continues; on continuation, each stream is chosen with probability $\frac{1}{2}$; the $-$ stream switches players, and game-value discount 2 compensates the split. At linear state $s^{(l,\mathrm{lin})}_{j,q,\kappa}$ for layer-$l$ neuron $j$ and sign stream $\kappa \in \{+,-\}$, the player $q$ in turn mixes over predecessors $i$, a bias node, and an outside option $\perp_{\mathrm{lin}}$ under policy $\pi_q(s^{(l,\mathrm{lin})}_{j,q,\kappa})$. The latter two absorb the trajectory with log-payoffs $\log[b^{(l)}_j]^\kappa$ and $\log \varepsilon$. In log space, player $r$'s advantage of moving to predecessor $i$ becomes the additive action–value

$$Q_r\big(s^{(l,\mathrm{lin})}_{j,q,\kappa}, i\big) = \begin{cases} \log W^{(l,\kappa)}_{ij} + V_r(s^{(l-1,\mathrm{act})}_{i,q}), & r{=}q, l{\geq}2, \\ \log\big[x_i W^{(1)}_{ij}\big]^\kappa, & r{=}q, l{=}1, \\ -\infty, & r \neq q, \end{cases} \tag{4.4}$$

where $V_r$ is the successor activation state's *game value* for player $r$. Shannon-entropy regularization of each linear routing subgame (Geist et al., 2019) yields the equilibrium value and Gibbs best response:

$$V_q\big(s^{(l,\mathrm{lin})}_{j,q,\kappa}\big) = \tau \log \textstyle\sum_i \exp\big(Q_q(s^{(l,\mathrm{lin})}_{j,q,\kappa}, i)/\tau\big), \tag{4.5}$$

$$\pi_q(s^{(l,\mathrm{lin})}_{j,q,\kappa})(i) \propto \exp\big(Q_q(s^{(l,\mathrm{lin})}_{j,q,\kappa}, i)/\tau\big), \tag{4.6}$$

where the sums include the bias node and $\perp_{\mathrm{lin}}$. For $\tau > 0$, an SPNE exists; at $\tau = 1$, its occupation measure represents $\alpha\beta$-LRP:

The Routing Game cumulative trajectory discount multiplies by $2\alpha$ on entering a positive linear-routing state, $2\beta$ on entering a negative one, and 1 otherwise, separately from game-value discounting.

**Theorem 4.2 ($\alpha\beta$-LRP representation, `Lean_54`).** *For $\tau > 0$, the Gibbs policies of Eq. (4.6) and stopping decisions from the corresponding continuation values yield a canonical SPNE of the entropy-regularized RG. We select stopping at every activation log tie and absorption for an empty routing stream, as specified in Remark S.1.2. At the anchor $\tau = 1$, the selected gate is $\mathbf{1}[z > 0]$ and the $\alpha\beta$-LRP attribution map in (4.1) equals $R^{(l)}_i = f(x)\,\big(\Gamma_x(s^{(l)}_{i,+}) - \Gamma_x(s^{(l)}_{i,-})\big)$.*

Section S.1.3 gives the full game definition, the proofs, and a worked example of the local routing dynamics (Figure S.1.2).

In practice, $\tau$ is a *myopic* control: each linear subgame uses its $\tau$-scaled Gibbs policy on its own action values, with downstream game values fixed at $\tau = 1$, preserving the network's actual forward activations. Re-solving downstream at $\tau$ would change them. Thus $\tau \neq 1$ deforms the occupation measure; low $\tau$ concentrates routing on dominant pathways.

## 4.4 Game-Theoretic Modes

Risk aversion and entropy regularization yield novel backward-rule modes for probing and tuning explanations (Table 4.1). Section 4.3's exact representations are the anchors; modes beyond the LRP parameters ($\alpha$,$\beta$,$\varepsilon$) are deformations, inactive at the anchor. We use five: temperature $\tau$ (focus), activation-state risk aversion $\lambda$ and its attention-logit variant $\lambda_{\mathrm{sm}}$, the probit gate $\tilde{\Phi}$ (smoothing), and equivariant averaging over $N$ input transformations (grid artifacts).

**Tab. 4.1:** Modes of the games. "Exact": the mode stays within a represented rule; "deform.": optional deformation, off (default value) at the anchor. $\sigma^2$: input variance of the ADF moment propagation; $N$: number of transformed inputs averaged by RG+Eq.

| Mode | Effect on the game | Default | Property |
|---|---|---|---|
| $(\alpha,\beta)$ | branch discounts of the two streams (exact) | (2,1) | support / oppose |
| $\varepsilon$ | outside option $\perp_{\text{lin}}$, payoff $\log\varepsilon$ (exact) | 0.5 | damps low-inflow neurons |
| $\tau$ | entropy regularization of routing (deform.) | 1 | sharp / diffuse |
| $\lambda$, $\sigma^2$ | values $\to$ lower-confidence bounds (deform.) | $\lambda$=0 | noise / fragility |
| $\lambda_{\text{sm}}$ | attention logits shifted by $\lambda_{\text{sm}}$ times a per-key ADF std (deform.) | 0 | stable attention |
| $\tilde{\Phi}$ | noise on player observations (deform.) | off | gate smoothing |
| $N$ | average over input transformations (deform.) | off | grid artifacts |

*Focus.* The myopic temperature of Section 4.3.2 rescales the Gibbs policy at every linear state while keeping the $\tau$=1 game values. Low $\tau$ concentrates routing on dominant predecessors; high $\tau$ spreads mass more uniformly, exposing broader contextual features as well as architectural priors. *Risk aversion. Assumed Density Filtering* (ADF; (Gast and Roth, 2018)) propagates a Gaussian mean $\mu$ and variance $\sigma^2$ for every pre-activation $z$ from an input variance $\sigma^2$ in one additional moment pass. At activation states, the risk-averse player replaces $z$ by the lower-confidence bound $\mu+\lambda\sigma$, $\lambda<0$, attenuating high-variance routes while leaving linear-state routing unchanged. This is the pessimistic counterpart of confidence-bound exploration (Auer et al., 2002; Fei et al., 2020). On attention blocks, $\lambda_{\text{sm}}$ applies the same shift to the oracle's attention logits before the softmax, downweighting high-variance key tokens. *Gate smoothing.* The risk-averse stop/continue rule is the hard gate $\mathbf{1}[\mu/\sigma+\lambda>0]$. The probit gate $\tilde{\Phi}$ lets the player instead continue with probability $\Phi(\mu/\sigma+\lambda)$, the marginal of a player who perceives the continuation payoff as $z+\eta$, $\eta\sim\mathcal{N}(0,1)$, and continues iff it is positive. At $\mu$=$z$, $\sigma$=1, $\lambda$=0 this probit-regularized choice recovers the Gaussian-smoothed ReLU $\mathbb{E}[\mathrm{ReLU}(z+\eta)]$, just as entropy regularization recovers Softplus (Section S.1.5); on ReLU networks it is a smoothing deformation. Because the gate uses the ADF $\sigma$, $\sigma^2$ is active whenever $\tilde{\Phi}$ is on. *Exact modes and cost.* $(\alpha,\beta)$ weight the two streams of the occupation measure, and the LRP stabilizer $\varepsilon$ adds an outside option $\perp_{\text{lin}}$ with log-score $\log\varepsilon$ at every linear state, leaking weak-evidence neurons into the cemetery while leaving strong-evidence ones near-unaffected (Theorem S.1.8). The exact modes and $\tau$ add no cost; $\lambda$, $\lambda_{\text{sm}}$ and $\tilde{\Phi}$ add one ADF pass (0.10 $\to$ 0.17 s per ViT-B/16 image); averaging multiplies the cost by $N$ (4.93 s at $N$=50, 8.66 s with $\lambda_{\text{sm}}$). Section S.1.4 derives the modes and gives defaults, costs and an order of use.

## 4.5 Signed affine output logits

The activation-output formulation above extends to the signed logits of a classifier by an *ungated affine readout.* Let $h_j=a_j^{(L)}\geq 0$ be the last hidden activations and

$$f_u(x)=\sum_j c_{uj}h_j(x)+b_u. \tag{4.7}$$

This output has no ReLU or sign-dependent stopping gate.

**Stopping Game.** Add a root with forced continuation, immediate bias payoff difference $b_u$, and structural discount $\gamma_u=\sum_j|c_{uj}|$. When $\gamma_u>0$, it moves to hidden neuron $j$ with probability

$|c_{uj}|/\gamma_u$, retaining the player for $c_{uj} \geq 0$ and switching players for $c_{uj} < 0$. Its value is therefore $b_u + \sum_j c_{uj} h_j = f_u(x)$, even when this value is negative. If $\gamma_u = 0$, the root pays $b_u$ and moves to the cemetery; the input derivative is zero. On the strict gate cells of Theorem 4.1, let $\Gamma^{\mathrm{SG}}_{j,\pm}(i)$ denote the input-state occupations from a unit source at hidden neuron $j$ with initial player $+$. The added transition gives

$$\partial_i f_u(x) = \sum_j c_{uj} \big(\Gamma^{\mathrm{SG}}_{j,+}(i) - \Gamma^{\mathrm{SG}}_{j,-}(i)\big). \tag{4.8}$$

This chain rule retains non-negative path occupations and the selected-backward convention at gate or pooling ties.

**Routing Game.** Append an unconditionally continuing, ungated source to the hidden Routing Game, with positive and negative branch masses including the bias: $\Sigma_u^\kappa = \sum_j [h_j c_{uj}]^\kappa + [b_u]^\kappa$. At $\tau = 1$, the sign-oracle probabilities $1/2$, trajectory discounts $2\alpha$ and $2\beta$, and the branchwise Gibbs policies give the coefficients

$$q_j^+ = \alpha \frac{[h_j c_{uj}]^+}{\Sigma_u^+ + \varepsilon}, \qquad q_j^- = \beta \frac{[h_j c_{uj}]^-}{\Sigma_u^- + \varepsilon}. \tag{4.9}$$

Bias and outside-option shares are absorbed. An empty branch is completed by a transition to the cemetery, and its ordinary-successor fractions are zero ($0/0 \coloneqq 0$). For unit-source hidden occupations $\Gamma^{\mathrm{RG}}_{j,\pm}(i)$, define

$$\widehat{\Gamma}_\pm(i) = \sum_j \big(q_j^+ \Gamma^{\mathrm{RG}}_{j,\pm}(i) + q_j^- \Gamma^{\mathrm{RG}}_{j,\mp}(i)\big). \tag{4.10}$$

Since $h_j = 0$ implies $q_j^+ = q_j^- = 0$, only positive hidden activations receive nonzero relevance. Subtracting the equations gives affine $\alpha\beta$ redistribution followed by Theorem S.1.8's hidden-layer recursion. For any real initial relevance $r_u$,

$$R_i = r_u \big(\widehat{\Gamma}_+(i) - \widehat{\Gamma}_-(i)\big). \tag{4.11}$$

The trajectory law and both occupations remain non-negative; the signed seed scales their *difference*. Logarithms apply only to non-negative branch masses, never the signed logit. Classical LRP seeds $r_u = f_u(x)$; the released Routing-Game routine uses $r_u = 1$, so the former map is $f_u(x)$ times the latter, reversing sign for negative logits. At zero logits, output-seeded relevance vanishes while unit-seeded relevance can remain nonzero.

# 5 Extensions: Attention and Transformers

This chapter extends the decomposition and routing-game constructions to multiplicative layers and Transformers. It distinguishes the sign-sorted monotone split inherited from Zimmermann & Loho (2026), an exact convex split of the full deterministic Transformer developed here, and the conditioned routing-game rules used in the experiments. Each construction has its own domain and preserves a different set of structural properties.

Two features separate a Transformer (Vaswani et al., 2017) from the pointwise-activation networks treated earlier. First, the attention products $QK^\top$ and $AV$ multiply input-dependent quantities. Second, softmax and LayerNorm couple several coordinates. Section 5.2 develops the sign-sorted product rule, which preserves monotonicity on non-negative split inputs. Section 5.3 instead combines polarization, the multivariate softmax pair of Theorem 3.12, and a reciprocal-square-root pair to preserve convexity as well. Section 5.5 then studies attention as a conditioned Value-routing game. Its implementation on ViT-B/16, described in Section 5.5.3, supplies the attribution results evaluated in Chapter 7; the new full Transformer DC construction is an analytical result.

## 5.1 Related Work: Interpretability in Transformers

Early analyses of Transformer head roles combined attention patterns with LRP-based importance estimates and pruning experiments (Voita et al., 2019). Abnar & Zuidema (2020) objected that single-layer weights ignore propagation through depth, and proposed *attention rollout* — multiplying the residual-corrected attention matrices across layers — and *attention flow*, a max-flow computation on the layered attention graph. Both use a routing graph: rollout treats attention matrices as transition kernels of a walk on tokens, whereas attention flow treats their entries as edge capacities. What they omit is everything outside the attention weights — value magnitudes, the MLPs, the sign structure of the projections — so rollout can assign high relevance to tokens whose values contribute nothing.

A second line adapts LRP (Bach et al., 2015) to the Transformer. Chefer et al. (2021b) combined LRP-style relevance with attention gradients to propagate class-specific relevance through self-attention, later generalized to co-attention and encoder–decoder architectures (Chefer et al., 2021a). Ali et al. (2022) showed that naive gradient propagation through attention and layer normalization can *violate conservation*; holding the attention matrix and LayerNorm denominator constant ("detached"), while retaining linear centering, gives conservative local rules. AttnLRP (Achtibat et al., 2024) introduced dedicated rules for the bilinear products and non-linearities, scaling to LLMs. Related LRP variants provide concept-level (Achtibat et al., 2023) and magnitude-aware (Vukadin et al., 2024) explanations. On the vision side, DAVE (Wróbel et al., 2026) targeted patch-grid attribution artifacts in ViTs (Dosovitskiy et al., 2021) with *conditioned* operators and spatial averaging. Separately, Darcet et al. (2024) mitigated high-norm artifact tokens with registers. The ResLRP preprint targets attribution instability caused by cancellation between opposing residual branches. Its residual $\gamma$-rule conserves relevance and bounds absolute relevance amplification by a factor of $1 + 2/\gamma$ at each backward merge, assuming a nonzero residual output and $\gamma > 0$ (Berend et al., 2026). At LLM scale, gradient approximations of activation patching (Kramár et al., 2024; Syed et al., 2024) localize circuits against interventions (Hanna et al., 2024), and attribution increasingly acts as a control signal for head identification and guided decoding (Kahardipraja et al., 2025; Komorowski et al., 2026).

These approaches motivate two separate questions. The structural question asks whether the full forward map, including the attention-generating operations, admits explicit convex streams; Sections 5.2 and 5.3 develop the relevant product and normalization rules. The attribution question asks how

relevance travels through the Value routes when the realized attention matrix is fixed. Section 5.5 formulates that conditioned question as a KL-regularized routing game and recovers the conservative attention-block rule of Ali et al. (2022). This gives a precise interpretation of a routing calculation of the kind motivated by Abnar & Zuidema (2020), with the conditioning assumptions stated explicitly.

## 5.2 Decomposing Multiplicative Operations

Gating layers, squeeze-and-excitation blocks, and attention multiply activations entrywise or contract tensors. The sign-sorted construction of Zimmermann & Loho (2026) preserves the difference and monotonicity of non-negative stream pairs. Section 5.3 gives a convexity-preserving product identity combining with Chapter 3's multivariate softmax pair.

**The gated product rule.** For $a = a^+ - a^-$ and $b = b^+ - b^-$ with non-negative streams, as guaranteed by Section 3.1, expand and sort the cross terms by sign:

$$(a^+ - a^-)(b^+ - b^-) \;=\; \underbrace{\left(a^+b^+ + a^-b^-\right)}_{=:\,(ab)^+} \;-\; \underbrace{\left(a^+b^- + a^-b^+\right)}_{=:\,(ab)^-}. \tag{5.1}$$

Both terms are non-negative and non-decreasing in each incoming stream, so multiplication preserves monotone pairs, analogously to routing $W^{(l,+)}$ and $W^{(l,-)}$ separately in Section 3.2.3. Applying (5.1) entrywise splits Hadamard layers; applying it to scalar contractions splits matrix products. Convexity need not survive products. The resulting streams are finite maxima of *posynomials*, polynomials with non-negative coefficients. Posynomials are closed under addition, multiplication and positive scaling, and are monotone on the non-negative orthant.

**Maxout multiplication networks.** We enlarge the architecture class for the splitting theorem. A rank-$k$ Maxout unit (Goodfellow et al., 2013) takes the maximum of $k$ affine forms; ReLU is the $k = 2$ case with one form zero.

**Definition 5.1 (Maxout multiplication network).** A *rank-$k$ Maxout multiplication network* is a finite directed acyclic graph of *input neurons*, *addition neurons*, and *multiplication neurons*. An addition neuron $v$ carries weight vectors $w^{uv} \in \mathbb{R}^k$ on its incoming arcs and computes the rank-$k$ Maxout activation $a^v = \max_{i\in[k]} \sum_u w_i^{uv} a^u$; a multiplication neuron carries natural exponents $w^{uv} \in \mathbb{N}$ and computes the monomial $a^v = \prod_u (a^u)^{w^{uv}}$. The network is called *monotone* if all weights are non-negative, and *input-convex* if all weights are non-negative except possibly those on arcs leaving input neurons.

This class includes linear and convolutional layers, biases via constant inputs, maxima and max-pooling, residual connections, Hadamard products, and matrix products. Natural exponents avoid division and fractional powers of possibly negative quantities. Definition 5.1's structural conditions give the following representation.

**Proposition 5.1 (Representation by maxima of posynomials, `Lean_72`).** *A monotone rank-$k$ Maxout multiplication network with non-negative intermediate quantities represents the maximum of finitely many posynomials in its non-negative input variables, and is monotone on the non-negative orthant. This representation does not imply ordinary convexity: the positive-coefficient monomial $xy$ is a counterexample. Allowing signed input-incident affine maps likewise supplies no general input-convexity guarantee in the presence of multiplication.*

*Proof sketch.* Induct in topological order on the non-negative orthant. Coordinate projections are posynomials; non-negative scaling, addition and maxima preserve finite maxima of posynomials. Products of non-negative factors distribute over finite maxima, covering every network operation. Convexity fails for $F(x,y) = xy$: $F(1,1) = 1$ exceeds $\frac{1}{2}F(2,0) + \frac{1}{2}F(0,2) = 0$. □

**The splitting theorem.** Two identities extend Section 3.2.3's split to this class. Maxout uses the maximum-shift identity

$$\max_{j\in[k]}\left(a_j^+ - a_j^-\right) \;=\; \max_{j\in[k]}\Big\{a_j^+ + \sum_{i\neq j} a_i^-\Big\} \;-\; \sum_{i\in[k]} a_i^-, \tag{5.2}$$

which generalizes both the ReLU splitting identity $\mathrm{ReLU}(a-b) = \max\{a,b\} - b$ of Section 3.2.3 and the gated max-pooling rule of Section 3.7; multiplication neurons are split by the multinomial generalization of (5.1), expanding $\prod_u (a^{u,+} - a^{u,-})^{w^{uv}}$ and sorting the resulting monomials by the parity of the number of $(-)$-stream factors.

**Theorem 5.2 (Monotone split of multiplicative ReLU networks, Lean_73).** *Let $N$ be a (residual) ReLU network containing linear, convolutional, maximum, matrix-product, and Hadamard-product components. Then, on non-negative split-input variables, $N$ has two* monotone *rank-2 Maxout multiplication networks $N^+$ and $N^-$ satisfying*

$$N(x) = N^+(x^+,x^-) - N^-(x^+,x^-), \qquad x = x^+ - x^-, \quad x^\pm \geq 0,$$

*Each component is monotone as a function of its non-negative split inputs and is representable there by a finite maximum of posynomials. This gives a difference-of-monotone representation; it does not establish a difference-of-convex representation in the original inputs. If the number of expanded monomials per multiplication neuron is uniformly bounded, the sizes of the split networks are linear in the size of $N$.*

*Proof sketch.* In topological order, replace each neuron by a function-preserving pair $(v^+, v^-)$, extending Hertrich & Loho (2026)'s extended-formulation argument. Supply *inputs* as non-negative $x^+,x^-$ (e.g. $\mathrm{ReLU}(x),\mathrm{ReLU}(-x)$), flipping output-weight signs toward the $(-)$-copy; unsplit inputs do not generally restore convexity with multiplication. *Addition (Maxout) neurons* split weights as $[w^{uv}]^+ - [w^{uv}]^-$ and apply (5.2): the maximum defines $v^+$ and the subtracted sum $v^-$, both with non-negative weights. For *multiplication neurons*, expand by signs and feed one auxiliary multiplication neuron per summand into collecting addition neurons $v^+$ (even parity) and $v^-$ (odd parity). These identities give $N = N^+ - N^-$; non-negative weights imply monotonicity and Proposition 5.1's representation. Growth is controlled by the expanded monomial count: a uniform bound suffices, whereas bounded fan-in with unbounded exponents does not. □

Theorem 5.2 preserves the computation through non-negative monotone streams. Section 5.3 replaces the sign-sorted product pair to preserve convexity too.

*Remark 5.1 (Smooth normalizers: softmax and sigmoid gates).* Theorem 5.2 covers projections, bilinear products and residual additions, but excludes softmax. Section 3.2.4 gives exact scalar sigmoid and GELU pairs; Corollary 3.5 covers exponentiation exactly on a fixed bounded score interval. Splitting softmax's coupled denominator requires Theorem 3.12's multivariate row pair, combined next with convex products and LayerNorm.

Conditioned attribution instead fixes the realized softmax matrix $A$, making $X \mapsto AV(X)$ affine (Ali et al., 2022; Wróbel et al., 2026). The routing-game sections study this conditioned calculation; the full forward decomposition retains $A$'s input dependence.

## 5.3 A Constructive DC Decomposition of Transformers

Replacing (5.1)'s sign-sorted product by a square identity preserves convexity and coordinatewise monotonicity on non-negative split inputs. With Theorem 3.12's softmax pair and a reciprocal-square-root extension, this yields a constructive DC decomposition of the full deterministic Transformer. This analytical extension is separate from the conditioned attribution implementation evaluated below.

**Proposition 5.3 (Convex product pair by polarization, `Lean_74`).** *For $A,B,C,D \geq 0$, let*

$$\begin{aligned} P(A,B,C,D) &= \tfrac{1}{2}\big((A+C)^2 + (B+D)^2\big), \\ Q(A,B,C,D) &= \tfrac{1}{2}\big((A+D)^2 + (B+C)^2\big). \end{aligned} \tag{5.3}$$

*Then $P - Q = (A-B)(C-D)$. Both functions are convex, non-negative, and coordinatewise non-decreasing on $\mathbb{R}^4_{\geq 0}$. Hence substituting non-negative convex, coordinatewise non-decreasing incoming streams preserves these properties. In particular,*

$$(A-B)^2 = 2(A^2+B^2) - (A+B)^2 \tag{5.4}$$

*is a valid convex, monotone square pair on $\mathbb{R}^2_{\geq 0}$.*

*Proof.* Expansion proves recovery. Each summand squares an affine form, hence is convex and non-negative. Its partial derivatives are non-negative on the stated orthant, permitting convex non-decreasing composition with convex components. Setting $C = A$ and $D = B$ gives (5.4). □

Monotonicity requires the orthant: the derivative of $(A+C)^2$ in $A$ is $2(A+C)$, possibly negative outside it. The Transformer theorem therefore uses non-negative doubled inputs, although the softmax pair has a larger domain. Apply (5.3) to scalar products and sum corresponding components to split matrix products.

### 5.3.1 Layer normalization with positive epsilon

For $\varepsilon > 0$ and $d \geq 1$, LayerNorm on $u \in \mathbb{R}^d$ uses

$$\begin{gathered} \bar{u} = \frac{1}{d}\sum_{j=1}^{d} u_j, \qquad c_i = u_i - \bar{u}, \qquad v = \frac{1}{d}\sum_{j=1}^{d} c_j^2 \geq 0, \\ \mathrm{LN}_\varepsilon(u)_i = \gamma_i c_i (\varepsilon + v)^{-1/2} + \beta_i. \end{gathered} \tag{5.5}$$

Centering and the learned scale and bias are signed affine; variance is a positive sum of squares. Only the reciprocal square root on non-negative arguments is new. Extending it to negative arguments gives a global convex pair when the variance streams vary independently.

**Proposition 5.4 (Reciprocal-square-root pair, Lean_75).** *Let $\varepsilon > 0$, put $M = (2\varepsilon^{3/2})^{-1}$, and define*

$$r_\varepsilon(t) = \begin{cases} \varepsilon^{-1/2} - Mt, & t \le 0, \\ (\varepsilon + t)^{-1/2}, & t \ge 0. \end{cases} \tag{5.6}$$

*Then*

$$R_+(x,y) = r_\varepsilon(x-y) + Mx, \qquad R_-(x,y) = Mx \tag{5.7}$$

*are convex and coordinatewise non-decreasing on $\mathbb{R}^2$, are non-negative on $\mathbb{R}^2_{\ge 0}$, and have difference $r_\varepsilon(x-y)$. Whenever $x - y \ge 0$, this difference is $(\varepsilon + x - y)^{-1/2}$.*

*Proof.* The extension is continuously differentiable at zero, with matching derivative $-M$. On $t > 0$ its second derivative is $3/(4(\varepsilon + t)^{5/2}) > 0$, whereas it is affine on $t < 0$. Thus $r_\varepsilon$ is convex and $-M \le r'_\varepsilon \le 0$. Affine precomposition and addition preserve convexity, and $\partial_x R_+ = r'_\varepsilon + M \ge 0$, $\partial_y R_+ = -r'_\varepsilon \ge 0$. The linear component $R_-$ is also monotone. Their origin values are $\varepsilon^{-1/2}$ and 0, giving non-negativity on the orthant; cancellation proves the stated differences. Equivalently, this is the scalar lift of Proposition 3.2 with profiles $p(t) = r_\varepsilon(t) + Mt$, $q(t) = Mt$, and slope bound $M$. □

To decompose (5.5), first split the affine centering map. Apply (5.4) to each centered coordinate and average the two streams separately to obtain $v = v^+ - v^-$. Apply (5.7) to $(v^+, v^-)$. Because the recovered variance is non-negative, the extension agrees exactly with the actual normalization factor. Finally use the polarized product pair to multiply each centered coordinate by that factor, followed by the signed affine scale and bias. Every intermediate stream is convex, monotone, and non-negative on the original split-input orthant. The hypothesis $\varepsilon > 0$ ensures that the normalization is defined for every input, including constant vectors.

### 5.3.2 An explicit pre-normalization architecture

To make the arbitrary-depth claim concrete, we specify the ordinary Transformer forward map before defining its split evaluation. Tokens are indexed by $t \in \{1, \dots, T\}$, with feature vectors in $\mathbb{R}^d$, where $T, d \ge 1$. Every matrix and bias below is a fixed real parameter and may have either sign.

**Definition 5.2 (Finite pre-normalization Transformer, Lean_76).** A Transformer is specified by a positional bias $P \in \mathbb{R}^{T \times d}$, a finite list of blocks $B_1, \dots, B_K$, an optional final LayerNorm, and a signed affine scalar readout. In block $k$, first normalize each token,

$$U_t = \mathrm{LN}_{k,1}(X_t).$$

Each head $h$ has positive query/key width $d_{q,h}$ and value width $d_{v,h}$, signed affine projections, a real score scale $\tau_h$, and a fixed nonempty allowed-key set $I_{h,t} \subseteq \{1, \dots, T\}$ for every query token. It computes

$$\begin{aligned} Q_{h,t} &= W_{Q,h}U_t + b_{Q,h}, & K_{h,t} &= W_{K,h}U_t + b_{K,h}, \\ V_{h,t} &= W_{V,h}U_t + b_{V,h}, & S_{h,ts} &= \tau_h \langle Q_{h,t}, K_{h,s} \rangle, \\ A_{h,ts} &= \frac{e^{S_{h,ts}}}{\sum_{r \in I_{h,t}} e^{S_{h,tr}}} \quad (s \in I_{h,t}), & O_{h,t} &= \sum_{s \in I_{h,t}} A_{h,ts} V_{h,s}. \end{aligned} \tag{5.8}$$

Masked coefficients outside $I_{h,t}$ are zero. The usual scaled dot product is the choice $\tau_h = 1/\sqrt{d_{q,h}}$. Summing the per-head output projections is equivalent to concatenation followed by one output

projection. The residual attention and pointwise feedforward updates are

$$\begin{aligned} Y_t &= X_t + \sum_h W_{O,h} O_{h,t} + b_O, \\ B_k(X)_t &= Y_t + W_2 \varrho_k(W_1 \operatorname{LN}_{k,2}(Y_t) + b_1) + b_2. \end{aligned} \tag{5.9}$$

Both LayerNorms have positive epsilon and arbitrary signed learned scale and bias as in (5.5). The activation $\varrho_k$ is chosen from ReLU, exact Gaussian GELU, Swish/SiLU, sigmoid, and softplus, with the positive parameters required in Chapter 3. Head widths, feedforward widths, masks, and parameters may differ between blocks, while the residual width is $d$.

For any finite depth $K \geq 0$, define

$$\begin{aligned} X^{(0)} &= X + P, \qquad X^{(k)} = B_k(X^{(k-1)}) \quad (1 \leq k \leq K), \\ \widehat{X}^{(K)} &= \begin{cases} \operatorname{LN}_{\text{out}}(X^{(K)}), & \text{if a final norm is present,} \\ X^{(K)}, & \text{otherwise,} \end{cases} \\ N(X) &= b_{\text{out}} + \sum_{t=1}^{T} \sum_{j=1}^{d} w_{tj}^{\text{out}} \widehat{X}_{tj}^{(K)}. \end{aligned} \tag{5.10}$$

The optional final norm is applied tokenwise with positive epsilon. The depth-zero case has no blocks and consists of the positional affine map, the optional final norm, and the readout. Choosing zero readout weights outside one token includes a class-token logit. This is a deterministic inference computation; a fixed dropout mask can additionally be represented by fixed coordinate scalings.

### 5.3.3 The recursive split algorithm

Expand the affine maps, dot products, and matrix contractions in Definition 5.2 into scalar expressions. The expression constructors are inputs, real constants, sums, signed scalar multiples, products, supported scalar activations, softmax coordinates, and LayerNorm coordinates. Their ordinary evaluation uses the usual real functions in (5.8)–(5.10). No convexity certificate is assumed as part of an ordinary expression.

Write $\mathscr{C}(e) = (e^+, e^-)$ for the recursively constructed pair, as a function of independent non-negative input streams $(x, y)$. Pair addition is componentwise, and signed scalar multiplication is

$$\operatorname{Scale}_c(p, q) = \begin{cases} (cp, cq), & c \geq 0, \\ (-cq, -cp), & c < 0. \end{cases} \tag{5.11}$$

The operators Prod, $\operatorname{Softmax}_i^{\pm}$, and $\operatorname{LN}_i^{\pm}$ below are the explicit constructions of (5.3), Theorem 3.12, and the LayerNorm recipe following Proposition 5.4. In particular, the LayerNorm pair computes the centered pairs, their square pairs, their average, the reciprocal-square-root pair, and the final product and affine parameters; it retains the input-dependent statistics.

The algorithm recurses over any finite expression in real arithmetic, independently of depth. Translation follows the finite block list: the empty list preserves its input expression; appending a block substitutes current token expressions into its formulas. Induction identifies ordinary evaluation with (5.10). Implementations may reuse shared expressions; correctness requires no bound on expansion or runtime.

**Algorithm 5.1:** Recursive DC evaluation of a finite Transformer expression

**Input:** A scalar output expression $e$ from Definition 5.2; split inputs $x,y \geq 0$.
**Output:** A pair $\mathscr{C}(e)(x,y)$ whose difference is the ordinary output at $x - y$.
For an input coordinate $i$, return $\mathscr{C}(X_i) = (x_i,y_i)$; for a constant $c$, return $(\max\{c,0\}, \max\{-c,0\})$.
Recursively compute the pair of each child expression before applying its parent operation.
For a sum $\sum_j e_j$, return $\sum_j \mathscr{C}(e_j)$; for $ce_1$, return $\mathrm{Scale}_c(\mathscr{C}(e_1))$.
For $e_1e_2$, return $\mathrm{Prod}(\mathscr{C}(e_1),\mathscr{C}(e_2))$ using the four-square formula (5.3).
For a scalar activation $\varrho(e_1)$, apply its explicit pair to $\mathscr{C}(e_1)$.
For a softmax coordinate, collect the compiled allowed-score pairs and apply $\mathrm{Softmax}_i^{\pm}$ to that vector of pairs.
For a LayerNorm coordinate, collect the compiled feature pairs and apply $\mathrm{LN}_i^{\pm}$ with its positive epsilon and signed scale and bias.
Return the pair at the requested affine readout.

### 5.3.4 The full Transformer construction

**Theorem 5.5 (Exact DC decomposition of a deterministic Transformer, Lean_77).** *Let $N\colon \mathbb{R}^{T\times d} \to \mathbb{R}$ be the Transformer of Definition 5.2, with any finite list of blocks, including the empty list, and identify its token-feature input with $\mathbb{R}^m$, where $m = Td$. Algorithm 5.1, applied to its output expression, terminates and exactly decomposes this forward map. It constructs functions $N_+,N_-\colon \mathbb{R}^{2m}_{\geq 0} \to \mathbb{R}_{\geq 0}$, convex and coordinatewise non-decreasing on that orthant, such that*

$$N(x-y) = N_+(x,y) - N_-(x,y) \qquad (x,y \in \mathbb{R}^m_{\geq 0}). \tag{5.12}$$

*In particular, the functions*

$$g(u) = N_+([u]^+,[u]^-), \qquad h(u) = N_-([u]^+,[u]^-) \tag{5.13}$$

*are convex on all of $\mathbb{R}^m$, are non-negative, and satisfy $N = g - h$. The functions $N_+,N_-$ are coordinatewise non-decreasing in the independent split inputs; $g,h$ need not be coordinatewise non-decreasing in the original signed variable $u$.*

*Proof.* The block list yields a finite expression, so syntax recursion terminates. Structural induction shows that each compiled expression has non-negative convex, coordinatewise non-decreasing heads on the input orthant, whose difference evaluates the expression at $x - y$.

Coordinate projections and positive/negative constant splits satisfy this invariant. Sums preserve it; signed scaling rescales by a non-negative coefficient, swapping heads for negative scalars. Proposition 5.3 handles products by exact recovery and convex, monotone composition on non-negative streams. The scalar pair theorems handle activations; Theorem 3.12 handles the nonempty allowed-key vector. LayerNorm follows its centering, square, reciprocal, product and affine recipe, with non-negative recovered variance and positive epsilon. Thus every constructor preserves the invariant.

Induction over the block list equates ordinary evaluation with the independently specified recurrence (5.10), proving (5.12) at every finite depth, including zero, with an ungated signed affine readout. Finally, $u \mapsto [u]^{\pm}$ has convex components valued in the orthant. Convex monotone composition gives globally convex $g,h$, and subtraction recovers $N(u)$. □

The construction retains input dependence through Queries, Keys, Values, softmax and LayerNorm statistics. Its real-arithmetic proof establishes a forward DC representation, without numerical-stability, attribution-quality, floating-point-error or runtime guarantees. The reciprocal pair's common coefficient $M = (2\varepsilon^{3/2})^{-1}$ grows as $\varepsilon$ shrinks; products and squares can amplify this mass, potentially requiring cancellation between enormous streams. The companion supplement's Section S.4.5 tests the same primitive pairs in a post-normalization block, whose LayerNorm placement differs from Definition 5.2. In this unstabilized float64 diagnostic, 15 of 20 cases exceed $10^{-8}$ output error and streams reach $5.95 \times 10^{111}$. The thesis's attribution experiments evaluate the conditioned rules below, not this convex construction.

## 5.4 Attention via a detached-logit oracle

For conditioned transport, our LayerNorm rule fixes both mean and scale, giving a pointwise affine map. CP-LRP and DAVE instead retain linear centering while fixing the scale (Ali et al., 2022; Wróbel et al., 2026). The QK-softmax block is replaced by an *oracle game* emitting a detached reference policy, the softmax row $A_{t\cdot}$, with occupation-measure trajectories flowing only through the Value route. Like conservative propagation (Ali et al., 2022), this explains the network conditioned on its realized attention pattern.

**Theorem 5.6 (Attention, informal).** *With reference policy $A_{t\cdot}$, the KL-regularized Value-routing subgame has an SPNE whose occupation measure at the block input equals $\alpha\beta$-LRP on the conditioned effective map $x \mapsto \sum_k A_{tk}\, x_k W_v + b_v$ (Theorem S.2.3).*

Our implementation applies this rule to the attention product and the Value projection as one composite step; a full gradient fallback through the attention block degrades localization (AL 0.624 vs. 0.711 at the localization-selected configuration). Equivariant Reynolds averaging optionally suppresses tokenization-grid artifacts (Wróbel et al., 2026) (RG+Eq). The target rule, oracle embedding and implementation are in Section 5.5; the full KL formulation and occupation-measure proof are in Chapter S.2 of the companion technical supplement.

## 5.5 Extension: Vision Transformer Attention

We state the target rule, oracle interpretation and implementation used by the ViT experiments. The full state-space formulation, KL derivation and occupation-measure proof are given in Chapter S.2 of the companion technical supplement; the risk-averse attention variant is in Section S.2.5.

### 5.5.1 Attention as fixed-reference KL Value routing

#### Target rule: $\alpha\beta$-LRP with detached logits

We first state the rule that the Routing-Game state-space construction in Section S.2.1 is designed to recover. We focus on a single attention head; the multi-head extension carries an additional head index $h$ and the recovery is per-head. Note that ViT-B/16 implements the Value projection as a single $D \times D$ linear $v = XW_v$ followed by a reshape into $n_h$ heads (the per-head matrix $W_v^{(h)} \in \mathbb{R}^{D \times d_h}$ is the head-$h$ column block of the full $W_v$), so one input embedding $X_{k,e}$ feeds Value entries of every head and the per-head $R^X$ contributions sum at $X_{k,e}$ to give the total. The per-head description below applies unchanged. We omit the $\varepsilon$ stabilizer inside the attention block — the surrounding ReLU recursion of §S.1.3.2 carries its own $\varepsilon$ on every linear-routing state and that mechanism transfers to

the $Av$+$W_v$ composite without modification, but tracking it through the tensor indexing here only obscures the structure.

**Forward pass.** The Value projection is applied per token: for each $k \in \{1,\dots,S\}$,

$$v_{k,d} \;=\; \sum_e W_{v,e,d}\, X_{k,e} + b_{v,d}, \qquad d \in \{1,\dots,d_h\}, \tag{5.14}$$

with shared $W_v \in \mathbb{R}^{D\times d_h}$ and $b_v \in \mathbb{R}^{d_h}$ applied independently per token. Detached softmax attention $A \in \mathbb{R}^{S\times S}_{\geq 0}$, with $A_{t,k} \geq 0$ and $\sum_k A_{t,k} = 1$, mixes Values across tokens independently for each feature $d$. For output token $t \in \{1,\dots,S\}$,

$$O_{t,d} \;=\; \sum_k A_{t,k}\, v_{k,d} \;=\; \sum_{k,e} A_{t,k}\, W_{v,e,d}\, X_{k,e} + b_{v,d}. \tag{5.15}$$

**Composite linear map and $\kappa$-stream split.** Holding $A$, the LayerNorm statistics and the GELU gates at their forward values (§5.5.3) is a modeling assumption about the explanation question—given the realized attention policy, through which Value routes does relevance flow?—and not a theorem about relevance through $Q$ and $K$. Our attention extension is therefore a Value-only attribution conditioned on the realized attention matrix, resembling the conservative propagation of Ali et al. (2022) (CP-LRP), which holds the attention matrix constant, and DAVE's conditioned operators (Wróbel et al., 2026). It differs from AttnLRP (Achtibat et al., 2024), which also propagates relevance through the attention-generating operations (the softmax and the bilinear QK product) and so attributes the Query and Key paths. With $A$ so conditioned, the attention block reduces to the affine *conditioned effective map* $X \mapsto O$ with composite weights $W^{\text{eff}}_{(k,e),\,(t,d)} = A_{t,k}\, W_{v,e,d}$ and bias $b_{v,d}$. Since $A_{t,k} \geq 0$, the sign of the composite weight is the sign of $W_{v,e,d}$:

$$\big[\, W^{\text{eff}}_{(k,e),\,(t,d)} \,\big]^{\kappa} \;=\; A_{t,k}\, W^{\kappa}_{v,e,d}, \qquad \kappa \in \{+,-\}. \tag{5.16}$$

For signed inputs $X$ (LayerNorm outputs), the per-token $\kappa$-stream split of the Value tensor (as in the fused input layer of the RG, with the bias as a bias node) and the per-output-feature stream sum are

$$\tilde{v}^{\kappa}_{k,d} \;\coloneqq\; \sum_e [X_{k,e} W_{v,e,d}]^{\kappa} + [b_{v,d}]^{\kappa}, \qquad Z^{\kappa}_{t,d} \;\coloneqq\; \sum_k A_{t,k}\, \tilde{v}^{\kappa}_{k,d} \;=\; \sum_{k,e} A_{t,k}\, [X_{k,e} W_{v,e,d}]^{\kappa} + [b_{v,d}]^{\kappa}. \tag{5.17}$$

Note that $\tilde{v}^{\kappa}_{k,d}$ is the $\kappa$-stream split inherited from $W_v$ and the sign of $X$ (not the standard scalar positive part $[v_{k,d}]^{+} = \max(0, v_{k,d})$); the two coincide only when $v_{k,d}$ has a single-signed predecessor decomposition.

**Target rule.** Let $R^{O}_{t,d}$ be relevance arriving at $O_{t,d}$ from the surrounding LRP recursion. Apply the $\alpha\beta$-LRP rule (4.1) to the composite map $X \mapsto O$ in two stages, retaining $W_v$'s stream split $\tilde{v}^{\kappa}_{k,d}$ through (5.16). Sequential $\alpha\beta$-LRP on $X \to v \to O$ instead splits netted Values by sign and applies $(\alpha,\beta)$ twice, giving a different rule for mixed-sign predecessors. All reported ViT results and the implementation (Section 5.5.3) use the composite rule; Section S.4.3.4's tests check it against (5.21).

**Zero-mass convention.** All ratios of non-negative contributions in this subsection use

$$\rho(u,D) := \begin{cases} u/D, & D > 0, \\ 0, & D = 0, \end{cases} \qquad 0 \le u \le D. \tag{5.18}$$

A zero denominator therefore gives zero. Zero-mass states route probability one to the cemetery and zero to ordinary successors, preserving a probability distribution. This applies to both $Z^{\kappa}_{t,d}$ and $\tilde{v}^{\kappa}_{k,d}$: zero $Z$ forces every $A_{t,k}\tilde{v}^{\kappa}_{k,d}$ to vanish; zero $\tilde{v}$ forces all matching input and bias contributions to vanish. The ideal formulas use $\varepsilon = 0$. The released implementation also supports $Z^{\kappa}_{t,d} + \varepsilon$ with positive stabilization, routing the remaining share to the outside option.

*AV mixing $O \to v$.* Applied to $O_{t,d} = \sum_k A_{t,k}\, v_{k,d}$, the rule gives the relevance at the Value entry $v_{k,d}$ as the weighted difference of two per-stream contributions:

$$R^{\mathrm{val}}_{k,d} \;=\; \alpha\, R^{\mathrm{val},+}_{k,d} \;-\; \beta\, R^{\mathrm{val},-}_{k,d}, \qquad R^{\mathrm{val},\kappa}_{k,d} \;:=\; \sum_t \frac{A_{t,k}\,\tilde{v}^{\kappa}_{k,d}}{Z^{\kappa}_{t,d}}\, R^{O}_{t,d}, \quad \kappa \in \{+,-\}. \tag{5.19}$$

*Value projection $v \to X$.* Applied to $v_{k,d} = \sum_e W_{v,e,d}\, X_{k,e} + b_{v,d}$, the rule routes each $\kappa$-stream contribution $R^{\mathrm{val},\kappa}_{k,d}$ to the input dim $e$ via the matching $\kappa$-stream ratio $[X_{k,e}W_{v,e,d}]^{\kappa}/\tilde{v}^{\kappa}_{k,d}$ (the bias share is absorbed):

$$R^{X}_{k,e} \;=\; \sum_d \left( \alpha\, \frac{[X_{k,e}W_{v,e,d}]^{+}}{\tilde{v}^{+}_{k,d}}\, R^{\mathrm{val},+}_{k,d} \;-\; \beta\, \frac{[X_{k,e}W_{v,e,d}]^{-}}{\tilde{v}^{-}_{k,d}}\, R^{\mathrm{val},-}_{k,d} \right). \tag{5.20}$$

*Aggregate.* Substitute (5.19) into (5.20). Positive $\tilde{v}^{\kappa}_{k,d}$ cancels; if it or $Z^{\kappa}_{t,d}$ is zero, both forms vanish by convention. Thus

$$R^{X}_{k,e} \;=\; \sum_{t,d} \left( \alpha\, \frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{+}}{Z^{+}_{t,d}} \;-\; \beta\, \frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{-}}{Z^{-}_{t,d}} \right) R^{O}_{t,d}, \tag{5.21}$$

is exactly $\alpha\beta$-LRP on the conditioned map $X \mapsto O$ with $W^{\mathrm{eff}}_{(k,e),(t,d)} = A_{t,k}\, W_{v,e,d}$ and bias $b_{v,d}$.

The state-space construction and the proof of Theorem S.2.3 are given in Chapter S.2 of the companion technical supplement.

### 5.5.2 The reference as the policy of a QK oracle player

§5.5.1 treated the softmax row $A_{t,\cdot}$ as a fixed reference policy. It is itself the entropy-regularized best response of an auxiliary *oracle* player whose payoff is the detached query–key logit $e_{tk} = Q_t K_k^{\top}/\sqrt{d_h}$: by (S.1.56) at temperature 1,

$$\begin{aligned} A_{t,\cdot} \;&=\; \mathrm{argmax}_{\pi \in \Delta(\{1,\dots,S\})} \Big\{ \textstyle\sum_k \pi(k)\, e_{tk} + \mathscr{H}(\pi) \Big\}, \\ \max_{\pi} \Big\{ \textstyle\sum_k \pi(k)\, e_{tk} + \mathscr{H}(\pi) \Big\} \;&=\; \log \textstyle\sum_k \exp(e_{tk}). \end{aligned} \tag{5.22}$$

The oracle acts in the forward pass only and hands $\bar{\pi}_t = A_{t,\cdot}$ to the Value-routing player; its states carry no occupation mass, so no relevance flows through $Q$ or $K$ (§5.5.3).

### 5.5.3 Implementation details

Four operator conventions align the RG forward and backward passes on ViT-B/16 with §S.2.1's explanatory walk. They define a DAVE-inspired *conditioned forward pass* (Wróbel et al., 2026) with the LayerNorm mean additionally frozen. Routing conservation includes bias and stabilizer absorption.

**LayerNorm detachment.** Standard backpropagation through $\mathrm{LN}(x) = w_{\mathrm{LN}} \odot (x - m(x))/s(x) + b_{\mathrm{LN}}$ traverses data-dependent $m(x)$,$s(x)$, coupling features within each token without mixing tokens, and can violate LRP conservation (Ali et al., 2022). Computing $m$,$s$ from $c = x.\mathrm{detach}()$ gives the pointwise affine map $x \mapsto w_{\mathrm{LN}} \odot (x - m(c))/s(c) + b_{\mathrm{LN}}$. Our rule freezes both mean and scale; CP-LRP and DAVE freeze only the scale.

**GELU gate detachment.** Implement GELU as $x \cdot \Phi(c)$, with $c = x.\mathrm{detach}()$ and standard normal CDF $\Phi$. Backward propagation then scales pointwise by constant $\Phi(c)$, realizing the elementwise LRP identity rule in modified Gradient×Input implementations (Arras et al., 2025, Appendix D.3).

**Softmax detachment.** Softmax outputs are detached. As in §5.5.2, the cached row $A_{t,\cdot}$ is an auxiliary entropy-regularized equilibrium policy supplied to the Value-routing player; explanatory trajectories never enter $Q$ or $K$.

**QK gradient detachment.** Softmax and QK-matmul backward hooks return zero, restricting QK to computing the detached KL reference $A_{t,\cdot}$; all attention-block explanatory mass follows Values. The composite rule of Theorem S.2.3 retains each Value entry's projection-stream masses $\tilde{v}^{\kappa}_{k,d}$. Attention routes stream $\kappa$ proportionally to $A_{t,k}\tilde{v}^{\kappa}_{k,d}$, then the Value projection routes to same-sign $[X_{k,e}W_{v,e,d}]^{\kappa}$ contributions. Discounts $(\alpha,\beta)$ therefore act once per block, as in (5.21). The released code implements only this rule (`av_rule='merged'`); no reported result uses §5.5.1's sequential variant. Section S.4.3.4's identity tests compare it with an independent reference for (5.21).

Together these conventions implement a conditioned RG for §5.5.1's Value-only attribution question, shared with CP-LRP (Ali et al., 2022); AttnLRP (Achtibat et al., 2024) also propagates relevance through softmax and QK.

### 5.5.4 Equivariant Spatial Averaging

The 16×16 patch tokenization breaks translation equivariance. Following DAVE (Wróbel et al., 2026), RG+Eq explains transformed inputs, maps explanations back, and averages the aligned maps (a Monte-Carlo Reynolds average),

$$\mathscr{M}_N(x) \;=\; \frac{1}{N}\sum_{n=1}^{N} T_n^{-1}\big[\mathscr{M}(T_n x)\big], \tag{5.23}$$

where $\mathscr{M}$ is the RG attribution and $T_n$ are i.i.d. random rotations (uniform in $\pm 10°$), translations (up to $\pm 5\%$ per image axis) and horizontal flips (probability 0.5), with $N = 50$ and no input noise. The conditioned pass (§5.5.3) separates operator variation from the effective transformation, following DAVE's argument (Wróbel et al., 2026). These transforms do not form a group, so this optional smoothing has no exactness guarantee.

# 6 Saliency and Attribution Methods

This chapter develops interpretability methods from the streams $g$,$h$ with $f = g - h$ constructed in Section 3.2.3. Section 6.1 reads the streams separately to explain pretrained networks through SplitCAM, SplitLRP and SplitGrad (Zimmermann and Loho, 2026). It also studies directly trained difference-of-convex input neural networks (DCINNs), whose MNIST examples exhibit class-dependent stream sensitivities. Section 6.2 collects practical *modes* that turn the game equilibria of Chapter 4 into tunable attribution maps, and the *technicalities* of equivariant averaging that improve attribution quality.

## 6.1 Split-Based and Difference-of-Convex Attribution

Post-hoc attribution assigns each input feature a score for its influence on a fixed network output. Established families include gradient saliency (Simonyan et al., 2014), deconvolution and guided backpropagation (Springenberg et al., 2015; Zeiler and Fergus, 2014), LRP and deep Taylor decomposition (Bach et al., 2015; Montavon et al., 2017), class-activation mapping (Selvaraju et al., 2017; Zhou et al., 2016), baseline-based methods such as Integrated Gradients and DeepLIFT (Shrikumar et al., 2017; Sundararajan et al., 2017), noise-averaged gradients (Smilkov et al., 2017), and perturbation- or surrogate-based explanations (Fel et al., 2023; Fong and Vedaldi, 2017; Kolek et al., 2023; Lundberg and Lee, 2017; Ribeiro et al., 2016). These explain the original network. We identify signal loss in rules using ordinary local derivatives, then develop SplitCAM, SplitLRP and SplitGrad from the separate signals preserved by the split-pair of Section 3.2.3, closing with by-design DCINNs.

PRCA and DRSA optimize relevance over activation subspaces; their virtual layer preserves the decision function when a full orthogonal basis is retained (Chormai et al., 2024). Our split representation separates contributions using the original network's signs and gates.

Consider a feedforward ReLU network with layers $1,\dots,L$, weights $W^{(l)}$, pre-activations $z^{(l)} = W^{(l)}a^{(l-1)}$, hidden activations $a^{(l)} = \mathrm{ReLU}(z^{(l)})$ for $l < L$, and ungated readout $a^{(L)} = z^{(L)}$. We attribute a fixed scalar class logit: $f^{(l)}\colon \mathbb{R}^{d^{(l)}} \to \mathbb{R}$ denotes layers $l{+}1,\dots,L$ acting on layer-$l$ activations. Biases are absorbed as in Section 3.2.3. The split streams $g^{(l)}$,$h^{(l)}\colon \mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}} \to \mathbb{R}$ act on $a^{(l,+)}$,$a^{(l,-)}$ and satisfy

$$f^{(l)}\big(a^+ - a^-\big) = g^{(l)}(a^+, a^-) - h^{(l)}(a^+, a^-) \qquad \text{for all } a^+, a^- \in \mathbb{R}^{d^{(l)}}, \tag{6.1}$$

and we write $g \coloneqq g^{(0)}$, $h \coloneqq h^{(0)}$ with balanced input split $a^{(0,\pm)} = \pm x/2$. Derivatives are row vectors (scalar Jacobians); derivative statements hold at differentiability points, hence Lebesgue-almost everywhere because all functions involved are CPWL.

A ReLU gate transmits positive pre-activations and otherwise returns zero, erasing their negative magnitude and blocking the local gradient. This *subthreshold signal loss* hides the suppressed pre-activation from the output through that gate and from backward rules retaining its ordinary derivative. Ordinary input gradients can therefore exhibit exact regional blindness.

**Proposition 6.1 (Regional blindness of the input gradient,** `Lean_67`**).** *Let $f$ be as above and $x_0$ an input with no zero pre-activations. If every path from input coordinate $i$ to the output passes through a neuron inactive at $x_0$, there is an open neighborhood $U \ni x_0$ with*

$$\frac{\partial f}{\partial x_i} \equiv 0 \quad \text{on } U,$$

*even if $f$ depends on $x_i$ elsewhere.*

*Proof.* Set $M^{(l,+)} := \operatorname{diag}\big(\mathbf{1}\{z^{(l)} > 0\}\big)$, $M^{(l,-)} := \operatorname{diag}\big(\mathbf{1}\{z^{(l)} \le 0\}\big)$ and $W^{(l,x)} := M^{(l,+)}W^{(l)}$ for hidden layers $1 \le l < L$. The affine logit readout is ungated and may be negative, so $W^{(L,x)} := W^{(L)} = \partial f/\partial a^{(L-1)}$. Continuity and nonzero hidden pre-activations at $x_0$ give an open $U \ni x_0$ with constant masks and affine $f$. Iterating the chain rule on $U$ yields

$$\frac{\partial f}{\partial x} = W^{(L,x)}\, W^{(L-1,x)} \cdots W^{(1,x)}, \tag{6.2}$$

and its $i$-th entry sums weight products over paths from input $i$ to the output, multiplied by indicators that every hidden neuron on the path is active; the output carries no mask. Every path contains an inactive hidden neuron, so every summand vanishes throughout $U$. □

The phenomenon already occurs in a single-hidden-neuron network.

*Remark 6.1 (The canonical counterexample).* Following Sundararajan et al. (2017), consider the one-dimensional ReLU network

$$f(x) = 1 - \mathrm{ReLU}(1 - x), \tag{6.3}$$

and compare input $x = 2$ with baseline $x' = 0$. The prediction changes by $f(2) - f(0) = 1$, making the sole feature relevant. Yet $f$ is constant on $x \ge 1$: at $x = 2$, gradients, Input×Gradient and rules retaining local ReLU derivatives assign zero attribution. The hidden neuron is inactive, so Proposition 6.1 applies with $U = (1,\infty)$.

Sundararajan et al. (2017) axiomatize this requirement.

**Definition 6.1 (Sensitivity(a)).** An attribution method $A$, mapping an input $x$ and a baseline $x'$ to feature scores $A_i(x, x') \in \mathbb{R}$, satisfies *Sensitivity* if for every pair $(x, x')$ that differs in exactly one feature $i$ and satisfies $f(x) \ne f(x')$, the attribution of that feature is non-zero, $A_i(x, x') \ne 0$.

Plain gradients violate Sensitivity (Remark 6.1). Integrated Gradients restores it by integrating the gradient along the segment from $x'$ to $x$, requiring many forward–backward passes and inheriting the baseline choice. The split-pair instead preserves subthreshold signals in the *representation* used for attribution.

Recall from Section 3.2.3 how the split-pair processes a ReLU gate: the pre-activation pair $z^{(l,+)}, z^{(l,-)}$ is mapped to

$$a^{(l,+)} = \max\{z^{(l,+)}, z^{(l,-)}\}, \qquad a^{(l,-)} = z^{(l,-)}, \tag{6.4}$$

a rank-2 Maxout unit in the $g$-stream and identity in the $h$-stream. Their difference reproduces $\mathrm{ReLU}(z^{(l)})$. At an inactive neuron, both activations equal $z^{(l,-)}$: the suppressed signal's magnitude remains in their common level. The backward pass likewise retains subthreshold information.

**Definition 6.2 (Local sensitivities).** For $s \in \{g, h\}$, $\sigma \in \{+, -\}$ and $0 \le l \le L$, the *local sensitivities* of the split-pair at layer $l$ are the row vectors

$$\delta^{(l,\sigma,s)} := \frac{\partial s^{(l)}}{\partial a^{(l,\sigma)}} \in \mathbb{R}^{1 \times d^{(l)}},$$

the derivatives of the stream outputs with respect to the positive and negative split activations of layer $l$. We further abbreviate the original-network sensitivity by $\delta^{(l)} \coloneqq \partial f^{(l)}/\partial a^{(l)}$ and define the *absolute weight product*

$$W_{\text{abs}}^{[l]} \coloneqq |W^{(L)}|\,|W^{(L-1)}|\cdots|W^{(l+1)}| \in \mathbb{R}^{1\times d^{(l)}}, \qquad W_{\text{abs}}^{[L]} \coloneqq \mathbf{1}^\top,$$

where $|\cdot|$ acts entrywise. The scalar output makes $W_{\text{abs}}^{[l]}$ a row vector whose $i$-th entry sums absolute weight products over all paths from layer-$l$ neuron $i$ to the output, active or inactive.

The four local sensitivities obey coupled backward recursions, the split analogue of backpropagation underlying this chapter's methods.

**Lemma 6.2 (Split backward recursions, `Lean_68`).** *With the activation masks $M^{(l,\pm)}$ of Proposition 6.1, extended by $M^{(L,+)} = I$ and $M^{(L,-)} = 0$ for the affine readout, for every $0 \le l < L$ and $s \in \{g, h\}$,*

$$\delta^{(l,+,s)} = \delta^{(l+1,+,s)}\big(M^{(l+1,+)}W^{(l+1,+)} + M^{(l+1,-)}W^{(l+1,-)}\big) + \delta^{(l+1,-,s)}\,W^{(l+1,-)}, \tag{6.5}$$

$$\delta^{(l,-,s)} = \delta^{(l+1,+,s)}\big(M^{(l+1,+)}W^{(l+1,-)} + M^{(l+1,-)}W^{(l+1,+)}\big) + \delta^{(l+1,-,s)}\,W^{(l+1,+)}, \tag{6.6}$$

*with base case $\delta^{(L,+,g)} = \delta^{(L,-,h)} = \mathbf{1}^\top$ and $\delta^{(L,-,g)} = \delta^{(L,+,h)} = 0$.*

These chain-rule recursions follow from the split forward equations of Section 3.2.3 (derivation in Section 3.6.2): the maximum in (6.4) selects $z^{(l+1,+)}$ exactly when the original neuron is active, giving $M^{(l+1,+)}$. They imply that total sensitivity mass is independent of the activation pattern.

**Proposition 6.3 (Sensitivity mass conservation, `Lean_69`).** *For every $0 \le l \le L$ and both streams $s \in \{g, h\}$,*

$$\delta^{(l,+,s)} + \delta^{(l,-,s)} = W_{\text{abs}}^{[l]}. \tag{6.7}$$

*Proof.* Induction on $l = L, \dots, 0$. The base case is $\mathbf{1}^\top + 0 = W_{\text{abs}}^{[L]}$. For the step, add (6.5) and (6.6): the mask terms combine via $M^{(l+1,+)} + M^{(l+1,-)} = I$ into

$$\delta^{(l,+,s)} + \delta^{(l,-,s)} = \big(\delta^{(l+1,+,s)} + \delta^{(l+1,-,s)}\big)\big(W^{(l+1,+)} + W^{(l+1,-)}\big) = \big(\delta^{(l+1,+,s)} + \delta^{(l+1,-,s)}\big)|W^{(l+1)}|,$$

since $W^{(l+1,+)} + W^{(l+1,-)} = |W^{(l+1)}|$ for the entrywise weight split. The induction hypothesis turns the right-hand side into $W_{\text{abs}}^{[l+1]}|W^{(l+1)}| = W_{\text{abs}}^{[l]}$. □

Differentiating split correctness recovers the original gradient.

**Corollary 6.4 (Backward invariant, `Lean_70`).** *For every $0 \le l \le L$,*

$$\delta^{(l,+,g)} - \delta^{(l,+,h)} = \delta^{(l)} = -\big(\delta^{(l,-,g)} - \delta^{(l,-,h)}\big), \quad \textit{hence} \quad \big(\delta^{(l,+,g)} - \delta^{(l,-,g)}\big) - \big(\delta^{(l,+,h)} - \delta^{(l,-,h)}\big) = 2\,\delta^{(l)}. \tag{6.8}$$

*Proof.* Differentiate the correctness identity (6.1) with respect to $a^+$ and $a^-$, respectively, and subtract the two resulting identities. □

Proposition 6.3 fixes the sum of partial sensitivities at input-independent $W_{\text{abs}}^{[l]}$; Corollary 6.4 recovers the original gradient from stream differences. Define $\delta^{(l,s)} := \frac{1}{2}\big(\delta^{(l,+,s)} - \delta^{(l,-,s)}\big)$. Then $\delta^{(l,g)} - \delta^{(l,h)} = \delta^{(l)}$, so regional blindness (Proposition 6.1) means equal combined stream gradients, possibly both zero. Individual partial sensitivities $\delta^{(l,\pm,s)}$ can differ from $\frac{1}{2}W_{\text{abs}}^{[l]}$; these deviations record routing at active and inactive ReLU gates. A path of nonzero weights from the feature to the output guarantees at least one positive partial sensitivity per head, but does not guarantee nonzero partial differences, combined gradients, or attribution. For example, with $f(x) = \mathrm{ReLU}(\mathrm{ReLU}(x) + \mathrm{ReLU}(-x) - 10)$ and balanced input split, both heads equal 10 locally near $x = 1$, and both combined gradients vanish.

*Remark 6.2 (The counterexample, revisited).* Split the network (6.3) with the balanced input split $a^{(0,\pm)} = \pm x/2$. A short computation along Section 3.2.3 gives, for $x > 1$,

$$g(x) = \frac{x}{2} + 1, \qquad h(x) = \frac{x}{2}, \qquad g(x) - h(x) = 1 = f(x),$$

so at $x = 2$ both stream gradients equal $\frac{1}{2}$, while $f'(2) = \frac{1}{2} - \frac{1}{2} = 0$. Thus the heads supply a nonzero signal consistent with $f(2) \neq f(0)$, even where their difference is constant. This example with $x' = 0$ does not establish Definition 6.1 for arbitrary attribution rules or baselines. The following methods turn the pair-valued signal into scalar maps; the shifted combinations of Section 6.1.3 avoid exact cancellation here and assign positive attribution for every $\alpha < \frac{1}{2}$.

Non-negative weights let each neuron only *add* evidence, a structure long associated with interpretability (Ayinde and Zurada, 2018; Lee and Seung, 1999). The split-pair gives an arbitrary pretrained network this structure by doubling its width; the following methods exploit its signal flow.

### 6.1.1 SplitCAM

CAM methods explain CNN decisions by combining spatial activation maps of a convolutional layer with class-specific weighting coefficients. The original CAM (Zhou et al., 2016) requires a global-average-pooling head; Grad-CAM (Selvaraju et al., 2017) extends it to CNNs without that head requirement by weighting each channel with the spatially pooled gradient of the class logit, and Grad-CAM++ (Chattopadhay et al., 2018) refines the pooling weights. LayerCAM (Jiang et al., 2021) drops the pooling altogether and weights every spatial location individually: for a layer $l$ with channel activation maps $a_c^{(l)}$ and class logit $y$,

$$\mathrm{LayerCAM}^{(l)} = \mathrm{ReLU}\Big(\sum_c \mathrm{ReLU}\Big(\frac{\partial y}{\partial a_c^{(l)}}\Big) \odot a_c^{(l)}\Big), \tag{6.9}$$

where $\odot$ denotes the elementwise product and the sum runs over the channels. This makes LayerCAM applicable to early layers with high spatial resolution. Inactive neurons contribute no activation-weighted term, and the positive-gradient weighting discards negative gradient evidence.

*SplitCAM* applies LayerCAM's spatial activation weighting to signed split-stream maps. Raw sensitivities sum *absolute* path products (Proposition 6.3) and become numerically enormous in deep split-pairs, requiring the stabilized sensitivities of Section 3.6.2.

**Definition 6.3 (Shifted local sensitivities, `Lean_71`).** Fix shift parameters $\alpha^{(l)} \in [0, \frac{1}{2}]$. For $s \in \{g, h\}$ define, backward from layer $L$, intermediate sensitivities $\delta_{\text{int}}^{(l,\pm,s)}$ by evaluating the recursions

(6.5)–(6.6) (respectively the base case of Lemma 6.2 at $l = L$) with the already-shifted sensitivities of the subsequent layer, and set

$$\delta_{\text{shift}}^{(l,\pm,s)} := \delta_{\text{int}}^{(l,\pm,s)} - \alpha^{(l)}\big(\delta_{\text{int}}^{(l,+,s)} + \delta_{\text{int}}^{(l,-,s)}\big), \qquad \delta_{\text{shift}}^{(l,s)} := \frac{\delta_{\text{shift}}^{(l,+,s)} - \delta_{\text{shift}}^{(l,-,s)}}{2}. \tag{6.10}$$

Both signs receive the *same* shift, so the informative differences are preserved layer by layer, while by Proposition 6.3 the subtracted direction is the pattern-independent bulk $\delta_{\text{int}}^{(l,+,s)} + \delta_{\text{int}}^{(l,-,s)} = \big[\prod_{i=l+1}^{L}(1-2\alpha^{(i)})\big] W_{\text{abs}}^{[l]}$ (the empty product at $l = L$ is 1); the factor $\frac{1}{2}$ in the combined sensitivity $\delta_{\text{shift}}^{(l,s)}$ matches the balanced input split. Unless stated otherwise all $\alpha^{(l)} \equiv \alpha$. For $\alpha = 0$ the shifted sensitivities are the raw ones; for $\alpha = \frac{1}{2}$ they collapse onto multiples of the original gradient (Chapter 3, Theorem 3.26), so the informative regime is $0 < \alpha < \frac{1}{2}$, where the split signal survives with a controlled, layer-dependent damping.

#### The SplitCAM attribution.

**Definition 6.4 (SplitCAM).** For a layer $l$ of the decomposed model, a stream $s \in \{g, h\}$ and a sign $\sigma \in \{+, -\}$, the SplitCAM attribution maps are

$$\text{SplitCAM}^{(l,\sigma,s)} := \sum_c \delta_{\text{shift}}^{(l,\sigma,s)}\big|_c \odot a_c^{(l,\sigma)}, \tag{6.11}$$

$$\text{SplitCAM}^{(l,s)} := \sum_c \delta_{\text{shift}}^{(l,s)}\big|_c \odot a_c^{(l)}, \tag{6.12}$$

where $a_c^{(l,\sigma)}$ denotes the channel-$c$ activation map of the $\sigma$-stream, $a_c^{(l)} = a_c^{(l,+)} - a_c^{(l,-)}$ the reconstructed original activation map, and $\cdot|_c$ the restriction of a sensitivity tensor to channel $c$.

SplitCAM *omits positive-gradient clipping and the outer ReLU*, retaining signed shifted sensitivities. It uses shifted sensitivities rather than raw derivatives: (6.10) changes each map only by multiples of the pattern-independent $W_{\text{abs}}^{[l]}$, keeping the quantities numerically meaningful.

Under Section 3.6, the forward pass scales split activations and restores their invariant against cached original activations. The backward pass applies Definition 6.3 using masks from those activations, preserving (6.8) against the cached original gradient.

**Layer choice and resolution.** Like LayerCAM, SplitCAM is defined at every layer, and the resulting map is bilinearly upsampled to the input resolution. Empirically the method peaks in the deep convolutional blocks — for VGG-16 (Simonyan and Zisserman, 2015) at layer 26 — where it concentrates on entire classified objects noticeably better than LayerCAM at the same layer; the full layer-ablation study and the quantitative comparison on the Quantus benchmark (Hedström et al., 2023) for VGG-16 and ResNet-18 (He et al., 2016) are reported in Chapter 7.

### 6.1.2 SplitLRP

LRP (Bach et al., 2015) propagates a relevance budget backward, distributing each neuron's relevance to its inputs in proportion to their contributions; Montavon et al. (2018) give an overview and Kohlbrenner et al. (2020) best practices. The $\gamma$-rule (Montavon et al., 2019) emphasizes positive

contributions. We use its bias-excluding, $\varepsilon$-stabilized variant: for $z_i^{(l+1)} = \sum_j W_{ij}^{(l+1)} a_j^{(l)} + b_i^{(l+1)}$, define

$$\tilde{z}_{ij}^{(l+1)} := a_j^{(l)} \Big( W_{ij}^{(l+1)} + \gamma \, [W_{ij}^{(l+1)}]^+ \Big), \qquad \gamma \geq 0, \tag{6.13}$$

and redistributes the relevances $R_i^{(l+1)}$ onto the inputs as

$$R_j^{(l)} = \sum_i \frac{\tilde{z}_{ij}^{(l+1)}}{\sum_{j'} \tilde{z}_{ij'}^{(l+1)} + \varepsilon \, \operatorname{sign}\big(\sum_{j'} \tilde{z}_{ij'}^{(l+1)}\big)} \, R_i^{(l+1)}, \tag{6.14}$$

with a small stabilizer $\varepsilon > 0$. Setting $\gamma = 0$ gives the corresponding bias-excluding $\varepsilon$-rule. For affine–ReLU blocks, ReLU activations pass relevance through unchanged, and redistribution acts on the affine layers.

**Relevance propagation on the split-pair.** *SplitLRP* treats both streams as neuron groups in one decomposed network. Each layer carries relevance tensors $R^{(l,+)}$,$R^{(l,-)}$, jointly propagated through the four coupling blocks built from $W^{(l,+)}$,$W^{(l,-)}$. Propagation starts at the class-specific positive $g$-output $a^{(L,+)}$. All weights are non-negative, so we use (6.14) with $\gamma = 0$, $\varepsilon = 10^{-6}$: the $\varepsilon$-rule needs no preference for positive over negative contributions. Paired Maxout gates (6.4) are transparent in the relevance pass.

The following conventions complete the specification.

- *Forward pass.* The split activations are computed with the same scaling stabilization and forward-invariant correction of Section 3.6 as SplitCAM.
- *Normalization.* The denominator of (6.14) renormalizes relevance layer-wise, preserving the combined budget of $R^{(l,+)}$,$R^{(l,-)}$ at $a^{(L,+)}$ up to the $\varepsilon$-stabilizer (Remark 6.3). This replaces backward shifting (Definition 6.3).
- *Pooling and normalization layers.* Relevance passes through max-pooling by the winner-takes-it-all scheme, assigning the full relevance to the location attaining the maximum; batch-normalization layers are merged into the adjacent linear or convolutional layers beforehand; residual additions split relevance evenly across branches.

**Definition 6.5 (Combined SplitLRP map).** The SplitLRP attribution at layer $l$ is the pair $\big(R^{(l,+)}, R^{(l,-)}\big)$; the *combined* map is

$$R^{(l,\mathrm{comb})} := R^{(l,+)} - R^{(l,-)}. \tag{6.15}$$

Positive-stream relevance traverses an even number of sign inversions and represents support; negative-stream relevance represents opposition. Equation (6.15) combines them into a signed saliency map.

*Remark 6.3 (A probabilistic reading).* The original network's $\varepsilon$-rule mixes signed contributions, producing a signed path measure (Chapter 8). In the split-pair, $\tilde{z}_{ij}^{(l+1)}$ multiplies non-negative activations and weights. Up to the stabilizer, its propagation matrices are stochastic: SplitLRP is a Markov chain on the doubled graph, with unsigned relevance measure $\big(R^{(l,+)},R^{(l,-)}\big)$. Doubling the state space absorbs weight signs, as in the stopping game's probabilistic backward flow (Chapter 4).

*Remark 6.4 (Sensitivity to learned parameters).* The parameter-randomization sanity check of Adebayo et al. (2018) differentiates the three split methods. After reinitializing the last three feedforward layers of VGG-16, every reported SplitCAM and SplitGrad configuration has lower SSIM than every listed

baseline, whereas the HOG and Spearman comparisons are mixed. This establishes sensitivity to the tested parameter changes, not faithfulness to the learned decision process. SplitLRP retains high HOG and Spearman similarity despite a decrease in SSIM. This retained structure is consistent with parameter-insensitivity concerns for some relevance-propagation rules (Sixt et al., 2020), but the comparison does not identify the mechanism in SplitLRP. Section S.4.1.4 reports the quantitative results and the caveats of Binder et al. (2023) on randomization-based checks.

### 6.1.3 SplitGrad

SplitGrad uses stabilized stream gradients directly. For the paired-input streams, fix the balanced embedding $a^{(0,\pm)} = \pm x/2$ and write $\tilde{g}^{(l)}(a) := g^{(l)}\big(\frac{a}{2}, -\frac{a}{2}\big)$, and $\tilde{h}^{(l)}$ analogously. The chain rule (Section 3.6.2) gives $\partial \tilde{s}^{(l)}/\partial a = \big(\delta^{(l,+,s)} - \delta^{(l,-,s)}\big)/2$, explaining the factor $\frac{1}{2}$ in (6.10). The combined shifted sensitivities $\delta^{(l,g)}_{\text{shift}}$,$\delta^{(l,h)}_{\text{shift}}$ are stabilized gradients of convex $\tilde{g}^{(l)}$,$\tilde{h}^{(l)}$; their difference recovers the exact original gradient because the common shift cancels sign-wise.

**Definition 6.6 (SplitGrad).** For a layer $l$ and stream $s \in \{g, h\}$, the SplitGrad attribution map is the channel mean of the combined shifted sensitivity,

$$\text{SplitGrad}^{(l,s)} := \frac{1}{C^{(l)}} \sum_{c=1}^{C^{(l)}} \delta^{(l,s)}_{\text{shift}}\big|_c, \tag{6.16}$$

where $C^{(l)}$ is the number of channels of layer $l$; the default configuration reads the $g$-stream, with forward and backward stabilization exactly as in Section 6.1.1.

**What SplitGrad measures.** Chapter 3 computes these stabilized gradients in closed form: by Theorem 3.26, the input-level stream gradients have original-gradient terms $\delta/2$ and $-\delta/2$, respectively, where $\delta := \partial f/\partial x$, and share a subtracted, geometrically damped sum of *subthreshold probe-gradients* — one per layer, each carrying the signal routed through the *inactive* neurons whose loss causes the blindness of Proposition 6.1. Both streams contain this probe sum with the same sign, so it cancels in their difference, which reproduces $\delta$ exactly. The probe sum can itself vanish; the decomposition makes subthreshold contributions available without guaranteeing a nonzero attribution for every input. SplitGrad therefore recovers the feature the plain gradient misses: in the counterexample of Remark 6.2, $\delta = 0$ while the probe term is non-zero, so the feature is attributed correctly for every $\alpha < \frac{1}{2}$. The layer-wise damping factors $\overline{\alpha}_j$ of Theorem 3.26 both counteract the rapid growth of the probe-gradient norms toward the input and let $\alpha$ interpolate between fidelity to the original gradient ($\alpha = \frac{1}{2}$) and full exposure of the split information ($\alpha = 0$); the sweep over $\alpha$ in Chapter 7 supports a metric- and configuration-dependent choice. For the two main-text configurations, localization optima occur at $\alpha = 0.35$ or 0.4, while Maximum Sensitivity is lowest at $\alpha = 0$; the full ablation is reported in Section S.4.2.

The three methods share one pipeline: a stabilized split forward pass caching the original activations, a stabilized backward pass — relevance-based for SplitLRP (initialized at the class-specific $a^{(L,+)}$ and propagated by the $\varepsilon$-rule), sensitivity-based otherwise (the shifted recursions of (6.10)) — and a method-specific readout, with the resulting map bilinearly upsampled to the input resolution. The network can be split once per model. In the reported implementation, SplitCAM takes approximately 14–29 times as long per image as the corresponding ordinary Grad-CAM and LayerCAM configurations; SplitGrad also takes substantially longer. Their timings are of the same order as the row labeled

Guided Grad-CAM, but the released VGG-16 configuration dispatches Guided LayerCAM, limiting identification of that baseline. Section S.4.1.5 details this ambiguity and the measurement scope.

### 6.1.4 Directly Trained Difference-of-Convex Input Neural Networks

To study mechanisms exposed by the DC decomposition of pretrained networks (Sections 6.1.1–6.1.3), we directly train small DC input neural networks (DCINNs) and their monotone relatives on MNIST. Their shared opposing-stream structure provides a controlled setting for examining class-dependent sensitivities on present and candidate missing strokes, with a modest, architecture-dependent accuracy cost.

**Architecture and training.** An ICNN (Amos et al., 2017) has convex, monotone activations and non-negative weights except possibly in its first layer, and represents a convex function (Section 2.2). Monotone activations with all weights non-negative give a monotone function. We view ICNNs as monotone networks composed with the affine transformation of an unconstrained first layer.

**Definition 6.7 (DCINN and difference-of-monotone models).** Fix a base architecture. A *DCINN* (in short: DIC model) is the function

$$f_{\mathrm{DIC}} := g - h,$$

where $g$ and $h$ are two ICNNs of the base architecture with independent parameters, trained *jointly* by standard empirical risk minimization on the difference. The *DM model* is defined analogously with both streams monotone networks. The *inverted-input* variants DICIN and DMIN feed the $h$-stream the complemented input $\mathbf{1} - x$ (for inputs normalized to $[0,1]^d$) instead of $x$.

As in the split-pair, two non-negative streams define the model by their difference. Here they learn convex (respectively monotone) functions directly: the difference structure is a training-time inductive bias. Related convex difference neural networks (CDiNN) address optimal control (Sankaranarayanan and Rengaswamy, 2022), with recurrent extensions for safety-critical model predictive control (Chen et al., 2025); we study their interpretability. A monotone $h$-stream accumulates evidence from input mass, so feeding it $\mathbf{1} - x$ lets it respond to missing mass.

Three implementation choices matter in practice.

- *Activations.* Both streams use rank-2 Maxout units (Goodfellow et al., 2013), which are convex and monotone as required, and which consistently outperformed ReLU streams in the difference setting. This aligns the trained models exactly with the hypothesis class of Section 3.2.3: the split-streams obtained there *are* rank-2 Maxout networks with non-negative weights, so training DIC and DM models searches the very class into which the splitting maps.
- *Non-negativity.* Constrained weights are parameterized as $p(\tilde{W})$ with free parameters $\tilde{W}$ and a positivity transform $p$; among softplus, exponential and squaring, $p = \mathrm{ReLU}$ with absolute-value-of-Gaussian initialization gave the best test accuracy. The transform is also applied to biases and to the scale and shift parameters of batch-normalization layers.
- *Regularization.* Since non-negative weights preclude cancellation, activations can grow quickly; a small initialization scale and a weight decay of $10^{-3}$ keep the streams stable during training.

The empirical study uses two MNIST base architectures: `cnn_maxout`, a CNN with four convolutional layers (32, 64, 128 and 128 channels with batch normalization) followed by three fully connected layers, roughly 1.2 million parameters in total, and `mlp_maxout`, a wide four-layer MLP ($784 \to 800 \to$

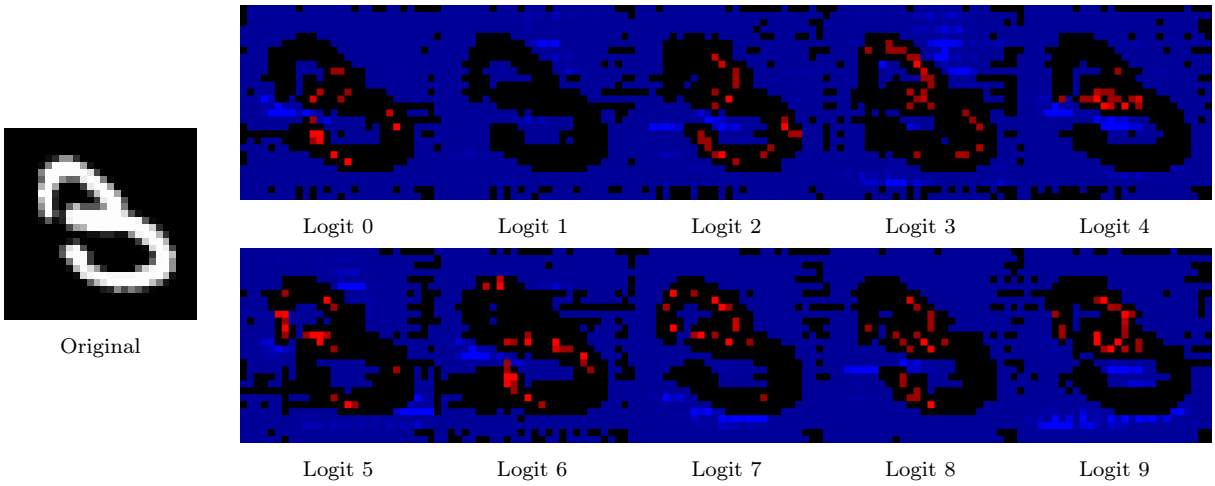



**Fig. 6.1:** Stream gradients of the inverted-input DIC model (`cnn_maxout` DICIN) on a correctly classified MNIST test digit "3", thresholded and overlaid per class logit. The $g$-stream gradient (red) highlights sensitivities on present strokes, while the $h$-stream gradient (blue) highlights sensitivities at candidate missing-stroke locations. These stream-local maps do not by themselves establish the effect of adding pixels on the complete model. Left: the original digit.

$800 \to 800 \to 10$) with roughly 4.2 million parameters — each in eight variants: unconstrained (None), single-stream constrained (IC, Mon), and difference-of-two models (DIC, DM, DICIN, DMIN, and an unconstrained difference baseline).

**Stream sensitivities and candidate counterfactual features.** The by-design route exposes the *plain input gradients* of the trained streams without shifting, relevance rules, or probe networks. For an inverted-input DIC class logit $f(x) = g(x) - h(\mathbf{1} - x)$, the two gradients describe complementary local sensitivities:

- the $g$-stream gradient $\nabla g(x)$ describes the sensitivity of the contribution $g(x)$ to changes in the input;
- the $h$-stream gradient $(\nabla h)(\mathbf{1} - x)$ describes the sensitivity of the contribution $-h(\mathbf{1} - x)$ to increasing the original input: the input complement and the subtraction cancel the two minus signs in the chain rule.

At differentiable inputs, the complete model gradient is therefore

$$\nabla f(x) = \nabla g(x) + (\nabla h)(\mathbf{1} - x).$$

The first affine layer of an ICNN is unconstrained, so convexity of the streams does not make either gradient non-negative in the original input coordinates. A positive entry of one stream gradient can be outweighed by a negative entry of the other. Even a positive entry of the complete gradient only establishes an increase for sufficiently small feasible increases in that coordinate at a differentiable input; it does not establish a necessary feature or a change in the predicted class. Finite counterfactual claims require evaluating the complete model under the proposed intervention. The displayed maps are thus stream-local sensitivities; their concentration on present and candidate missing strokes motivates a factual/counterfactual interpretation to be tested by interventions.

Figure 6.1 overlays both gradients for each class logit of a test "3". The $g$-gradient best traces strokes for the ground-truth logit; for competing "6", "8" and "9", $h$-gradients concentrate *between the arches*,

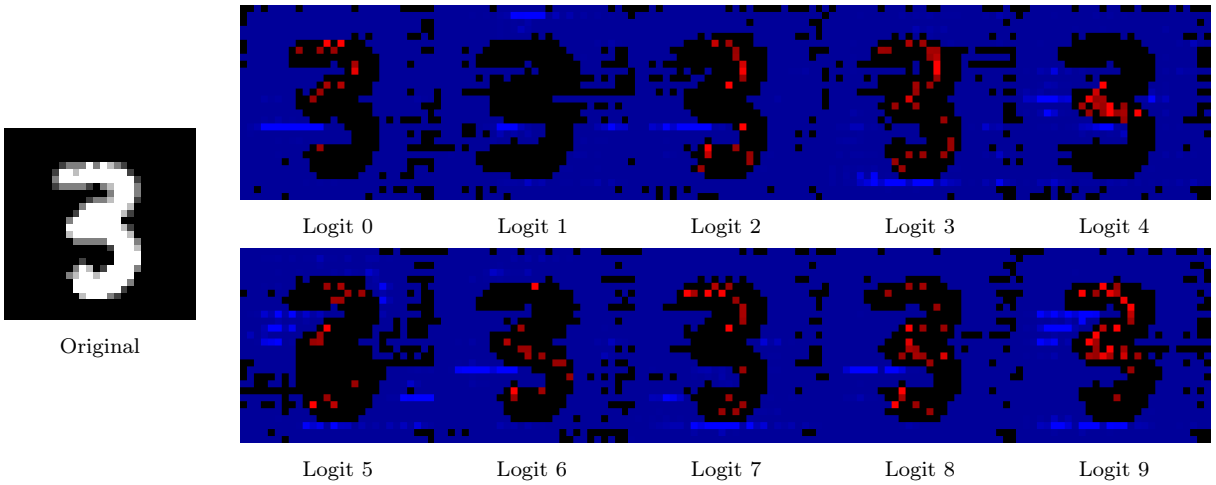



**Fig. 6.2:** Second example of stream gradients of the inverted-input DIC model (`cnn_maxout` DICIN) on a correctly classified MNIST test digit "3", thresholded and overlaid per class logit. The $g$-stream gradient (red) highlights sensitivities on present strokes, while the $h$-stream gradient (blue) highlights sensitivities at candidate missing-stroke locations. These stream-local maps do not by themselves establish the effect of adding pixels on the complete model. Left: the original digit.

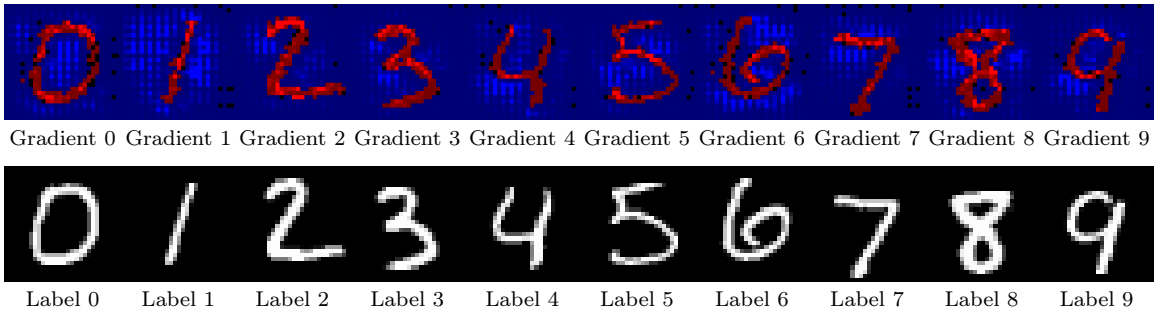



**Fig. 6.3:** Input-gradient visualizations of the `cnn_maxout` IC model for ten MNIST digits, each with respect to its ground-truth label. Top: saliency maps; bottom: input images. As in the source figure, positive and negative gradient values above threshold 0.2 are colored red and blue, with ranking colorized using exponential decay.

where added pixels would geometrically close loops. A second correctly classified "3" (Figure 6.2) shows the same qualitative pattern. The single `cnn_maxout` IC model's gradients are mostly positive on strokes and negative on background, with less class-dependent directional variation. These examples suggest a role for the difference structure without establishing general disentanglement.

**Qualitative comparison with a single input-convex model.** Figure 6.3 reproduces the source paper's ten-class input-gradient comparison for the single `cnn_maxout` IC model: positive local logit sensitivities mostly follow strokes, negative sensitivities the background, with little class-dependent directional variation. The DICIN maps (Figures 6.1, 6.2) instead show class-dependent stream patterns, with $h$ highlighting candidate missing strokes. This motivates a factual/counterfactual reading, without establishing finite-intervention effects or separation across all inputs and models.

**Accuracy trade-offs.** Architectural constraints can reduce expressivity (Liu et al., 2020; Sill, 1997), but for the difference-of-two construction the accuracy cost is small on the convolutional base model

and even negative on the MLP, where every difference-of-two variant outperforms the unconstrained baseline; the full accuracy table across ten seeds and all eight variants is reported in Chapter 7.

*Remark 6.5 (Post-hoc and by-design, reconciled).* Both routes test whether difference-of-monotone representations make sensitivities more legible. The MNIST study shows that directly trained differences of constrained networks achieve competitive accuracy and exhibit class-dependent stream sensitivities on present and candidate missing strokes. These qualitative examples establish neither faithful counterfactuals nor general explanatory-role separation for the trained models or pretrained VGG-16/ResNet-18 split-pairs. Complete-model interventions and larger-architecture tests remain open (Chapter 9).

## 6.2 Game-Based Attribution: Modes and Technicalities

Chapter 4 offers a complementary route to the fixed DC readouts: the per-neuron difference of the players' discounted occupation measures under equilibrium trajectories defines attribution. It recovers gradients in the SG and the full $\alpha\beta$-LRP family in the RG at $\tau = 1$ (Theorem 4.2). With these equivalences stated there and proved in the companion supplement, we turn to *modes* that deform the games and input-side *technicalities* that improve their maps.

### 6.2.1 Attribution Modes

Chapter 4 varies equilibrium trajectory laws through routing and stopping temperatures $\tau$,$\theta$, the policy-entropy bonus, risk $\lambda \leq 0$, stream balance $(\alpha,\beta)$, and LRP stabilization $\epsilon \geq 0$. Section 4.4 gives an exact dictionary from each parameter to its backward rule. These controls yield tunable attribution maps and form the sweeps of Chapter 7.

Here $(\alpha,\beta)$ denotes the $\alpha\beta$-LRP discount pair, with $\alpha - \beta = 1$ and anchor (2,1) weighting support against opposition; a bare $\alpha$ is this stream weight, unrelated to the backward shift $\alpha \in [0,\frac{1}{2}]$ of Section 6.1. Routing temperature $\tau$ recovers $\alpha\beta$-LRP exactly only at $\tau = 1$ (Theorem 4.2); its myopic application preserves the forward pass. It is distinct from the spatial transforms $\tau_t$ below.

### 6.2.2 Technicalities

Input-transform averaging and a detached-operator (DAVE-style) reading of the gradient decomposition improve maps without changing the game. They matter especially for ViTs, whose $16 \times 16$ patch tokenization breaks translation equivariance and introduces grid artifacts (Dosovitskiy et al., 2021; Wróbel et al., 2026).

**Reynolds-inspired averaging.** Following DAVE (Wróbel et al., 2026), compute attributions on $N$ randomly transformed inputs (small rotations, translations, horizontal flips), map them back by $\tau_t^{-1}$, and average. These spatial transforms $\tau_1,\ldots,\tau_N$ are distinct from routing temperature $\tau$. Reynolds-inspired averaging can suppress transform-dependent architectural artifacts while retaining common image structure; $N$ trades computation for smoothness and is ablated in Chapter 7.

**Connection to DAVE-style gradient decomposition.** The averaging pairs with a *conditioned forward pass* inspired by DAVE (Wróbel et al., 2026). Our game fixes both LayerNorm statistics, the GELU gate and the softmax row; DAVE retains linear mean-centering while fixing the normalization scale. Conservation concerns the game's propagation rule with its bias accounting. The attribution takes the transform-averaged input×gradient form $\frac{1}{N}\sum_t \tau_t^{-1}(W_t) \odot X$, where $W_t$ is the conditioned backward signal on the $t$-th transformed input. Chapter 5 specifies the conditioned operators used by the game-based maps.

# 7 Experiments and Evaluation

This chapter evaluates two design choices: exposing split streams to attribution and changing backward routing through game policies. The split-method and game-based campaigns examine localization, faithfulness, robustness, and numerical or computational cost. We retain their checkpoints, sample exclusions, metric conventions, and selection protocols, comparing scores within each campaign. Model-evaluation scores come from the papers; Pascal VOC uncertainty estimates and comparison statistics are recomputed locally using images as independent sampling units.

## 7.1 Experimental Setup

The split campaign evaluates pretrained VGG-16 and ResNet-18 on a custom 50-class ImageNet-S subset at $224 \times 224$ resolution. The source paper reports 50 validation images (one per class) for layer–formula selection and 566 test images. Its released loader selects locally available image–mask pairs without the game campaign's label-agreement filter; these counts alone cannot recover historical sample membership. Explanations target the predicted class, including for misclassified images; localization uses the annotated object mask. Baselines use Captum, Zennit for $\gamma$-LRP, and separate Grad-CAM++ and LayerCAM implementations. Each method retains localization-selected and Pixel-Flipping-selected configurations (full settings and results: Section S.4.1 of the supplement).

The split tables report Pixel Flipping as score-drop AUC ($\mathrm{PF}_5^{\mathrm{drop}}$, $\mathrm{PF}_{20}^{\mathrm{drop}}$; ↑), perturbing 112 scalar input features per step, and Maximum Sensitivity with 25 perturbations. The revised game campaign reports remaining-logit AUC ($\mathrm{PF}_5$, $\mathrm{PF}_{20}$; ↓), with 1,568 scalar features per step, and sensitivity with 50 perturbations on a common 200-image test subset. Both Pixel-Flipping implementations broadcast spatial attribution over color channels, indexing the flattened $C \times H \times W$ input: 150,528 features for RGB images at this resolution. Quantus measures Selectivity, Attribution Localization and Pointing Game. Both campaigns include VGG-16; their ResNet-18 and ResNet-50 scores are not interchangeable. Table abbreviations are AL (Attribution Localization), PG (Pointing Game), Sel (Selectivity), MaxS (Maximum Sensitivity), AvgS (Average Sensitivity), GCAM (Grad-CAM), GCAM++ (Grad-CAM++), LCAM (LayerCAM), IntGrad (Integrated Gradients), and SmGrad (SmoothGrad); Guided GCAM denotes Guided Grad-CAM. Dark blue shading and bold type mark the best values within each caption's comparison; n/r means unreported in the source paper, and n/a means inapplicable.

## 7.2 Split Methods: VGG-16

**Qualitative analysis.** Figure 7.1 compares split and baseline saliency maps at their best-performing layers. Mid-layer SplitCAM resembles earlier-layer classical $\gamma$-LRP. Against same-layer LayerCAM, it gives more weight across object pixels while emphasizing key features, balancing interpretability and focus (low entropy). For $\alpha \ll 0.5$, SplitGrad yields less focused, finely aligned gradients following image details. We attribute this to Maxout retaining pre-activation information that ReLU discards at inactive neurons. Further examples appear in Section S.4.1 of the supplement.

**Quantitative analysis.** Tables 7.1 and 7.2 compare selected VGG-16 and ResNet-18 configurations on five Quantus metrics: Selectivity, Attribution Localization, Pointing Game accuracy, Pixel Flipping (score-drop AUC, $\mathrm{PF}_5^{\mathrm{drop}}$ and $\mathrm{PF}_{20}^{\mathrm{drop}}$), and Maximum Sensitivity. *Abs* indicates absolute saliency

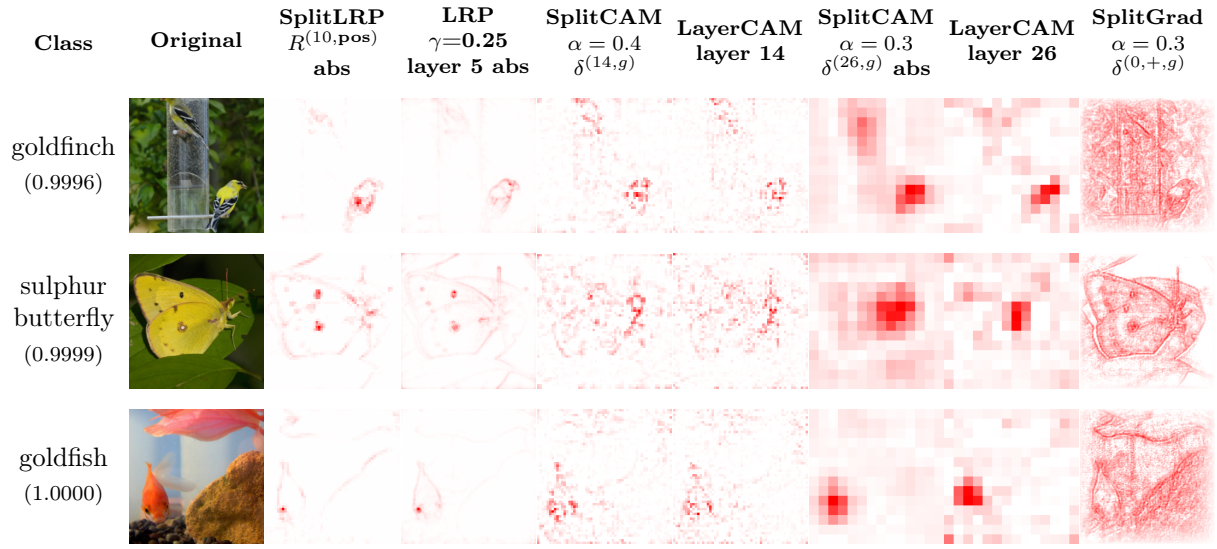



**Fig. 7.1:** Saliency map comparison for VGG-16 on our custom ImageNet-S test split. Each row shows the predicted class (with confidence), original image, and saliency maps from different methods.

values before Quantus evaluation. Full results, explanations, layers and parameter configurations appear in Section S.4.1 of the supplement.

The backpropagation-based methods depend strongly on layer choice (ablations: Section S.4.2 of the supplement).

VGG-16 gains depend on metric and configuration. Table 7.1 gives SplitCAM Pointing Game 0.938 versus Guided Backprop's 0.887, Attribution Localization 0.651 versus Guided Backprop's 0.632, and Selectivity 2.727 versus LayerCAM's 2.792, from different SplitCAM layers. LayerCAM has the highest displayed Pixel Flipping score-drop AUCs, 0.317 and 2.474, versus SplitCAM's 0.300 and 2.404. SplitLRP has the lowest Maximum Sensitivity, 0.282, versus Guided Backprop's 0.375, and higher Pointing Game but lower Attribution Localization and Pixel Flipping scores than the displayed classical $\gamma$-LRP configurations.

## 7.3 Split Methods: ResNet-18

Table 7.2 reports selected ResNet-18 configurations on the same five metrics, with *Abs* indicating absolute saliency before evaluation (full tables: Section S.4.1 of the supplement). SplitCAM has the lowest reported Selectivity, 3.503, versus 3.517 for the strongest baseline, Guided Backprop: a difference of 0.014. SplitCAM and SplitGrad reach Pointing Game 0.972 versus LayerCAM's 0.971. Attribution Localization is 0.673 for SplitCAM and 0.664 for LayerCAM; Guided Grad-CAM leads the full comparison at 0.742 (Table S.4.2). Baselines retain the best Pixel Flipping and Maximum Sensitivity scores: the gains are metric-specific.

## 7.4 Split-Method Stabilization Ablations

### 7.4.1 Backward-Pass Stabilization of the Split Methods

Table 7.3 varies the shift $\alpha$ for two selected SplitGrad and SplitCAM configurations on the custom ImageNet-S test split. As in Section 3.6, $\alpha = 0$ leaves the backward pass unstabilized; $\alpha = 0.5$ gives

**Tab. 7.1:** Selected interpretability evaluation configurations (VGG-16, ImageNet-S test split). The *wta* abbreviation indicates the use of the "winner-takes-it-all" max-pooling implementation; *sc* emphasizes the scaling procedure in the forward pass (Section 3.6). Best values per metric among the displayed rows are bold with dark blue shading. The bracket notation in the layer column follows the upper indices of the split methods as introduced in Chapter 6.

| Method | Layer | Abs | Sel↓ | AL↑ | PG↑ | $PF_5^{drop}$↑ | $PF_{20}^{drop}$↑ | MaxS↓ |
|---|---|---|---|---|---|---|---|---|
| *Split Methods* | | | | | | | | |
| SplitCAM (sc, $\alpha = 0.4$) | (12, g) | n | 4.720 | **0.651** | 0.912 | 0.213 | 1.794 | 0.389 |
| SplitCAM (sc, $\alpha = 0.4$) | (14, g) | n | **2.727** | 0.563 | 0.836 | 0.300 | 2.404 | 1.012 |
| SplitCAM (sc, $\alpha = 0.4$, wta) | (26, +, g) | y | 4.711 | 0.635 | **0.938** | 0.165 | 1.590 | 0.456 |
| SplitGrad ($\alpha = 0.4$, wta) | (12, +, g) | n | 4.735 | 0.635 | 0.883 | 0.186 | 1.756 | 0.651 |
| SplitGrad ($\alpha = 0.4$, wta) | (14, +, g) | y | 3.287 | 0.540 | 0.818 | 0.249 | 2.186 | 0.857 |
| SplitLRP (sc, wta) | (26, pos) | y | 5.168 | 0.588 | 0.871 | 0.151 | 1.473 | **0.282** |
| *Baselines (With Layer Optimization)* | | | | | | | | |
| Guided Backprop | 2 | n | 3.276 | 0.632 | 0.883 | 0.281 | 2.302 | 0.571 |
| Guided Backprop | 5 | n | 3.448 | 0.614 | 0.887 | 0.240 | 2.091 | 0.375 |
| LCAM | 14 | n/a | 2.792 | 0.576 | 0.798 | **0.317** | **2.474** | 2.656 |
| LCAM | 2 | n/a | 3.128 | 0.580 | 0.855 | 0.300 | 2.191 | 5.508 |
| LRP $\gamma = 0.25$ | 0 | n | 3.189 | 0.615 | 0.680 | 0.218 | 1.918 | 0.519 |
| LRP $\gamma = 0.25$ | 14 | n | 3.593 | 0.577 | 0.611 | 0.279 | 2.255 | 0.491 |
| *Baselines* | | | | | | | | |
| DeepLIFT | 0 | y | 4.447 | 0.551 | 0.726 | 0.281 | 1.945 | 1.053 |
| Feature Ablation | 0 | y | 6.029 | 0.479 | 0.624 | 0.123 | 1.250 | 0.859 |
| GradientSHAP | 0 | y | 4.808 | 0.552 | 0.783 | 0.261 | 1.862 | 1.142 |
| GCAM++ | 30 | n | 5.955 | 0.564 | 0.827 | 0.130 | 1.245 | 0.401 |
| IntGrad | 0 | y | 4.838 | 0.554 | 0.797 | 0.262 | 1.851 | 1.069 |
| Occlusion | 0 | y | 5.214 | 0.499 | 0.716 | 0.130 | 1.217 | 0.699 |
| Random Baseline | 0 | y | 3.570 | 0.425 | 0.458 | 0.203 | 1.585 | n/r |

local sensitivities resembling scaled original-network gradients. Reducing $\alpha$ from 0.5 to 0.4 improves both configurations' Pointing Game and Attribution Localization, while SplitCAM's $PF_{20}^{drop}$ falls from 1.607 to 1.585. SplitGrad peaks in $PF_{20}^{drop}$ at $\alpha = 0.4$ and localization at $\alpha = 0.35$; SplitCAM's Pointing Game and Attribution Localization peak at 0.4 and 0.35, respectively. Both minimize Maximum Sensitivity at $\alpha = 0$. The sweep supports metric-dependent $\alpha$ choices; numerical stabilization and attribution-rule changes remain confounded. Section S.4.2 of the supplement (Table S.4.5) gives the full grid and compares convex with winner-takes-it-all max-pooling.

For the same SplitCAM configuration, Tables 7.1 and 7.3 retain conflicting source values: $PF_{20}^{drop} = 1.590$ versus 1.585 and MaxS= 0.456 versus 0.457. Resolving this discrepancy requires unavailable per-image results.

**Forward-pass stabilization.** Without forward stabilization (Section 3.6), late VGG-16 split activations grow to order $10^{18}$, producing errors of order $10^9$; SplitLRP produces nearly uniform maps. Scaling outperforms shifting on every reported metric in the two early-layer SplitLRP configurations. At later layers, shifting gives higher Attribution Localization and Pixel-Flipping score-drop

**Tab. 7.2:** Selected interpretability evaluation configurations (ResNet-18, ImageNet-S test split). Highlighting identifies the best values among the displayed rows; the full comparison is in Table S.4.2. The triples $x.y.z$ are a ResNet-18-internal indexing of the convolutional layer at which the attribution maps are calculated. See the caption of Table 7.1 for further abbreviations and table explanation.

| Method | Layer | Abs | Sel↓ | AL↑ | PG↑ | $PF_5^{drop}$↑ | $PF_{20}^{drop}$↑ | MaxS↓ |
|---|---|---|---|---|---|---|---|---|
| *SplitCAM Methods* | | | | | | | | |
| SplitCAM (sc, $\alpha = 0.3$, wta) | (1.0.2, g) | n | **3.503** | 0.487 | 0.747 | 0.168 | 1.395 | 1.277 |
| SplitCAM (sc, $\alpha = 0.3$, wta) | (4.1.2, g) | n | 5.654 | **0.673** | **0.972** | 0.081 | 0.880 | 0.233 |
| SplitGrad ($\alpha = 0.3$, wta) | (1.0.2, g) | y | 4.449 | 0.513 | 0.777 | 0.149 | 1.266 | 0.813 |
| SplitGrad ($\alpha = 0.3$, wta) | (4.1.2, g) | y | 5.663 | 0.649 | 0.972 | 0.082 | 0.882 | 0.219 |
| SplitLRP (sc, wta) | (conv1, pos) | y | 6.313 | 0.506 | 0.587 | 0.056 | 0.697 | 0.755 |
| *Baselines (With Layer Optimization)* | | | | | | | | |
| GCAM | 4.1.2 | n | 6.070 | 0.554 | 0.945 | 0.076 | 0.829 | **0.131** |
| LCAM | 2.0.2 | n | 3.690 | 0.549 | 0.865 | 0.144 | 1.428 | 0.913 |
| LCAM | 4.1.1 | n | 5.661 | 0.664 | 0.971 | 0.083 | 0.890 | 0.227 |
| LRP $\gamma = 0.25$ | conv1 | y | 3.550 | 0.656 | 0.852 | **0.213** | 1.737 | 0.521 |
| *Baselines* | | | | | | | | |
| Guided Backprop | conv1 | n | 3.517 | 0.621 | 0.867 | 0.212 | **1.757** | 0.459 |
| GradientSHAP | conv1 | y | 5.269 | 0.527 | 0.779 | 0.166 | 1.227 | 1.074 |
| IntGrad | conv1 | y | 5.265 | 0.528 | 0.777 | 0.167 | 1.226 | 0.994 |
| Feature Ablation | conv1 | n | 4.655 | 0.136 | 0.684 | 0.188 | 1.388 | 1.077 |
| Occlusion | conv1 | n | 5.300 | 0.179 | 0.726 | 0.130 | 1.030 | 0.785 |
| Deconvolution | conv1 | n | 3.934 | 0.484 | 0.666 | 0.147 | 1.203 | 0.751 |
| Random Baseline | n/a | y | 4.071 | 0.425 | 0.440 | 0.147 | 1.067 | n/r |

**Tab. 7.3:** Comparison of different $\alpha$ parameters with two SplitGrad and SplitCAM base configurations, both in *wta* and *abs* mode, evaluated on the custom ImageNet-S test split.

| **Method** | **Metric** | $\alpha$ .50 | $\alpha$ .45 | $\alpha$ .40 | $\alpha$ .35 | $\alpha$ .30 | $\alpha$ .00 |
|---|---|---|---|---|---|---|---|
| **SplitGrad** (26, g) | PG↑ | 0.898 | 0.906 | 0.947 | **0.956** | 0.889 | 0.401 |
| | AL↑ | 0.543 | 0.548 | 0.605 | **0.617** | 0.535 | 0.449 |
| | MaxS↓ | 0.616 | 0.607 | 0.440 | 0.238 | 0.334 | **0.028** |
| | $PF_{20}^{drop}$ ↑ | 1.527 | 1.534 | **1.581** | 1.448 | 1.317 | 1.247 |
| **SplitCAM** (26, +, g) | PG↑ | 0.868 | 0.868 | **0.938** | 0.933 | 0.613 | 0.855 |
| | AL↑ | 0.550 | 0.554 | 0.635 | **0.663** | 0.403 | 0.513 |
| | MaxS↓ | 0.691 | 0.682 | 0.457 | 0.265 | 0.251 | **0.120** |
| | $PF_{20}^{drop}$ ↑ | 1.607 | **1.609** | 1.585 | 1.499 | 0.853 | 1.507 |

AUC, scaling lowers Maximum Sensitivity, and Pointing Game depends on configuration (supplement Table S.4.6; unstabilized and shift-stabilized SplitCAM grids: Tables S.4.7 and S.4.8).

## 7.5 Game-Based Case Study

**Setup.** We evaluate RG and SG against standard backward baselines (Section S.4.3.1) on VGG-16 (Simonyan and Zisserman, 2015), ResNet-50 (He et al., 2016), and ViT-B/16 (Dosovitskiy et al., 2021), using PyTorch (Paszke et al., 2019) and torchvision ImageNet-1K checkpoints (Deng et al.,

2009; Russakovsky et al., 2015)[1], on a 50-class ImageNet-S subset (Gao et al., 2023) at 224×224 (50-image validation split, 565-image test split), and on Pascal VOC 2012 (Everingham et al., 2010)[2]. For the game campaign, the 752 released ImageNet-S50 annotation records are filtered to 616 whose class agrees with the official ILSVRC2012 label; one image per class is selected for validation with seed 42, and one of the remaining 566 test images is excluded because its object mask is outside the center crop. The explanation target is the dataset class, including on misclassified images (Section A.2). ImageNet-S provides the pixel masks required by the localization metrics; ImageNet boxes over-credit mass anywhere inside the box. All metrics are computed through Quantus (Hedström et al., 2023) (definitions in Section A.2.2): localization (*Attribution Localization* (AL), *Pointing Game* (PG), Top-$k$ Intersection (TK); ↑), faithfulness (Pixel Flipping area under the remaining-logit curve over the first 5% and 20% of flips ($\mathrm{PF}_5$, $\mathrm{PF}_{20}$), *Selectivity* (Sel); ↓), robustness (Maximum Sensitivity (MaxS), Average Sensitivity (AvgS); ↓). Baselines run in their reference implementations (lxt, zennit, captum, pytorch-grad-cam, and the authors' code for TIBAV and GAE) at the authors' settings, plus declared grids (Achtibat et al., 2024; Anders et al., 2026; Chefer et al., 2021a,b; Gildenblat and contributors, 2021; Kokhlikyan et al., 2020). RG analytical modes are reported alongside each configuration: temperature $\tau$, risk aversion $\lambda$ (softmax-shift variant $\lambda_{\mathrm{sm}}$ on ViT attention logits), LRP stabilizer $\varepsilon$, equivariant-Reynolds sample count $N$.

**Protocol.** Methods are evaluated under a two-stage protocol that treats baselines and our games alike. Stage 1 runs a grid search on the validation split (one image per class) for every method with tunable parameters, including DAVE (Section S.4.3.2, Table S.4.9; 2,021 configurations in total). Stage 2 picks, for each method, the *loc* configuration with the best localization rank-sum over $\{\mathrm{AL}, \mathrm{PG}, \mathrm{TK}\}$ and, independently, the *faith* configuration with the best faithfulness rank-sum over $\{\mathrm{PF}_5, \mathrm{PF}_{20}, \mathrm{Sel}\}$, and evaluates both once on the test split, with 95% percentile-bootstrap confidence intervals (Efron and Tibshirani, 1993) and Holm-corrected (Holm, 1979) paired tests against the best row of each metric (Section A.2.4); for RG / RG+Eq we additionally show the $\alpha\beta$-LRP anchor $(\alpha,\beta,\varepsilon,\tau)=(2,1,0.5,1)$ as a theorem-driven reference (Theorem 4.2). Localization is a natural target for modes designed to control routing focus, and object annotations provide external ground truth: Kohlbrenner et al. (2020) compare LRP rule composites by the share of positive relevance inside ground-truth bounding boxes; we use absolute attribution mass and pixel masks. The main table reports the loc row of every method; since mask agreement is not faithfulness, the faith rows of all methods are reported in Section S.4.4.

[1] Checkpoint documentation: https://docs.pytorch.org/vision/stable/models.html; the weight identifiers used here are listed in Section S.4.3.

[2] Release documentation: https://www.robots.ox.ac.uk/~vgg/projects/pascal/VOC/voc2012/.

**Tab. 7.4:** ViT-B/16 on the ImageNet-S test split ($n$=565; MaxS/AvgS on a fixed random subset of 200 test images). **Bold**, dark shading: best in column; ≈, light shading: not significantly different from the best (paired bootstrap / McNemar, Holm-corrected, $p \geq 0.05$); small numbers: 95% percentile-bootstrap confidence intervals over test images (Section A.2.4). One row per method, selected on the validation split by the localization rank-sum (faithfulness-selected rows and further baselines: Section S.4.4); the RG anchor is the exact $\alpha\beta$-LRP representation (Theorem 4.2). $t$/img: seconds per image on one GPU. Config: SmoothGrad averages $N$ samples at noise level $\sigma$ relative to the input range; LRP-$\gamma$ has stabilizer $\varepsilon$ and positive-weight boost $\gamma$; AttnLRP has layer-type parameters $\gamma_{\text{conv}}, \gamma_{\text{lin}}, \varepsilon_{\text{attn}}$; KernelSHAP draws $N$ samples for $M$ superpixels; RG modes as in Table 4.1.

| Method | Config | AL↑ | PG↑ | TK↑ | $PF_5$↓ | $PF_{20}$↓ | Sel↓ | MaxS↓ | AvgS↓ | $t$/img |
|---|---|---|---|---|---|---|---|---|---|---|
| Gradient | fixed | 0.411 [0.389,0.432] | 0.361 [0.322,0.402] | 0.386 [0.359,0.413] | 8.45 [8.33,8.56] | 8.08 [7.94,8.22] | 6.033 [5.905,6.154] | 1.290 [1.233,1.351] | 0.922 [0.893,0.953] | **0.02** |
| SmGrad | $N$=50,$\sigma$/range=0.2 | 0.517 [0.498,0.536] | 0.706 [0.667,0.743] | 0.675 [0.654,0.694] | 8.10 [7.96,8.24] | 7.02 [6.83,7.20] | 5.029 [4.894,5.163] | 0.597 [0.569,0.630] | 0.468 [0.451,0.485] | 1.12 |
| LRP-$\gamma$ | $\varepsilon$=0.1,$\gamma$=2 | 0.644 [0.628,0.660] | 0.873 [0.844,0.899] | 0.816≈ [0.800,0.832] | 7.93 [7.78,8.08] | 6.64 [6.44,6.83] | **4.451** [4.324,4.572] | 0.300 [0.287,0.314] | 0.218 [0.210,0.226] | 0.08 |
| AttnLRP | $\gamma_{\text{conv}}$=0.25,$\gamma_{\text{lin}}$=0.05,$\varepsilon_{\text{attn}}$=0.25 | 0.644 [0.628,0.660] | 0.855 [0.825,0.883] | 0.809 [0.794,0.825] | **7.77** [7.61,7.93] | **6.43** [6.23,6.62] | 4.582 [4.452,4.706] | 0.456 [0.430,0.484] | 0.342 [0.324,0.364] | 0.10 |
| TIBAV | fixed | 0.615 [0.597,0.634] | 0.878 [0.851,0.904] | 0.782 [0.759,0.805] | 8.13 [8.01,8.24] | 7.60 [7.46,7.74] | 4.906 [4.774,5.033] | 1.062 [1.001,1.126] | 0.512 [0.486,0.540] | 0.15 |
| GAE | fixed | 0.533 [0.514,0.551] | 0.743 [0.708,0.779] | 0.649 [0.623,0.677] | 8.13 [8.01,8.25] | 7.62 [7.47,7.76] | 5.098 [4.961,5.231] | 0.525 [0.494,0.558] | 0.368 [0.346,0.391] | 0.03 |
| KernelSHAP | $N$=2$M$+2048, 80 superpixels | 0.565 [0.548,0.583] | **0.961** [0.945,0.977] | 0.823≈ [0.799,0.848] | 8.29 [8.18,8.40] | 7.50 [7.35,7.65] | 5.140 [5.012,5.261] | 0.897 [0.868,0.928] | 0.650 [0.632,0.667] | 6.71 |
| RG anchor | $\tau$=1 | 0.509 [0.490,0.528] | 0.724 [0.685,0.759] | 0.699 [0.678,0.720] | 8.19 [8.05,8.31] | 6.97 [6.78,7.15] | 4.854 [4.732,4.972] | **0.166** [0.161,0.172] | 0.120 [0.117,0.124] | 0.10 |
| RG | $\tau$=0.4,$\lambda_{\text{sm}}$=−10,$\sigma^2$=5,$\varepsilon$=0.1 | **0.711** [0.692,0.730] | 0.825 [0.793,0.855] | 0.788 [0.768,0.808] | 8.27 [8.14,8.39] | 7.48 [7.31,7.64] | 5.109 [4.987,5.226] | 0.729 [0.685,0.777] | 0.494 [0.471,0.518] | 0.17 |
| RG+Eq anchor | $\tau$=1,$N$=50 | 0.520 [0.501,0.539] | 0.761 [0.726,0.796] | 0.752 [0.729,0.775] | 8.41 [8.29,8.52] | 7.72 [7.56,7.87] | 4.864 [4.742,4.981] | 0.170≈ [0.165,0.175] | **0.115** [0.112,0.118] | 4.93 |
| RG+Eq | $\tau$=0.5,$\lambda_{\text{sm}}$=−10,$\sigma^2$=2,$N$=50 | 0.698 [0.680,0.717] | 0.855 [0.827,0.883] | **0.836** [0.815,0.857] | 8.39 [8.28,8.50] | 7.80 [7.65,7.94] | 5.010 [4.889,5.127] | 0.441 [0.421,0.463] | 0.301 [0.290,0.313] | 8.66 |

**ViT-B/16 (Table 7.4).** Under the localization-selected protocol, the RG family beats every baseline on attribution localization, including the prior transformer-specific backward methods AttnLRP (Achtibat et al., 2024), our DAVE implementation (Wróbel et al., 2026) (Section S.4.3.1), TIBAV (Chefer et al., 2021b) and GAE (Chefer et al., 2021a): the selected mode reaches AL 0.711 against 0.644 for the best baseline (LRP-$\gamma$), and RG+Eq 0.698, both significantly ahead of every baseline row after Holm correction. RG+Eq also has the highest observed TK (0.836), but the Holm-corrected tests do not detect a difference from KernelSHAP at 0.823 or LRP-$\gamma$ at 0.816. The pointing game is led by KernelSHAP (0.961), whose maps are constant on superpixels: a hit needs only one pixel of the maximal region inside the mask. Scoring a single pixel drawn from that region instead, KernelSHAP falls below RG+Eq, which is then not significantly below the best baseline (LRP-$\gamma$; Section S.4.4.6). Among the localization-selected rows, faithfulness favors the LRP rules: AttnLRP is best on $PF_5$ and $PF_{20}$ and LRP-$\gamma$ on Sel ($PF_{20}$ 6.43 versus 7.48 for RG), and the faithfulness-selected RG row narrows but does not close the gap ($PF_{20}$ 6.78). Across both selection objectives, faithfulness-selected AttnLRP has the lowest $PF_5$, $PF_{20}$ and Selectivity, at 7.64, 6.12 and 4.441, respectively. The two anchors are the most robust rows (MaxS 0.166 and 0.170 versus 0.208 for AR), whereas the localization-selected mode reaches MaxS 0.729: sharper routing trades robustness and faithfulness for localization. ADF adds modest overhead (RG: 0.17 s vs. 0.10 s base), still below sampling baselines like SmoothGrad or IntGrad; Reynolds averaging is the only expensive variant (8.66 s), exceeding even the perturbation method KernelSHAP (6.71 s).

**Higher $\tau$ spreads attribution into context.** Localization-optimal modes concentrate mass on the few patches aligning with the mask. Increasing $\tau$ spreads equilibrium routing over more predecessors: the displayed examples retain object focus while extending attribution into context, without establishing

which contextual features the model uses. Around the selected ViT-B/16 setting, AL decreases monotonically from 0.723 at $\tau$=0.3 to 0.461 at $\tau$=1; faithfulness is non-monotonic. High $\tau$ shows how localization scores favor concentrated maps within the same game. User-perceived informativeness remains untested.

**Cross-dataset transfer (Pascal VOC).** A natural worry with validation-tuned modes is that the chosen $\tau$, $\lambda_{\text{sm}}$ and $\sigma^2$ are overfitted to the ImageNet-S 50-class subset. We test transfer on a per-image disjoint split of Pascal VOC 2012 trainval (2,743 test (image, foreground-class) samples with pixel-accurate segmentation masks): RG and RG+Eq use the *same* mode parameters as Table 7.4 without any VOC-side re-tuning, while every baseline gets a fresh per-method validation grid on the new pool. This tests the portability of the selected parameters across these datasets for the same pretrained model, without identifying whether the underlying structure is model-specific or dataset-specific. The localization lead carries over (AL 0.459 for RG and 0.456 for RG+Eq versus 0.413 for TIBAV, both significant against every baseline row), while TK and PG are led by TIBAV and the perturbation methods, and faithfulness by the LRP rules (Section S.4.4.4).

**VGG-16 and ResNet-50 (Section S.4.4).** On CNNs, where LRP-type rules are strong, localization-selected RG leads AL within that selection: clearly on VGG-16 (0.685 versus 0.613 for LRP-$\varepsilon$) and narrowly on ResNet-50 (0.719 versus 0.711 for LRP-$\gamma$). The ResNet-50 difference is not Holm-significant (adjusted $p = 0.420$), so its numerical lead does not establish superiority. Across both objectives, VGG-16's strongest baseline AL is faithfulness-selected LRP-$\varepsilon$ at 0.639, below RG's 0.685. Pixel Flipping favors LRP-$\varepsilon$ for VGG-16 $\text{PF}_5$ (12.43), LRP-$\gamma$ for VGG-16 $\text{PF}_{20}$ (7.89 versus LRP-$\varepsilon$'s 8.17), and LRP-$\gamma$ for both ResNet-50 metrics; CAM and perturbation methods lead TK and PG. VGG-16's risk-averse SG is most robust (MaxS 0.278 versus 0.314 for GCAM++).

### 7.5.1 Game-Method Ablations

Section S.4.4.5 varies one mode around each validation-selected configuration on 565 test images; these diagnostics do not affect selection. Section S.4.4.6 controls Pointing Game for map granularity, and Section S.4.4.7 compares composite Value-routing attention with a gradient fallback. The analyses reveal trade-offs among mask agreement, perturbation faithfulness and robustness.

# 8 The Trajectory Distribution View

Chapter 6 turns internal signals and backward routing choices into attribution maps; Chapter 7 evaluates them. Their use in interventions also requires examining the underlying calculation. The equilibrium policies of Chapter 4 induce backward-trajectory distributions, enabling comparison before projection onto input pixels. We study their Hellinger distance, its exact per-input-state disagreement map, and complete versus survival-conditioned laws. Spectral analysis diagnoses amplification and target discrimination through the attribution redistribution operator's singular spectrum at a fixed input (Maeß et al., 2026); our experiments compare normalized routing under input noise and parameter randomization.

## 8.1 Comparison with Normalized Relevance Measures

The NRM framework of Xiong et al. (2026), by Ping Xiong, Thomas Schnake, Grégoire Montavon, Klaus-Robert Müller, and Shinichi Nakajima, studies backward calculations through a normalized signed measure on complete neuron walks. Summing walk relevance through selected neurons or concepts gives their relevance; conditional relevance divides joint by nonzero marginal relevance. With suitable normalized propagation rules, this covers ordinary LRP, concept relevance, and Concept Relevance Propagation (CRP) (Achtibat et al., 2023). Their CRP quantity is joint relevance of an input neuron, intermediate concept neuron, and target output, without an automatic conditional-probability interpretation.

Both views retain network paths before forming attribution maps; their factorization supports sum-product computation and queries across layers. In NRM, individual walk relevance may be negative despite unit total. Our game policies define a non-negative probability law $\Pi$ on the expanded graph, including player labels and termination. Signed differences of its discounted occupation measures recover gradients and relevance scores (Chapter 4), generally distinct from unweighted marginals. Player expansion thus separates trajectory probability from contribution sign and scale, without identifying $\Pi$ with a given NRM measure.

The NRM construction normalizes propagation columns under positive-sum assumptions; identifying it with globally normalized original walk relevance additionally requires the constant-sum conditions of Xiong et al. (2026, Section 3.5). Our transitions remain normalized; discounts retain scale, with explicit bias and stabilizer absorption. Hellinger distance applies directly to these probability laws, but generally not to signed relevance measures. Conditioning $\Pi$ on a concept requires positive probability; signed relevance can instead vanish by cancellation.

Symbolic XAI scores logical feature queries by selecting signed terms from a multi-order prediction decomposition (Schnake et al., 2025). SemanticLens combines component attributions and semantic embeddings to audit expected and spurious visual concepts (Dreyer et al., 2025). These address feature relationships and human interpretation; trajectory probabilities describe normalized routing. We propose querying game trajectories through concepts, retaining attribution signs and discounts, and comparing their laws across inputs or models. Equivalence with a particular NRM/CRP construction remains unproved and would require matching propagation rules, normalization, and absorption conventions.

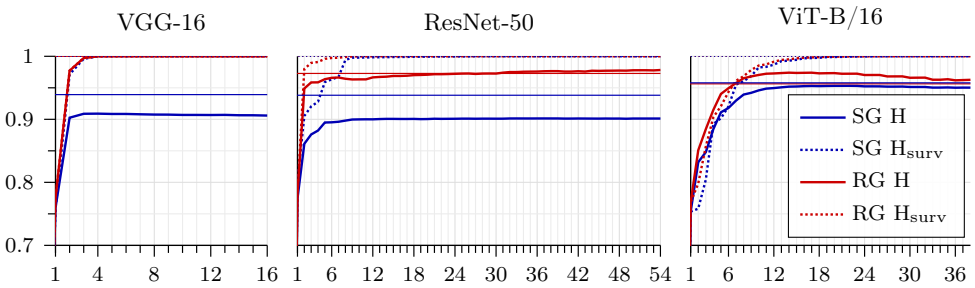



**Fig. 8.1:** Hellinger trajectory distance under cascading parameter randomization (Adebayo et al., 2018). Hellinger distance H (vertical axis, from 0.7) against the number of cascadingly re-initialized layers, output to input (horizontal axis; H = 0 before the first layer is randomized). Solid: plain H; dotted: $H_{surv}$, conditioned on survival to an ordinary input-layer state. Thin horizontal reference lines mark the double-random baseline values from Table 8.3 (solid: plain H; dotted: $H_{surv}$), in the color of the corresponding game.

## 8.2 Trajectory Sanity Check

The cascading-randomization sanity check of Adebayo et al. (2018) tests explanations' dependence on learned parameters by comparing pixel-space maps. Here we examine normalized backward routing: a trajectory distribution $\Pi$ whose model-dependent signed discounted occupation projection recovers attribution. The **Hellinger distance** (Hellinger, 1909) $H(\Pi_A, \Pi_B) = (1 - BC)^{1/2}$, with Bhattacharyya coefficient (Bhattacharyya, 1943) $BC = \sum_\zeta \sqrt{\Pi_A(\zeta)\,\Pi_B(\zeta)}$, directly compares routing. We test whether parameter randomization moves $\Pi$ even when attribution maps remain stable. The metric satisfies $H = 0$ iff $\Pi_A = \Pi_B$ and $H = 1$ iff their supports are disjoint. It remains finite when routes open or close, unlike KL-type divergences. For layered Markov laws on the same graph, BC factorizes: one backward pass computes H exactly in edge-linear time, and $H^2$ decomposes into per-input-state contributions (Section 8.3).

### 8.2.1 Trajectories change, maps need not

Under cascading randomization (layers cumulatively re-initialized output→input), SG and RG diverge from the first randomized layer and plateau near the double-random reference (Table 8.3, Figure 8.1). This reference lies below H = 1 because models randomized over the tested layers still share short trajectories: both may stop at structurally dead neurons and enter the common cemetery. Such paths coincide more readily than full input-reaching trajectories. Conditioning on survival removes shared cemetery mass, yielding $H_{surv} \approx 1$. Trajectory and map readouts have weak observed per-image rank association (Spearman $|\rho| \leq 0.19$ between H and pixel-level Gradient or $\alpha\beta$-LRP similarity): similar maps need not imply similar calculations. Section 8.4 details per-game values, conditioning, the floor mechanism, and per-image analysis.

**What the map hides.** Map-level checks under parameter randomization reproduce Adebayo et al. (2018) and Sixt et al. (2020): some methods retain structure at the cascade endpoint. On ViT-B/16, the RG anchor retains Spearman $|\rho| = 0.454$, highest among all methods, and HOG $r = 0.469$; VGG-16 LRP-$\varepsilon$ retains 0.341 and 0.457 (Table 8.2). Localization-selected RG modes are slightly more model sensitive than the anchor on all three models. In Figure 8.1, H reaches the double-random reference while maps retain structure. The map-level failure remains; H detects routing changes obscured by the map. Since discounted occupation projections also vary across models, this discrepancy does not isolate loss under a fixed projection.

## 8.3 Hellinger Distance and Disagreement Map

We compare two layered Markov processes on a common state graph and seek both a scalar divergence of the trajectory distributions and an exact per-pixel decomposition of this divergence at the input layer. We use the squared Hellinger distance because the geometric mean $\sqrt{PQ}$ preserves the multiplicative structure of Markov trajectory distributions and yields an exact linear-time dynamic program. Unlike Jeffreys divergence, it remains finite under support mismatch; unlike Jensen–Shannon, it admits an exact recursion on the layered graph. Later, we decompose the squared Hellinger distance as a sum of non-negative per-pixel contributions over all input-layer states, giving a disagreement map at the input layer (Theorem 8.1).

The formal foundations and proofs are given in companion Chapter S.3. We compare two processes on a common finite layered state graph with a shared deterministic starting state $s_0$. At layer $l$, $\mathscr{S}_+^{(l)} = \mathscr{S}^{(l)} \cup \{\dagger\}$ contains the ordinary states and an absorbing cemetery state; $T_M^{(l)}$ is the transition kernel from layer $l$ to $l-1$, and $m_M^{(l)}(s)$ is the marginal probability of visiting $s$. The full trajectory law is $\Pi_M = \Pi_M^{(0)}$. For the RG, full-law diagnostics retain the gate, branch, and routing substeps; only ordinary surviving paths admit exact act-only elimination (Remark S.3.1).

### 8.3.1 Bhattacharyya Coefficient and the Geometric-Mean Kernel

**Definitions.** For two probability distributions $P, Q$ on a common finite set, the *Bhattacharyya coefficient* is the geometric overlap

$$\mathrm{BC}(P,Q) \;:=\; \sum_y \sqrt{P(y)\,Q(y)} \;\in\; [0,1], \tag{8.1}$$

and the *squared Hellinger distance* is

$$\mathrm{H}^2(P,Q) \;:=\; \tfrac{1}{2}\sum_y \big(\sqrt{P(y)} - \sqrt{Q(y)}\big)^2 \;=\; 1 - \mathrm{BC}(P,Q) \;\in\; [0,1]. \tag{8.2}$$

The Hellinger distance $\mathrm{H}(P,Q) = \sqrt{1-\mathrm{BC}(P,Q)}$ is a true metric: $0 \le \mathrm{H} \le 1$, with $\mathrm{H} = 0$ iff $P = Q$ and $\mathrm{H} = 1$ iff $P$ and $Q$ have disjoint supports. Crucially, $\sqrt{0 \cdot Q(y)} = 0$, so mismatched supports cause no infinity—only a smaller overlap.

**Geometric-mean kernel.** For each layer $l \ge 1$ and pair of states $(s, s') \in \mathscr{S}_+^{(l)} \times \mathscr{S}_+^{(l-1)}$, define the *geometric-mean kernel*

$$\widetilde{T}^{(l)}(s,s') \;:=\; \sqrt{T_A^{(l)}(s \to s')\; T_B^{(l)}(s \to s')}. \tag{8.3}$$

This kernel is non-negative but in general sub-stochastic. Its row sum

$$\widetilde{R}^{(l)}(s) \;:=\; \sum_{s' \in \mathscr{S}_+^{(l-1)}} \widetilde{T}^{(l)}(s,s') \tag{8.4}$$

satisfies $0 \le \widetilde{R}^{(l)}(s) \le 1$ by Cauchy–Schwarz, with equality iff $T_A^{(l)}(s \to \cdot) = T_B^{(l)}(s \to \cdot)$. Cemetery handling is automatic: $T_A^{(l)}(\dagger \to \dagger) = T_B^{(l)}(\dagger \to \dagger) = 1$, so $\widetilde{T}^{(l)}(\dagger,\dagger) = 1$ and $\widetilde{R}^{(l)}(\dagger) = 1$.

### 8.3.2 Hellinger backward pass

The chain rule (Lemma S.3.1) yields a backward DP that propagates the per-state cumulative $\widetilde{T}$-mass $g^{(l)}(s)$ from the start state $s_0$ down through the layers. The prefix Bhattacharyya coefficient at each layer is the row sum $\mathrm{BC}^{(l)} = \sum_s g^{(l)}(s)$, the prefix Hellinger distance is $\mathrm{H}^{(l)} = \sqrt{1 - \mathrm{BC}^{(l)}}$, and the per-terminal-state disagreement map is read off at $l = 0$ from (8.5). The map is defined on the full terminal set $\mathscr{S}_+^{(0)} = \mathscr{S}^{(0)} \cup \{\dagger\}$; the visualized pixel attribution map is its restriction to ordinary pixels $\mathscr{S}^{(0)}$, and its sum is therefore $\mathrm{H}^2_{\mathrm{ord}} = \mathrm{H}^2 - \mathrm{H}^2(\dagger)$, not the full $\mathrm{H}^2$ (Theorem 8.1(iii)).

**Algorithm 8.1:** Hellinger backward pass (with per-terminal-state disagreement map)

Initialize the occupation marginals $m_M^{(L)}$ and overlap mass $g^{(L)}$ at the target output.;
For $l = L, L-1, \ldots, 1$, propagate each model's marginals with its kernel and propagate overlap by $g^{(l-1)}(s') = \sum_s g^{(l)}(s) \sqrt{T_A^{(l)}(s,s') T_B^{(l)}(s,s')}$.;
At each layer compute $\mathrm{BC}^{(l)} = \sum_s g^{(l)}(s)$ and $\mathrm{H}^{(l)} = \sqrt{1 - \mathrm{BC}^{(l)}}$.;
At every terminal state return $\mathrm{H}^2(s) = \frac{1}{2}(m_A^{(0)}(s) + m_B^{(0)}(s)) - g^{(0)}(s)$. The pixel disagreement map restricts this output to ordinary input states.

### 8.3.3 Per-Pixel Disagreement Map

The squared Hellinger distance $\mathrm{H}^2$ admits an exact non-negative decomposition over input-layer states: each input pixel carries a calibrated share of the total trajectory disagreement (Theorem 8.1), giving a disagreement *map* that is itself an attribution of the trajectory mismatch back to the input. The conditioned-on-survival variant removes the cemetery contribution that otherwise bounds plain H from below on randomized models.

#### Per-Pixel Decomposition: The Disagreement Map

The squared Hellinger distance at layer 0 decomposes exactly over terminal pixels.

**Theorem 8.1 (Per-pixel decomposition, `Lean_85`).** *Let* $\mathrm{H}^2 := \mathrm{H}^2(\Pi_A^{(0)}, \Pi_B^{(0)})$. *For* $s \in \mathscr{S}_+^{(0)}$, *define*

$$\mathrm{H}^2(s) \;:=\; \tfrac{1}{2}\big(m_A^{(0)}(s) + m_B^{(0)}(s)\big) \;-\; g^{(0)}(s), \tag{8.5}$$

*where* $g^{(0)}(s)$ *is the cumulative* $\widetilde{T}$*-mass at* $s$. *Then:*

*(i)* $\mathrm{H}^2(s) \geq 0$ *for every* $s \in \mathscr{S}_+^{(0)}$.
*(ii)*

$$\mathrm{H}^2 \;=\; \sum_{s \in \mathscr{S}_+^{(0)}} \mathrm{H}^2(s). \tag{8.6}$$

*(iii)*

$$\mathrm{H}^2_{\mathrm{ord}} \;:=\; \sum_{s \in \mathscr{S}^{(0)}} \mathrm{H}^2(s) \;=\; \mathrm{H}^2 - \mathrm{H}^2(\dagger) \tag{8.7}$$

Proofs of the backward pass and the decomposition are given in companion Chapter S.3.

**The disagreement map.** Theorem 8.1 defines $\mathrm{H}^2(\cdot)$ on the full terminal set $\mathscr{S}_+^{(0)} = \mathscr{S}^{(0)} \cup \{\dagger\}$: this is the *full terminal-state decomposition*, with $\sum_{s\in\mathscr{S}_+^{(0)}} \mathrm{H}^2(s) = \mathrm{H}^2_{(0)}$, and the cemetery contribution $\mathrm{H}^2(\dagger)$ is part of it. The *Hellinger disagreement map* is the ordinary-pixel restriction $s \mapsto \mathrm{H}^2(s)$ on $\mathscr{S}^{(0)}$, which is the quantity we visualize. It is non-negative, sums to $\mathrm{H}^2_{\mathrm{ord}}$ over ordinary pixels, and is computed from $g^{(0)}, m_A^{(0)}, m_B^{(0)}$. The quantity $\mathrm{H}^2_{\mathrm{ord}}$ is an unnormalized ordinary-state subsum; the conditioned-on-survival Hellinger distance that normalizes it is introduced below. Recall from Section S.3.1.2 that layer-0 ordinary states of the two games are *tagged* pixel pairs $s^{(0)}_{j,\pm}$, tagged by the player $q$ for the SG and by the stream sign $\kappa$ for the RG; the per-pixel attribution displayed in figures aggregates the two tags,

$$\mathrm{H}^2_{\mathrm{pixel}}(j) \;=\; \mathrm{H}^2\big(s^{(0)}_{j,+}\big) + \mathrm{H}^2\big(s^{(0)}_{j,-}\big), \tag{8.8}$$

which remains non-negative and sums to $\mathrm{H}^2_{\mathrm{ord}}$ over pixels. By the Cauchy–Schwarz step in the proof of Theorem 8.1(i), $g^{(0)}(s) \le \sqrt{m_A^{(0)}(s)\, m_B^{(0)}(s)}$, so $\mathrm{H}^2(s)$ is bounded below by the marginal surrogate $\frac{1}{2}\big(m_A^{(0)}(s) + m_B^{(0)}(s)\big) - \sqrt{m_A^{(0)}(s)\, m_B^{(0)}(s)}$, the per-pixel squared Hellinger distance between the layer-0 marginal occupations; the trajectory map is strictly larger whenever the conditional distributions of trajectories reaching pixel $s$ differ between models.

#### Conditioning on Survival to the Ordinary Input Layer

When the cemetery absorbs a large fraction of trajectory mass, the total Hellinger distance mixes two effects: disagreement about *whether* a trajectory survives and disagreement about *where surviving trajectories go*. To isolate the second effect, we condition each model's trajectory law on the event that the trajectory reaches an ordinary input-layer state. The resulting scalar compares the two *surviving* trajectory distributions directly and removes the common cemetery floor. Because survival to the input layer factorizes along the layered graph, this conditioning remains Markov and is implemented by a per-state Doob $h$-transform (with $h = \psi_M$, the remaining survival probability).

Let $\mathsf{Surv}$ denote this event and assume that both survival masses are positive. With

$$Z_M = \mathbb{P}_M(\mathsf{Surv}) = \sum_{s\in\mathscr{S}^{(0)}} m_M^{(0)}(s) > 0, \qquad \mathrm{BC}^{(0)}_{\mathrm{ord}} = \sum_{s\in\mathscr{S}^{(0)}} g^{(0)}(s),$$

the conditioned-survival Hellinger distance is

$$\mathrm{H}^2_{\mathrm{surv}} = \mathrm{H}^2\big(\Pi_A(\,\cdot \mid \mathsf{Surv}), \Pi_B(\,\cdot \mid \mathsf{Surv})\big) = 1 - \frac{\mathrm{BC}^{(0)}_{\mathrm{ord}}}{\sqrt{Z_A Z_B}}.$$

Theorem S.3.4 gives the conditioned kernels and the proof; Equation (S.3.46) gives the corresponding non-negative map. Post-hoc normalization becomes numerically infeasible for deep networks when either survival probability gets too small; the conditioned-kernel computation avoids this final division.

### 8.3.4 Experiments

With the model fixed, we test sensitivity to input perturbations and separation among ImageNet-S classes and from random noise. We use the Stopping-Game kernel (S.3.4) without risk aversion ($\lambda = 0$; "ADF" in code) and Routing-Game kernel (S.3.5) at $\tau = 1$. The diagnostic RG omits the bias action and renormalizes the remaining branches, so it may differ from the bias-inclusive attribution game.

### Input-Noise Sensitivity

For each architecture and $\sigma \in \{0.005, 0.01, 0.02, 0.03, 0.05, 0.1, 0.2, 0.3, 0.5\}$, we draw one standard-Gaussian tensor $\eta$ per image and compute $\mathrm{H}(\Pi_A(x), \Pi_A(x + \sigma\eta))$ for both kernels, targeting the clean input's argmax. Mean H and $\mathrm{H}_{\mathrm{surv}}$ increase with $\sigma$ for both games on all three architectures (Figure 8.2). They coincide for VGG-16 RG, whose kernel never routes to dead-neuron states. ResNet-50 and ViT-B/16 skip connections reach such states; ViT-B/16's GELU gate also sends some trajectories to the cemetery at every state. Generalised Explanation Faithfulness (GEF) correlates explanation and model-output distortion along perturbation paths (Hedström et al., 2025), anchoring explanation changes to model responses. Our Hellinger curves measure routing sensitivity, without establishing this alignment or explanation faithfulness.

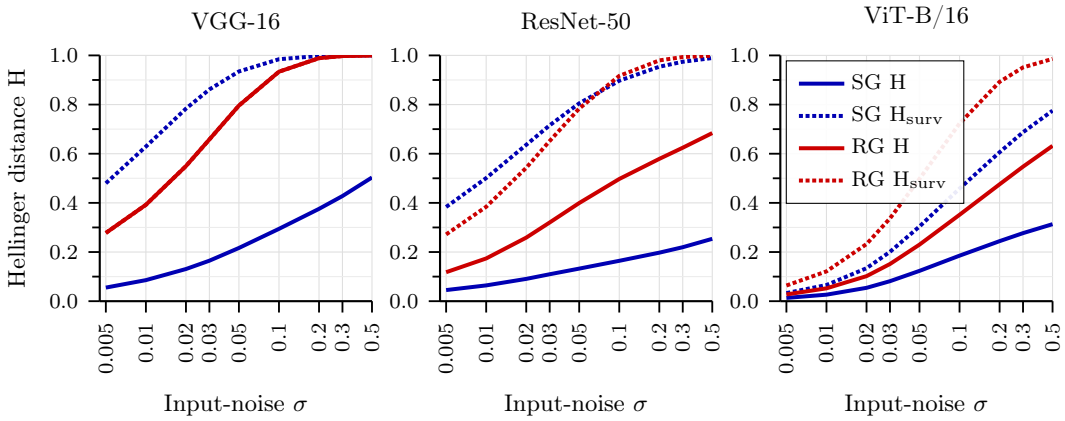



**Fig. 8.2:** Input-noise Hellinger trajectory distance. Solid: plain H; dotted: $\mathrm{H}_{\mathrm{surv}}$, conditioned on survival to an ordinary input-layer state. $x$-axis is log-scale input-noise $\sigma$; same $(A,A)$ model on both sides, only the input differs.

### Class Separability and Reference Baselines

Class separation depends on architecture and kernel (Table 8.1):

- On **VGG-16**, SG gives clean-intraclass H = 0.513 below clean-interclass H = 0.607. RG saturates at H = 1 throughout, resolving no class structure.
- On **ResNet-50**, RG separates clean-intraclass H = 0.786 from clean-interclass H = 0.902; SG does not (intraclass 0.272 vs. interclass 0.268).
- On **ViT-B/16**, RG gives intraclass 0.789 vs. interclass 0.882; the SG gap ($0.422 \pm 0.023$ vs. $0.435 \pm 0.017$) is smaller than the spread over pairs and does not separate classes.

Random-vs-random is lowest in every nonconstant column. Clean-vs-random is highest in each plain-H column except ResNet-50 SG (0.269 vs. intraclass 0.272) and saturated VGG-16 RG, consistent with different routing for Gaussian and in-distribution inputs. Conditioned $\mathrm{H}_{\mathrm{surv}}$ is at least 0.99 throughout VGG-16, ResNet-50, and ViT-B/16 RG, providing no class-separability signal. ViT-B/16 SG separates random-vs-random (0.852) from clean rows (0.957–0.968), but neither intraclass from interclass (0.963 vs. 0.968) nor clean-vs-random (0.957) from clean pairs.

| Architecture | Baseline | Stopping Game | | Routing Game | |
|---|---|---|---|---|---|
| | | H | $H_{surv}$ | H | $H_{surv}$ |
| VGG-16 | clean-intraclass | $0.513 \pm 0.071$ | $1.000 \pm 0.000$ | $1.000 \pm 0.000$ | $1.000 \pm 0.000$ |
| | clean-interclass | $0.607 \pm 0.032$ | $1.000 \pm 0.000$ | $1.000 \pm 0.000$ | $1.000 \pm 0.000$ |
| | clean-vs-random | $0.663 \pm 0.020$ | $1.000 \pm 0.000$ | $1.000 \pm 0.000$ | $1.000 \pm 0.000$ |
| | random-vs-random | $0.320 \pm 0.019$ | $1.000 \pm 0.000$ | $1.000 \pm 0.000$ | $1.000 \pm 0.000$ |
| ResNet-50 | clean-intraclass | $0.272 \pm 0.056$ | $0.999 \pm 0.000$ | $0.786 \pm 0.044$ | $1.000 \pm 0.000$ |
| | clean-interclass | $0.268 \pm 0.045$ | $1.000 \pm 0.000$ | $0.902 \pm 0.033$ | $1.000 \pm 0.000$ |
| | clean-vs-random | $0.269 \pm 0.035$ | $1.000 \pm 0.000$ | $0.943 \pm 0.010$ | $1.000 \pm 0.000$ |
| | random-vs-random | $0.152 \pm 0.008$ | $0.992 \pm 0.000$ | $0.739 \pm 0.011$ | $0.999 \pm 0.000$ |
| ViT-B/16 | clean-intraclass | $0.422 \pm 0.023$ | $0.969 \pm 0.008$ | $0.789 \pm 0.067$ | $1.000 \pm 0.000$ |
| | clean-interclass | $0.435 \pm 0.017$ | $0.974 \pm 0.005$ | $0.882 \pm 0.010$ | $1.000 \pm 0.000$ |
| | clean-vs-random | $0.439 \pm 0.014$ | $0.963 \pm 0.006$ | $0.891 \pm 0.007$ | $1.000 \pm 0.000$ |
| | random-vs-random | $0.377 \pm 0.013$ | $0.867 \pm 0.013$ | $0.476 \pm 0.018$ | $0.997 \pm 0.000$ |

**Tab. 8.1:** Reference baselines for the input-noise sweep, averaged over $R$ pairs drawn from the 565-image ImageNet-S test split (clean rows: $R = 200$ sampled pairs; clean-vs-random: $R = 565$ images paired with a fresh Gaussian each; random-vs-random: $R = 200$ Gaussian pairs); entries are mean $\pm$ standard deviation over pairs. **clean-intraclass**: $\mathrm{H}(\Pi_A(x_1), \Pi_A(x_2))$ with $x_1, x_2$ from the same ground-truth class; **clean-interclass**: same, with distinct classes; **clean-vs-random**: one side clean, other Gaussian; **random-vs-random**: both sides Gaussian.

## 8.4 Randomization Sanity Checks

This section supports the cascading-randomization experiments of Section 8.2: §8.4.1 extends the protocol of Adebayo et al. (2018) to trajectory Hellinger diagnostics (Section 8.3); §8.4.2 tests main-paper attribution maps; §8.4.3 gives the double-random reference; and §8.4.4 compares trajectory and map diagnostics per image.

### 8.4.1 Cascading Randomization in Trajectory Space

Replacing a weight matrix in the randomization test of Adebayo et al. (2018) yields two kernels on the same graph; the Hellinger pass of Section 8.3 quantifies their trajectory-space change. With its geometric-mean kernel $\widetilde{T}^{(l)}$ and overlap mass $g^{(l)}(s)$, $\mathrm{BC}^{(l)} = \sum_{s \in \mathscr{S}_+^{(l)}} g^{(l)}(s)$; $\mathrm{BC}^{(l)}_{\mathrm{ord}}$ restricts this sum to ordinary states $s \in \mathscr{S}^{(l)}$.

The Stopping-Game contraction bound assumes a conditional alive probability bounded away from one; the Routing-Game bound assumes a uniform row-overlap bound strictly below one. The precise hypotheses and proofs are in Theorems S.3.6 and S.3.7; the cemetery-floor analysis is in Remark S.3.2.

The discounted occupation measures at the input layer recover gradient and $\alpha\beta$-LRP maps (Theorems S.1.6 and S.1.8). The undiscounted endpoint marginal $m_M^{(0)}$ is generally different: Hellinger compares the trajectory routing laws and does not include the path discounts. Under their respective contraction hypotheses, Theorems S.3.6–S.3.7 show that $\Pi_M$ changes drastically under randomization, while Table 8.2 shows that the resulting attribution maps can remain similar. The two diagnostics therefore detect different aspects of parameter sensitivity: the map-level test of Adebayo et al. (2018) asks whether the explanation delivered to the user depends on the learned parameters, the trajectory-level test whether its normalized routing does. A map can fail the first while its routing passes the

second; this discrepancy does not isolate a fixed projection loss, since the discounts can also change across models, and it does not make such a map adequate for model debugging (Adebayo et al., 2018; Sixt et al., 2020).

### 8.4.2 Heatmap similarity at the cascade endpoint

We apply the cascading randomization test of Adebayo et al. (2018) to Table 8.2's attribution methods. Starting pretrained, we cumulatively re-initialize convolutional and linear layers output-to-input: weights Kaiming normal (He et al., 2015), biases $\mathcal{N}(0, 0.01^2)$. BatchNorm (ResNet-50), LayerNorm, positional embeddings, and class token (ViT-B/16) retain pretrained values. The ViT attribution cascade randomizes decomposed query/key/value projections; the Hellinger cascade (Section 8.2.1) retains packed query/key/value parameters. At each depth, we compare original and randomized models' attributions for every test image. The test asks whether maps change, without requiring monotonic similarity decay. Persistent similarity can reflect input structure or architecture.

**Metrics.** We report complementary map-similarity measures:

- **Spearman** $\rho(|a|)$: rank correlation of absolute attributions, measuring agreement in pixel importance. Absolute values accommodate differing attribution sign conventions; the correlation itself remains signed.
- **HOG** $r$: Pearson correlation of Histograms of Oriented Gradients (HOG) descriptors (Dalal and Triggs, 2005), measuring local-gradient-orientation structure.

We omit SSIM. Binder et al. (2023) show that input-dependent structure and activation scales can survive top-down randomization, while low similarity can reflect noise; this noise critique also covers Spearman correlation of absolute attributions. Values average over the 565-image ImageNet-S test split (50 classes). Constant maps receive similarity 0 by convention: at ResNet-50's cascade endpoint, LIME and KernelSHAP outputs no longer depend on superpixels. Table 8.2 reports this endpoint, with all tested convolutional and linear layers re-initialized and the parameter groups above retained.

**Discussion.** Gradient and Grad-CAM stay closest to zero on the CNNs, as does LayerCAM on VGG-16; on ViT-B/16, Gradient, SmoothGrad and GAE do. These scores record map changes under the tested intervention, not faithfulness. LRP-$\varepsilon$ retains the most VGG-16 rank structure (0.341). ResNet-50's LRP rules (LRP-$\gamma$ 0.254, LRP-$\varepsilon$ 0.250) approach Integrated Gradients and LayerCAM, which retain most there. LIME and KernelSHAP retain HOG correlation about 0.86 with low Spearman correlation on VGG-16 and ViT-B/16: input-derived superpixels preserve edge structure. On ResNet-50, endpoint outputs no longer depend on superpixels, yielding constant maps ($^{\ddagger}$). The RG rows lie between Gradient and LRP-$\varepsilon$ on VGG-16, slightly below LRP on ResNet-50, and highest in ViT-B/16 Spearman correlation (0.454 anchor, 0.450 selected mode). Grad-CAM++ on ResNet-50 and TIBAV on ViT-B/16 become anti-correlated, rather than unrelated. Section 8.4.1 complements map similarity with trajectory-distribution divergence.

**Tab. 8.2:** Parameter randomization sanity check adapted from Adebayo et al. (2018). Each cell reports the average similarity between the attribution of the original pretrained model and the attribution of a copy randomized over the tested convolutional and linear layers (every convolutional and linear layer re-initialized, including the decomposed attention query/key/value projections; normalization layers and the ViT embeddings stay pretrained). Values near zero indicate low map similarity under this parameter intervention; the Spearman correlation is signed, so negative values occur. Bold marks the value closest to zero per metric within each architecture, among maps that are not constant; ‡: the map is constant at the cascade endpoint (similarity 0 by convention). Rows: the reported configuration of every method (Section 7.5).

| Method | Spear. $\rho(\lvert a\rvert)$ | HOG $r$ |
|---|---|---|
| **VGG-16** | | |
| Gradient | 0.013 | **0.009** |
| SmGrad | 0.084 | 0.101 |
| IntGrad | 0.207 | 0.348 |
| DeepLIFT | 0.216 | 0.340 |
| LRP-$\varepsilon$ | 0.341 | 0.457 |
| LRP-$\gamma$ | 0.092 | 0.370 |
| GCAM | **-0.007** | 0.069 |
| GCAM++ | 0.180 | 0.129 |
| LCAM | -0.011 | 0.052 |
| LIME | 0.257 | 0.864 |
| KernelSHAP | 0.075 | 0.868 |
| SG ($\lambda$=$-1$,$\sigma^2$=5, gate on) | 0.289 | 0.456 |
| RG anchor | 0.299 | 0.535 |
| RG ($\tau$=0.3,$\varepsilon$=0.1) | 0.284 | 0.287 |
| **ResNet-50** | | |
| Gradient | 0.148 | 0.134 |
| SmGrad | 0.233 | 0.214 |
| IntGrad | 0.277 | 0.424 |
| DeepLIFT | 0.224 | 0.351 |
| LRP-$\varepsilon$ | 0.250 | 0.393 |
| LRP-$\gamma$ | 0.254 | 0.130 |
| GCAM | **0.020** | **0.072** |
| GCAM++ | -0.502 | 0.151 |
| LCAM | 0.266 | 0.145 |
| LIME‡ | 0.000 | 0.000 |
| KernelSHAP‡ | 0.000 | 0.000 |
| RG anchor | 0.217 | 0.344 |
| RG ($\tau$=0.9,$\varepsilon$=1) | 0.180 | 0.316 |
| **ViT-B/16** | | |
| Gradient | **-0.003** | 0.041 |
| SmGrad | 0.010 | **0.038** |
| IntGrad | 0.272 | 0.362 |
| LRP-$\varepsilon$ | 0.165 | 0.300 |
| LRP-$\gamma$ | 0.027 | 0.234 |
| AttnLRP | 0.124 | 0.176 |
| DAVE | 0.178 | 0.323 |
| TIBAV | -0.175 | 0.150 |
| GAE | -0.009 | 0.130 |
| AR | 0.120 | 0.153 |
| LIME | 0.248 | 0.861 |
| KernelSHAP | 0.131 | 0.867 |
| RG anchor | 0.454 | 0.469 |
| RG ($\tau$=0.4,$\lambda_{\text{sm}}$=$-10$,$\sigma^2$=5,$\varepsilon$=0.1) | 0.450 | 0.310 |

Spear. $\rho(\lvert a\rvert)$ = Spearman rank correlation of absolute attribution values; HOG $r$ = Pearson correlation of HOG descriptors. RG anchor = the $\alpha\beta$-LRP representation ($\tau$=1, $\varepsilon$=0.5); the second RG row and the SG row are the localization-selected configurations and list the modes that differ ($\tau$ = temperature; $\lambda$, $\lambda_{\text{sm}}$ = risk aversion; $\sigma^2$ = ADF input variance; $\varepsilon$ = LRP stabilizer; $\tilde{\Phi}$ = noisy-observation gate).

### 8.4.3 Double-Random Empirical Baseline

Our cascade-endpoint reference compares two copies of each architecture with independently randomized convolutional and linear layers, retaining the pretrained parameter groups below. This differs from Adebayo et al. (2018)'s trained-versus-randomized comparison.

**Protocol.** For each architecture, $R = 500$ independent rounds comprise: *(i)* copy the pretrained checkpoint to $A_r, B_r$ and independently re-initialize every `Conv2d`/`Linear` layer as at the cascade endpoint. Both retain normalization parameters, packed attention query/key/value projections, positional embedding, and class token. We use checkpoints for all architectures: `weights=None` gives ViT an identically zero classification head, making trajectory laws trivially indistinguishable. *(ii)* Draw one image with replacement from the 565-image ImageNet-S test split (336 distinct images among 500 draws with our fixed seed). *(iii)* Compute plain $\mathrm{H}(\Pi_{A_r}, \Pi_{B_r})$ and survival-conditioned $\mathrm{H}_{\mathrm{surv}}$ for both Stopping and Routing Game kernels, targeting the ground-truth class. Table 8.3 reports round-to-round means and standard deviations; variance mixes image selection and random-model-pair realization.

| | Stopping Game | | Routing Game | |
|---|---|---|---|---|
| Architecture | H | $\mathrm{H}_{\mathrm{surv}}$ | H | $\mathrm{H}_{\mathrm{surv}}$ |
| VGG-16 | $0.939 \pm 0.003$ | $1.000 \pm 0.000$ | $1.000 \pm 0.000$ | $1.000 \pm 0.000$ |
| ResNet-50 | $0.938 \pm 0.003$ | $1.000 \pm 0.000$ | $0.973 \pm 0.002$ | $1.000 \pm 0.000$ |
| ViT-B/16 | $0.958 \pm 0.002$ | $1.000 \pm 0.000$ | $0.956 \pm 0.001$ | $1.000 \pm 0.000$ |

**Tab. 8.3:** Hellinger trajectory distance between two copies of the same architecture with independently randomized convolutional and linear layers and the remaining pretrained parameters retained, averaged over $R = 500$ rounds (one fresh ImageNet-S test image per round). Plain H concentrates at an architecture-dependent cemetery floor of 0.94–0.97; conditioned-survival $\mathrm{H}_{\mathrm{surv}}$ saturates at unity. The VGG-16 Routing Game cell is the one case in which the plain distance also saturates at 1.000.

**Interpretation.** Plain H concentrates at a model-dependent floor of 0.94–0.97, matching the cemetery-floor prediction of Remark S.3.2: once both random kernels produce uncorrelated dead masks, the shared dead-in-both coincidence rate bounds $\mathrm{BC}^{(0)}$ below by a strictly positive constant and plain H cannot reach unity. The conditioned-survival distance removes the cemetery floor by renormalizing on survival and reports the two randomized trajectory laws as essentially support-disjoint on ordinary input states ($\mathrm{H}_{\mathrm{surv}} \approx 1$ throughout). The pretrained-vs-randomized Hellinger distance that the cascading sanity check reports should therefore be read against this architecture-specific empirical reference: values close to the double-random baseline indicate a trajectory distance comparable to that between two copies independently randomized over the same layers.

### 8.4.4 Per-Image Association Between Trajectory and Map Diagnostics

Theorems S.3.6–S.3.7 and the double-random baseline of Table 8.3 bound the *aggregate* behavior of the Hellinger trajectory distance and the pixel-level attribution-map similarity. A complementary question, motivated by the argument of §8.4.1, is whether the two diagnostics agree *per image*: at fixed randomization depth, does a larger displacement of an image's trajectory distribution correspond to a more dissimilar attribution map? Table 8.4 tests this directly.

**Protocol.** For each architecture we pick a cascading step at the same network depth in the Hellinger and attribution-sanity runs: the layers down to `features.26` on VGG-16 and down to `layer4.1.conv3` on ResNet-50 (identical sets of re-initialized layers), and the last encoder block on ViT-B/16, where the Hellinger cascade re-initializes the MLP and the output projection of the block and the attribution cascade additionally its $Q$, $K$ and $V$ projections. The interventions therefore match on the CNNs; on ViT-B/16, the comparison also reflects this difference in the randomized operators. The mean H / $H_{surv}$ at these steps can be read off Figure 8.1. For each of the 565 ImageNet-S test images we then record four Hellinger scalars (H plain/surv for the Stopping and Routing Games) and four attribution-map scalars (Spearman $\rho(|a|)$ and HOG, each computed between the pretrained and partially-randomized model, for Gradient and for the $\alpha\beta$-RG anchor at $\tau$=1; these are the attribution methods whose marginal laws line up with the Hellinger game kernels). Table 8.4 reports Spearman rank correlations across the 565 images.

| | | Within Hellinger | | | | Gradient | | RG anchor | |
|---|---|---|---|---|---|---|---|---|---|
| Architecture | Hellinger variant | $H_{RG,pl}$ | $H_{RG,surv}$ | $H_{SG,pl}$ | $H_{SG,surv}$ | Spear. $\rho(\|a\|)$ | HOG $r$ | Spear. $\rho(\|a\|)$ | HOG $r$ |
| VGG-16 | $H_{RG,plain}$ | 1.00 | +1.00 | −0.33 | +0.85 | +0.07 | +0.10 | −0.01 | +0.04 |
| | $H_{RG,surv}$ | +1.00 | 1.00 | −0.33 | +0.85 | +0.07 | +0.10 | −0.01 | +0.04 |
| | $H_{SG,plain}$ | −0.33 | −0.33 | 1.00 | −0.30 | +0.12 | +0.11 | +0.00 | −0.02 |
| | $H_{SG,surv}$ | +0.85 | +0.85 | −0.30 | 1.00 | +0.01 | +0.05 | −0.10 | −0.10 |
| ResNet-50 | $H_{RG,plain}$ | 1.00 | +0.58 | −0.08 | +0.70 | +0.05 | −0.01 | −0.08 | −0.19 |
| | $H_{RG,surv}$ | +0.58 | 1.00 | +0.16 | +0.89 | +0.15 | +0.08 | +0.05 | −0.05 |
| | $H_{SG,plain}$ | −0.08 | +0.16 | 1.00 | +0.16 | −0.04 | +0.07 | −0.01 | +0.02 |
| | $H_{SG,surv}$ | +0.70 | +0.89 | +0.16 | 1.00 | +0.16 | +0.07 | −0.01 | −0.10 |
| ViT-B/16 | $H_{RG,plain}$ | 1.00 | +0.96 | +0.29 | +0.30 | +0.02 | +0.00 | −0.04 | −0.08 |
| | $H_{RG,surv}$ | +0.96 | 1.00 | +0.25 | +0.28 | +0.02 | +0.02 | −0.07 | −0.07 |
| | $H_{SG,plain}$ | +0.29 | +0.25 | 1.00 | +0.99 | −0.12 | −0.06 | −0.03 | −0.05 |
| | $H_{SG,surv}$ | +0.30 | +0.28 | +0.99 | 1.00 | −0.12 | −0.06 | −0.04 | −0.05 |

**Tab. 8.4:** Per-image Spearman rank correlations across the 565-image ImageNet-S test split at the selected cascading steps of the Hellinger and attribution-sanity pipelines. The randomized layers match on the CNNs; the ViT-B/16 attribution cascade additionally randomizes $Q$, $K$, and $V$. Left block: within-Hellinger cross-correlations of the plain ($H_{\cdot,pl}$) and survival-conditioned ($H_{\cdot,surv}$) distances under the RG and SG kernels (self-entries shown as −; trivially +1). Right block: Hellinger-vs-map cross-correlations. The Hellinger-vs-map block is uniformly $|\rho| \leq 0.19$: per-image Hellinger and per-image attribution-map similarity have weak observed rank association.

**Readings.** The Hellinger-vs-map block of Table 8.4 is uniformly near zero ($|\rho| \leq 0.19$ across all three architectures): trajectory displacement and map drift have weak observed per-image rank association. Statistical independence and nonlinear dependence remain unresolved; ViT-B/16's unequal interventions further limit conclusions about the diagnostics alone.

Within Hellinger, per-image H gives a stable ordering. When little mass reaches the cemetery, plain and conditioned distances rank images almost identically: $\rho = +0.96$ (RG) and +0.99 (SG) on ViT-B/16. VGG-16 RG gives identical values because it never routes to dead-neuron states. For SG on ReLU backbones, the normalizations decouple ($\rho = -0.30$ VGG-16, +0.16 ResNet-50): image-dependent dead masks change the cemetery share, which renormalization by $1/\sqrt{Z_A Z_B}$ removes. The two games' conditioned distances agree strongly on ReLU backbones ($\rho = +0.85$ VGG-16, +0.89 ResNet-50), but weakly on ViT-B/16 (+0.28), where the kernels capture partly different per-image structure.

Within an attribution method, Spearman $\rho(|a|)$ and HOG agree well ($\rho = 0.65$–$0.82$, from the same per-image joins), whereas Gradient and $\alpha\beta$-RG similarities agree weakly across images ($\rho$ from −0.10 to +0.30, depending on architecture and metric). Together with the near-zero Hellinger-vs-map block,

this supports §8.4.1: trajectory and pixel diagnostics yield weakly associated per-image rankings here, and the attribution method changes which images a pixel-level sanity check flags.

# 9 Conclusion and Future Work

Section 9.1 summarizes the contributions, Section 9.2 their theoretical and empirical limits, and Section 9.4 future research.

## 9.1 Summary of Contributions

The starting point was the growing role of interpretability in improving and controlling models. When attribution guides a correction or an intervention, both the internal representation it reads and the backward calculation it performs matter. This thesis connects those two choices through *network decomposition*: expose structure a trained ReLU network already contains, preserve its predictions, and study how attribution propagates through that structure.

The common mathematical foundation is the decomposition theory of Chapter 3. A *valid decomposition* replaces each layer map by a pair of coordinated split-streams whose difference reproduces the network exactly, and the chapter populated the definition with instances of increasing structure. The mixing family $f^{\eta}_{\text{mix}}$ interpolates between trivial and fully separated splits; its endpoint $\eta = 1$ recovers the classical DC decomposition of a ReLU network into a difference $f = g - h$ of two convex, monotone split-streams, routing the weight parts $W^{(l,+)}, W^{(l,-)}$ into separate channels (Zimmermann and Loho, 2026). Beyond the piecewise-linear case, Theorem 3.13 established a DC–monotone splitting for every activation with integrable curvature — GELU (Hendrycks and Gimpel, 2016), Swish (Ramachandran et al., 2018), Mish, tanh, sigmoid, softplus — with a canonicity statement singling out the split of minimal curvature, and a bounded-variation extension recovering ReLU as the Dirac limit. The stopping decomposition $f_{\text{stop}}$ added an early-termination channel, and the regularized family $f^{\Omega}_{\text{reg}}$ showed that a convex regularizer on the stop/continue decision generates regularized decompositions systematically, the softplus decomposition $f^{\theta}_{\text{soft}}$ arising as the binary-entropy subcase — placing the softplus replacement of Dombrowski et al. (2022) inside the framework rather than beside it. The chapter closed with the stabilization that makes split passes usable at depth and a full CNN extension: convolutions, gated pooling, batch normalization, and residual connections.

The game semantics of Chapter 4 gives the stopping instance a second, complementary reading. Building on the ReLU-network games of Gaubert & Vlassopoulos (2025), the stopping decomposition was reinterpreted as a two-player zero-sum SG on a doubled state graph: weight signs route between the players $P^+$ and $P^-$, gradients and $\alpha\beta$-LRP attributions emerge as discounted occupation measures under equilibrium policies, and the deformation parameters of Chapter 3 — temperature, risk aversion, stabilizers — become principled modifications of the game. Chapter 8 takes the decisive step: the equilibrium policies induce a genuine *probability distribution* $\Pi$ over complete backward trajectories, and this distribution records normalized backward routing. Recovering attribution additionally requires its signs and model-dependent discounts. The comparison with the NRM framework of Xiong et al. (2026) in Section 8.1 identifies their shared path perspective: NRM assigns normalized signed relevance to neuron walks, while the game separates a probability law on expanded states from the signs and discounts used to recover attribution. A direct identification requires matching the propagation and normalization conventions. The probability law makes trajectory-level statistical comparison possible: the Hellinger distance between two trajectory laws is computable in one linear-time pass, and the parameter-randomization sanity check of Adebayo et al. (2018) lifts from attribution maps to trajectory space, where Theorem S.3.6 shows Bhattacharyya mass decays geometrically in the number of randomized layers — detecting collapse even when heatmaps remain deceptively similar (Sixt et al., 2020).

These representations lead to the attribution methods of Chapter 6. SplitCAM, SplitLRP, and SplitGrad apply established attribution principles to the decomposed streams, exposing separate positive and negative contributions; the difference-of-convex input neural networks (DCINNs) of Section 6.1.4 added a counterpart trained directly in split form, with visually distinct stream sensitivities in the displayed MNIST examples. Chapter 5 extends the framework to Transformers (Vaswani et al., 2017) in two distinct ways. The constructive DC decomposition of Section 5.3 retains the full input dependence of Queries, Keys, Values, softmax, LayerNorm, and GELU, producing convex streams that are monotone in the independent non-negative split inputs. Separately, the conditioned Value-routing game supplies the attribution implementation evaluated on ViT-B/16 (Dosovitskiy et al., 2021).

The shared evaluation questions in Chapter 7 concern localization, faithfulness, robustness, and the cost of obtaining these signals. The split methods were evaluated on VGG-16 and ResNet-18 on ImageNet-S; the revised game study covers VGG-16, ResNet-50, and ViT-B/16 on ImageNet-S, with a ViT-B/16 transfer study on Pascal VOC. No single method dominates every metric. On ViT-B/16, one adapted $\alpha\beta$-LRP configuration outperforms all evaluated baselines in attribution localization, and this lead transfers to Pascal VOC without re-tuning. The LRP baselines remain stronger on the faithfulness metrics, while the anchor configurations are particularly robust. Independent validation selections for localization and faithfulness, bootstrap confidence intervals, and Holm-corrected paired tests make these trade-offs explicit. Hellinger diagnostics complement pixel-level comparisons: they distinguish trained from randomized calculations even where heatmaps remain similar, increase with Gaussian input noise, and separate intra- from inter-class pairs for at least one game on each tested architecture.

These results substantiate the central claim: convex, monotone stream representations and backward games can be constructed from a trained ReLU network while preserving its predictions.

## 9.2 Limitations

The theory and evidence have the following limits.

**Computational overhead.** A split pass propagates two streams per channel and currently materializes duplicated weight matrices. Reported SplitCAM runtimes are approximately 14–29 times those of corresponding ordinary Grad-CAM and LayerCAM configurations; Section S.4.1.5 specifies the measurement scope. Historical artifacts omit hardware, precision, and exact configuration IDs. Masked views could avoid weight duplication while still propagating two streams; this optimization remains unimplemented and unbenchmarked here.

**Stabilization hyperparameters.** Stabilization (Section 3.6) is essential at depth. The theory constrains choices of thresholds, shifting schemes, game temperatures $\tau$, $\theta$, and risk aversion $\lambda$ without determining them. Declared validation grids yield different localization and faithfulness winners, and Pascal VOC transfer uses configurations fixed on ImageNet-S. Optimal settings and SplitCAM/SplitLRP layers therefore depend on the metric.

**Transformer structure and numerical stability.** The exact constructive DC theorem (Section 5.3) covers fixed finite inference graphs of the stated operations, with positive LayerNorm $\varepsilon$ and nonempty masked-softmax rows. The evaluated Value-routing game conditions on attention, LayerNorm statistics, and GELU gates, representing a specified backward rule that omits the full Q/K derivative; it

establishes no unique explanatory game. The full DC construction retains that input dependence and is exact in real arithmetic, yet its numerical prototype suffers severe cancellation. A practical stable implementation remains open. The ViT-B/16 attribution results concern the conditioned game alone.

**Benchmark scope.** Evidence is limited to the two studies' architectures (VGG-16, ResNet-18, ResNet-50, ViT-B/16), datasets (ImageNet-S, Pascal VOC), and metrics. Human-mask localization measures spatial agreement, leaving decision faithfulness unresolved; sharper maps may discard decision-relevant evidence outside the object. Medical-image attributions, for example, highlighted a microscope artifact's boundary rather than its full annotated area (Pahde et al., 2025). Faithfulness metrics assess specified perturbation protocols, with rankings that can depend on occlusion strategy (Blücher et al., 2024); neither metric family establishes a general causal explanation. Transfer beyond vision and downstream task gains remain untested. Trajectory diagnostics were validated through sanity checks and controlled perturbations; dedicated calculation-level benchmarks do not yet exist.

**Stream interpretation.** The MNIST stream-gradient examples are qualitative. In an inverted-input DCINN, the full logit gradient sums the two plotted stream sensitivities, so either stream can suggest a direction the other outweighs. Convexity permits this when the first affine layer is unconstrained. Concentration on present and candidate missing strokes motivates complete-model interventions to assess the interpretation; it certifies neither finite counterfactuals nor necessary features.

## 9.3 Broader Impact

The work is methodological: it studies attribution methods for existing image classifiers and does not introduce a deployed decision system, a new dataset of people, or a high-risk generative model. Its intended positive impact is to make interpretability methods more diagnostically accountable. In particular, trajectory-level comparisons can reveal cases where visually similar attribution maps arise from different backward calculations, and can therefore support more careful auditing of interpretability pipelines used in scientific or engineering workflows.

Sharper or more stable maps can also create unwarranted confidence when treated as causal guarantees. In safety-critical or fairness-sensitive settings, improved-looking attributions could justify classifiers whose predictions remain brittle, biased, or dataset-dependent. Hellinger distance measures internal calculation paths; a small distance establishes neither social validity nor fairness, robustness, or deployment suitability.

The framework is an analysis tool, without deployment certification. Experiments on public benchmark classifiers and ImageNet-derived evaluation data provide evidence about attribution behavior under those benchmarks, without establishing suitability for consequential use.

## 9.4 Future Research Directions

Four directions appear most promising.

**Concept queries on the expanded state graph.** Section 8.1 proposes a connection to NRM (Xiong et al., 2026): its neuron-set queries suggest events for game trajectories. Their probabilities and separate signed discounted occupation measures could support cross-layer concept analysis. Correspondence with an NRM or CRP rule requires matching propagation factors, normalization, and absorption conventions; discounts cannot replace probability normalization. Hellinger comparisons of trajectories conditioned on positive-probability concept events could then supply calculation-level benchmarks (Section 9.2). This extension remains proposed.

**Regularized decompositions for smooth activations.** Theorem 3.13 gives canonical DC–monotone splits for GELU, Swish, and related activations; $f_{\mathrm{reg}}^{\Omega}$ (Section 3.5) gives softplus as an entropy-regularized gate. Joining these constructions remains open. GELU and Swish cannot be identified directly with a conjugate activation

$$\Omega^*(z) = \sup_{\pi \in [0,1]} \{\pi z - \Omega(\pi)\}$$

which is always convex, non-decreasing, and 1-Lipschitz, properties those activations lack globally. Possible approaches combine their exact DC–monotone decompositions with games on non-negative streams or formulate local games conditional on the realized activation pattern. Transferring trajectory diagnostics would require corresponding recovery and backward-occupation theorems.

**DCINNs at scale.** Section 6.1.4's small-model MNIST experiments show qualitative, class-dependent stream sensitivities on present and candidate missing strokes. Reliable counterfactual interpretation requires complete-model interventions, potentially guided by fidelity, understandability, and sufficiency criteria for visual counterfactual explainers (Bender et al., 2026). Scaling also raises questions: does difference-of-monotone parametrization train stably at ImageNet scale, can DC-programming (Le Thi and Pham Dinh, 2018) or ICNN ideas (Amos et al., 2017) accelerate training, and how does capacity affect the accuracy–interpretability trade-off? Positive results would strengthen split architectures as a design principle for interpretable models.

**Transformers, LLMs, and efficient split passes.** The constructive theorem covers fixed causal masks with nonempty allowed-key sets. Autoregressive LLMs applications require stable stream evaluation, efficient key–value caching, and robustness testing; trajectory diagnostics additionally require a specified backward rule and recovery guarantees. Masked views could avoid materialized weight duplicates (Section 9.2), while non-negative split weights invite hardware-level exploitation. Layer-dependent attribution quality calls for principled layer selection or multi-layer aggregation, expressible through layers as intermediate marginals of the trajectory measure $\Pi$.

The unifying lesson is that a trained network is a richer object than the function it computes. One set of weights carries a difference of convex monotone parts, a zero-sum game between cooperating adversaries, and a probability distribution over backward trajectories — none of which requires changing the network. Interpretability, on this view, provides structured access to computations that can be inspected, compared, and modified. The decomposition and game perspectives supply a mathematical basis for that access; evaluating how these tools improve a model through downstream correction, training, or steering is the next step.

# Appendix A

# Experimental Protocol and Reproducibility

The companion supplement, Chapter S.4, collects complete evaluation tables, validation grids, ablations, and qualitative examples.

## A.1 Split-Method Evaluation

The split-method campaign explains the predicted class, including misclassifications, while localization compares it with the annotated object mask. For misclassified images, target and mask therefore refer to different classes. The game campaign explains the dataset target class. Both retain misclassified images under these distinct target conventions.

### A.1.1 Artifact Availability and Reproduction Scope

The Hidden Monotonicity tables retain their published-source provenance. The code checkout lacks original per-image outputs, the complete historical evaluation environment, and six advertised MNIST multi-seed training configurations. The runnable notebook-derived example cannot reconstruct those missing settings, so the available artifacts cannot establish exact reproduction of the historical tables.

Deterministic CPU fixtures exercise double-precision reconstruction, input derivatives, attribution, timing, and Pixel Flipping with an independently evaluated initial target score. They establish the tested implementation contracts; agreement with the original ImageNet results remains unverified without a rerun. The historical training runner used the test loader for epoch validation. The current runner reserves a deterministic 10% of training data for validation and evaluates the test set after training, recording partition indices and seed. This does not establish the checkpoint-selection procedure behind the historical MNIST figures.

## A.2 Evaluation Protocol

### A.2.1 Data and splits

The game-method campaign uses the released ImageNet-S50 validation annotations (Gao et al., 2023), which contain 752 images from 50 classes. We exclude 136 images whose annotation class differs from the official ILSVRC2012 label, leaving 616 eligible images. With seed 42, one image per class forms the **50-image validation split**, used only for configuration selection; the remaining 566 form the test pool. Images whose object mask lies entirely outside the 224×224 center crop are excluded after the split, because the mask-based metrics are undefined for them. This removes one test image and leaves **565 test images**. The explanation target is the dataset class, including when the model misclassifies the image. This population and target convention belong to the game-method protocol and need not coincide with the split-method campaign. The Pascal VOC 2012 transfer split is described in Section S.4.4.4.

### A.2.2 Metric definitions

Let $x \in \mathbb{R}^{C\times H\times W}$ be an image, let $y$ be its target class, and let $a \in \mathbb{R}^{H\times W}$ be the scalar attribution map produced by an explanation method after channel reduction. Write $d \coloneqq HW$ and let $\mathrm{vec}(a) \in \mathbb{R}^d$ denote the flattened map. For any binary segmentation mask $m \in \{0,1\}^{H\times W}$, write $\mathrm{vec}(m) \in \{0,1\}^d$. When a metric is configured with `abs=true` and `normalise=true`, the evaluator applies the max-abs normalization (not min–max)

$$\psi(a) = \frac{|\mathrm{vec}(a)|}{\max_i |\mathrm{vec}(a)_i|}, \tag{A.1}$$

with the convention that the all-zero map remains zero. We denote the preprocessed attribution vector by $\bar{a}$. All metrics below use $\bar{a}$; faithfulness metrics use raw logits (no softmax) to avoid saturation effects, and perturbations use the Quantus per-sample mean replacement baseline.

**Attribution localization (AL).** We adapt Kohlbrenner et al. (2020)'s positive-relevance ratio within ground-truth boxes to absolute mass, pixel masks, and a stabilized denominator:

$$\mathrm{AL}(a,m) = \frac{\sum_{i=1}^{d} \bar{a}_i m_i}{\sum_{i=1}^{d} \bar{a}_i + 10^{-9}}. \tag{A.2}$$

**Pointing game (PG).** We use the pointing-game criterion of Zhang et al. (2018a); our configuration applies `abs=true` without normalization. Let $M(a) \coloneqq \{i \in \{1,\dots,d\} : \bar{a}_i = \max_j \bar{a}_j\}$ contain all maximizers. Then

$$\mathrm{PG}(a,m) = \mathbf{1}\big\{M(a) \cap \{i : m_i = 1\} \neq \varnothing\big\}. \tag{A.3}$$

A tie scores a hit if any maximizer lies inside the mask. PG is binary per image–class record; its ImageNet-S test mean is a multiple of $1/565$.

**Top-$k$ intersection (TK).** Following Theiner et al. (2022), with $k = 1000$ and the permutation $\pi$ that sorts $\bar{a}$ in descending order:

$$\mathrm{TK}(a,m) = \frac{1}{1000} \sum_{r=1}^{1000} m_{\pi_r}. \tag{A.4}$$

**Pixel-flipping AUC ($\mathbf{PF}_5$, $\mathbf{PF}_{20}$).** Pixel flipping (Bach et al., 2015; Samek et al., 2017) removes the most relevant features first and tracks the class score. The evaluator uses `pixel_flipping_abs` with `features_in_step = 1568` and `perturb_baseline = mean`. Unlike the spatial localization metrics above, pixel flipping operates on $D \coloneqq CHW$ scalar input features. The spatial attribution is broadcast over the color channels; let $\widetilde{\pi}$ sort the flattened broadcast attribution in descending order. For $T \coloneqq \lceil D/1568 \rceil$, the input $x^{(t)}$ is obtained from $x^{(t-1)}$, starting at $x^{(0)} = x$, by replacing features $\widetilde{\pi}_{(t-1)\cdot 1568+1}, \dots, \widetilde{\pi}_{\min\{t\cdot 1568, D\}}$ with the scalar mean of the current flattened input, as implemented by Quantus. The evaluator computes

$$f_t \coloneqq F(x^{(t)})_y, \qquad t = 1,\dots,T, \tag{A.5}$$

and $\mathrm{PF}_p$ is the trapezoidal area over the first $p\%$ of this flipping curve, i.e. over its first $K_p$ points with the abscissa rescaled to [0,1]: $\mathrm{PF}_p = \mathrm{trapz}\big(\{(t-1)/(K_p-1), f_t\}_{t=1}^{K_p}\big)$, $K_p = \max(2, \lceil pT/100 \rceil)$. For $C = 3$ and $H = W = 224$, $D = 150{,}528$ and $T = 96$ steps, giving $K_5 = 5$ and $K_{20} = 20$. The curve contains only post-perturbation scores; the original input score is not an additional point. Lower is better.

**Selectivity (Sel).** Following Montavon et al. (2018) and Quantus (Hedström et al., 2023), sum $\bar{a}$ over square 39×39 patches (side $\lfloor\sqrt{1568}\rfloor$). Replace non-overlapping patches by the per-sample mean in decreasing attribution order, recording the target logit after each removal. Sel is the trapezoidal area under this curve against the fraction of removed patches. Lower is better.

**Max and average sensitivity (MaxS, AvgS).** Following Yeh et al. (2019)'s max-sensitivity and its mean analogue, regenerate attributions for both metrics on the same $N_s = 50$ inputs $x + \xi_k$, with uniform noise $\xi_k \sim \mathscr{U}[-r, r]^{C\times H\times W}$, $r = 0.2$. Report the maximum (MaxS) or mean (AvgS) over $k$ of $\|\psi(a(x+\xi_k)) - \psi(a(x))\|_F / \|\psi(a(x))\|_F$; lower values indicate greater stability to input noise. Per-image seeds give every row identical perturbations. Both metrics use the same fixed random subset of 200 test images (seed 0) across rows and models; all other metrics use the full test split.

### A.2.3 Selection

Every tunable method, including baselines, selects two configurations on validation: `loc` minimizes rank-sum over $\{\mathrm{AL}, \mathrm{PG}, \mathrm{TK}\}$, and `faith` over $\{\mathrm{PF}_5, \mathrm{PF}_{20}, \mathrm{Sel}\}$. Ranks are within-method and respect metric directions. Ties favor the default (the anchor for RG), then fewer non-default parameters, then grid order. Admission requires non-zero maps and defined metrics on every validation image. Before ranking, we exclude 5 of the 40 VGG-16 SG configurations, all with strong risk aversion and deterministic gates, whose maps vanish on some images; all others qualify. MaxS and AvgS are excluded from selection because each evaluation re-explains every image $2N_s$ times. Main tables report `loc`; Section S.4.4 reports both selections. Localization suits modes controlling routing focus, has external mask ground truth, and has precedent for comparing LRP rules using bounding boxes (Kohlbrenner et al., 2020). Perturbation metrics evaluate off-distribution inputs. Reporting both selections preserves the distinction and trade-off between localization and faithfulness.

### A.2.4 Statistics

The game-method test tables report 95% percentile bootstrap confidence intervals (Efron and Tibshirani, 1993) using $B = 10{,}000$ resamples and seed 0. On ImageNet-S, the sampling units are images, with the same resampled image indices used across method rows. MaxS and AvgS intervals resample the fixed 200-image subset of Section A.2.2. Continuous metrics use a two-sided paired bootstrap test of the mean per-image difference: the bootstrap distribution is centered at zero, and $p$ is the fraction of centered resampled differences at least as large in absolute value as the observed difference. Binary PG uses McNemar's test (McNemar, 1947) with continuity correction.

On Pascal VOC, the estimand remains the mean over image–class records, but the independent sampling units are the $G = 2{,}293$ original test images. Each bootstrap replicate samples $G$ images with replacement and retains every class record of each sampled image. Its mean is the sum over these records divided by their replicate-specific count. Thus, classes from the same image remain together, while images with more evaluated classes retain their contribution to the reported record-weighted mean. Paired comparisons use common finite records and common image draws. Every VOC metric,

including binary PG, uses the corresponding two-sided, null-centered paired cluster-bootstrap test. For observed mean difference $\widehat{\Delta}$ and bootstrap differences $\Delta_b^*$, its finite-resample $p$-value is

$$p = \frac{1 + \sum_{b=1}^{B} \mathbf{1}\{|\Delta_b^* - \widehat{\Delta}| \geq |\widehat{\Delta}|\}}{B + 1}. \tag{A.6}$$

For the main and full test tables, Holm correction (Holm, 1979) is applied to all unordered pairs of eligible rows within each metric and reported table, before extracting comparisons with the observed best row or declaring a row ahead of every baseline. The observed winner therefore does not determine the comparison family. A difference is called significant at adjusted $p < 0.05$; otherwise, no difference was detected at this threshold. This does not establish practical equivalence. These intervals and tests condition on the fixed checkpoints, selected configurations and realized attribution evaluations; they do not quantify training or configuration-selection variability. The split-method tables report the source point estimates without corresponding confidence intervals.

## A.3 Formal Verification and Mathematical Audit

The thesis repository contains TeX sources, pinned links to both official code repositories, and a Lean 4 project with mathlib pinned to 4.35.0. Checked statements have explicit assumptions. Formalization remains partial: a related lemma cannot certify an entire theorem or paper.

Numbered `Lean_1`, `Lean_2`, ... links in theorem and lemma headings follow the planar-features paper's convention. Each targets the corresponding checked declaration or components, distinguished in `formalization/coverage_map.json`, which records exact assumptions and remaining parts by source label. Selected equation numbers link to checked identities or definitions. Following arXiv:2610.01728, links pin the Git commit and declaration line, keeping targets stable under default-branch edits.

The companion supplement, Section S.4.6, details the checked developments and mathematical audit.

# Appendix B

# Publication Provenance and Tool Use

This thesis integrates two coauthored works: *Hidden Monotonicity*, published at CVPR 2026 as a Highlight (Zimmermann and Loho, 2026), and the revised *Playing the Network Backward* preprint, arXiv:2605.06212v2 (Zimmermann et al., 2026). The thesis author is first author of both. The game paper retains preprint status, without a claim of AISTATS acceptance. A shared decomposition framework organizes their technical content. Model-evaluation outputs and point estimates are reused; VOC intervals and paired comparisons are recomputed by whole-image resampling (Section A.2.4). The thesis adds common definitions, mathematical extensions, connections between constructions, and Lean formalization.

## B.1 Author's Contribution

My principal contributions to *Hidden Monotonicity* and *Playing the Network Backward* were developing the theory, developing and running experiments, and writing all appendices and most paper text outside the introductions. These responsibilities were part of collaborations with the coauthors identified in Section 1.1.

The thesis unifies their mathematical exposition, connects their decompositions, and adds Lean formalization. The following disclosure distinguishes original-paper preparation from later thesis integration, formalization, and revision.

## B.2 Disclosure in the Revised Game Paper

The revised game paper contains the following AI-use statement, reproduced here as a statement about the preparation of that publication:

> In this work, we used generative AI tools for implementing standard baseline methods and evaluation code, for producing plots and tables of results, for drafting parts of the text, and for editing (grammar, spelling, word choice, and reformatting). We have not used generative AI tools to generate research ideas, to formulate the games, or to produce the theorems and their proofs. All AI-assisted code was reviewed and tested by the authors, including unit tests that check the game-theoretic occupation measures against reference gradient and $\alpha\beta$-LRP implementations; all AI-assisted text was checked by the authors for correctness, and every reference was verified manually. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI.

## B.3 AI Assistance in the Thesis Revision

The November 2026 thesis revision used OpenAI Codex, supplied by OpenAI, for mathematical exploration and proof development, Lean implementation and checking, code review and repair, statistical reaggregation, and thesis drafting and editing. Mathematical assistance included developing and checking extensions of the convex, monotone stream construction. The original paper's historical

statement about theorem and proof development applies to its preparation, excluding this later thesis work.

The formalization repository records reviewed theorem types, proof-checking receipts, and coverage limits. Lean certifies the stated formal propositions; source correspondence requires mathematical review, and kernel checks do not certify experiments.

# List of References


Abnar, S., W. Zuidema (2020): Quantifying Attention Flow in Transformers. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 4190–4197. DOI: 10.18653/v1/2020.acl-main.385. URL: http://dx.doi.org/10.18653/v1/2020.acl-main.385 (cit. on pp. 6, 43, 49, 50).

Achtibat, R., M. Dreyer, I. Eisenbraun, S. Bosse, T. Wiegand, W. Samek, S. Lapuschkin (2023): From Attribution Maps to Human-Understandable Explanations through Concept Relevance Propagation. *Nature Machine Intelligence* 5 (9), 1006–1019. DOI: 10.1038/s42256-023-00711-8. URL: https://www.nature.com/articles/s42256-023-00711-8 (cit. on pp. 49, 79).

Achtibat, R., S. M. V. Hatefi, M. Dreyer, A. Jain, T. Wiegand, S. Lapuschkin, W. Samek (2024): AttnLRP: Attention-Aware Layer-Wise Relevance Propagation for Transformers. *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 135–168. URL: https://proceedings.mlr.press/v235/achtibat24a.html (cit. on pp. 6, 43, 44, 49, 57, 59, 76, 77).

Adebayo, J., J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, B. Kim (2018): Sanity Checks for Saliency Maps. *Advances in Neural Information Processing Systems*. Vol. 31. URL: https://proceedings.neurips.cc/paper/2018/hash/294a8ed24b1ad22ec2e7efea049b8737-Abstract.html (cit. on pp. 1, 5, 65, 80, 85–88, 91).

Ali, A., T. Schnake, O. Eberle, G. Montavon, K.-R. Müller, L. Wolf (2022): XAI for Transformers: Better Explanations through Conservative Propagation. *Proceedings of the 39th International Conference on Machine Learning (ICML)*. Vol. 162. Proceedings of Machine Learning Research. PMLR, 435–451. URL: https://proceedings.mlr.press/v162/ali22a.html (cit. on pp. 6, 43, 44, 49, 50, 52, 56, 57, 59).

Alvarez-Melis, D., T. S. Jaakkola (2018): On the Robustness of Interpretability Methods. *ICML 2018 Workshop on Human Interpretability in Machine Learning*, 66–71. DOI: 10.48550/arXiv.1806.08049. arXiv: 1806.08049 [cs.LG]. URL: https://arxiv.org/abs/1806.08049 (cit. on p. 5).

Amos, B., L. Xu, J. Z. Kolter (2017): Input Convex Neural Networks. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 146–155. URL: https://proceedings.mlr.press/v70/amos17b.html (cit. on pp. 2, 5, 9, 19, 22, 42, 67, 94).

Ancona, M., E. Ceolini, C. Öztireli, M. Gross (2018): Towards better understanding of gradient-based attribution methods for Deep Neural Networks. *International Conference on Learning Representations*. URL: https://openreview.net/forum?id=Sy21R9JAW (cit. on p. 43).

Anders, C. J., D. Neumann, W. Samek, K.-R. Müller, S. Lapuschkin (2026): Software for dataset-wide XAI: From local explanations to global insights with Zennit, CoRelAy, and ViRelAy. *PLOS One* 21 (1), e0336683. DOI: 10.1371/journal.pone.0336683. arXiv: 2106.13200. URL: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0336683 (cit. on pp. 44, 76).

Anders, C. J., L. Weber, D. Neumann, W. Samek, K.-R. Müller, S. Lapuschkin (2022): Finding and removing Clever Hans: Using explanation methods to debug and improve deep models. *Information Fusion* 77, 261–295. DOI: 10.1016/j.inffus.2021.07.015. URL: https://www.sciencedirect.com/science/article/pii/S1566253521001573 (cit. on p. 1).

Arora, R., A. Basu, P. Mianjy, A. Mukherjee (2018): Understanding Deep Neural Networks with Rectified Linear Units. *International Conference on Learning Representations*. arXiv: 1611.01491 [cs.LG]. URL: https://openreview.net/forum?id=B1J_rgWRW (cit. on pp. 5, 8, 29, 30).

Arras, L., B. Puri, P. Kahardipraja, S. Lapuschkin, W. Samek (2025): *A Close Look at Decomposition-based XAI-Methods for Transformer Language Models*. Preprint, version 1. DOI: 10.48550/arXiv.2502.15886. arXiv: 2502.15886v1 [cs.CL]. URL: https://arxiv.org/abs/2502.15886v1 (cit. on p. 59).

Auer, P., N. Cesa-Bianchi, P. Fischer (2002): Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47 (2-3), 235–256. ISSN: 1573-0565. DOI: 10.1023/a:1013689704352. URL: http://dx.doi.org/10.1023/A:1013689704352 (cit. on p. 47).

Awasthi, P., A. Mao, M. Mohri, Y. Zhong (2026): DC-programming for neural network optimizations. *Journal of Global Optimization* 95. Published online 2 January 2024, 5–21. DOI: 10.1007/s10898-023-01344-2. URL: https://link.springer.com/article/10.1007/s10898-023-01344-2 (cit. on pp. 5, 21).

Ayinde, B. O., J. M. Zurada (2018): Deep Learning of Constrained Autoencoders for Enhanced Understanding of Data. *IEEE Transactions on Neural Networks and Learning Systems* 29 (9), 3969–3979. DOI: 10.1109/TNNLS.2017.2747861. URL: https://doi.org/10.1109/TNNLS.2017.2747861 (cit. on pp. 2, 63).

Bach, F. (2017): Breaking the Curse of Dimensionality with Convex Neural Networks. *Journal of Machine Learning Research* 18 (19), 1–53. URL: http://jmlr.org/papers/v18/14-546.html (cit. on p. 5).

Bach, S., A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, W. Samek (2015): On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation. *PLOS ONE* 10 (7), e0130140. DOI: 10.1371/journal.pone.0130140. URL: https://doi.org/10.1371/journal.pone.0130140 (cit. on pp. 2, 5, 43, 44, 49, 60, 64, 96).

Balestriero, R., R. G. Baraniuk (2021): Mad Max: Affine Spline Insights Into Deep Learning. *Proceedings of the IEEE* 109 (5), 704–727. DOI: 10.1109/JPROC.2020.3042100. URL: https://doi.org/10.1109/JPROC.2020.3042100 (cit. on p. 5).

Bender, S., J. Herrmann, K.-R. Müller, G. Montavon (2026): Towards desiderata-driven design of visual counterfactual explainers. *Pattern Recognition* 174, 112811. DOI: 10.1016/j.patcog.2025.112811. URL: https://www.sciencedirect.com/science/article/pii/S0031320325014748 (cit. on p. 94).

Berend, J., R. Achtibat, D. Schäffer, A. Binder, W. Samek, S. Lapuschkin, M. Dreyer (2026): *ResLRP: The Role of Residual Cancellation in Attribution Instability in Vision Transformers*. Preprint, version 1. DOI: 10.48550/arXiv.2609.17152. arXiv: 2609.17152v1 [cs.CV]. URL: https://arxiv.org/abs/2609.17152v1 (cit. on p. 49).

Bhatt, U., A. Weller, J. M. F. Moura (2020): Evaluating and Aggregating Feature-based Model Explanations. *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 3016–3022. DOI: 10.24963/ijcai.2020/417. URL: https://www.ijcai.org/proceedings/2020/417 (cit. on p. 5).

Bhattacharyya, A. K. (1943): On a Measure of Divergence between Two Statistical Populations Defined by Their Probability Distributions. *Bulletin of the Calcutta Mathematical Society* 35, 99–109. URL: https://dspace.bcu-iasi.ro/handle/123456789/43660 (cit. on p. 80).

Binder, A., G. Montavon, S. Lapuschkin, K.-R. Müller, W. Samek (2016): Layer-wise Relevance Propagation for Neural Networks with Local Renormalization Layers. *International Conference on Artificial Neural Networks (ICANN)*. Vol. 9887. Lecture Notes in Computer Science. Springer, 63–71. DOI: 10.1007/978-3-319-44781-0_8. URL: https://link.springer.com/chapter/10.1007/978-3-319-44781-0_8 (cit. on p. 43).

Binder, A., L. Weber, S. Lapuschkin, G. Montavon, K.-R. Müller, W. Samek (2023): Shortcomings of Top-Down Randomization-Based Sanity Checks for Evaluations of Deep Neural Network Explanations. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 16143–16152. DOI: 10.1109/cvpr52729.2023.01549. URL: http://dx.doi.org/10.1109/CVPR52729.2023.01549 (cit. on pp. 66, 86).

Blücher, S., J. Vielhaben, N. Strodthoff (2024): Decoupling Pixel Flipping and Occlusion Strategy for Consistent XAI Benchmarks. *Transactions on Machine Learning Research*. ISSN: 2835-8856. arXiv: 2401.06654. URL: https://openreview.net/forum?id=bIiLXdtUVM (cit. on p. 93).

Boyd, S., L. Vandenberghe (2004): *Convex Optimization*. Cambridge University Press. ISBN: 9780511804441. DOI: 10.1017/cbo9780511804441. URL: http://dx.doi.org/10.1017/CBO9780511804441 (cit. on p. 15).

Brandenburg, M.-C., M. Grillo, C. Hertrich (2025): Decomposition Polyhedra of Piecewise Linear Functions. *International Conference on Learning Representations*. Ed. by Y. Yue, A. Garg, N. Peng, F. Sha, R. Yu. Vol. 2025, 62808–62837. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/9d66f74820f11ce037fb5f711ab9acd4-Abstract-Conference.html (cit. on p. 5).

Calafiore, G. C., S. Gaubert, C. Possieri (2020): A Universal Approximation Result for Difference of Log-Sum-Exp Neural Networks. *IEEE Trans. Neural Networks Learn. Syst.* 31 (12), 5603–5612. DOI: 10.1109/TNNLS.2020.2975051. URL: https://doi.org/10.1109/TNNLS.2020.2975051 (cit. on p. 5).

Chattopadhay, A., A. Sarkar, P. Howlader, V. N. Balasubramanian (2018): Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks. *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 839–847. DOI: 10.1109/WACV.2018.00097. URL: https://doi.org/10.1109/WACV.2018.00097 (cit. on pp. 5, 63).

Chefer, H., S. Gur, L. Wolf (2021a): Generic Attention-Model Explainability for Interpreting Bi-Modal and Encoder-Decoder Transformers. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 397–406. DOI: 10.1109/ICCV48922.2021.00045. URL: https://openaccess.thecvf.com/content/ICCV2021/html/Chefer_Generic_Attention-Model_Explainability_for_Interpreting_Bi-Modal_and_Encoder-Decoder_Transformers_ICCV_2021_paper.html (cit. on pp. 6, 49, 76, 77).

Chefer, H., S. Gur, L. Wolf (2021b): Transformer Interpretability Beyond Attention Visualization. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 782–791. DOI: 10.11

09/CVPR46437.2021.00084. URL: https://openaccess.thecvf.com/content/CVPR2021/html/Chefer_Transformer_Interpretability_Beyond_Attention_Visualization_CVPR_2021_paper.html (cit. on pp. 6, 43, 49, 76, 77).

Chen, H., Y. Wang, W. Lin, Z. Ding (2025): Recurrent Convex Difference Neural Networks for Safety-Critical Model Predictive Control. *IEEE Robotics and Automation Letters* 10 (6), 6400–6407. DOI: 10.1109/LRA.2025.3568315. URL: https://ieeexplore.ieee.org/document/10993304/ (cit. on pp. 5, 67).

Chormai, P., J. Herrmann, K.-R. Müller, G. Montavon (2024): Disentangled Explanations of Neural Network Predictions by Finding Relevant Subspaces. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46 (11), 7283–7299. DOI: 10.1109/TPAMI.2024.3388275. URL: https://doi.org/10.1109/TPAMI.2024.3388275 (cit. on p. 60).

Clevert, D., T. Unterthiner, S. Hochreiter (2016): Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*. Ed. by Y. Bengio, Y. LeCun. URL: http://arxiv.org/abs/1511.07289 (cit. on p. 41).

Dalal, N., B. Triggs (2005): Histograms of Oriented Gradients for Human Detection. *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*. Vol. 1. IEEE, 886–893. DOI: 10.1109/CVPR.2005.177. URL: https://doi.org/10.1109/CVPR.2005.177 (cit. on p. 86).

Daniels, H., M. Velikova (2010): Monotone and partially monotone neural networks. *Trans. Neur. Netw.* 21 (6), 906–917. ISSN: 1045-9227. DOI: 10.1109/TNN.2010.2044803. URL: https://doi.org/10.1109/TNN.2010.2044803 (cit. on pp. 5, 9).

Darcet, T., M. Oquab, J. Mairal, P. Bojanowski (2024): Vision Transformers Need Registers. *ICLR*. OpenReview.net. URL: https://proceedings.iclr.cc/paper_files/paper/2024/hash/0b408293619f725fd30162af057e531a-Abstract-Conference.html (cit. on pp. 6, 49).

Deng, J., W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei (2009): ImageNet: A Large-Scale Hierarchical Image Database. *2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 248–255. DOI: 10.1109/CVPR.2009.5206848. URL: https://doi.org/10.1109/CVPR.2009.5206848 (cit. on p. 75).

Dombrowski, A.-K., M. Alber, C. J. Anders, M. Ackermann, K.-R. Müller, P. Kessel (2019): Explanations Can Be Manipulated and Geometry Is to Blame. *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. URL: https://papers.nips.cc/paper/2019/hash/bb836c01cdc9120a9c984c525e4b1a4a-Abstract.html (cit. on p. 5).

Dombrowski, A.-K., C. J. Anders, K.-R. Müller, P. Kessel (2022): Towards robust explanations for deep neural networks. *Pattern Recognition* 121, 108194. ISSN: 0031-3203. DOI: 10.1016/j.patcog.2021.108194. URL: http://dx.doi.org/10.1016/j.patcog.2021.108194 (cit. on pp. 6, 43, 91).

Doshi-Velez, F., B. Kim (2017): *Towards A Rigorous Science of Interpretable Machine Learning*. DOI: 10.48550/arXiv.1702.08608. arXiv: 1702.08608. URL: https://arxiv.org/abs/1702.08608 (cit. on p. 1).

Dosovitskiy, A., L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby (2021): An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*. OpenReview.net. URL: https://openreview.net/forum?id=YicbFdNTTy (cit. on pp. 49, 70, 75, 92).

Dreyer, M., J. Berend, T. Labarta, J. Vielhaben, T. Wiegand, S. Lapuschkin, W. Samek (2025): Mechanistic understanding and validation of large AI models with SemanticLens. *Nature Machine Intelligence* 7 (9), 1572–1585. DOI: 10.1038/s42256-025-01084-w. URL: https://www.nature.com/articles/s42256-025-01084-w (cit. on p. 79).

Efron, B., R. J. Tibshirani (1993): *An Introduction to the Bootstrap*. Vol. 57. Monographs on Statistics and Applied Probability. Chapman & Hall. ISBN: 9780412042317. DOI: 10.1007/978-1-4899-4541-9. URL: https://www.routledge.com/An-Introduction-to-the-Bootstrap/Efron-Tibshirani/p/book/9780429246593 (cit. on pp. 76, 97).

Everingham, M., L. Van Gool, C. K. I. Williams, J. Winn, A. Zisserman (2010): The PASCAL Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision* 88 (2), 303–338. DOI: 10.1007/s11263-009-0275-4. URL: https://doi.org/10.1007/s11263-009-0275-4 (cit. on p. 76).

Eykholt, K., I. Evtimov, E. Fernandes, B. Li, A. Rahmati, C. Xiao, A. Prakash, T. Kohno, D. Song (2018): Robust Physical-World Attacks on Deep Learning Visual Classification. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1625–1634. DOI: 10.1109/CVPR.2018.00175. URL: https://openaccess.thecvf.com/content_cvpr_2018/html/Eykholt_Robust_Physical-World_Attacks_CVPR_2018_paper.html (cit. on p. 1).

Fei, Y., Z. Yang, Y. Chen, Z. Wang, Q. Xie (2020): Risk-Sensitive Reinforcement Learning: Near-Optimal Risk-Sample Tradeoff in Regret. *NeurIPS*. URL: https://papers.neurips.cc/paper_files/paper/2020/hash/fdc42b6b0ee16a2f866281508ef56730-Abstract.html (cit. on pp. 6, 47).

Fel, T., M. Ducoffe, D. Vigouroux, R. Cadène, M. Capelle, C. Nicodème, T. Serre (2023): Don't Lie to Me! Robust and Efficient Explainability With Verified Perturbation Analysis. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 16153–16163. DOI: 10.1109/CVPR52279.2023.01550. URL: https://openaccess.thecvf.com/content/CVPR2023/html/Fel_Dont_Lie_to_Me_Robust_and_Efficient_Explainability_With_Verified_CVPR_2023_paper.html (cit. on pp. 5, 60).

Fong, R. C., A. Vedaldi (2017): Interpretable Explanations of Black Boxes by Meaningful Perturbation. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 3429–3437. DOI: 10.1109/ICCV.2017.371. URL: https://openaccess.thecvf.com/content_iccv_2017/html/Fong_Interpretable_Explanations_of_ICCV_2017_paper.html (cit. on pp. 5, 60).

Gao, S., Z.-Y. Li, M.-H. Yang, M.-M. Cheng, J. Han, P. Torr (2023): Large-Scale Unsupervised Semantic Segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45 (6), 7457–7476. DOI: 10.1109/TPAMI.2022.3218275. arXiv: 2106.03149 [cs.CV]. URL: https://doi.org/10.1109/TPAMI.2022.3218275 (cit. on pp. 4, 76, 95).

Gast, J., S. Roth (2018): Lightweight Probabilistic Deep Networks. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 3369–3378. DOI: 10.1109/cvpr.2018.00355. URL: http://dx.doi.org/10.1109/cvpr.2018.00355 (cit. on p. 47).

Gaubert, S., Y. Vlassopoulos (2025): *Relu and softplus neural nets as zero-sum turn-based games*. DOI: 10.48550/arXiv.2512.20582. arXiv: 2512.20582 [cs.LG]. URL: https://arxiv.org/abs/2512.20582 (cit. on pp. 3, 6, 13–15, 43, 91).

Geirhos, R., J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, F. A. Wichmann (2020): Shortcut learning in deep neural networks. *Nature Machine Intelligence* 2 (11), 665–673. DOI: 10.1038/s42256-020-00257-z. URL: https://www.nature.com/articles/s42256-020-00257-z (cit. on p. 1).

Geist, M., B. Scherrer, O. Pietquin (2019): A Theory of Regularized Markov Decision Processes. *Proceedings of the 36th International Conference on Machine Learning*. Ed. by K. Chaudhuri, R. Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2160–2169. URL: https://proceedings.mlr.press/v97/geist19a.html (cit. on p. 46).

Ghorbani, A., A. Abid, J. Zou (2019): Interpretation of Neural Networks Is Fragile. *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (01), 3681–3688. ISSN: 2159-5399. DOI: 10.1609/aaai.v33i01.33013681. URL: http://dx.doi.org/10.1609/aaai.v33i01.33013681 (cit. on pp. 1, 5).

Gildenblat, J., contributors (2021): *PyTorch library for CAM methods*. https://github.com/jacobgil/pytorch-grad-cam, 2021. URL: https://github.com/jacobgil/pytorch-grad-cam (cit. on p. 76).

Goodfellow, I., D. Warde-Farley, M. Mirza, A. Courville, Y. Bengio (2013): Maxout Networks. *Proceedings of the 30th International Conference on Machine Learning*. Ed. by S. Dasgupta, D. McAllester. Vol. 28. Proceedings of Machine Learning Research 3. Atlanta, Georgia, USA: PMLR, 1319–1327. URL: https://proceedings.mlr.press/v28/goodfellow13.html (cit. on pp. 10, 21, 50, 67).

Griewank, A., A. Walther (2020): Polyhedral DC Decomposition and DCA Optimization of Piecewise Linear Functions. *Algorithms* 13 (7), 166. DOI: 10.3390/a13070166. URL: https://www.mdpi.com/1999-4893/13/7/166 (cit. on p. 5).

Gu, J., Y. Yang, V. Tresp (2019): Understanding Individual Decisions of CNNs via Contrastive Backpropagation. *Computer Vision – ACCV 2018, Revised Selected Papers, Part III*. Vol. 11363. Lecture Notes in Computer Science. Springer, 119–134. DOI: 10.1007/978-3-030-20893-6_8. URL: https://link.springer.com/chapter/10.1007/978-3-030-20893-6_8 (cit. on p. 43).

Guillemot, M., C. Heusèle, R. Korichi, S. Schnebert, L. Chen (2020): Breaking Batch Normalization for better explainability of Deep Neural Networks through Layer-wise Relevance Propagation. *Conférence sur l'Apprentissage Automatique (CAp)*. Accepted paper published online; conference event cancelled. arXiv: 2002.11018 [cs.LG]. URL: https://cap-rfiap2020.sciencesconf.org/data/pages/CAp2020_paper_1.pdf (cit. on p. 40).

Gupta, M., A. Cotter, J. Pfeifer, K. Voevodski, K. Canini, A. Mangylov, W. Moczydlowski, A. van Esbroeck (2016): Monotonic Calibrated Interpolated Look-Up Tables. *Journal of Machine Learning Research* 17 (109), 1–47. URL: http://jmlr.org/papers/v17/15-243.html (cit. on p. 5).

Hanna, M., S. Pezzelle, Y. Belinkov (2024): Have Faith in Faithfulness: Going Beyond Circuit Overlap When Finding Model Mechanisms. *First Conference on Language Modeling*. arXiv: 2403.17806 [cs.LG]. URL: https://openreview.net/forum?id=TZ0CCGDcuT (cit. on pp. 6, 49).

He, K., X. Zhang, S. Ren, J. Sun (2015): Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, 1026–1034. DOI: 10.1109/ICCV.2015.123. URL: https://openaccess.thecvf.com/content_iccv_2015/html/He_Delving_Deep_into_ICCV_2015_paper.html (cit. on p. 86).

He, K., X. Zhang, S. Ren, J. Sun (2016): Deep Residual Learning for Image Recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 770–778. DOI: 10.1109/cvpr.2016.90. URL: http://dx.doi.org/10.1109/CVPR.2016.90 (cit. on pp. 39, 64, 75).

Hedström, A., P. L. Bommer, T. F. Burns, S. Lapuschkin, W. Samek, M. M.-C. Höhne (2025): Evaluating Interpretable Methods via Geometric Alignment of Functional Distortions. *Transactions on Machine Learning Research*. ISSN: 2835-8856. URL: https://openreview.net/forum?id=ukLxqA8zXj (cit. on p. 84).

Hedström, A., L. Weber, D. Krakowczyk, D. Bareeva, F. Motzkus, W. Samek, S. Lapuschkin, M. M.-C. Höhne (2023): Quantus: An Explainable AI Toolkit for Responsible Evaluation of Neural Network Explanations and Beyond. *Journal of Machine Learning Research* 24 (34), 1–11. URL: https://jmlr.org/papers/v24/22-0142.html (cit. on pp. 1, 5, 64, 76, 97).

Hellinger, E. (1909): Neue Begründung der Theorie quadratischer Formen von unendlichvielen Veränderlichen. *Journal für die reine und angewandte Mathematik* 136, 210–271. DOI: 10.1515/crll.1909.136.210. URL: https://doi.org/10.1515/crll.1909.136.210 (cit. on p. 80).

Hendrycks, D., K. Gimpel (2016): Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*. DOI: 10.48550/arXiv.1606.08415. URL: https://arxiv.org/abs/1606.08415 (cit. on pp. 23, 35, 91).

Hertrich, C., G. Loho (2026): Neural Networks and (Virtual) Extended Formulations. *SIAM Journal on Mathematics of Data Science* 8 (4), 1350–1369. DOI: 10.1137/25M180130X. arXiv: 2411.03006 [math.CO]. URL: https://epubs.siam.org/doi/10.1137/25M180130X (cit. on p. 51).

Holm, S. (1979): A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6 (2), 65–70. URL: https://www.jstor.org/stable/4615733 (cit. on pp. 76, 98).

Huchette, J., G. Muñoz, T. Serra, C. Tsay (2026): When Deep Learning Meets Polyhedral Theory: A Survey. *INFORMS Journal on Computing*. Articles in Advance. DOI: 10.1287/ijoc.2024.0902. arXiv: 2305.00241 [math.OC]. URL: https://pubsonline.informs.org/doi/10.1287/ijoc.2024.0902 (cit. on pp. 5, 9).

Hui, L. Y. W., A. Binder (2019): BatchNorm Decomposition for Deep Neural Network Interpretation. *Advances in Computational Intelligence – IWANN 2019*. Vol. 11507. Lecture Notes in Computer Science. Springer, 280–291. DOI: 10.1007/978-3-030-20518-8_24. URL: https://link.springer.com/chapter/10.1007/978-3-030-20518-8_24 (cit. on p. 40).

Jiang, P.-T., C.-B. Zhang, Q. Hou, M.-M. Cheng, Y. Wei (2021): LayerCAM: Exploring Hierarchical Class Activation Maps for Localization. *IEEE Transactions on Image Processing* 30, 5875–5888. DOI: 10.1109/TIP.2021.3089943. URL: https://doi.org/10.1109/TIP.2021.3089943 (cit. on pp. 5, 63).

Kahardipraja, P., R. Achtibat, T. Wiegand, W. Samek, S. Lapuschkin (2025): The Atlas of In-Context Learning: How Attention Heads Shape In-Context Retrieval Augmentation. *NeurIPS*. DOI: 10.52202/085713-3940. URL: https://proceedings.nips.cc/paper_files/paper/2025/hash/ab49f5e61eadbe979373c133232a96a0-Abstract-Conference.html (cit. on pp. 1, 6, 49).

Kauffmann, J., K.-R. Müller, G. Montavon (2020a): Towards explaining anomalies: A deep Taylor decomposition of one-class models. *Pattern Recognition* 101, 107198. ISSN: 0031-3203. DOI: 10.1016/j.patcog.2020.107198. URL: https://www.sciencedirect.com/science/article/pii/S0031320320300054 (cit. on p. 1).

Kauffmann, J., L. Ruff, G. Montavon, K.-R. Müller (2020b): *The Clever Hans Effect in Anomaly Detection*. DOI: 10.48550/arXiv.2006.10609. arXiv: 2006.10609 [cs.LG]. URL: https://arxiv.org/abs/2006.10609 (cit. on p. 1).

Kazda, K., X. Li (2024): A linear programming approach to difference-of-convex piecewise linear approximation. *European Journal of Operational Research* 312 (2), 493–511. ISSN: 0377-2217. DOI: 10.1016/j.ejor.2023.07.026. URL: https://www.sciencedirect.com/science/article/pii/S0377221723005647 (cit. on p. 5).

Kindermans, P.-J., K. T. Schütt, M. Alber, K.-R. Müller, D. Erhan, B. Kim, S. Dähne (2018): Learning How to Explain Neural Networks: PatternNet and PatternAttribution. *International Conference on Learning Representations (ICLR)*. URL: https://openreview.net/forum?id=Hkn7CBaTW (cit. on pp. 1, 5).

Kohlbrenner, M., A. Bauer, S. Nakajima, A. Binder, W. Samek, S. Lapuschkin (2020): Towards Best Practice in Explaining Neural Network Decisions with LRP. *2020 International Joint Conference on Neural Networks*

*(IJCNN)*. IEEE, 1–7. DOI: 10.1109/IJCNN48605.2020.9206975. URL: https://doi.org/10.1109/IJCNN48605.2020.9206975 (cit. on pp. 43, 64, 76, 96, 97).

Kokhlikyan, N., V. Miglani, M. Martin, E. Wang, B. Alsallakh, J. Reynolds, A. Melnikov, N. Kliushkina, C. Araya, S. Yan, O. Reblitz-Richardson (2020): *Captum: A unified and generic model interpretability library for PyTorch*. DOI: 10.48550/arXiv.2009.07896. arXiv: 2009.07896 [cs.LG]. URL: https://arxiv.org/abs/2009.07896 (cit. on p. 76).

Kolek, S., R. Windesheim, H. Andrade-Loarca, G. Kutyniok, R. Levie (2023): Explaining Image Classifiers With Multiscale Directional Image Representation. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 18600–18609. DOI: 10.1109/CVPR52729.2023.01784. URL: https://openaccess.thecvf.com/content/CVPR2023/html/Kolek_Explaining_Image_Classifiers_With_Multiscale_Directional_Image_Representation_CVPR_2023_paper.html (cit. on pp. 5, 60).

Komorowski, P., E. Golimblevskaia, R. Achtibat, T. Wiegand, S. Lapuschkin, W. Samek (2026): Attribution-Guided Decoding. *The Fourteenth International Conference on Learning Representations*. arXiv: 2509.26307 [cs.LG]. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/da30215ee52c1daaaaddada8137cfd0b-Abstract-Conference.html (cit. on pp. 1, 6, 49).

Koutschan, C., B. Moser, A. Ponomarchuk, J. Schicho (2025): Representing piecewise linear functions by functions with small arity. *Applicable Algebra in Engineering, Communication and Computing* 36 (4), 595–610. DOI: 10.1007/s00200-023-00627-1. URL: https://doi.org/10.1007/s00200-023-00627-1 (cit. on p. 5).

Kramár, J., T. Lieberum, R. Shah, N. Nanda (2024): *AtP*: An efficient and scalable method for localizing LLM behaviour to components*. DOI: 10.48550/arXiv.2403.00745. arXiv: 2403.00745 [cs.LG]. URL: https://arxiv.org/abs/2403.00745 (cit. on pp. 6, 49).

Kripfganz, A., R. Schulze (1987): Piecewise affine functions as a difference of two convex functions. *Optimization* 18 (1), 23–29. DOI: 10.1080/02331938708843210. URL: https://www.tandfonline.com/doi/abs/10.1080/02331938708843210 (cit. on pp. 5, 8, 29).

Lapuschkin, S., S. Wäldchen, A. Binder, G. Montavon, W. Samek, K.-R. Müller (2019): Unmasking Clever Hans predictors and assessing what machines really learn. *Nature Communications* 10 (1), 1096. DOI: 10.1038/s41467-019-08987-4. URL: https://doi.org/10.1038/s41467-019-08987-4 (cit. on p. 1).

Le Thi, H. A., T. Pham Dinh (2018): DC programming and DCA: thirty years of developments. *Mathematical Programming* 169 (1), 5–68. DOI: 10.1007/s10107-018-1235-y. URL: https://link.springer.com/article/10.1007/s10107-018-1235-y (cit. on pp. 5, 94).

Lee, D. D., H. S. Seung (1999): Learning the Parts of Objects by Non-Negative Matrix Factorization. *Nature* 401 (6755), 788–791. DOI: 10.1038/44565. URL: https://doi.org/10.1038/44565 (cit. on pp. 2, 63).

Lipton, Z. C. (2018): The Mythos of Model Interpretability. *Communications of the ACM* 61 (10), 36–43. DOI: 10.1145/3233231. URL: https://arxiv.org/abs/1606.03490 (cit. on p. 1).

Liu, X., X. Han, N. Zhang, Q. Liu (2020): Certified Monotonic Neural Networks. *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 15427–15438. URL: https://proceedings.neurips.cc/paper_files/paper/2020/hash/b139aeda1c2914e3b579aafd3ceeb1bd-Abstract.html (cit. on pp. 5, 9, 69).

Lundberg, S. M., S.-I. Lee (2017): A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett. Vol. 30. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html (cit. on pp. 5, 6, 43, 60).

Maeß, J., G. Montavon, S. Nakajima, K.-R. Müller, T. Schnake (2026): Uncovering the Structure of Explanation Quality with Spectral Analysis. *Explainable Artificial Intelligence*. Vol. 2578. Communications in Computer and Information Science. Springer, 289–309. DOI: 10.1007/978-3-032-08327-2_14. URL: https://link.springer.com/chapter/10.1007/978-3-032-08327-2_14 (cit. on p. 79).

McNemar, Q. (1947): Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12 (2), 153–157. DOI: 10.1007/BF02295996. URL: https://doi.org/10.1007/BF02295996 (cit. on p. 97).

Melzer, D. (1986): On the expressibility of piecewise-linear continuous functions as the difference of two piecewise-linear convex functions. In: *Quasidifferential Calculus*. Ed. by V. F. Demyanov, L. C. W. Dixon. Vol. 29. Mathematical Programming Studies. Berlin, Heidelberg: Springer, 118–134. DOI: 10.1007/BFb0121142. URL: https://link.springer.com/chapter/10.1007/BFb0121142 (cit. on pp. 5, 8, 29).

Montavon, G., A. Binder, S. Lapuschkin, W. Samek, K.-R. Müller (2019): Layer-Wise Relevance Propagation: An Overview. In: *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Ed. by W. Samek, G. Montavon, A. Vedaldi, L. K. Hansen, K.-R. Müller. Cham: Springer International Publishing, 193–209.

ISBN: 978-3-030-28954-6. DOI: 10.1007/978-3-030-28954-6_10. URL: https://doi.org/10.1007/978-3-030-28954-6_10 (cit. on p. 64).

Montavon, G., S. Lapuschkin, A. Binder, W. Samek, K.-R. Müller (2017): Explaining nonlinear classification decisions with deep Taylor decomposition. *Pattern Recognition* 65, 211–222. DOI: 10.1016/j.patcog.2016.11.008. URL: https://www.sciencedirect.com/science/article/pii/S0031320316303582 (cit. on pp. 5, 43, 44, 60).

Montavon, G., W. Samek, K.-R. Müller (2018): Methods for interpreting and understanding deep neural networks. *Digital Signal Processing* 73, 1–15. DOI: 10.1016/j.dsp.2017.10.011. URL: https://www.sciencedirect.com/science/article/pii/S1051200417302385 (cit. on pp. 5, 64, 97).

Murdoch, W. J., C. Singh, K. Kumbier, R. Abbasi-Asl, B. Yu (2019): Definitions, methods, and applications in interpretable machine learning. *Proceedings of the National Academy of Sciences* 116 (44), 22071–22080. DOI: 10.1073/pnas.1900654116. URL: https://doi.org/10.1073/pnas.1900654116 (cit. on p. 1).

Muttenthaler, L., K. Greff, F. Born, B. Spitzer, S. Kornblith, M. C. Mozer, K.-R. Müller, T. Unterthiner, A. K. Lampinen (2025): Aligning machine and human visual representations across abstraction levels. *Nature* 647 (8089), 349–355. DOI: 10.1038/s41586-025-09631-6. URL: https://doi.org/10.1038/s41586-025-09631-6 (cit. on p. 1).

Nam, W., S. Gur, J. Choi, L. Wolf, S. Lee (2020): Relative Attributing Propagation: Interpreting the Comparative Contributions of Individual Units in Deep Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 34. 3. AAAI Press, 2501–2508. DOI: 10.1609/aaai.v34i03.5632. URL: https://ojs.aaai.org/index.php/AAAI/article/view/5632 (cit. on p. 43).

Nguyen, A.-P., D. L. Moreno, N. Le-Bel, M. Rodríguez Martínez (2023): MonoNet: enhancing interpretability in neural networks via monotonic features. *Bioinformatics Advances* 3 (1), vbad016. ISSN: 2635-0041. DOI: 10.1093/bioadv/vbad016. eprint: https://academic.oup.com/bioinformaticsadvances/article-pdf/3/1/vbad016/50156829/vbad016.pdf. URL: https://doi.org/10.1093/bioadv/vbad016 (cit. on p. 5).

Nie, W., Y. Zhang, A. B. Patel (2018): A Theoretical Explanation for Perplexing Behaviors of Backpropagation-based Visualizations. *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 3809–3818. URL: https://proceedings.mlr.press/v80/nie18a.html (cit. on pp. 1, 5).

Osborne, M. J., A. Rubinstein (1994): *A Course in Game Theory*. MIT Press. ISBN: 9780262650403. URL: https://mitpress.mit.edu/9780262150415/a-course-in-game-theory/ (cit. on p. 45).

Pahde, F., M. Dreyer, W. Samek, S. Lapuschkin (2023): Reveal to Revise: An Explainable AI Life Cycle for Iterative Bias Correction of Deep Models. *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023*. Vol. 14221. Lecture Notes in Computer Science. Springer Nature Switzerland, 596–606. DOI: 10.1007/978-3-031-43895-0_56. URL: https://doi.org/10.1007/978-3-031-43895-0_56 (cit. on pp. 1, 6).

Pahde, F., T. Wiegand, S. Lapuschkin, W. Samek (2025): Ensuring medical AI safety: interpretability-driven detection and mitigation of spurious model behavior and associated data. *Machine Learning* 114 (9), 206. DOI: 10.1007/s10994-025-06834-w. URL: https://link.springer.com/article/10.1007/s10994-025-06834-w (cit. on p. 93).

Paszke, A., S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala (2019): PyTorch: an imperative style, high-performance deep learning library. *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html (cit. on pp. 39, 75).

Phuong, M., C. H. Lampert (2020): Functional vs. parametric equivalence of ReLU networks. *ICLR*. OpenReview.net. URL: https://iclr.cc/virtual_2020/poster_Bylx-TNKvH.html (cit. on p. 5).

Puterman, M. L. (1994): *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons. DOI: 10.1002/9780470316887. URL: https://doi.org/10.1002/9780470316887 (cit. on p. 11).

Ramachandran, P., B. Zoph, Q. V. Le (2018): Searching for Activation Functions. *International Conference on Learning Representations (ICLR), Workshop Track*. Preprint first posted in 2017. arXiv: 1710.05941 [cs.NE]. URL: https://openreview.net/forum?id=Hkuq2EkPf (cit. on pp. 23, 41, 91).

Ribeiro, M. T., S. Singh, C. Guestrin (2016): "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. San Francisco, California, USA: Association for Computing Machinery, 1135–1144.

ISBN: 9781450342322. DOI: 10.1145/2939672.2939778. URL: https://doi.org/10.1145/2939672.2939778 (cit. on pp. 5, 60).

Ross, A. S., F. Doshi-Velez (2018): Improving the Adversarial Robustness and Interpretability of Deep Neural Networks by Regularizing Their Input Gradients. *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. DOI: 10.1609/aaai.v32i1.11504. URL: https://arxiv.org/abs/1711.09404 (cit. on p. 1).

Ross, A. S., M. C. Hughes, F. Doshi-Velez (2017): Right for the Right Reasons: Training Differentiable Models by Constraining their Explanations. *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, 2662–2670. DOI: 10.24963/ijcai.2017/371. URL: https://ijcai.org/proceedings/2017/371 (cit. on p. 1).

Rudin, C. (2019): Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence* 1 (5), 206–215. DOI: 10.1038/s42256-019-0048-x. URL: https://doi.org/10.1038/s42256-019-0048-x (cit. on p. 2).

Runje, D., S. M. Shankaranarayana (2023): Constrained Monotonic Neural Networks. *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 29338–29353. URL: https://proceedings.mlr.press/v202/runje23a.html (cit. on pp. 5, 9, 10).

Russakovsky, O., J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, L. Fei-Fei (2015): ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision* 115 (3), 211–252. ISSN: 1573-1405. DOI: 10.1007/s11263-015-0816-y. URL: http://dx.doi.org/10.1007/s11263-015-0816-y (cit. on p. 76).

Samek, W., A. Binder, G. Montavon, S. Lapuschkin, K.-R. Müller (2017): Evaluating the Visualization of What a Deep Neural Network Has Learned. *IEEE Transactions on Neural Networks and Learning Systems* 28 (11), 2660–2673. DOI: 10.1109/TNNLS.2016.2599820. URL: https://doi.org/10.1109/TNNLS.2016.2599820 (cit. on p. 96).

Sankaranarayanan, P., R. Rengaswamy (2022): CDiNN – Convex difference neural networks. *Neurocomput.* 495 (C), 153–168. ISSN: 0925-2312. DOI: 10.1016/j.neucom.2022.01.024. URL: https://doi.org/10.1016/j.neucom.2022.01.024 (cit. on pp. 5, 67).

Schlüter, N., M. Schulze Darup (2020): Novel Convex Decomposition of Piecewise Affine Functions. *21st IFAC World Congress*. Late-breaking results contribution; reprinted as arXiv:2108.03950 in 2021. arXiv: 2108.03950 [eess.SY]. URL: https://arxiv.org/abs/2108.03950 (cit. on p. 5).

Schnake, T., F. Rezaei Jafari, J. Lederer, P. Xiong, S. Nakajima, S. Gugler, G. Montavon, K.-R. Müller (2025): Towards symbolic XAI — explanation through human understandable logical relationships between features. *Information Fusion* 118, 102923. DOI: 10.1016/j.inffus.2024.102923. URL: https://doi.org/10.1016/j.inffus.2024.102923 (cit. on p. 79).

Selvaraju, R. R., M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra (2017): Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *2017 IEEE International Conference on Computer Vision (ICCV)*, 618–626. DOI: 10.1109/ICCV.2017.74. URL: https://openaccess.thecvf.com/content_iccv_2017/html/Selvaraju_Grad-CAM_Visual_Explanations_ICCV_2017_paper.html (cit. on pp. 5, 60, 63).

Shapley, L. S. (1953): Stochastic Games. *Proceedings of the National Academy of Sciences* 39 (10), 1095–1100. DOI: 10.1073/pnas.39.10.1095. URL: https://doi.org/10.1073/pnas.39.10.1095 (cit. on p. 13).

Shrikumar, A., P. Greenside, A. Kundaje (2017): Learning Important Features Through Propagating Activation Differences. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 3145–3153. URL: https://proceedings.mlr.press/v70/shrikumar17a.html (cit. on pp. 5, 60).

Siahkamari, A., A. Gangrade, B. Kulis, V. Saligrama (2020): Piecewise Linear Regression via a Difference of Convex Functions. *Proceedings of the 37th International Conference on Machine Learning*. Ed. by H. D. III, A. Singh. Vol. 119. Proceedings of Machine Learning Research. PMLR, 8895–8904. URL: https://proceedings.mlr.press/v119/siahkamari20a.html (cit. on pp. 5, 21).

Sill, J. (1997): Monotonic Networks. *Advances in Neural Information Processing Systems*. Vol. 10. MIT Press, 661–667. URL: https://papers.nips.cc/paper_files/paper/1997/file/83adc9225e4deb67d7ce42d58fe5157c-Paper.pdf (cit. on pp. 2, 5, 9, 69).

Simonyan, K., A. Vedaldi, A. Zisserman (2014): Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. *2nd International Conference on Learning Representations, ICLR*

*2014, Banff, AB, Canada, April 14-16, 2014, Workshop Track Proceedings*. Ed. by Y. Bengio, Y. LeCun. URL: http://arxiv.org/abs/1312.6034 (cit. on pp. 5, 60).

Simonyan, K., A. Zisserman (2015): Very Deep Convolutional Networks for Large-Scale Image Recognition. *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*. Ed. by Y. Bengio, Y. LeCun. URL: http://arxiv.org/abs/1409.1556 (cit. on pp. 39, 64, 75).

Sivaprasad, S., A. Singh, N. Manwani, V. Gandhi (2021): The Curious Case of Convex Neural Networks. *Machine Learning and Knowledge Discovery in Databases. Research Track*. Ed. by N. Oliver, F. Pérez-Cruz, S. Kramer, J. Read, J. A. Lozano. Cham: Springer International Publishing, 738–754. ISBN: 978-3-030-86486-6. DOI: 10.1007/978-3-030-86486-6_45. URL: https://link.springer.com/chapter/10.1007/978-3-030-86486-6_45 (cit. on p. 5).

Sivaraman, A., G. Farnadi, T. Millstein, G. V. den Broeck (2020): Counterexample-Guided Learning of Monotonic Neural Networks. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. Method/tool: COMET. URL: https://proceedings.neurips.cc/paper/2020/hash/8ab70731b1553f17c11a3bbc87e0b605-Abstract.html (cit. on p. 5).

Sixt, L., M. Granz, T. Landgraf (2020): When Explanations Lie: Why Many Modified BP Attributions Fail. *ICML*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 9046–9057. URL: https://proceedings.mlr.press/v119/sixt20a.html (cit. on pp. 1, 5, 66, 80, 86, 91).

Smilkov, D., N. Thorat, B. Kim, F. Viégas, M. Wattenberg (2017): *SmoothGrad: removing noise by adding noise*. DOI: 10.48550/arXiv.1706.03825. arXiv: 1706.03825 [cs.LG]. URL: https://arxiv.org/abs/1706.03825 (cit. on pp. 5, 60).

Springenberg, J. T., A. Dosovitskiy, T. Brox, M. Riedmiller (2015): Striving for Simplicity: The All Convolutional Net. *International Conference on Learning Representations (Workshop)*. DOI: 10.48550/arXiv.1412.6806. URL: https://arxiv.org/abs/1412.6806 (cit. on pp. 5, 60).

Sriperumbudur, B. K., G. R. G. Lanckriet (2009): On the Convergence of the Concave-Convex Procedure. *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://papers.nips.cc/paper/3646-on-the-convergence-of-the-concave-convex-procedure (cit. on p. 5).

Sundararajan, M., A. Taly, Q. Yan (2017): Axiomatic Attribution for Deep Networks. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 3319–3328. URL: https://proceedings.mlr.press/v70/sundararajan17a.html (cit. on pp. 5, 60, 61).

Sutton, R. S., A. G. Barto (2018): *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press. URL: https://mitpress.mit.edu/9780262039246/reinforcement-learning/ (cit. on p. 11).

Syed, A., C. Rager, A. Conmy (2024): Attribution Patching Outperforms Automated Circuit Discovery. *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*. Association for Computational Linguistics, 407–416. DOI: 10.18653/v1/2024.blackboxnlp-1.25. URL: http://dx.doi.org/10.18653/v1/2024.blackboxnlp-1.25 (cit. on pp. 6, 49).

Tao, P. D., L. T. H. An (1997): Convex Analysis Approach to D.C. Programming: Theory, Algorithms and Applications. *Acta Mathematica Vietnamica* 22 (1), 289–355. URL: https://math.ac.vn/uploads/files/9701289.pdf (cit. on p. 5).

Theiner, J., E. Müller-Budack, R. Ewerth (2022): Interpretable Semantic Photo Geolocation. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 1474–1484. DOI: 10.1109/WACV51458.2022.00154. URL: https://openaccess.thecvf.com/content/WACV2022/html/Theiner_Interpretable_Semantic_Photo_Geolocation_WACV_2022_paper.html (cit. on p. 96).

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin (2017): Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30, 5998–6008. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html (cit. on pp. 49, 92).

Voita, E., D. Talbot, F. Moiseev, R. Sennrich, I. Titov (2019): Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 5797–5808. DOI: 10.18653/v1/P19-1580. URL: https://aclanthology.org/P19-1580/ (cit. on pp. 6, 49).

Vukadin, D., P. Afrić, M. Šilić, G. Delač (2024): Advancing Attribution-Based Neural Network Explainability through Relative Absolute Magnitude Layer-Wise Relevance Propagation and Multi-Component Evaluation.

*ACM Transactions on Intelligent Systems and Technology* 15 (3), 1–30. ISSN: 2157-6912. DOI: 10.1145/3649458. URL: http://dx.doi.org/10.1145/3649458 (cit. on pp. 43, 49).

Wang, S., M. Gupta (2020): Deontological Ethics By Monotonicity Shape Constraints. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*. Ed. by S. Chiappa, R. Calandra. Vol. 108. Proceedings of Machine Learning Research. PMLR, 2043–2054. URL: https://proceedings.mlr.press/v108/wang20e.html (cit. on p. 2).

Wróbel, A., S. Gairola, J. Tabor, B. Schiele, B. M. Zieliński, D. D. Rymarczyk (2026): DAVE: Distribution-Aware Attribution via ViT Gradient Decomposition. *Proceedings of the 43rd International Conference on Machine Learning*. Vol. 306. Proceedings of Machine Learning Research. PMLR, 135468–135511. arXiv: 2602.06613 [cs.CV]. URL: https://proceedings.mlr.press/v306/wrobel26a.html (cit. on pp. 6, 44, 49, 52, 56, 57, 59, 70, 71, 77).

Xiong, P., T. Schnake, G. Montavon, K.-R. Müller, S. Nakajima (2026): Normalized Relevance Measure as a Unifying Framework to Explain Neural Network Latent Structures. *arXiv preprint arXiv:2606.00557*. DOI: 10.48550/arXiv.2606.00557. arXiv: 2606.00557 [cs.LG]. URL: https://arxiv.org/abs/2606.00557 (cit. on pp. 6, 43, 79, 91, 94).

Yeh, C.-K., C.-Y. Hsieh, A. S. Suggala, D. I. Inouye, P. Ravikumar (2019): On the (In)fidelity and Sensitivity of Explanations. *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 10965–10976. URL: https://proceedings.neurips.cc/paper/2019/hash/a7471fdc77b3435276507cc8f2dc2569-Abstract.html (cit. on pp. 5, 97).

Zalgaller, V. A. (2000): Representation of functions of several variables by differences of convex functions. *Journal of Mathematical Sciences* 100, 2209–2227. DOI: 10.1007/s10958-000-0006-4. URL: https://link.springer.com/article/10.1007/s10958-000-0006-4 (cit. on p. 5).

Zech, J. R., M. A. Badgeley, M. Liu, A. B. Costa, J. J. Titano, E. K. Oermann (2018): Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study. *PLoS Medicine* 15 (11), e1002683. DOI: 10.1371/journal.pmed.1002683. URL: https://doi.org/10.1371/journal.pmed.1002683 (cit. on p. 1).

Zeiler, M. D., R. Fergus (2014): Visualizing and Understanding Convolutional Networks. *Computer Vision – ECCV 2014*. Ed. by D. Fleet, T. Pajdla, B. Schiele, T. Tuytelaars. Cham: Springer International Publishing, 818–833. ISBN: 978-3-319-10590-1. DOI: 10.1007/978-3-319-10590-1_53. URL: https://link.springer.com/chapter/10.1007/978-3-319-10590-1_53 (cit. on pp. 5, 60).

Zhang, J., S. A. Bargal, Z. Lin, J. Brandt, X. Shen, S. Sclaroff (2018a): Top-Down Neural Attention by Excitation Backprop. *International Journal of Computer Vision* 126 (10), 1084–1102. DOI: 10.1007/s11263-017-1059-x. URL: https://doi.org/10.1007/s11263-017-1059-x (cit. on pp. 5, 6, 43, 44, 96).

Zhang, L., G. Naitzat, L.-H. Lim (2018b): Tropical Geometry of Deep Neural Networks. *Proceedings of the 35th International Conference on Machine Learning*. Ed. by J. Dy, A. Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, 5824–5832. URL: https://proceedings.mlr.press/v80/zhang18i.html (cit. on pp. 5, 21).

Zhou, B., A. Khosla, A. Lapedriza, A. Oliva, A. Torralba (2016): Learning Deep Features for Discriminative Localization. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2921–2929. DOI: 10.1109/CVPR.2016.319. URL: https://openaccess.thecvf.com/content_cvpr_2016/html/Zhou_Learning_Deep_Features_CVPR_2016_paper.html (cit. on pp. 60, 63).

Zimmermann, J. P., J. Berend, G. Loho, S. Lapuschkin, W. Samek (2026): *Playing the Network Backward: A Game-Theoretic Attribution Framework*. Preprint, version 2. DOI: 10.48550/arXiv.2605.06212. arXiv: 2605.06212v2 [cs.LG]. URL: https://arxiv.org/abs/2605.06212v2 (cit. on pp. 3, 4, 99).

Zimmermann, J. P., G. Loho (2026): Hidden Monotonicity: Explaining Deep Neural Networks via their DC Decomposition. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Highlight, 24107–24117. arXiv: 2601.07700 [cs.CV]. URL: https://openaccess.thecvf.com/content/CVPR2026/html/Zimmermann_Hidden_Monotonicity_Explaining_Deep_Neural_Networks_via_their_DC_Decomposition_CVPR_2026_paper.html (cit. on pp. 2, 4, 21, 29, 30, 35, 43, 49, 50, 60, 91, 99).